

Базы данных

Тема 1

Введение

Цели курса

- изучение концепций построения и технологий приложений баз данных различного предназначения;
- практическое овладение технологиями баз данных на основе Microsoft SQL Server 2012 и ADO.NET

Применение баз данных

- основная цель создания приложений баз данных - хранение, накопление, обработка и представление информации различного рода;
- области и масштабы применения баз данных могут быть очень обширными – от небольших приложений для настольных компьютеров до крупных хранилищ в центрах обработки данных (ЦОД), доступ к которым осуществляется в том числе с использованием интернет-технологий.

Примеры приложений БД

- цель: хранение, накопление, обработка и представление информации различного рода:
 - адресная/телефонная книга;
 - учет клиентов и заказов фирмы;
 - каталог продукции;
 - электронный каталог библиотеки;
 - «Электронный Университет»;
 - геоинформационные системы (ГИС);
- очевидно, что требования к приложениям могут зависеть как от предметной области, так и от масштабов системы.

Информационная система (ИС)

- Информационная система (*information system*) – варианты определения:
 - система, предназначенная для сбора, обработки, хранения и распространения информации.
 - совокупность содержащейся в базах данных информации и обеспечивающих ее обработку информационных технологий и технических средств (п. 3 ст. 2 Федерального закона № 149-ФЗ "Об информации, информационных технологиях и о защите информации").
 - система обработки информации, работающая совместно с организационными ресурсами, такими как люди, технические средства и финансовые ресурсы, которые обеспечивают и распределяют информацию (стандарт ISO/IEC 2382-1)

Предметная область (domain knowledge)

- часть реального мира, рассматриваемая в пределах данного контекста (области, которая является объектом некоторой деятельности);
- определяет множество рассматриваемых объектов, а также их свойства, отношения и функциональные связи

Бизнес-логика

- Бизнес-логика (*domain logic, business logic*)
 - совокупность правил, принципов, зависимостей поведения объектов предметной области.
- Бизнес-процесс (*business process, business method, or business function*)
 - совокупность взаимосвязанных и структурированных задач и мероприятий (производственных, административных и т.п.), направленных на достижение определенной цели или результата в рамках организации.
- Бизнес-правила (*business rules*)
 - абстракция политик и практик организации (в рамках бизнес-процессов).

Требования

- Требование (*requirement*) – некоторое условие, выраженное чётко и однозначно и часто имеющее количественное описание, выполнение которого необходимо для того, чтобы результат работы считался приемлемым.
- Спецификация (*specification*) – набор требований (используемый при разработке и тестировании).
- Функциональные требования (*functional requirements*) – детальное описание функций (возможностей) и данных, необходимых для реализации автоматизируемых информационной системой бизнес-процессов.
- Нефункциональные требования (*non-functional requirements*) – детальное описание условий и ограничений, при которых система должна сохранять свою эффективность.

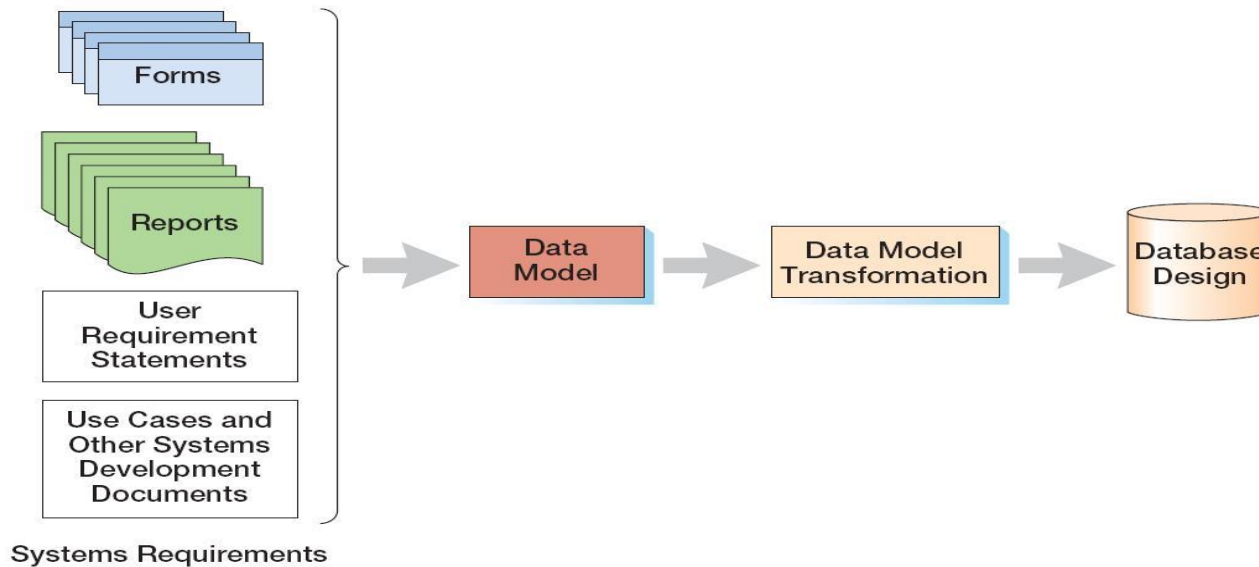
Функциональные требования

- пользовательские сценарии (*use cases*) являются одним из средств представления функциональных требований;
- примеры (в обобщённом виде):
 - выполнение аутентификации пользователей;
 - ввод данных;
 - проверка корректности ввода / операций
 - загрузка / выгрузка данных;
 - обработка данных;
 - представление данных (построение определённых форм и отчётов);
 - поиск информации;
 - ...

Нефункциональные требования

- производительность;
- масштабируемость:
 - объем накапливаемой информации;
 - многопользовательский режим и количество пользователей;
 - производительность;
- надежность и отказоустойчивость;
- низкая сложность администрирования и поддержки;
- низкие накладные расходы при модификации базы данных и приложения (рефакторинге);
- переносимость;
- требования к квалификации пользователей;
- соответствие стандартам;
- ...

Три уровня моделей информационных систем



- внешние модели (*external schema*): представление пользователей о системе (user view);
- концептуальная модель (*conceptual schema*): абстрактное представление системы (данных);
- внутренние модели (*internal schema*).

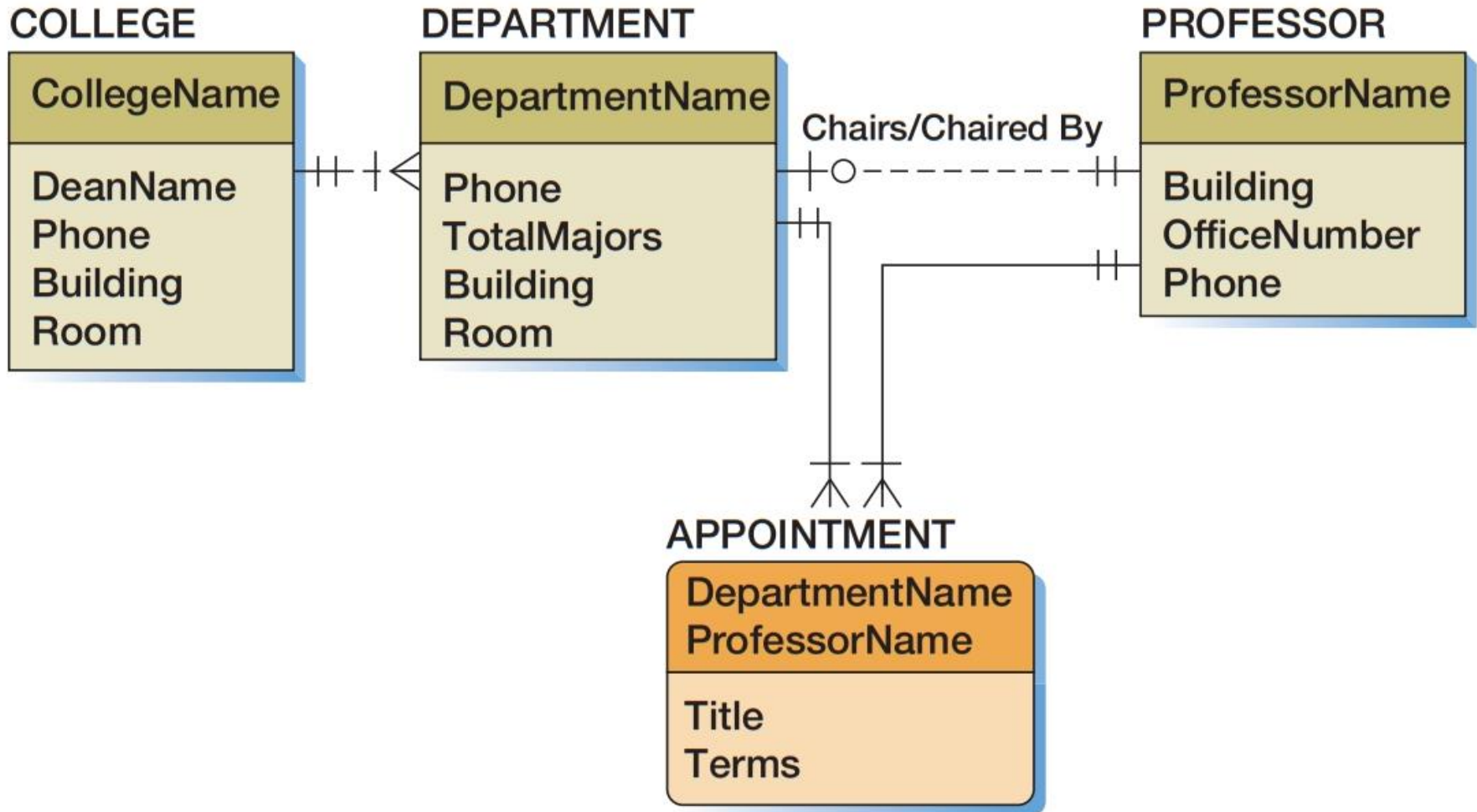
Модели данных (ANSI)

- внешняя модель (*external schema*) – набор представлений пользователей о той части предметной области, с которой он сталкивается;
- концептуальная модель (*conceptual schema / conceptual data model*) – набор ключевых объектов предметной области, их взаимосвязей (взаимодействий) и ограничений, накладываемых на объекты;
- логическая модель (*logical schema / logical data model / information schema*) – представление концептуальной модели в соответствии с логической моделью, используемой для хранения данных;
- физическая модель (*physical data model*) – описание физического представления, хранения и обработки данных.

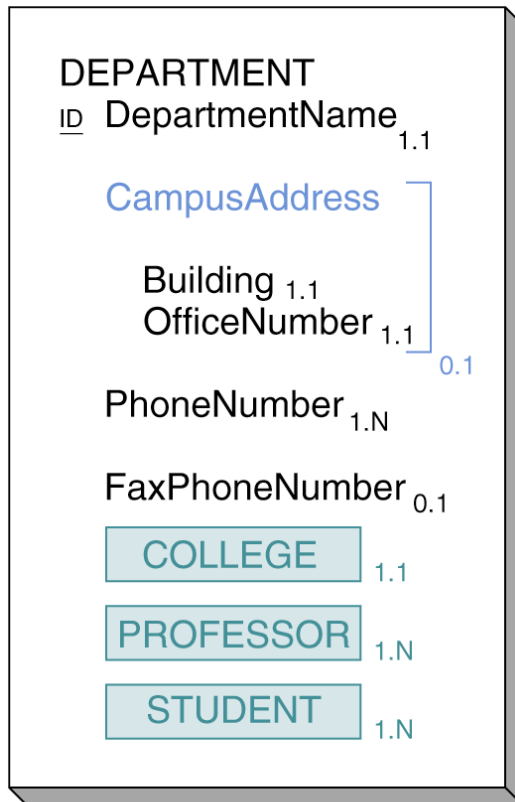
Моделирование данных

- база данных является «моделью модели», то есть через объекты базы данных описывает представление пользователей о конкретной предметной области;
- необходимо сформулировать требования и построить на их основе модель данных, что является во многом творческой задачей, решение которой основано на опыте и интуиции;
- моделирование данных (*data modeling*) - процесс создания логического представления базы данных (пользовательская модель данных */user data modell*, модель требований к данным */requirements data modell*, концептуальная модель */conceptual data modell*);
- модель данных содержит языковые и изобразительные стандарты для своего представления:
 - модель сущность – связь (*entity-relationship model*);
 - модель семантических объектов (*semantic object model*);

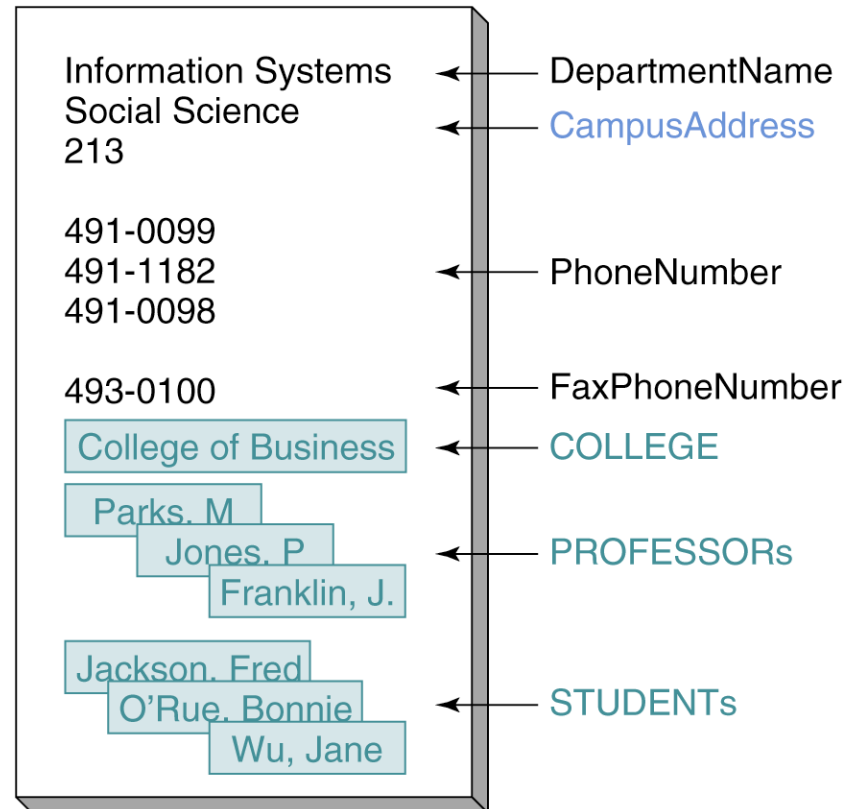
Модель сущность-связь



Модель семантических объектов

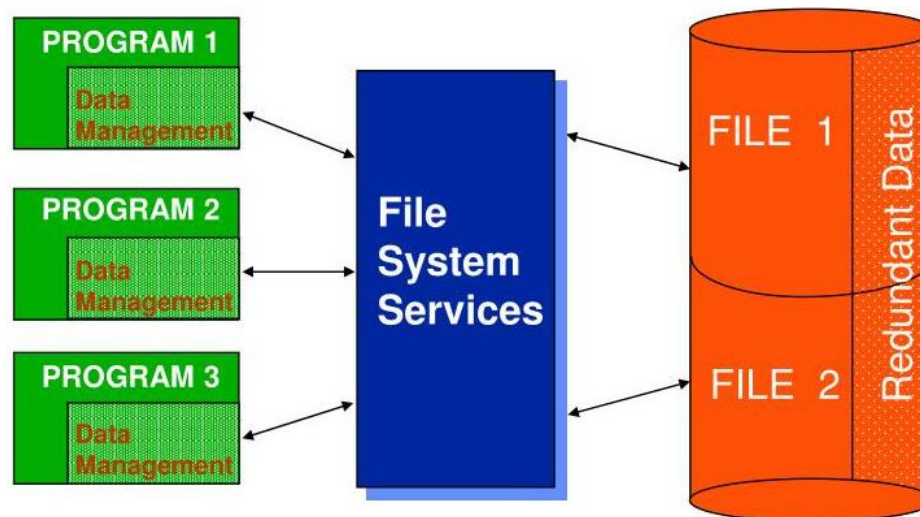


(b)



Системы обработки файлов

- разделенные и изолированные файлы, каждому приложению может соответствовать свой набор файлов;
- зависимость прикладных программ от форматов файлов (в том числе, даже если некоторые поля не используются) и несовместимость файлов;
- дублирование данных (*data duplication*) – возможное нарушение целостности данных (*data integrity*);
- трудность представления данных в файлах в различных видах / совместного использования данных (в том числе из-за отсутствия явно определенной связи между данными);
- сложность разработки и поддержки.

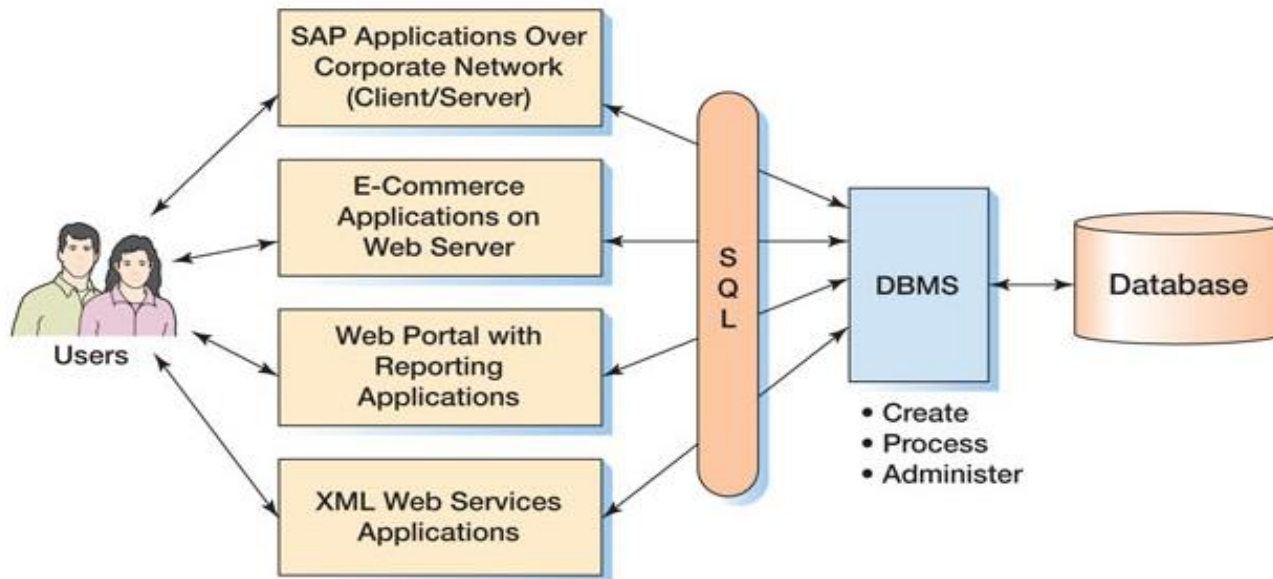


База данных

- база данных – самодокументированный набор интегрированных записей;
- самодокументированность: база данных содержит не только данные, но и их описание – метаданные (metadata):
 - возможность определить структуру и содержимое базы данных путем обращения к самой базе данных;
 - изменение структуры в первую очередь затрагивает метаданные, и, следовательно, ограничивает количество программ, на которые влияют эти изменения;
- интегрированность:
 - байты → поля → записи / объекты → файлы;
 - метаданные;
 - вспомогательные структуры данных (например, индексы);
 - метаданные приложения (дополнительные данные для построения форм и отчетов);

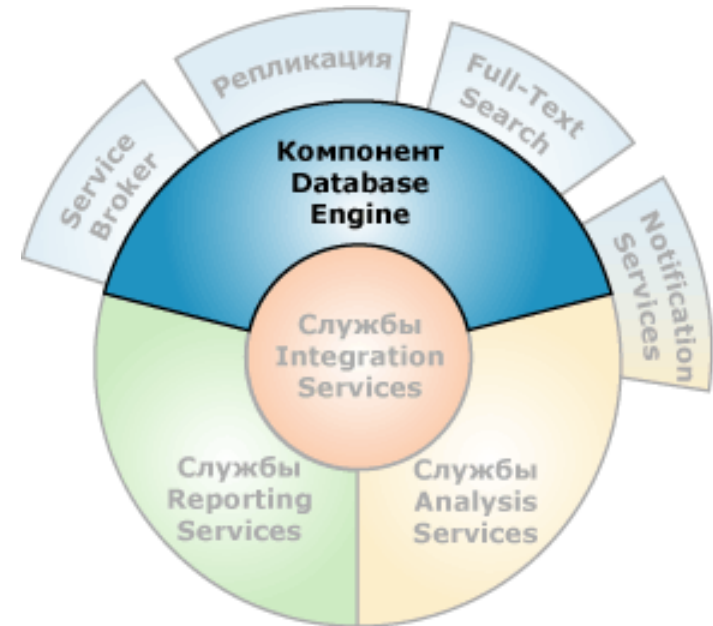
СУБД

- синонимичные термины:
 - система обработки баз данных (*database processing system*);
 - СУБД – система управления базами данных (DBMS, *database management system*);
- СУБД - является посредником между приложением и данными:
 - изоляция от физической организации хранения данных: реализация операций CRUD (create, read, update, delete) посредством языка запросов (query language) или программного интерфейса (API, application programming interface);
 - данные интегрированы;
 - уменьшение дублирования данных;
 - независимость программ от форматов файлов;
 - возможность гибкого выбора формы представления данных;



Компоненты современных СУБД

- ядро СУБД (*database engine*) обеспечивает хранение, обработку и защиту данных:
 - преобразование запросов в действия над данными;
 - управление совместным доступом к данным (транзакциями);
 - разграничение доступа к данным;
 - резервное копирование и восстановление;
- средства интерактивной аналитической обработки (*OLAP – online analytic processing*) и средства интеллектуального анализа данных;
- средства, обеспечивающие интеграцию инструментов и алгоритмов машинного обучения;
- средства интеграции данных (*ETL – extract-transform-load*), обеспечивающие экспорт и импорт данных;
- средства создания и публикации форм и отчетов;
- средства репликации, обеспечивающие копирование и распространение данных и объектов баз данных из одной базы данных в другую с возможностью последующей синхронизации между базами данных для поддержания согласованности данных для поддержания согласованности);
- средства выполнения полнотекстовых запросов к неструктурированным текстовым данным;
- ...



Управление параллельной обработкой в БД

- управление параллельной обработкой (*concurrency control*) направлено на то, чтобы исключить *непредусмотренное* влияние действий одного пользователя на действия другого;
- управление параллельной обработкой, как правило, предполагает компромисс между уровнем изоляции различных операций друг от друга и производительностью системы;
- транзакция (*transaction*) является последовательностью операций, выполненных как одна логическая единица работы (*logical unit of work*), т.е. либо все действия внутри транзакции выполняются успешно, либо не выполняется ни одно из них.

Реализация требований

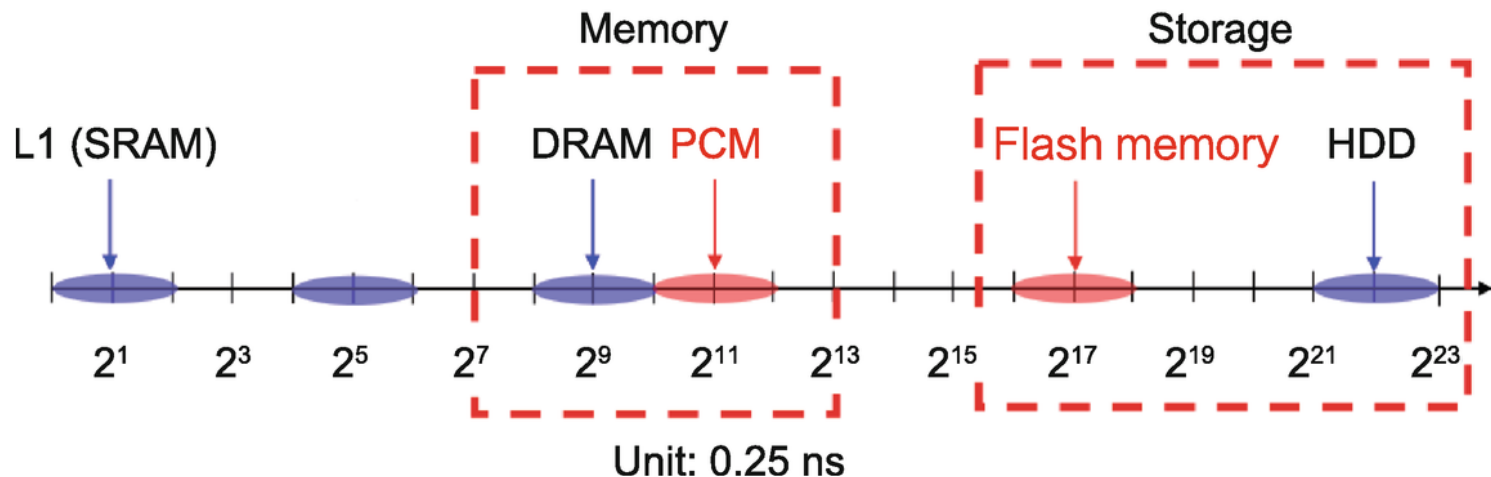
- функциональные требования:
 - выбор СУБД / конфигурации СУБД с учётом характера накапливаемой информации (числовые и текстовые данные, аудио и видео, изображения и т.п.) и её обработки;
 - схема базы данных (*database schema* – набор объектов, которые отражают разработанную модель данных предметной области), в т.ч. ограничения целостности данных (*data integrity constraints*);
- нефункциональные требования:
 - выбор архитектуры и конфигурации системы (для оптимизация производительности, достижения заданных параметров масштабируемости, отказоустойчивости и т.д.);
 - выбор СУБД (формализуется алгоритмом оценки альтернативных вариантов);
- бизнес-логика (набор ограничений на возможные действия с данными, хранящимися в БД, может быть реализована:
 - на уровне СУБД: независимо от источника действий, ограничения выполняются в любом случае;
 - на уровне приложения (создание форм и отчетов): ограничения выполняются только в рамках конкретного приложения.

OLTP vs OLAP

- транзакционные базы данных (OLTP – *Online Transaction Processing*, оперативная обработка транзакций):
 - приложениями пользуются многие пользователи, которые одновременно совершают транзакции, изменяя таким образом текущие данные;
 - хотя отдельные запросы пользователей обращаются лишь к небольшому числу записей, при этом одновременно производится большое количество таких обращений;
 - управление параллелизмом в системе базы данных гарантирует, что два пользователя не могут одновременно изменять одни и те же данные и что пользователь не может изменить какие-либо данные, пока другой пользователь не закончит работу с ними;
- аналитические базы данных, хранилища данных (OLAP – *Online Analytical Processing*, поддержка принятия решений):
 - предназначены для организации хранения большого количества преимущественно неизменных данных с целью упрощения их получения и анализа;
 - отдельный запрос может потребовать обращения к существенным объёмам и доле хранящихся данных.

Хранение данных

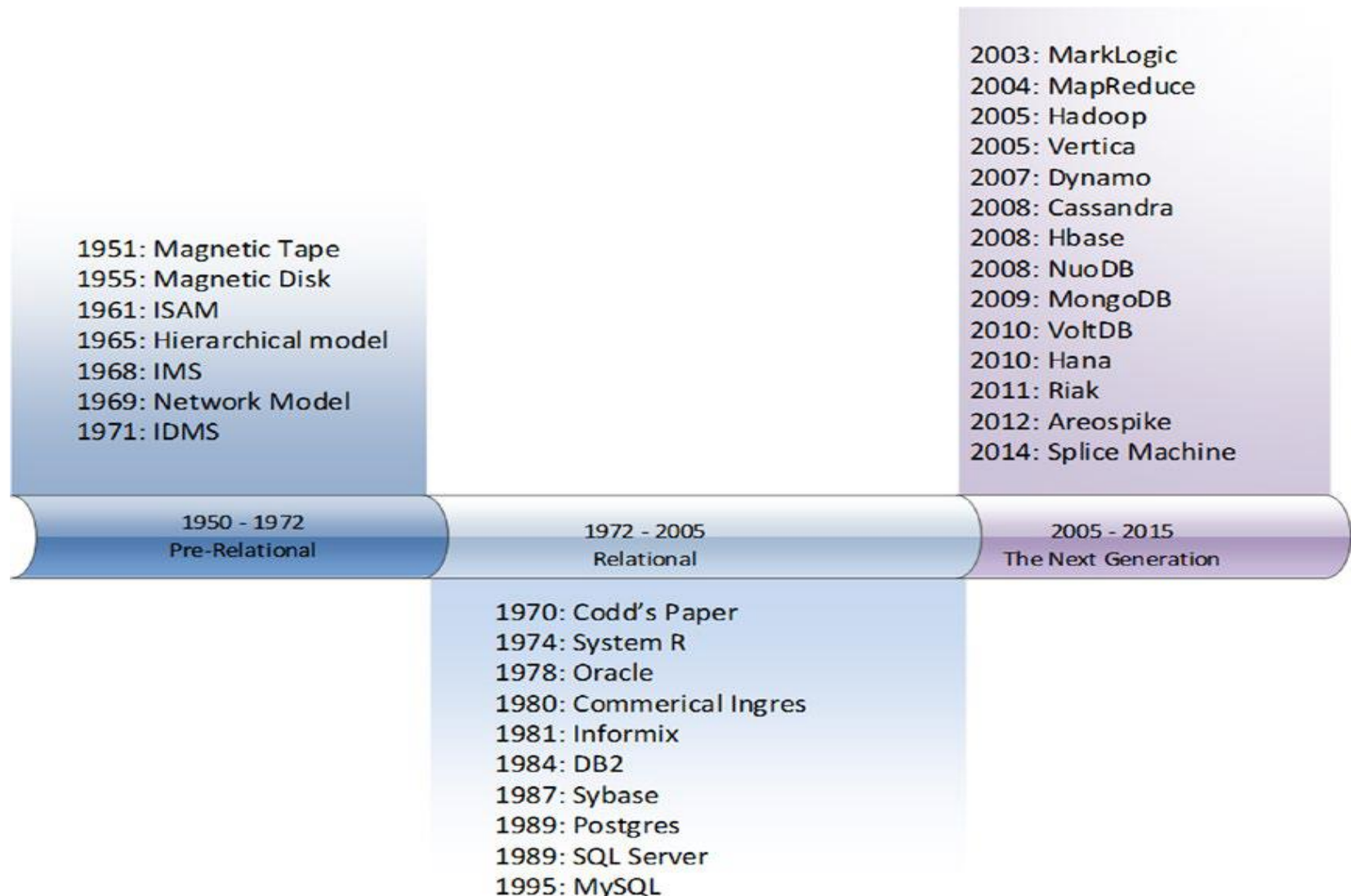
- СУБД в памяти
- Дисковые СУБД
 - локальные диски (DAS – *directly attached storage*), дисковые массивы (RAID - *Redundant Array of Inexpensive Disks*)
 - NAS (*network-attached storage*)
 - SAN (*storage area network*)



Логические модели баз данных

- логическая модель (*logical data model, information model*) базы данных определяет:
 - набор поддерживаемых типов структур данных;
 - набор допустимых операций над поддерживаемыми структурами данных;
 - набор общих правил целостности данных, явно или неявно определяющих корректные состояния базы данных или их изменения;
- модели баз данных:
 - иерархическая (*hierarchical model*);
 - сетевая (*network model*);
 - реляционная (*relational model*);
 - объектно-ориентированные (*object-oriented model*);
 - пост-реляционные модели, NoSQL (Not only SQL) базы данных:
 - хранилища ключей и значений (*KV-store, key-value store, KVP, key-value pairs*);
 - документо-ориентированные базы данных (*document-oriented databases*);
 - графовые базы данных (*graph databases*);
 - столбцовые базы данных (*column store, wide-column store*);
 - временные базы данных (*time-series databases*);
- ряд СУБД поддерживают несколько логических моделей данных.

Модели баз данных: развитие



Рейтинг СУБД: db-engines.com/en/ranking

423 systems in ranking, January 2025

Rank			DBMS	Database Model	Score		
Jan 2025	Dec 2024	Jan 2024			Jan 2025	Dec 2024	Jan 2024
1.	1.	1.	Oracle	Relational, Multi-model	1258.76	-5.03	+11.27
2.	2.	2.	MySQL	Relational, Multi-model	998.15	-5.61	-125.31
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	798.55	-7.14	-78.05
4.	4.	4.	PostgreSQL	Relational, Multi-model	663.41	-2.97	+14.45
5.	5.	5.	MongoDB	Document, Multi-model	402.50	+2.12	-14.98
6.	7.	9.	Snowflake	Relational	153.90	+6.54	+27.98
7.	6.	6.	Redis	Key-value, Multi-model	153.36	+3.08	-6.03
8.	8.	7.	Elasticsearch	Multi-model	134.92	+2.60	-1.15
9.	9.	8.	IBM Db2	Relational, Multi-model	122.97	+0.19	-9.43
10.	10.	11.	SQLite	Relational	106.69	+4.97	-8.51
11.	11.	12.	Apache Cassandra	Wide column, Multi-model	99.19	+1.26	-11.84
12.	12.	10.	Microsoft Access	Relational	92.70	+1.88	-24.97
13.	13.	17.	Databricks	Multi-model	87.85	+0.16	+7.31
14.	15.	13.	MariaDB	Relational, Multi-model	85.58	+1.81	-13.65
15.	14.	14.	Splunk	Search engine	83.09	-2.27	-9.63
16.	16.	15.	Microsoft Azure SQL Database	Relational, Multi-model	73.78	-2.59	-7.29
17.	17.	16.	Amazon DynamoDB	Multi-model	73.00	+0.27	-7.94
18.	18.	18.	Apache Hive	Relational	56.87	+3.78	-10.08
19.	19.	19.	Google BigQuery	Relational	53.04	+0.75	-10.44
20.	20.	22.	Neo4j	Graph	43.69	+0.62	-4.49
21.	21.	21.	FileMaker	Relational	41.50	-1.36	-10.55
22.	23.	20.	Teradata	Relational, Multi-model	36.25	-1.35	-16.93
23.	22.	23.	SAP HANA	Relational, Multi-model	36.06	-1.59	-10.38
24.	24.	24.	Apache Solr	Search engine, Multi-model	32.32	-0.15	-12.81
25.	25.	25.	SAP Adaptive Server	Relational, Multi-model	29.05	-1.26	-10.38
26.	26.	26.	Apache HBase	Wide column	24.75	-0.25	-9.11
27.	27.	27.	Microsoft Azure Cosmos DB	Multi-model	22.96	-0.10	-10.51
28.	28.	28.	InfluxDB	Time Series, Multi-model	21.54	+0.31	-6.02
29.	29.	30.	PostGIS	Spatial, Multi-model	19.70	+0.26	-6.95
30.	31.	37.	ClickHouse	Relational, Multi-model	18.56	+1.12	+1.21

Реляционная модель

- реляционная модель (*relational database model*)
 - введена в 1970г. Э.Ф.Коддом (E.F.Codd, A Relational Model of Data for Large Shared Databanks);
 - основана на применении концепции реляционной алгебры к проблеме хранения больших объемов данных;
 - определяет способ хранения данных в виде таблиц со строками и столбцами, при этом минимизируется дублирование и исключаются определенные типы ошибок обработки;
 - дает стандартный способ структурирования и обработки данных;
- нормализация (*normalization*) – последовательность преобразований, обеспечивающих определенный набор требований.

EmpNum	EmpName	DeptNum	DeptName
100	Jones	10	Accounting
150	Lau	20	Marketing
200	McCauley	10	Accounting
300	Griffin	10	Accounting

(a) One-Table Design

OR?

DeptNum	DeptName
10	Accounting
20	Marketing

EmpNum	EmpName	DeptNum
100	Jones	10
150	Lau	20
200	McCauley	10
300	Griffin	10

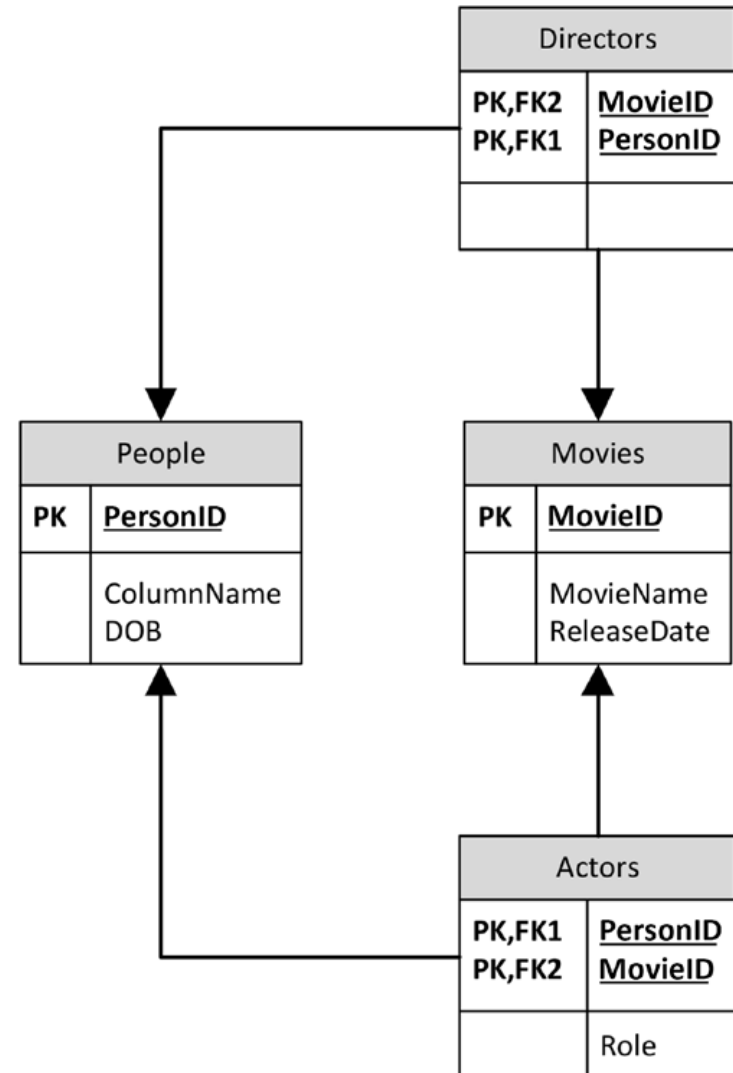
(b) Two-Table Design

Объекты реляционной базы данных

- основной объект – отношение (*relation*), представляющее собой двумерную таблицу:
 - столбцы (поля, атрибуты) – определяют набор данных, которыми обладает описываемый объект предметной области;
 - строки (записи) – содержат набор значений атрибутов для конкретного объекта;
- домен (*domain*) – множество допустимых значений атрибута:
 - физическое описание: тип данных и дополнительные наложенные ограничения;
 - семантическое (логическое) описание: описывает назначение данного атрибута;
- ключ (*key*) – набор из одного или нескольких атрибутов, однозначно идентифицирующий конкретную запись.

Реляционная БД: пример

- структура таблиц (отношений), выбор ключей и процедура нормализации напрямую зависит от модели данных, которая была сформирована на основе требований пользователей к системе



Метаданные в реляционной базе данных

- хранение метаданных осуществляется в системных таблицах (*system tables*), которые могут располагаться в специализированной системной базе данных

USER_TABLES Table

TableName	NumberColumns	PrimaryKey
STUDENT	3	StudentNumber
CLASS	4	ClassNumber
GRADE	3	(StudentNumber, ClassNumber)

USER_COLUMNS Table

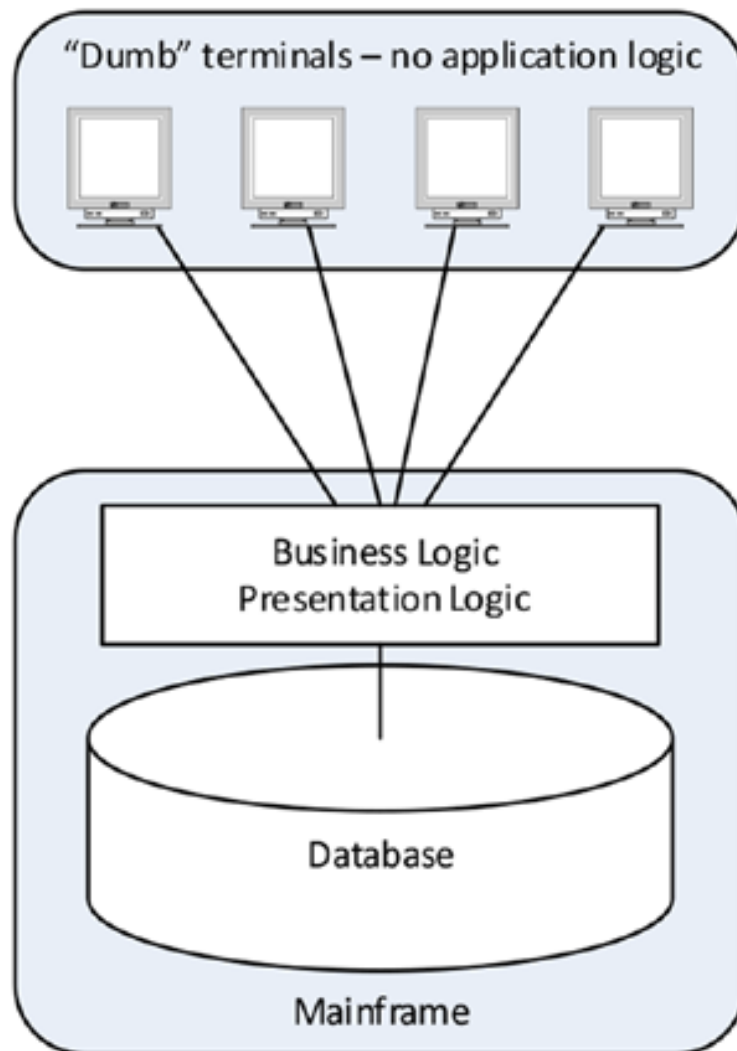
ColumnName	TableName	DataType	Length (bytes)
StudentNumber	STUDENT	Integer	4
StudentName	STUDENT	Text	50
EmailAddress	STUDENT	Text	50
ClassNumber	CLASS	Integer	4
Name	CLASS	Text	50
Term	CLASS	Text	5
Section	CLASS	SmallInteger	2
StudentNumber	GRADE	Integer	4
ClassNumber	GRADE	Integer	4
Grade	GRADE	Decimal	(3, 2)

Архитектуры доступа к данным

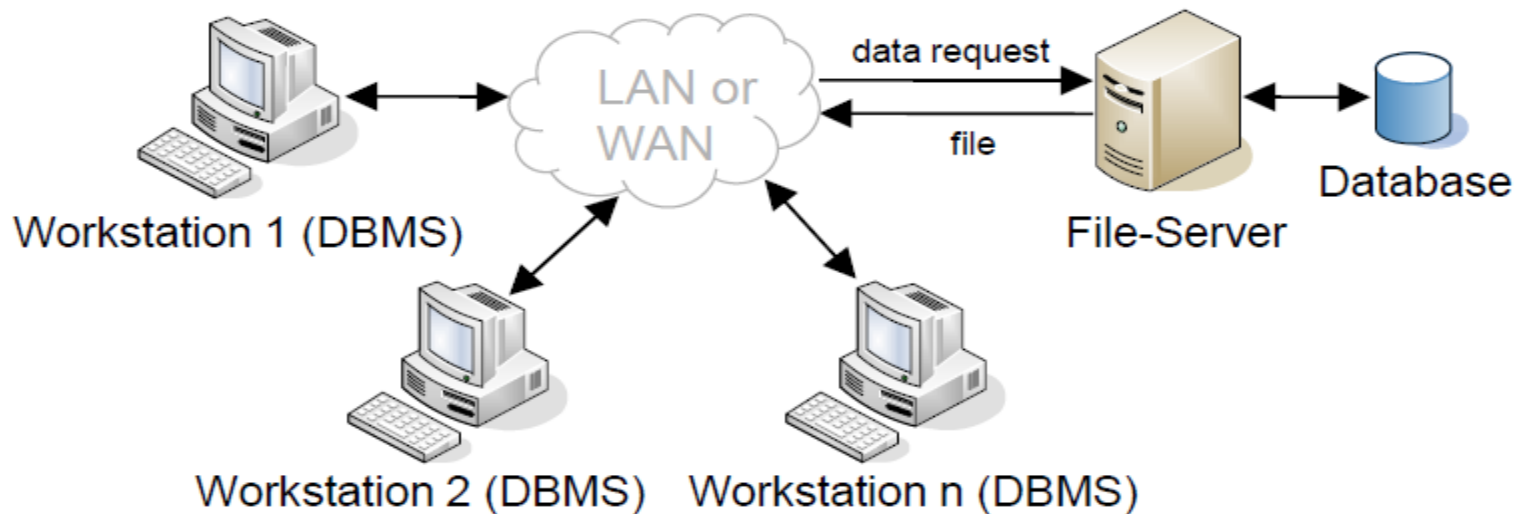
- терминальный доступ (teleprocessing);
- доступ в режиме разделения файлов (file-sharing);
- встраиваемые базы данных (embedded databases);
- клиент-серверные системы (client-server systems):
 - двухуровневая / двухзвенная архитектура (two-tier);
 - трёхуровневая / трехзвенная архитектура (three-tier);
 - многоуровневая архитектура (N-tier, multitier);
- архитектура «точка-точка» (peer-to-peer);
- облачное хранение / облачные вычисления (cloud storage / cloud computing).

Терминальный доступ

- единственный мейнфрейм (mainframe) и множество терминалов (предназначенных исключительно для ввода-вывода);
- недостаток: высокая нагрузка на мейнфрейм, т.к. на нем:
 - выполняются приложения и СУБД;
 - производится подготовка информации для отображения на терминалах.



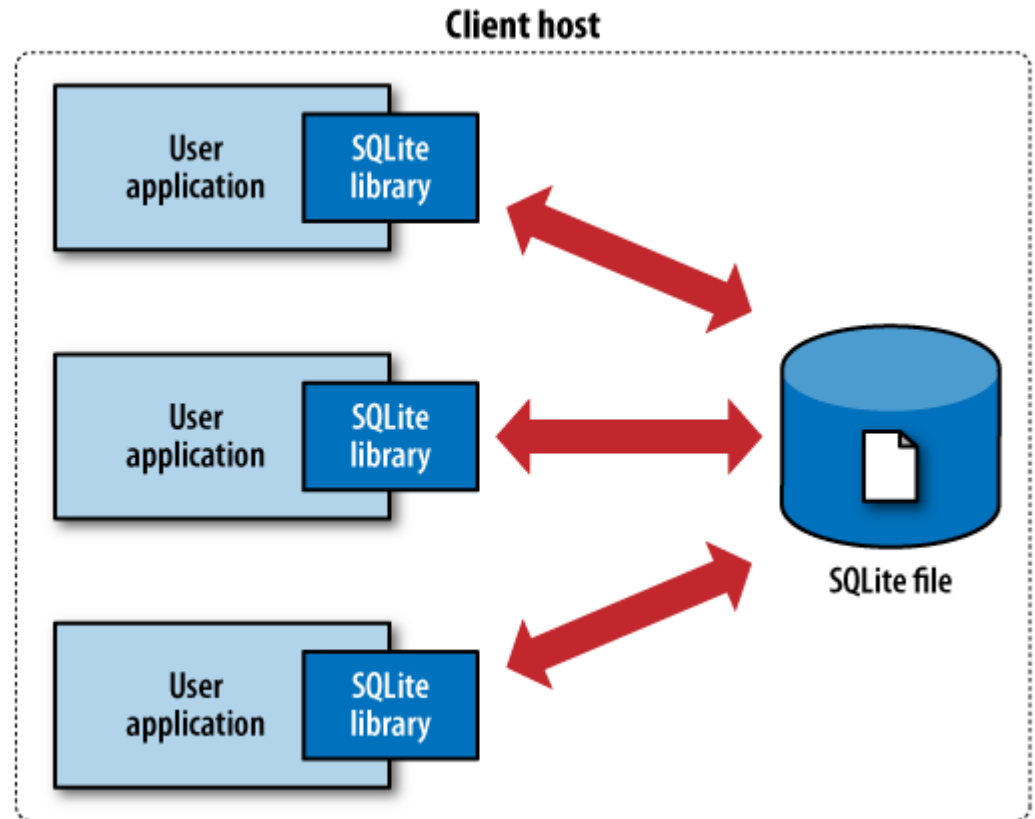
Доступ в режиме разделения файлов



- файловый сервер (*file-server*) – компьютер, преимущественно использующийся для разделяемого хранения файлов;
- все или часть компонентов СУБД выполняются на компьютере пользователя, для этого может понадобиться передача файлов;
- недостатки:
 - высокая нагрузка на сеть;
 - сложные схемы обеспечения целостности данных, совместного доступа к данным и восстановления;
 - высокая стоимость владения (установка и обслуживание всех компонентов СУБД на каждом компьютере);

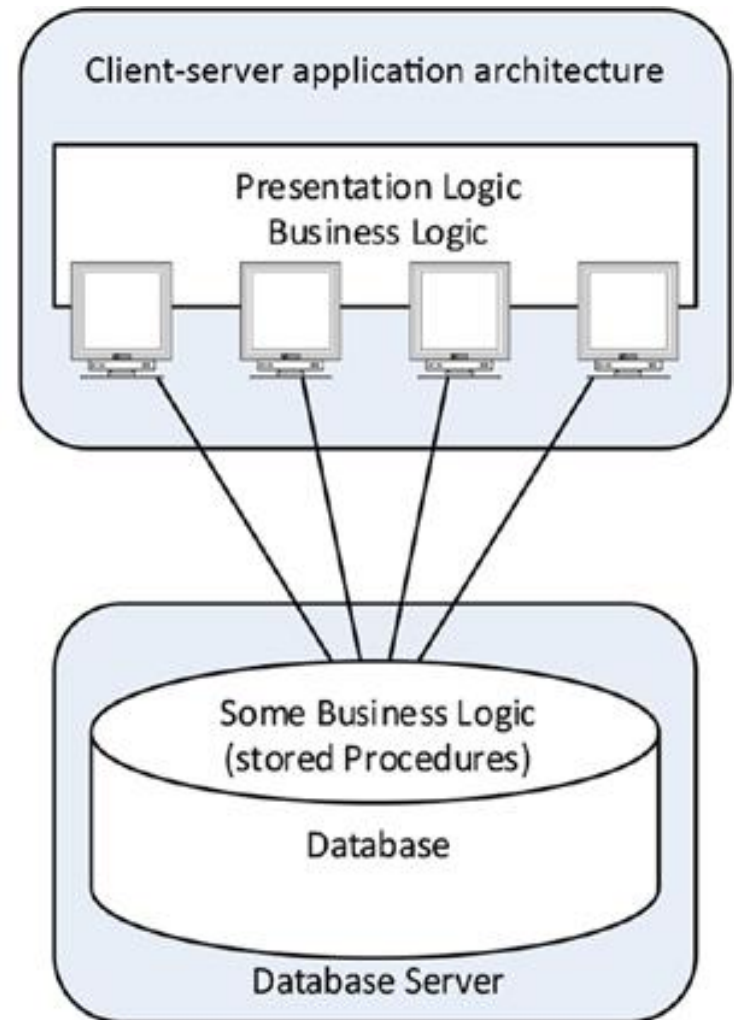
Встраиваемые СУБД

- СУБД встраивается непосредственно в приложение, работающее с данными – одноуровневая архитектура (*single-tier*)
- SQLite, Berkeley DB, LevelDB, RocksDB
- актуальны для мобильных устройств / IoT



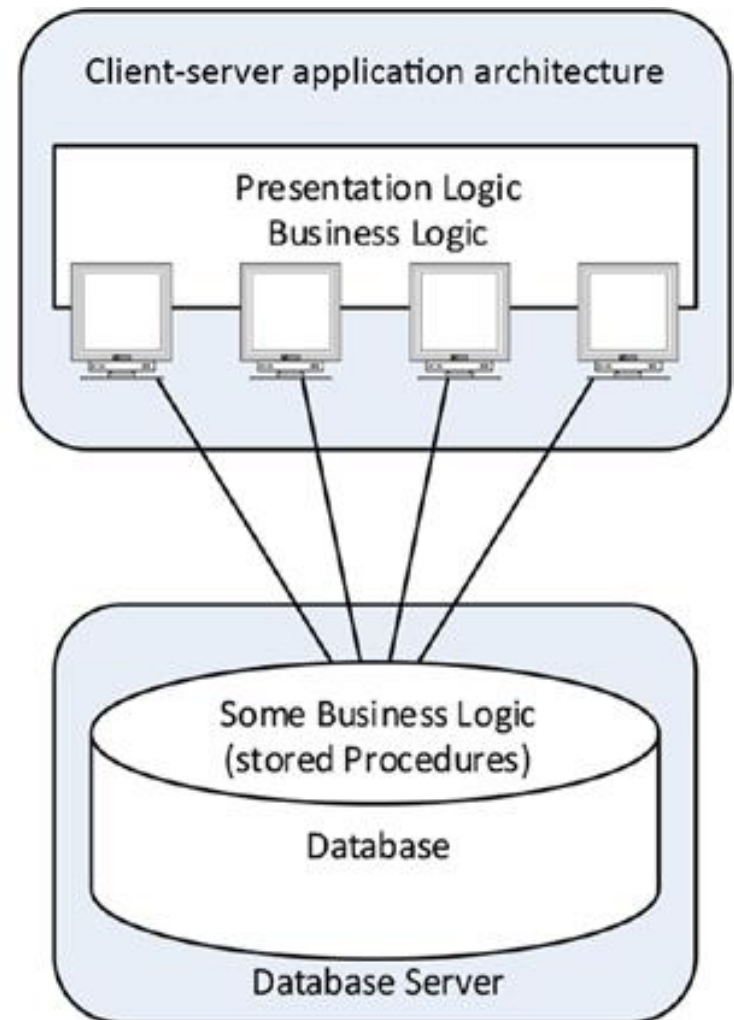
Двухуровневая архитектура («клиент-сервер»)

- на клиенте выполняется приложение пользователя, которое обращается к СУБД на сервере: толстый (thick) / тонкий (thin) клиент;
- функции клиента:
 - представление данных (пользовательский интерфейс);
 - выполнение некоторой бизнес-логики;
 - отправка запросов и прием результатов их выполнения;
- функции сервера:
 - выполнение бизнес-логики;
 - обеспечение совместного доступа к данным;



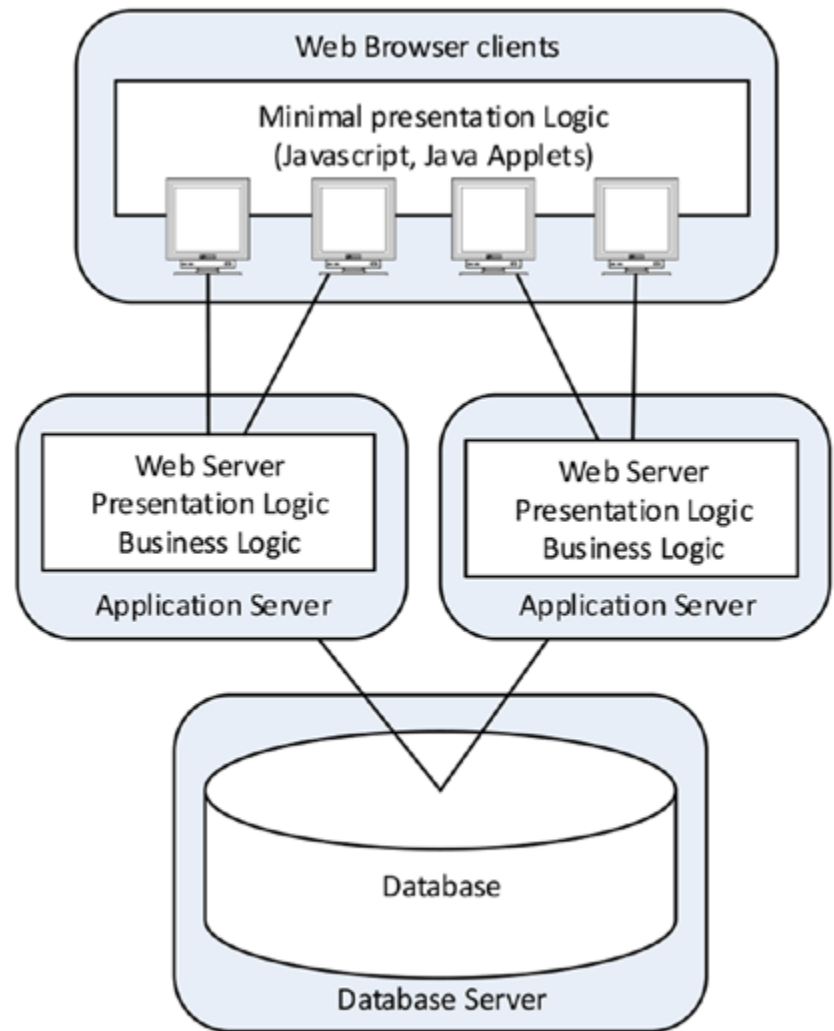
Двухуровневая архитектура («клиент-сервер»): преимущества и недостатки

- преимущества:
 - возможность увеличения производительности (параллельное выполнение запросов и настройка сервера под конкретную СУБД);
 - возможность обеспечения целостности данных (централизованная проверка ограничений) и безопасности;
 - снижение сетевого трафика;
 - снижение стоимости владения;
- недостатки:
 - ограниченная возможность масштабирования;
 - большие издержки на поддержание клиентских приложений и требований к производительности (в случае толстого клиента);



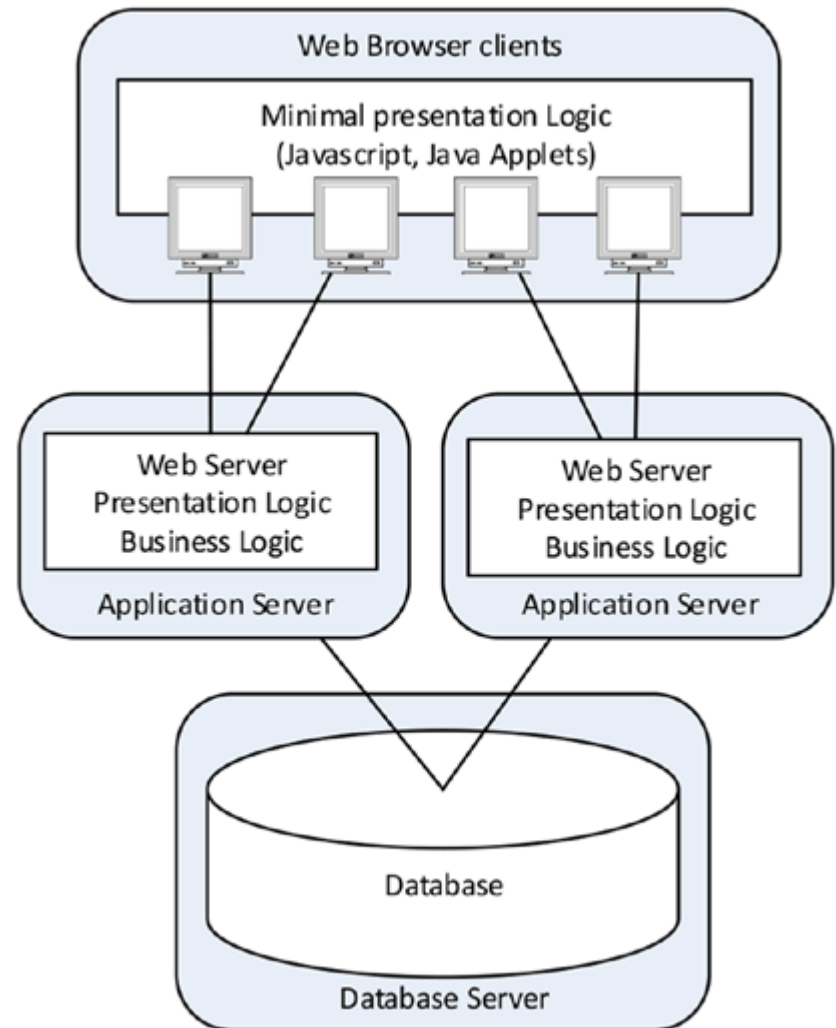
Трехуровневая архитектура

- появилась в 90-х для обеспечения требований масштабируемости (например, для веб-приложений);
- приложение включает три уровня:
 - представления (*presentation tier*) – клиент (*client*):
 - представление данных (пользовательский интерфейс);
 - базовая проверка ввода (тонкий клиент);
 - отправка запросов и прием результатов их выполнения;
 - бизнес-логики (*logic tier*) – сервер приложений (*application server*):
 - выполнение бизнес-логики и обработка данных;
 - данных (*data tier*) – сервер баз данных (*database server*):
 - базовая проверка корректности данных;
 - обеспечение совместного доступа к данным;
 - обеспечение целостности данных.

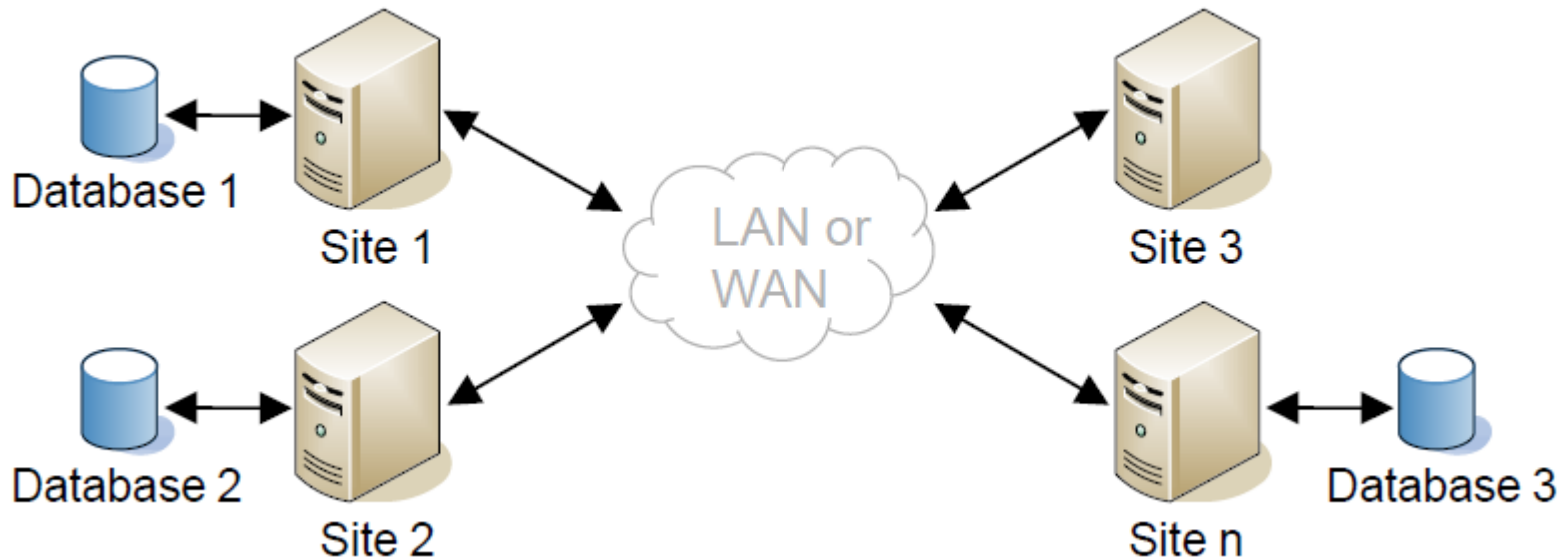


Трехуровневая архитектура: преимущества и недостатки

- преимущества:
 - снижение требований к производительности клиентов;
 - централизация бизнес-логики на сервере приложений (упрощается задача поддержки и установки обновлений);
 - увеличение модульности;
 - возможность балансировки нагрузок на каждом из уровней;
- пример: архитектура веб-приложений (браузер выступает в качестве тонкого клиента);
- возможно увеличение количества промежуточных уровней для повышения гибкости системы (например, повышения эффективности балансировки нагрузок).

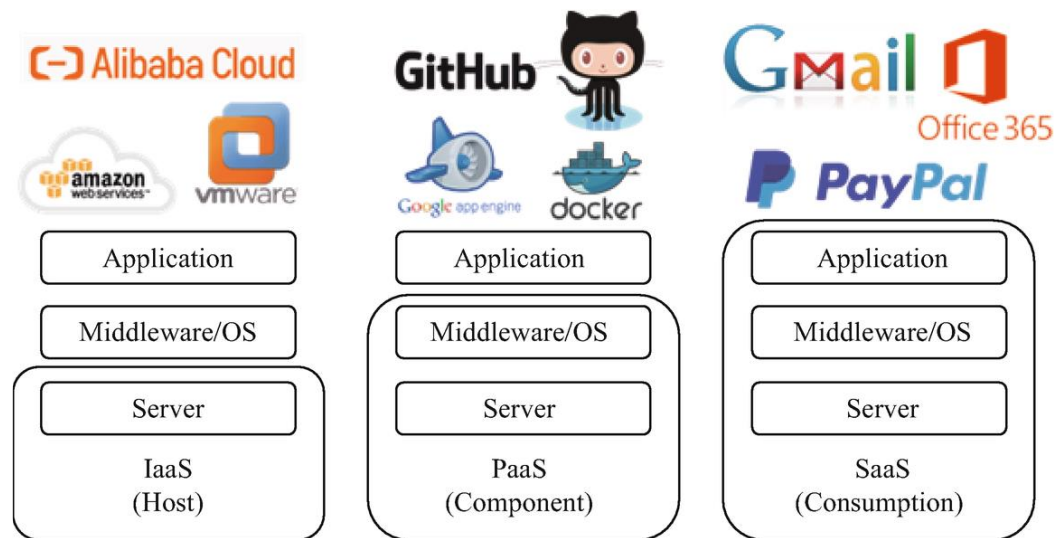


Архитектура «точка-точка»



- отсутствует четко выраженное разделение клиентов и серверов – связи могут формироваться динамически для обмена данными и услугами;
- в связи с отсутствием выделенного центрального узла и глобальной схемы данных могут потребоваться дополнительные средства для скрытия разнородности данных/систем и обеспечения унифицированного представления (mediation) услуг и данных.

Облачное хранение

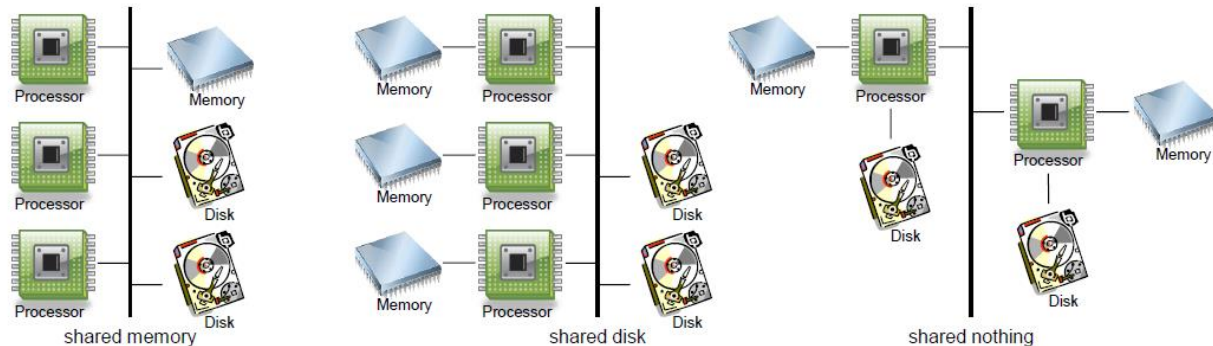


- модели служб:
 - инфраструктура как услуга (IaaS, Infrastructure as a Service): виртуализация физических ресурсов;
 - платформа как услуга (PaaS, Platform as a Service): операционная система, СУБД, веб-сервер;
 - программное обеспечение как услуга (SaaS, Software as a Service): приложения и базы данных;
- в области баз данных:
 - изменяются подходы к надежности, производительности, резервному копированию, восстановлению и т.п.
 - IaaS / PaaS: Amazon EC2, Google Cloud, Microsoft Azure, VK Cloud Solutions (MySQL, PostgreSQL, MongoDB);
 - DBaaS: MS Azure SQL Database, Amazon SimpleDB.

Масштабируемость

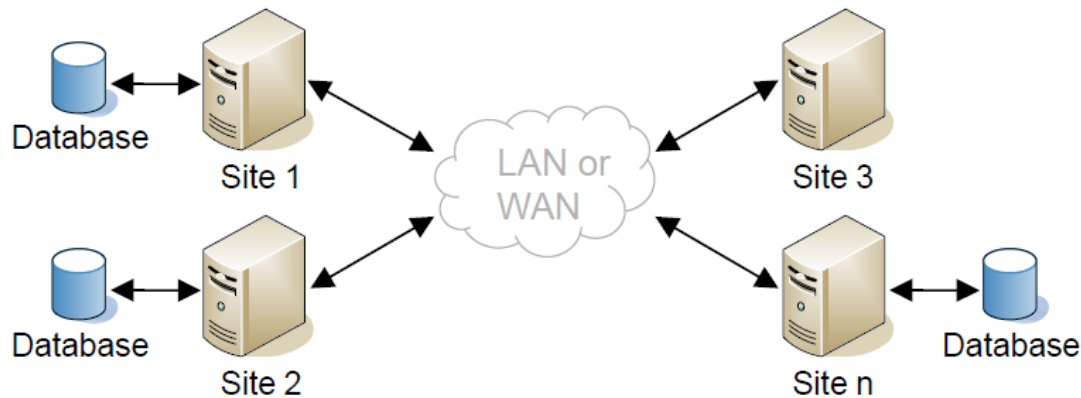
- централизованные / монолитные СУБД:
 - Oracle, PostgreSQL, MS SQL Server, ...
- параллельные СУБД (parallel database systems):
 - Teradata, Oracle Real Application Cluster, ...
- распределенные СУБД (distributed database systems);
 - на базе монолитных СУБД
 - MongoDB, Cassandra, HBase, Google BigTable, Neo4j, Redis, Memcached, Tarantool...

Параллельные базы данных



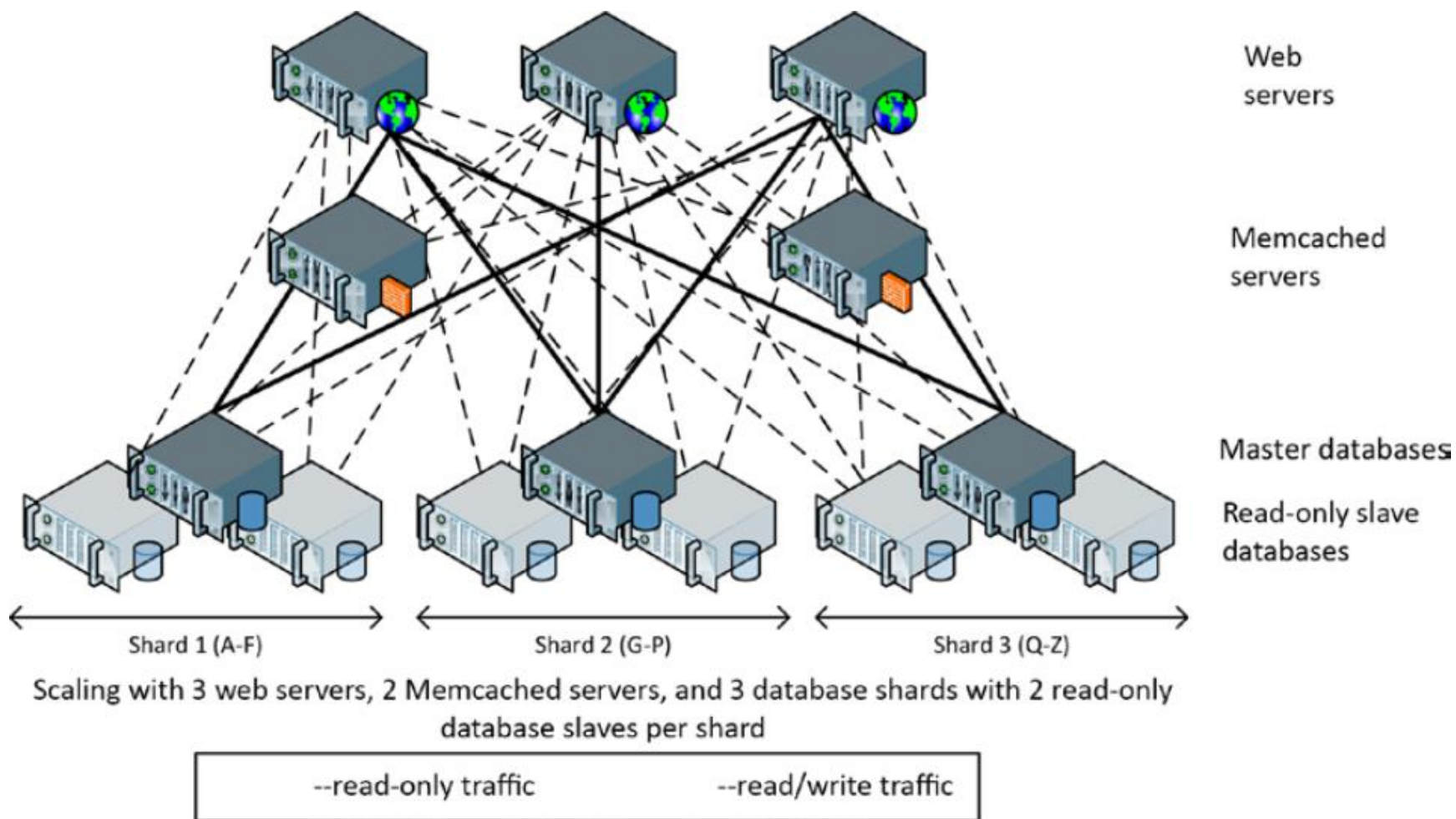
- позволяют повысить производительность за счет выполнения запроса (транзакции) на нескольких узлах:
 - разделяемая память (shared memory):
 - крайне эффективный обмен между процессорами;
 - невозможность масштабирования (шина является узким местом);
 - разделяемое дисковое пространство (shared disk):
 - определенная степень отказоустойчивости при отказе памяти / процессора + отказоустойчивость дисковой подсистемы за счет организации RAID-массивов;
 - узким местом является обмен с дисковой подсистемой;
 - нет разделяемых ресурсов (shared nothing):
 - наиболее масштабируемая;
 - неравномерная скорость доступа к различным дискам;
 - иерархическая (hierarchical): комбинация вышеперечисленных подходов.

Распределенные базы данных

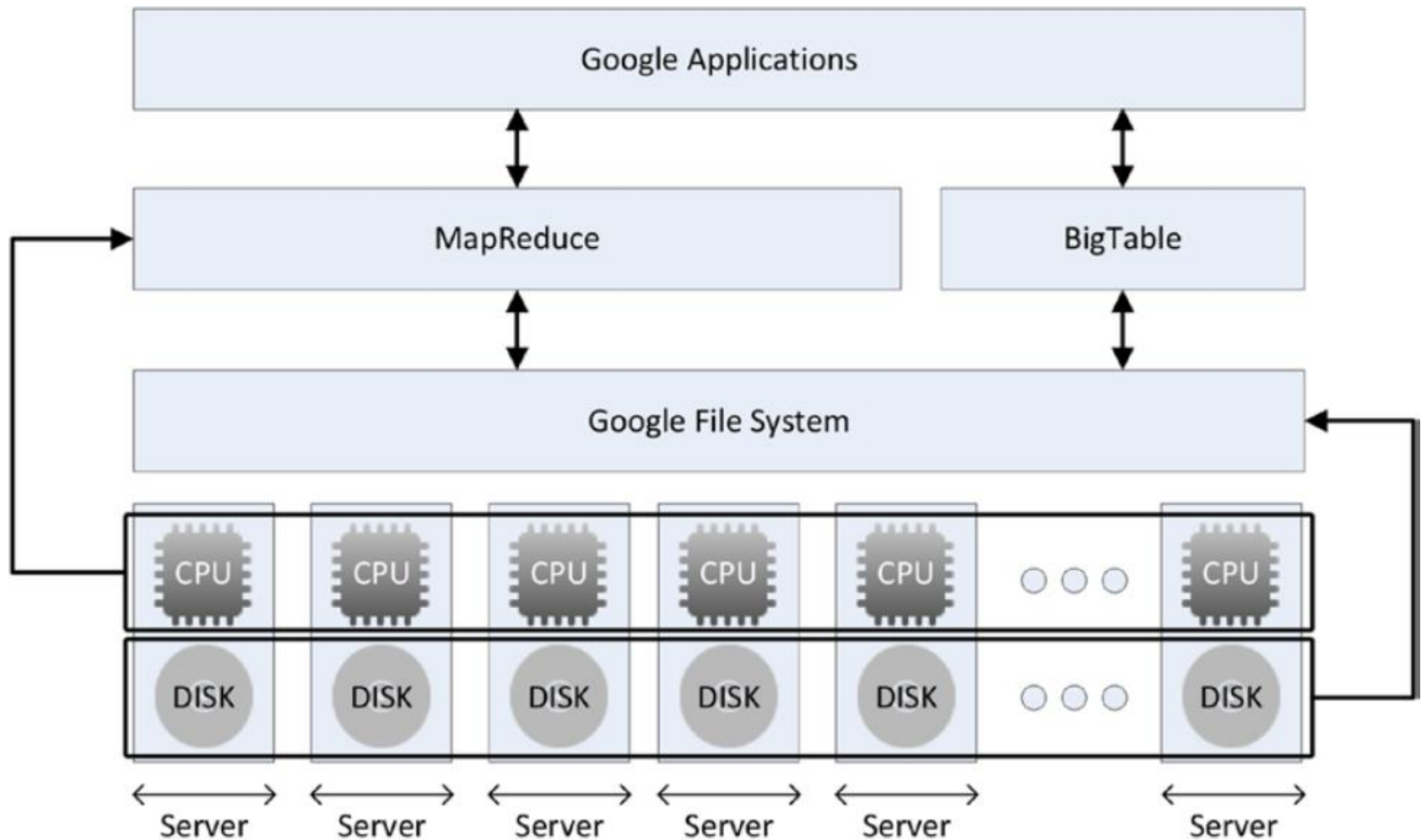


- распределенная база данных: логическая совокупность данных и метаданных, распределенная внутри сети;
- распределенная СУБД: система, обеспечивающая прозрачное управление распределенной базой данных;
 - средства распределенного выполнения запросов / транзакций;
 - средства согласования метаданных в случае объединения разнородных подсистем;
- преимущества:
 - возможность интеграции и прозрачного доступа к данным других подсистем;
 - высокая степень доступности данных (*availability*), особенно при использовании репликации;
 - более дешевые и масштабируемые, чем кластерные системы.

Сегментирование данных в web- системах



Программно-аппаратная архитектура Google



Выводы

- создание приложений баз данных является сложной инженерной задачей, при решении которой:
 - эффективность созданного программно-аппаратного комплекса во многом зависит от адекватного понимания разработчиком тех требований, которые предъявляет пользователь к данному комплексу;
 - необходимы знания в областях, включающих:
 - аппаратное обеспечение,
 - алгоритмы и структуры данных,
 - системное программирование,
 - сетевые технологии,
 - разработка пользовательского интерфейса.

Вопросы к экзамену

- Информационная система. Разработка и формализация требований. Функциональные и нефункциональные требования.
- Моделирование информационных систем. Моделирование данных. Модель «сущность-связь» и модель семантических объектов: основные понятия.
- Системы обработки файлов и базы данных. Системы управления базами данных (СУБД).
- Системы управления базами данных (СУБД). Ядро СУБД. Язык SQL. Дополнительные компоненты современных СУБД.
- Моделирование информационных систем. Логические модели баз данных. Понятие реляционной модели и нормализации. Метаданные в реляционных базах данных.
- Управление параллельной обработкой в БД. Приложения баз данных для оперативной обработки транзакций (OLTP) и поддержки принятия решений (OLAP).
- Архитектуры доступа к данным. Терминальный доступ. Доступ в режиме разделения файлов. Встраиваемые СУБД.
- Архитектуры доступа к данным. Архитектура «клиент-сервер».
- Архитектуры доступа к данным. Архитектура «точка-точка». Модели служб облачных вычислений. Облачное хранение.
- Масштабирование баз данных. Параллельные и распределённые СУБД.

Вопросы к экзамену (2016)

- Приложения баз данных. Требования к приложениям баз данных. Процесс разработки баз данных.
- Моделирование данных. Модель «сущность-связь» и модель семантических объектов: основные понятия.
- История развития баз данных. Системы обработки файлов и базы данных. Системы управления базами данных (СУБД).
- Понятие реляционной модели и нормализации. Реляционные базы данных. Метаданные в реляционных базах данных.
- Системы управления базами данных (СУБД). Ядро СУБД. Язык SQL. Дополнительные компоненты современных СУБД.
- Системы управления базами данных (СУБД). Создание баз данных и основные объекты ядра СУБД.
- Модели баз данных. Иерархическая и сетевая модели. Реляционная модель.
- Модели баз данных. Реляционная модель. Обзор постреляционных моделей (объектно-ориентированные, документо-ориентированные, графовые и столбцовые базы данных).
- Схемы доступа к данным. Терминальный доступ. Доступ в режиме разделения файлов. Архитектура «клиент-сервер».
- Схемы доступа к данным. Архитектура «клиент-сервер». Распределенные базы данных. Параллельные базы данных.
- Схемы доступа к данным. Архитектура «точка-точка». Облачные вычисления. Базы данных для мобильных устройств.
- Управление параллельной обработкой в БД. Приложения баз данных для оперативной обработки транзакций (OLTP) и поддержки принятия решений (OLAP).

Базы данных

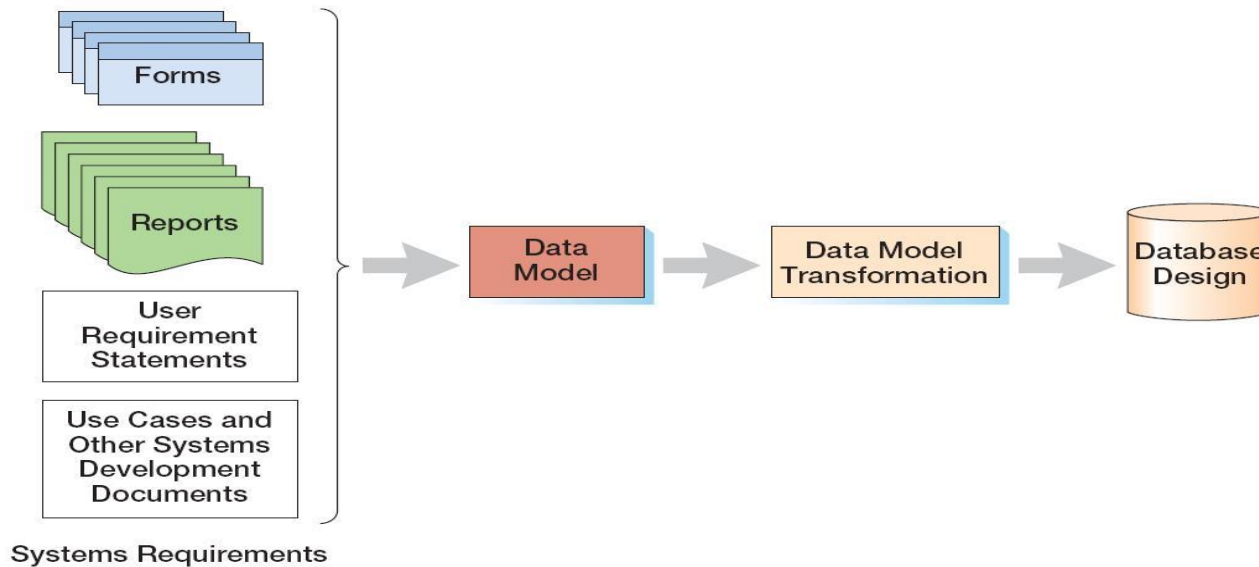
Тема 2

Моделирование данных: модель сущность-связь

Процесс разработки БД

- общие стратегии:
 - разработка сверху вниз (*top-down database development*)
 - получение абстрактной модели данных исходя из анализа стратегических целей организации, способов их достижения, а также информации и способов ее представления, которые необходимы для достижения этих целей;
 - лучшее взаимодействие подсистем, глобальный охват;
 - разработка снизу вверх (*bottom-up database development*)
 - для разработки выбирается конкретная система, которая выполняет только часть функций предприятия;
 - исходя из сформированных требований, выбранная подсистема создается быстрее и с меньшими рисками;
- одна из основ эффективной реализации приложения базы данных – ясное представление модели предметной области

Три уровня моделей информационных систем



- внешние модели (*external schema*): представление пользователей о системе (user view);
- концептуальная модель (*conceptual schema*): абстрактное представление системы (данных);
- внутренние модели (*internal schema*).

Модели данных (ANSI)

- внешняя модель (*external schema*) – набор представлений пользователей о той части предметной области, с которой он сталкивается;
- концептуальная модель (*conceptual schema / conceptual data model*) – набор ключевых объектов предметной области, их взаимосвязей (взаимодействий) и ограничений, накладываемых на объекты;
- логическая модель (*logical schema / logical data model / information schema*) – представление концептуальной модели в соответствии с логической моделью, используемой для хранения данных;
- физическая модель (*physical data model*) – описание физического представления, хранения и обработки данных.

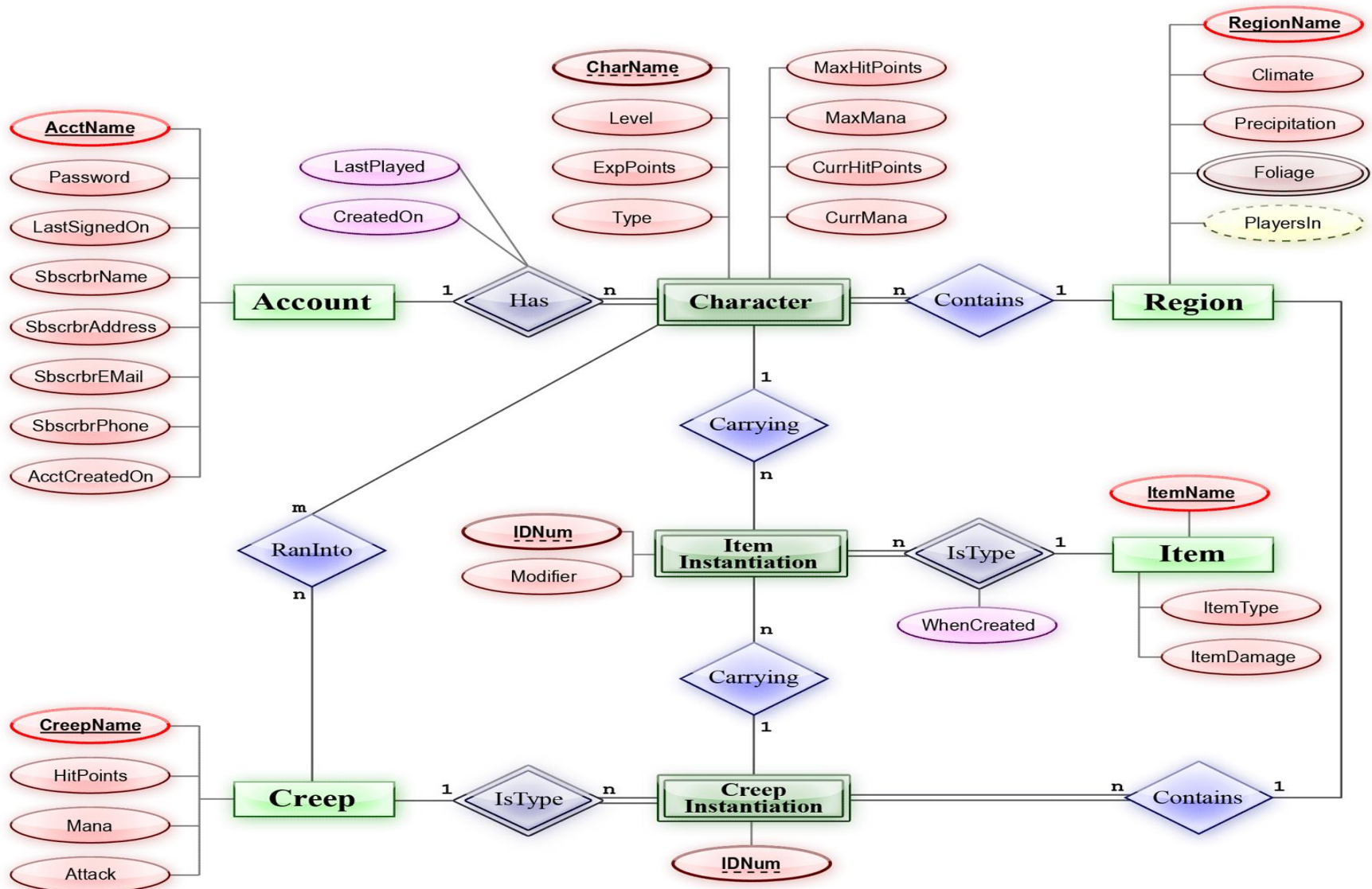
Моделирование данных

- база данных является «моделью модели», то есть через объекты базы данных описывает представление пользователей о конкретной предметной области;
- при любом процессе разработки необходимо сформулировать требования и построить на их основе модель данных, что является во многом творческой задачей, решение которой основано на опыте и интуиции;
- моделирование данных (*data modeling*) - процесс создания логического представления базы данных (пользовательская модель данных - *user data model*, модель требований к данным - *requirements data model*);
- модель данных представляет собой совокупность понятий и графических символов для построения концептуальных схем, т.е. содержит языковые и изобразительные стандарты для своего представления;
 - модель сущность – связь (*entity-relationship model, E-R model*);
 - модель семантических объектов (*semantic object model*);

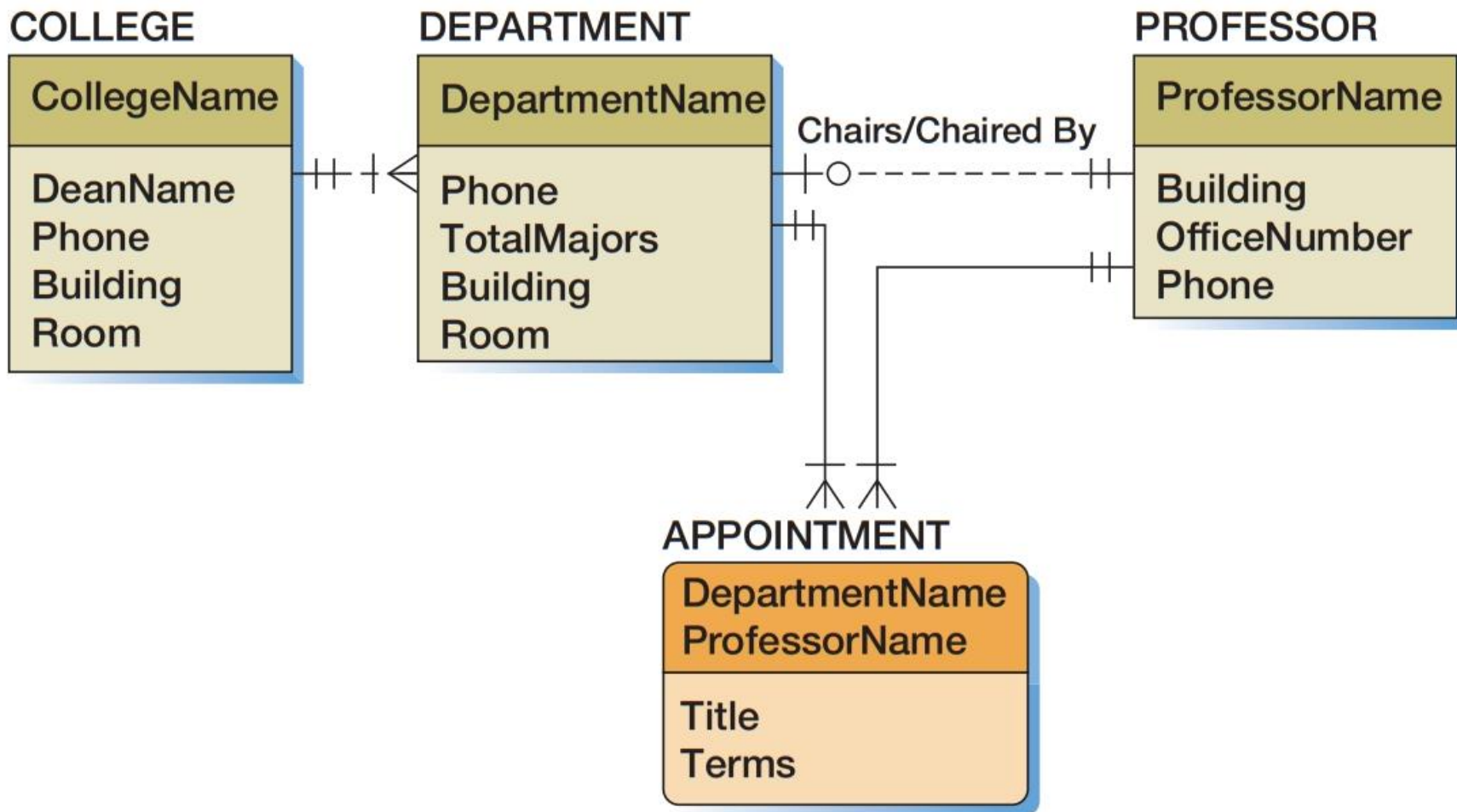
ER-модель: варианты

- **исходная ER-модель:** предложена Питером Ченом в 1976 году ("The Entity-Relationship Model--Toward a Unified View of Data". In: *ACM Transactions on Database Systems* 1/1/1976 ACM-Press ISSN 0362-5915, S. 9–36);
- **расширенная ER-модель:** дополненная модель П.Чена;
- **информационная инженерия (Information Engineering, IE)**
– предложена Джеймсом Мартином в 1990 году, использует нотацию "crow's foot";
- **IDEF1X (Integrated DEFinition, 1, eXtended)** – национальный стандарт (NIST, National Institute of Standards and Technology - <http://www.itl.nist.gov/fipspubs/idef1x.doc>);
- **Unified Modeling Language (UML, универсальный язык моделирования)** – стандарт OMG (Object Management Group).

ER-модель: пример (нотация П.Чена)



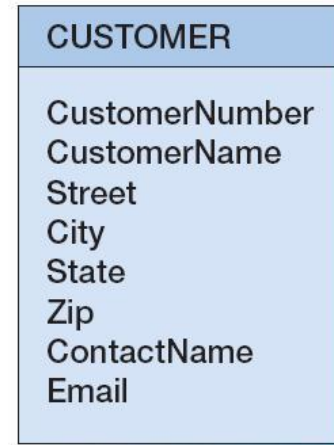
Модель сущность-связь (нотация Дж. Мартина)



Сущности

- *сущность (entity)* – некоторый объект, который способен существовать независимо и может быть идентифицирован;
 - класс сущностей – набор сущностей определенного типа;
 - экземпляр сущности – сущность некоторого класса с заданными значениями атрибутов;
- сущность может являться абстракцией:
 - реального физического объекта;
 - события, действия или процесса (заказ, услуга, транзакция).

CUSTOMER Entity

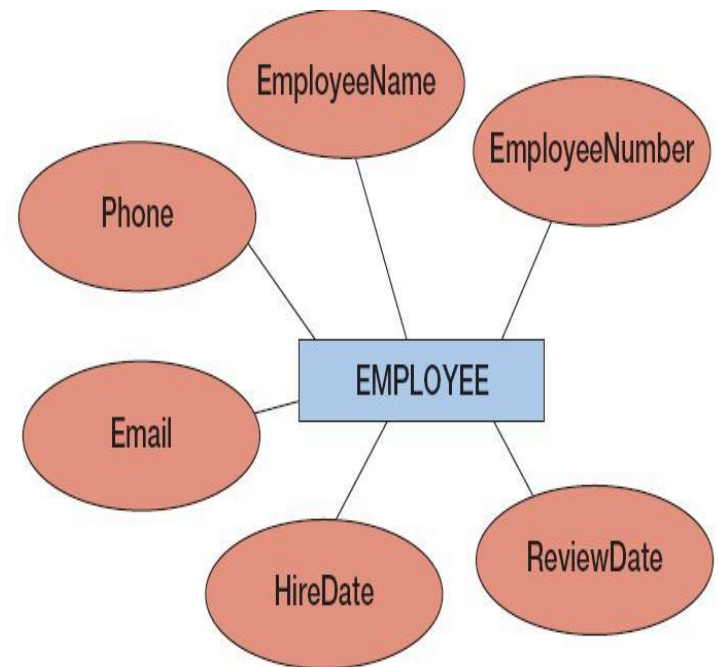


Two CUSTOMER Instances

1234 Ajax Manufacturing 123 Elm Street Memphis TN 32455 P_Schwartz P_S@Ajax.com	99890 Jones Brothers 434 10th Street Boston MA 01234 Fritz Billingsley Fritz@JB.com
--	--

Атрибуты

- *атрибу́т (attribute)* – свойство, которым обладает сущность;
- все сущности определенного класса обладают одинаковым набором атрибутов, но различаются значениями данных атрибутов;
- в общем случае, атрибуты могут быть:
 - составными (*composite*);
 - многозначными (*multi-value*).
- идентификатор сущности (*entity identifier*) – набор атрибутов, которые определяют, именуют сущность;
- идентификаторы могут быть составными, т.е. состоять более, чем из одного атрибута;
- заметим, идентификаторы в модели данных становятся ключами в физических моделях;



EMPLOYEE	
EmployeeNumber	
EmployeeName	
Phone	
Email	
HireDate	
ReviewDate	

СВЯЗИ

- *связь (relationship)* определяет, как сущности соотносятся друг с другом;
- подобно классам и экземплярам сущностей, можно выделить классы связей и экземпляры связей;
- *степень (degree)* связи показывает, сколько сущностей определяют эту связь (участвуют в связи):
 - бинарные связи (*binary*);
 - тернарные связи (*ternary*);
- оригинальная ER-модель позволяет назначать атрибуты также и связям, но в более современных нотациях от такой возможности отказались;
- ER-модель для описания данных **не** оперирует понятиями ключей, что упрощает создание модели в условиях неопределенности на начальных этапах проектирования.

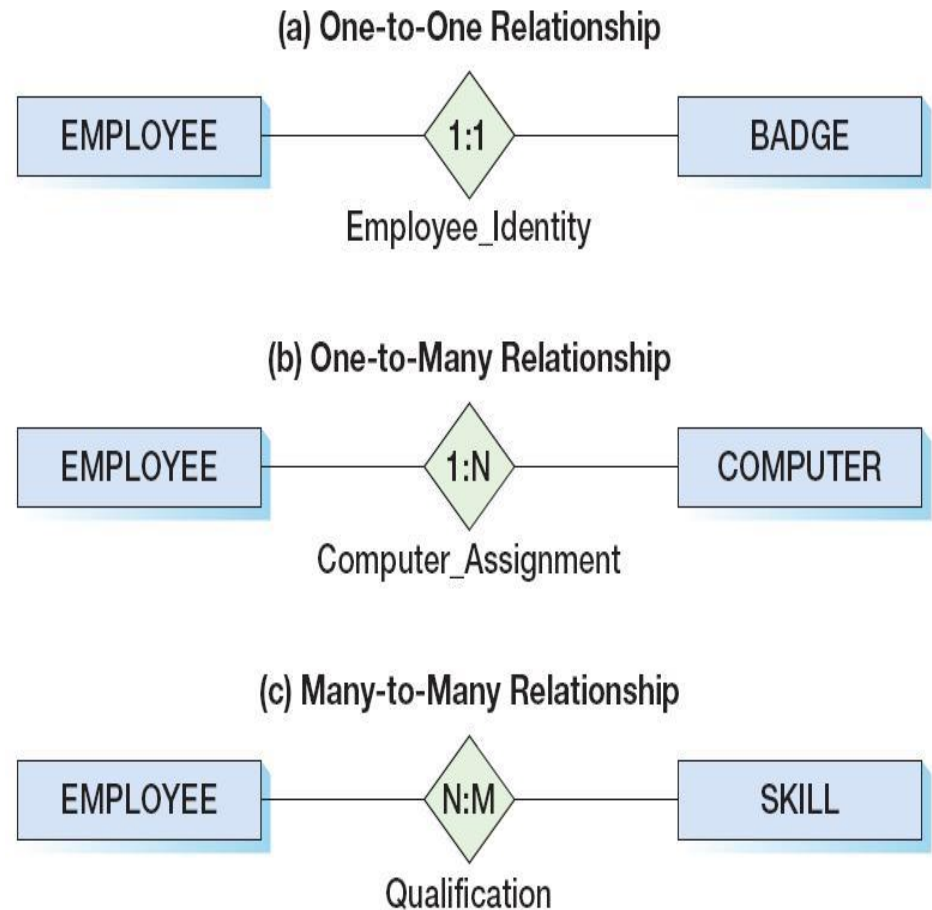


Кардинальные числа связей

- *кардинальное число (cardinality, мощность)* указывает число экземпляров сущностей, которые участвуют в связи;
- *максимальное кардинальное число (maximum cardinality)* - максимальное количество экземпляров сущностей, которые *могут* участвовать в связи;
- *минимальное кардинальное число (minimum cardinality)* - минимальное количество экземпляров сущностей, которые *должны* участвовать в связи.

Максимальное кардинальное число

- существует три типа бинарных связей, в зависимости от максимального кардинального числа:
 - связь типа «один-к-одному» (*one-to-one*): [1:1];
 - связь типа «один-ко-многим» (*one-to-many*): [1:N];
 - связь типа «многие-ко-многим» (*many-to-many*): [N:M];
- рассматриваемые связи также называются связями типа *принадлежность* (*HAS-A relationships*);
- в связях типа «один-ко-многим» сущность на единичной стороне часто называют родительской (*parent*), а на множественной стороне – дочерней (*child*).



Минимальное кардинальное число

- минимальное кардинальное число, как правило, указывается как ноль или единица:
 - если минимальное кардинальное число равно 0, то участие некоторого экземпляра сущности в связи *не является* обязательным (*optional*);
 - если минимальное кардинальное число равно 1, то участие некоторого экземпляра сущности в связи *является* обязательным (*mandatory*);

(a) Mandatory-to-Mandatory (M-M) Relationship



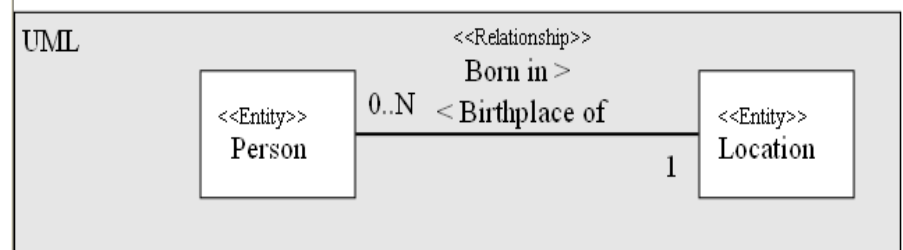
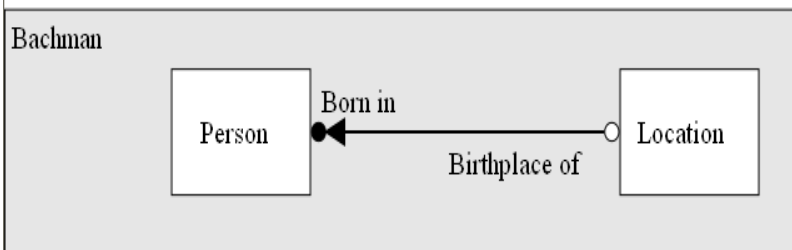
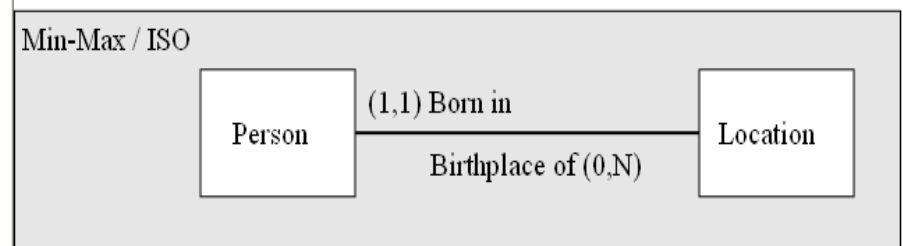
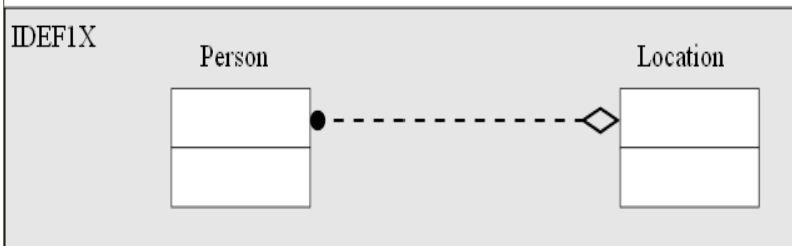
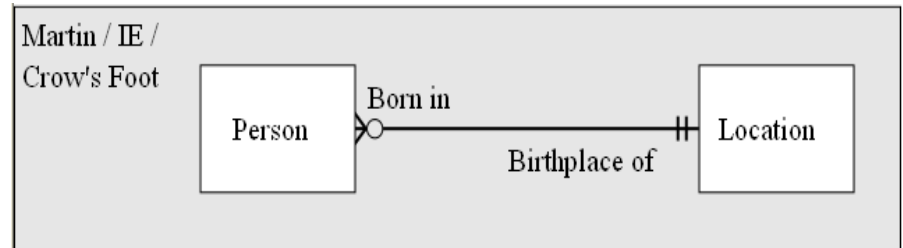
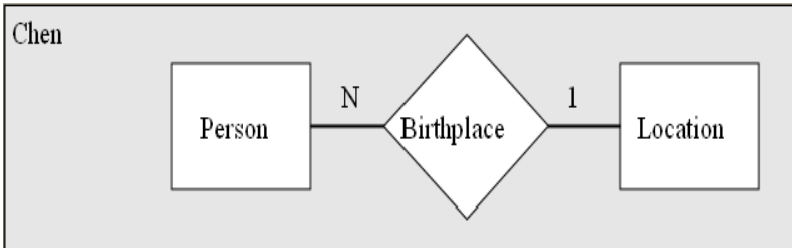
(b) Optional-to-Optional (O-O) Relationship



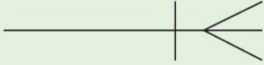
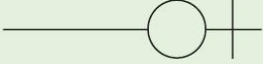
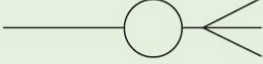
(c) Optional-to-Mandatory Relationship



Представление бинарных связей в различных нотациях

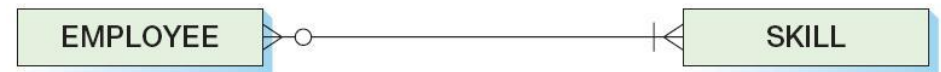


Нотация Information Engineering (crow's foot, Дж.Мартин)

Symbol	Meaning
	One—Mandatory
	Many—Mandatory
	One—Optional
	Many—Optional

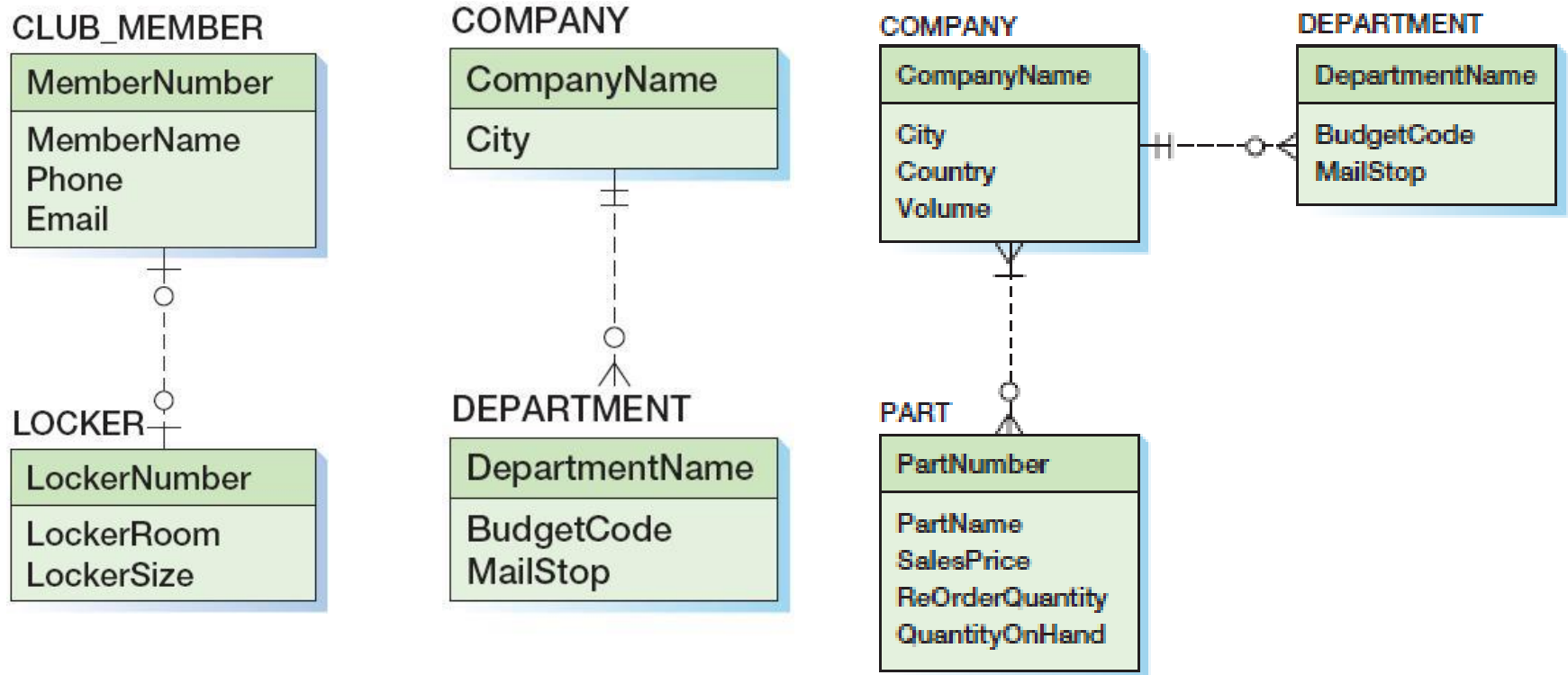


(a) Original E-R Model Version



(b) Crow's Foot Version

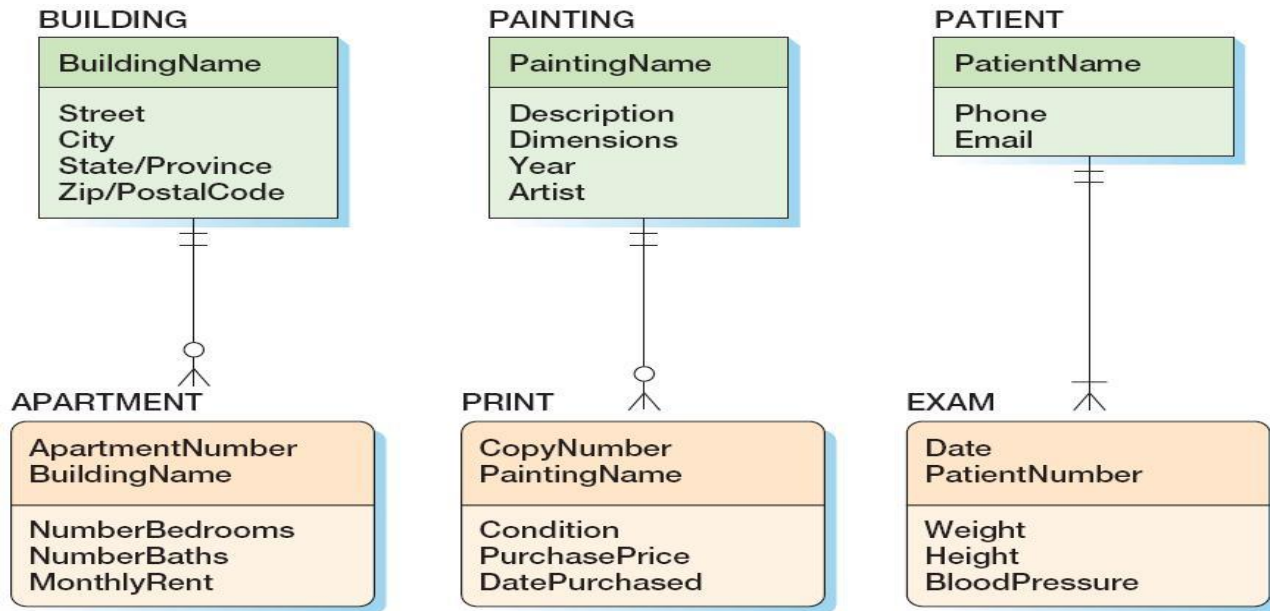
Сильные сущности: примеры



- СВЯЗЬ 1:1;
- СВЯЗЬ 1:N;
- СВЯЗЬ N:M.

Идентификационно-зависимые сущности

- идентификационно-зависимой (*ID-dependent*) называется такая дочерняя сущность, идентификатор которой содержит идентификатор родительской сущности (дочерняя сущность является логическим расширением или составной частью родительской);
- минимальное кардинальное число родительской сущности для идентификационно-зависимой сущности всегда равно *единице*.



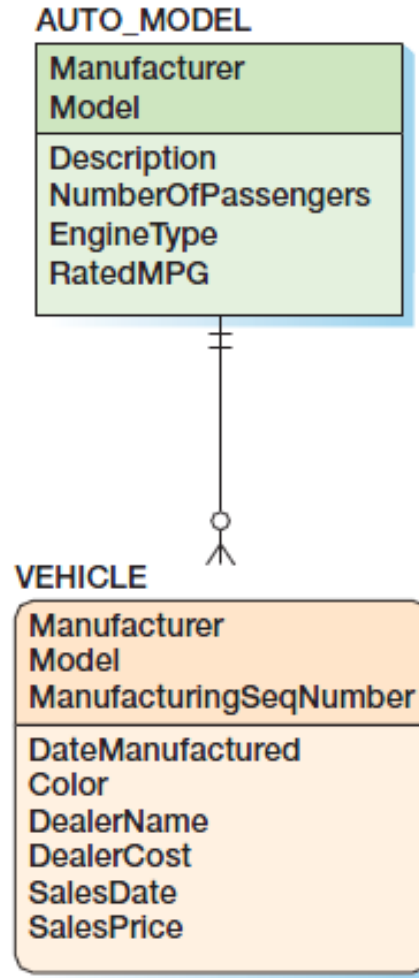
(a) APARTMENT is
ID-Dependent on
BUILDING

(b) PRINT is
ID-Dependent
on PAINTING

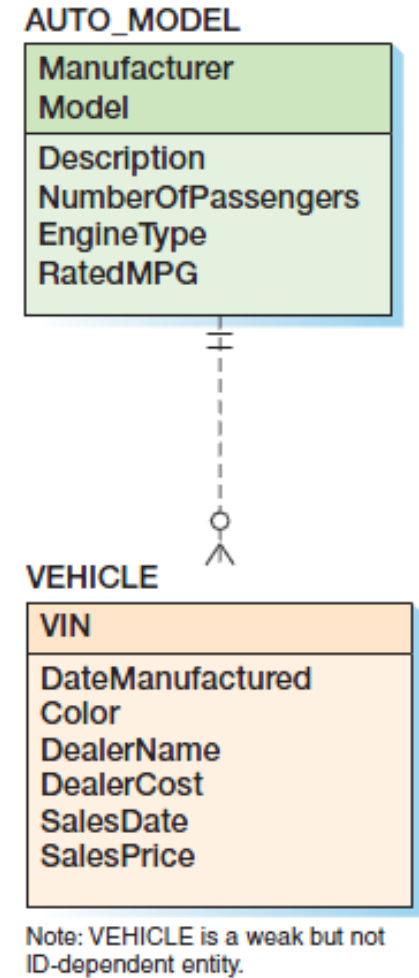
(c) EXAM is
ID-Dependent
on PATIENT

Слабые сущности

- *слабой сущностью (weak entity)* называется сущность, существование которой зависит от наличия родительской сущности;
- все идентификационно-зависимые сущности являются слабыми;
- возможно наличие в модели слабых сущностей, которые при этом не являются идентификационно-зависимыми, в этом случае условия зависимости реализуются дополнительными средствами бизнес-логики.



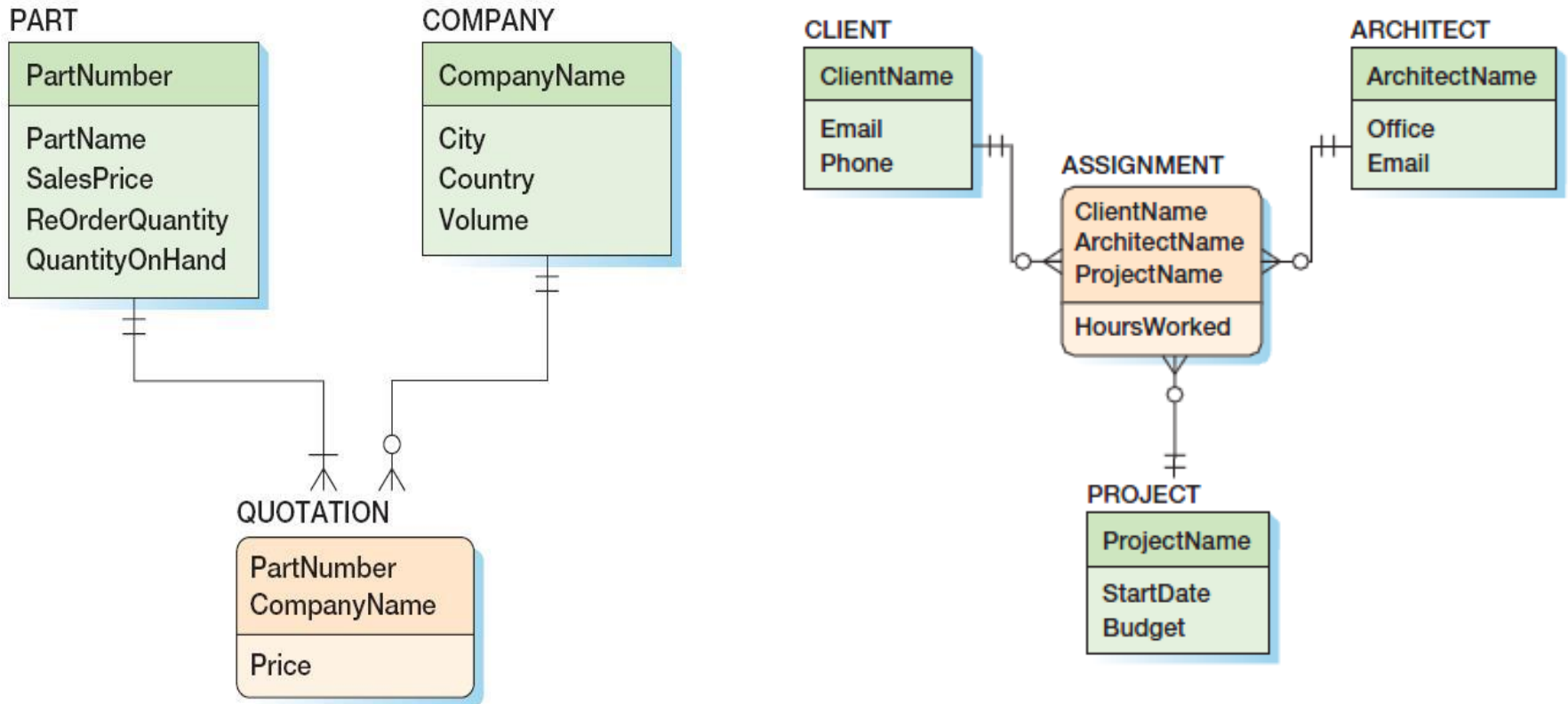
(a) ID-Dependent Entity



Note: VEHICLE is a weak but not ID-dependent entity.

(b) Non-ID-Dependent Weak Entity

Идентификационно-зависимые сущности: шаблон «сопряжение» (The Association Pattern)



- сопряжение может быть реализовано с использованием слабой сущности (при наличии собственного идентификатора)

Идентификационно-зависимые сущности: многозначные атрибуты (The Multivalued Attribute Pattern)

COMPANY

CompanyName: Ajax Manufacturing

City: Sydney

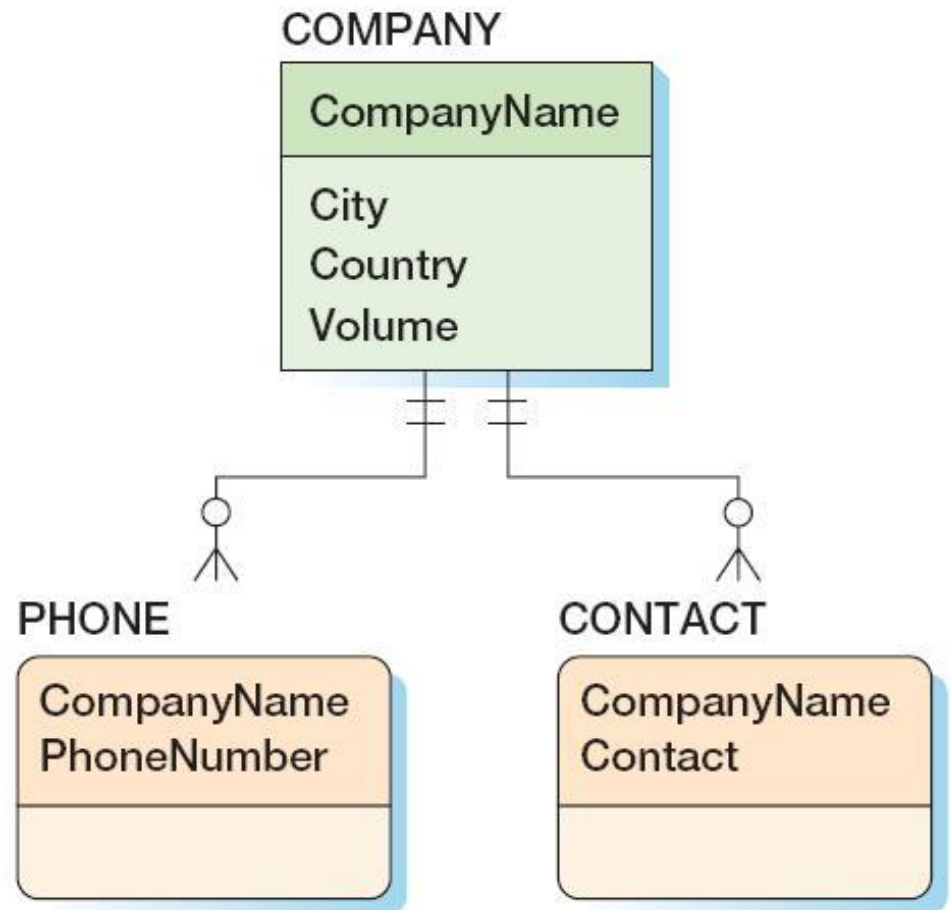
Country: Australia

Volume (USD): \$187,500.00

PHONE

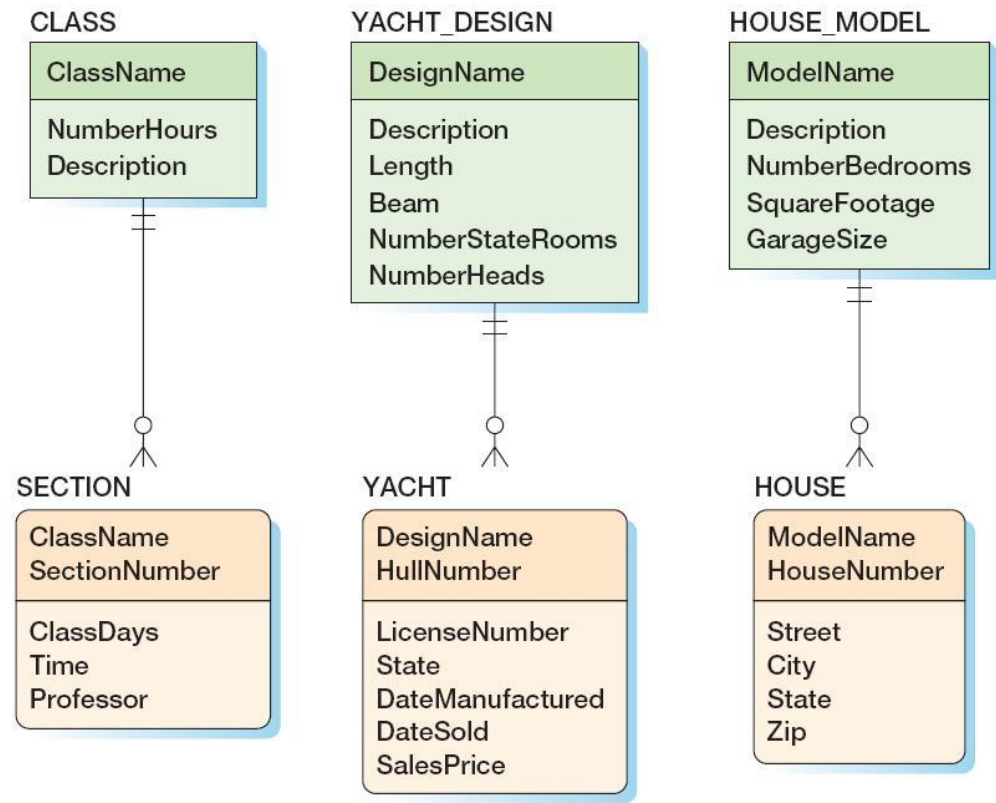
PhoneNumber
1.100.334.8000
1.100.444.9988
800-123-4455

Record: 4 of 1

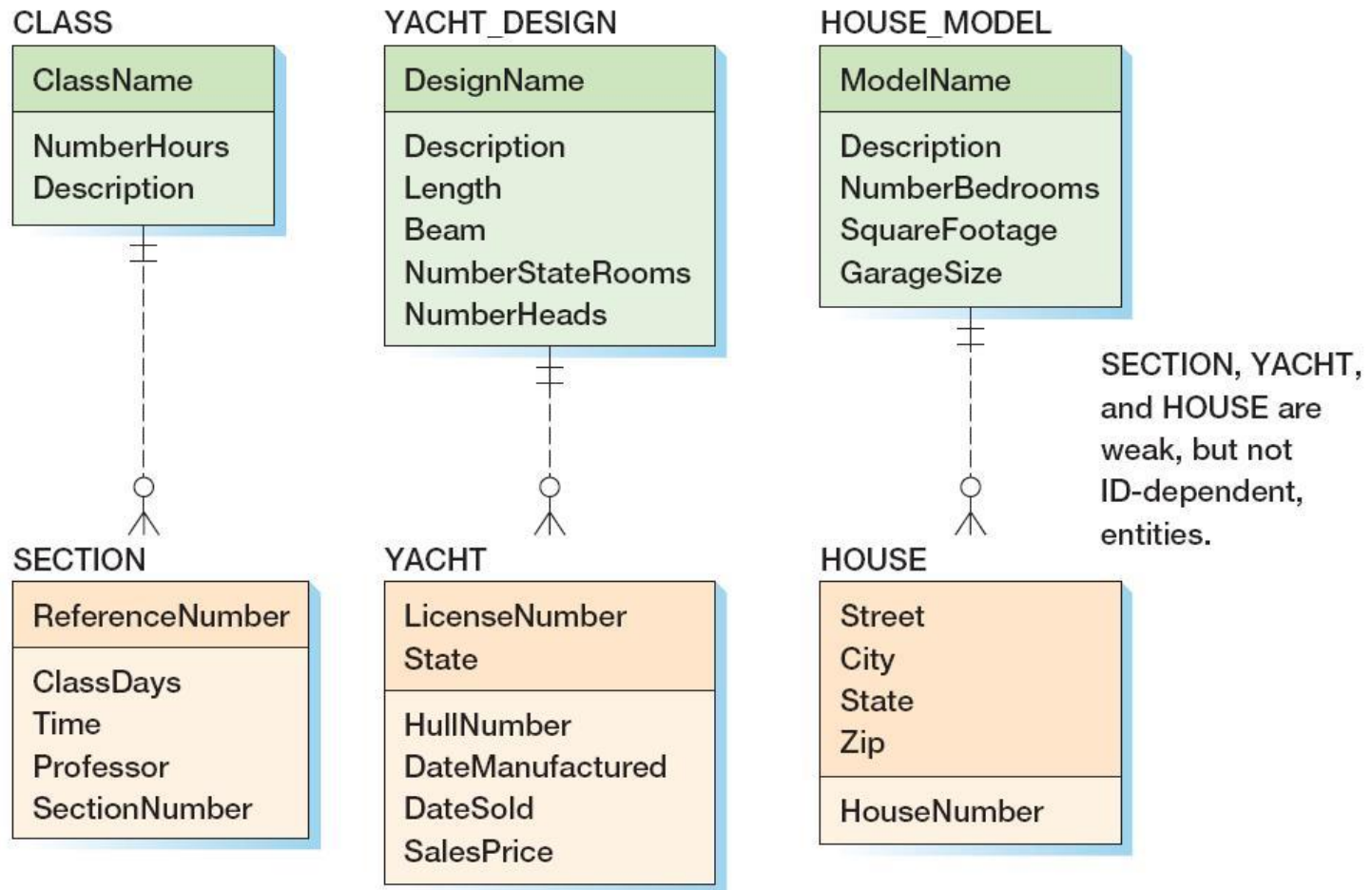


Идентификационно-зависимые сущности: шаблон «прототип-экземпляр» (The Archetype/Instance Pattern)

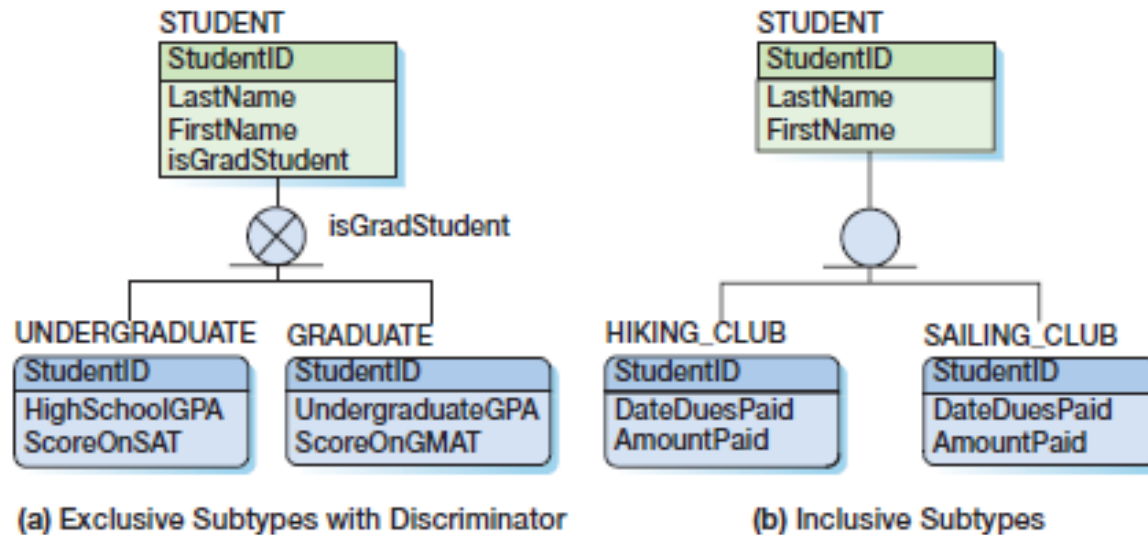
- данный шаблон необходим в том случае, когда идентификационно-зависимая дочерняя сущность является физической реализацией некоторой абстрактной или логической родительской сущности.



Реализация шаблона «прототип-экземпляр» с использованием слабых сущностей

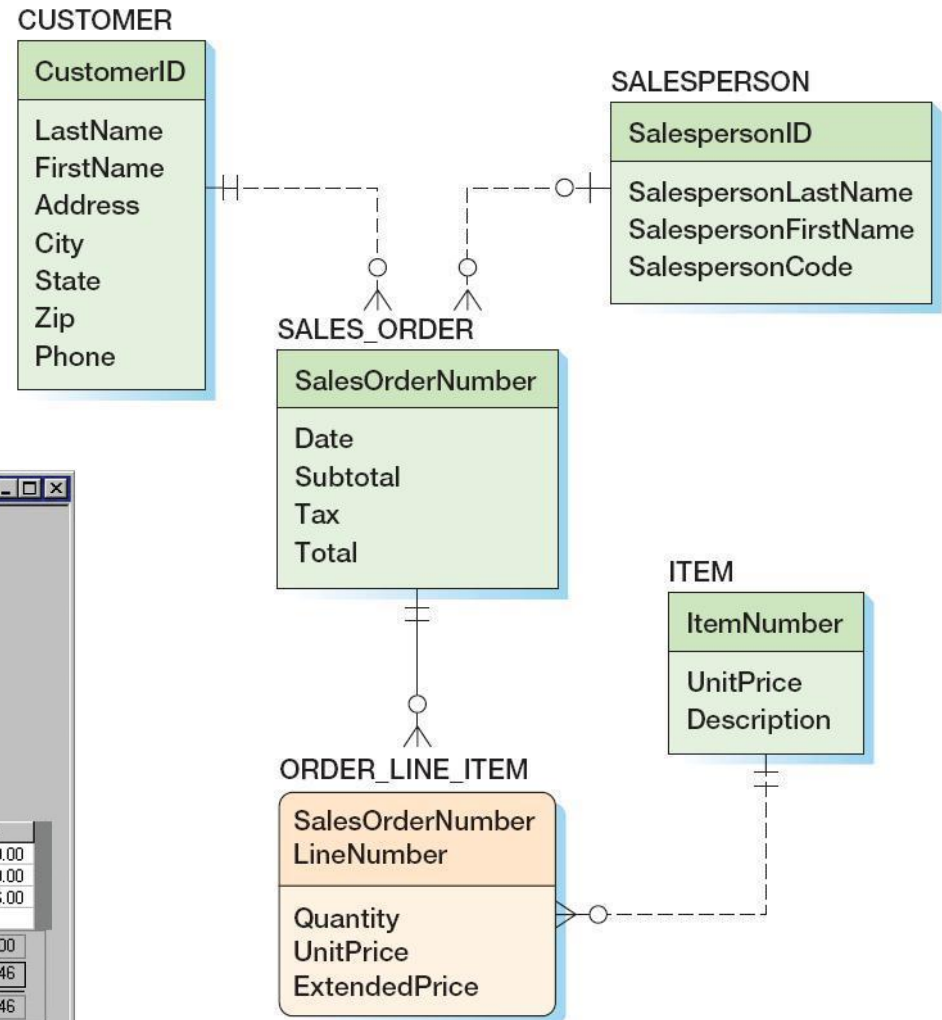


Сущности категория-подтип



- родительская сущность (*supertype entity*) содержит атрибуты, общие для нескольких подтипов;
- подтип (*subtype entity*) расширяет набор атрибутов родительской сущности;
- в ряде случаев один из атрибутов супертипа (*дискриминатор, discriminator*) указывает на то, каким из подтипов является конкретный экземпляр сущности;
- подтипы могут быть *взаимоисключающими (exclusive)* и *невзаимоисключающими (inclusive)*.

Шаблон «строка-элемент» (The Line-Item Pattern)



Carbon River Furniture Sales Order Form

Sales Order Number: 10643 Date: 25-Sep-08

Customer Name: Carbon River Bookshop

Address: 1145 Elm Street

City: Carbon River State: IL Zip: 02234

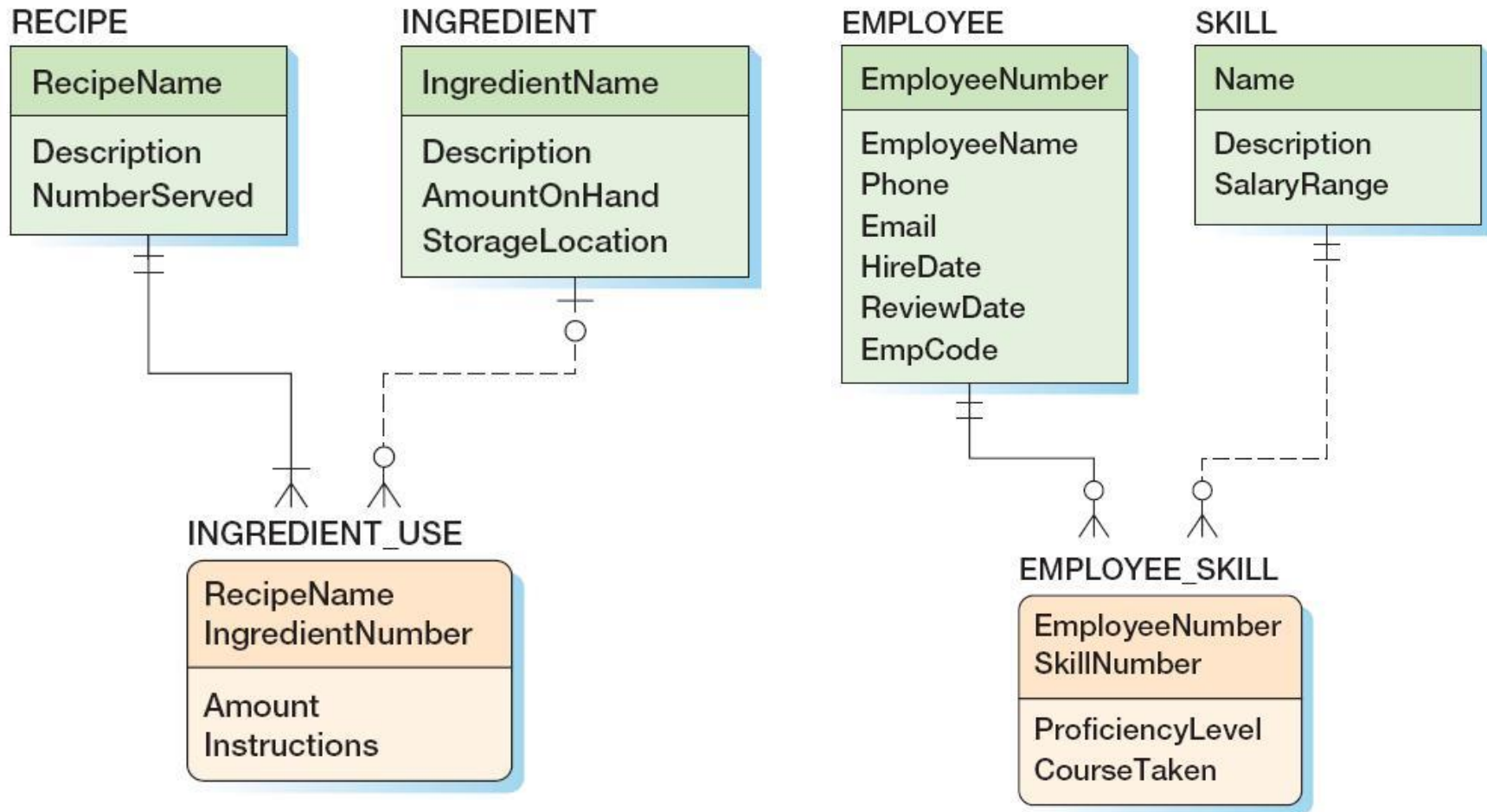
Phone: 232-0010

Salesperson Name: Dodsworth, Anne Salesperson Code: EZ-1

Quantity	Item Number	Description	Unit Price	Extended Price
1	78	Executive Desk	\$959.00	\$959.00
1	79	Conference Table	\$1,750.00	\$1,750.00
4	80	Side Chair	\$99.00	\$396.00

Subtotal: \$3,105.00
Tax: \$29.46
Total: \$3,134.46

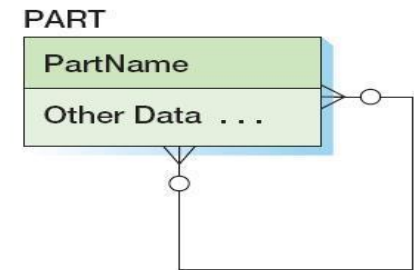
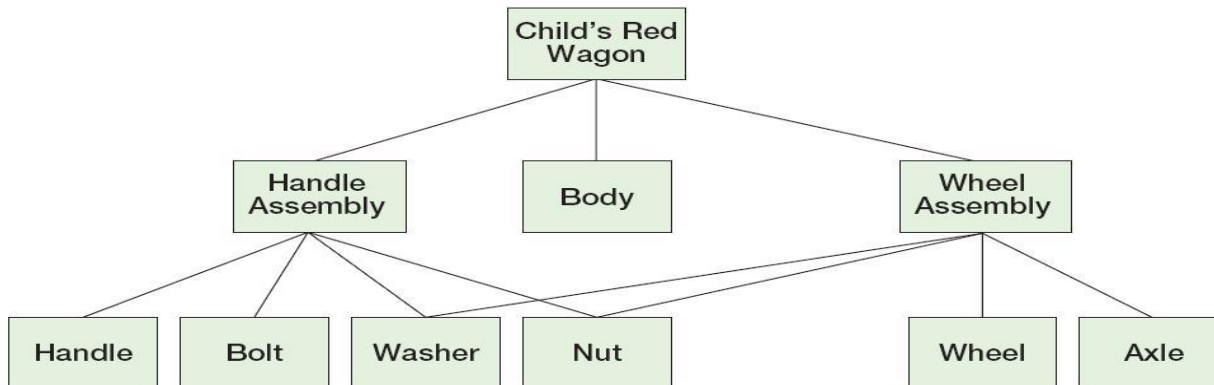
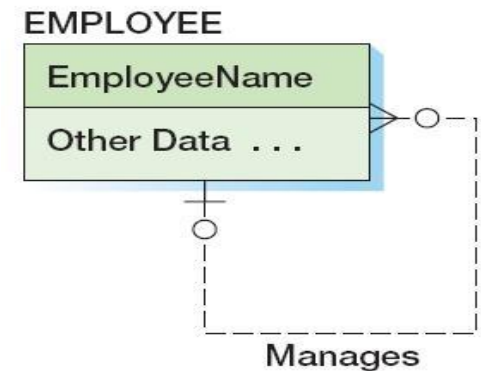
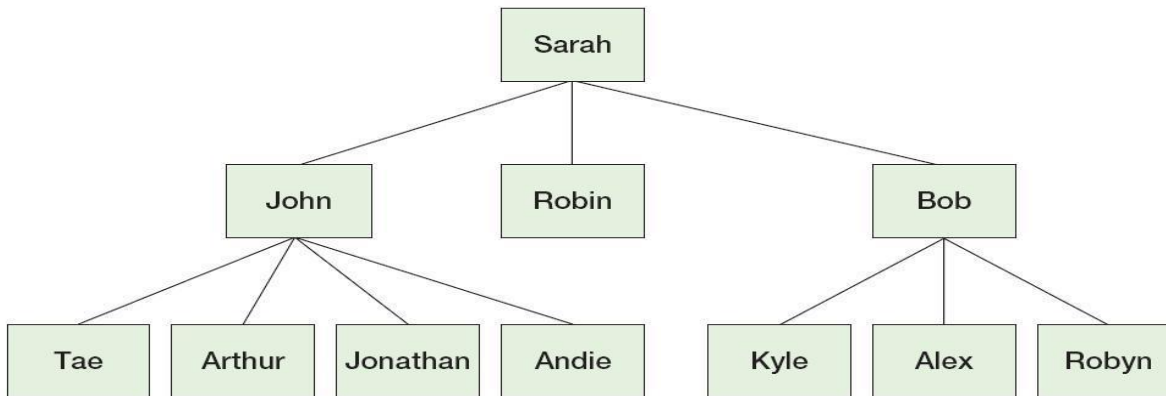
Смешанные шаблоны



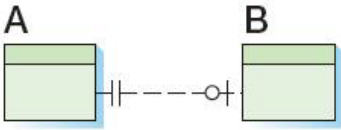
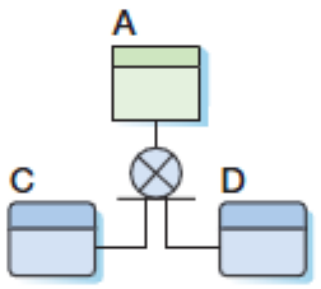
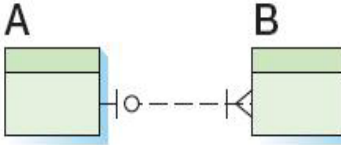
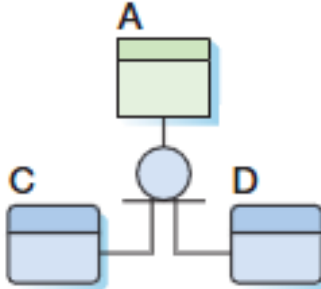
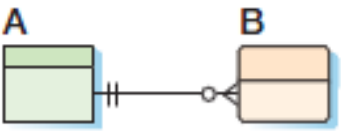
Фактически: многозначные атрибуты, связанные с сильной сущностью

Рекурсивные связи

- рекурсивная (*recursive*) связь возникает, когда некоторый класс сущностей имеет связь с самим собой



Резюме по нотации

DEPARTMENT DepartmentName BudgetCode OfficeNumber		сущность DEPARTMENT; DepartmentName – идентификатор; BudgetCode, OfficeNumber – атрибуты; Идентификационно-зависимые сущности изображаются прямоугольниками со скругленными углами	
	неидентифицирующая связь 1:1 сущности A соответствует 0 или 1 сущности B, сущности B соответствует ровно одна сущность A		сущность A является супертипом, C и D являются взаимоисключающими подтипами, дискриминатор не указан
	неидентифицирующая связь 1:N сущности A соответствует 1..N сущностей B, сущности B соответствует 0 или одна сущность A		сущность A является супертипом, C и D являются неэксклюзивными подтипами
	идентифицирующая связь 1:N сущности A соответствует 0..N сущностей B, сущности B соответствует ровно одна сущность A		

Вопросы к экзамену

- Процесс разработки баз данных и уровни моделирования информационных систем. Моделирование данных. Модель «сущность-связь» и ее основные элементы (сущность, связь, атрибуты).
- Модель «сущность-связь» и ее основные элементы (сущность, связь, атрибуты). Кардинальные числа связей.
- Модель «сущность-связь». Идентификационно-зависимые, слабые и сильные сущности. Сущности категория-подтип.
- Модель «сущность-связь»: шаблоны моделирования «сопряжение» и «прототип-экземпляр», представление многозначных атрибутов и рекурсивных связей.

Лабораторная работа №1

Моделирование данных с использованием модели сущность-связь

1. Выбрать простейшую предметную область, соответствующую 4-5 сущностям.
2. Сформировать требования к предметной области.
3. Создать модель «сущность-связь» для предметной области с обоснованием выбора кардинальных чисел связей.

Базы данных

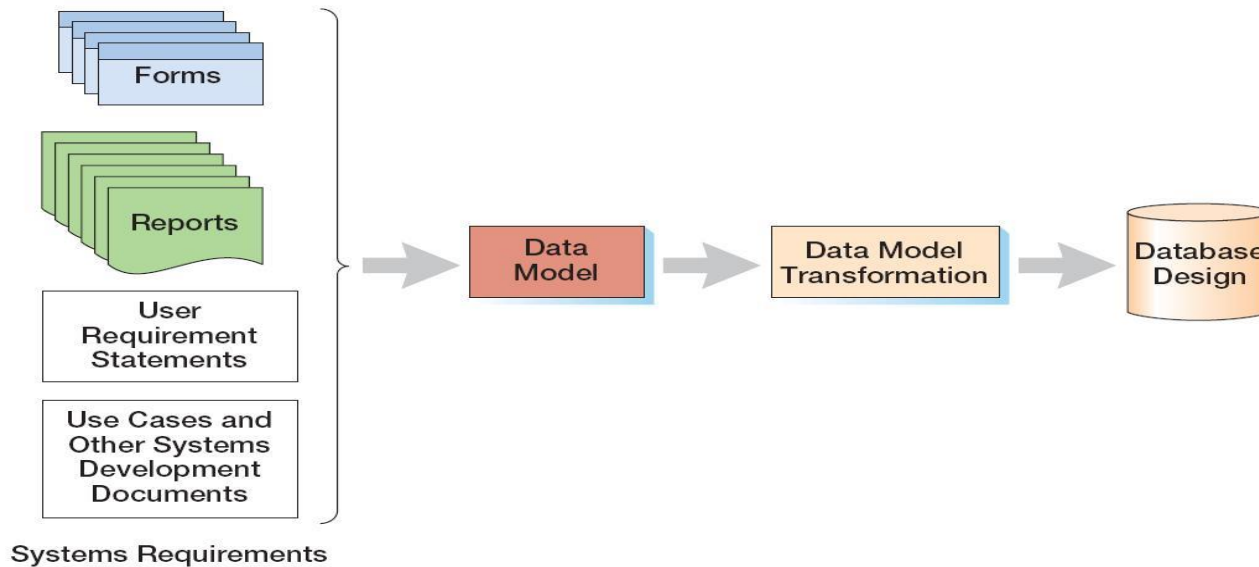
Тема 2

Моделирование данных:
модель семантических объектов

Процесс разработки БД

- общие стратегии:
 - разработка сверху вниз (*top-down database development*)
 - получение абстрактной модели данных исходя из анализа стратегических целей организации, способов их достижения, а также информации и способов ее представления, которые необходимы для достижения этих целей;
 - лучшее взаимодействие подсистем, глобальный охват;
 - разработка снизу вверх (*bottom-up database development*)
 - для разработки выбирается конкретная система, которая выполняет только часть функций предприятия;
 - исходя из сформированных требований, выбранная подсистема создается быстрее и с меньшими рисками;
- одна из основ эффективной реализации приложения базы данных – ясное представление модели предметной области

Три уровня моделей информационных систем



- внешние модели (*external schema*): представление пользователей о системе (user view);
- концептуальная модель (*conceptual schema*): абстрактное представление системы (данных);
- внутренние модели (*internal schema*).

Модели данных (ANSI)

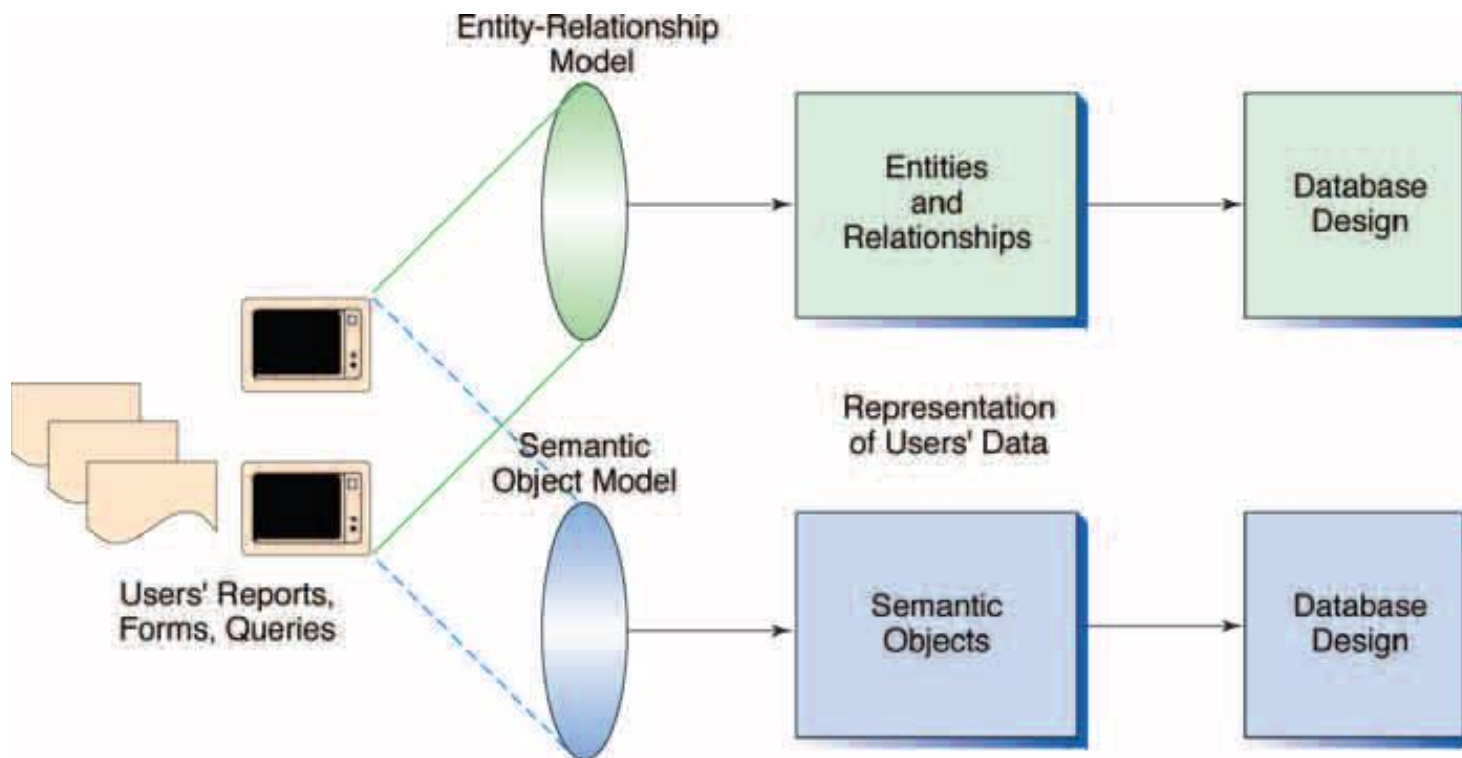
- внешняя модель (*external schema*) – набор представлений пользователей о той части предметной области, с которой он сталкивается;
- концептуальная модель (*conceptual schema / conceptual data model*) – набор ключевых объектов предметной области, их взаимосвязей (взаимодействий) и ограничений, накладываемых на объекты;
- логическая модель (*logical schema / logical data model / information schema*) – представление концептуальной модели в соответствии с логической моделью, используемой для хранения данных;
- физическая модель (*physical data model*) – описание физического представления, хранения и обработки данных.

Моделирование данных

- база данных является «моделью модели», то есть через объекты базы данных описывает представление пользователей о конкретной предметной области;
- при любом процессе разработки необходимо сформулировать требования и построить на их основе модель данных, что является во многом творческой задачей, решение которой основано на опыте и интуиции;
- моделирование данных (*data modeling*) - процесс создания логического представления базы данных (пользовательская модель данных - *user data model*, модель требований к данным - *requirements data model*);
- модель данных представляет собой совокупность понятий и графических символов для построения концептуальных схем, т.е. содержит языковые и изобразительные стандарты для своего представления;
 - модель сущность – связь (*entity-relationship model, E-R model*);
 - модель семантических объектов (*semantic object model*);

Модель семантических объектов

- модель семантических объектов – альтернативный способ представления модели данных;
- семантика (от греч. *semantikos* - обозначающий) - значение единиц языка.

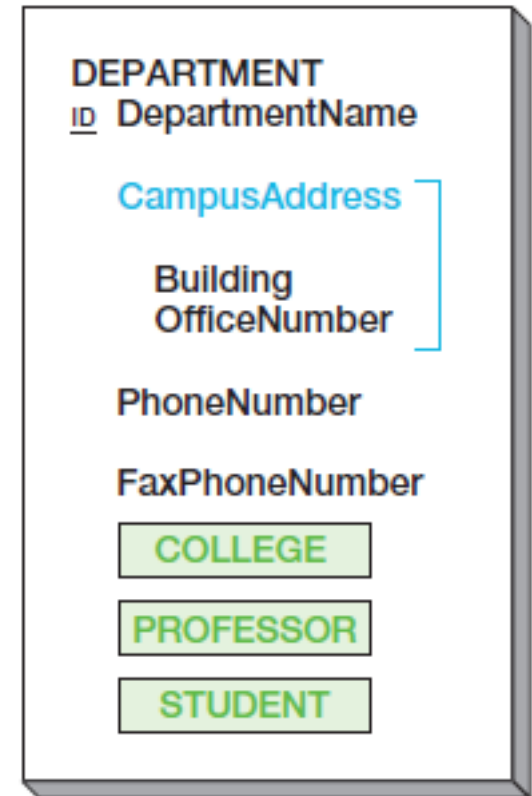


Семантический объект

- семантический объект (*semantic object*) – некоторый объект, который способен существовать независимо и может быть идентифицирован (именованный набор атрибутов, исчерпывающе описывающий отдельную сущность);
- аналогично сущностям ER-модели,
 - можно определить:
 - класс объектов (*object class*) – набор объектов определенного типа;
 - экземпляр объекта (*object instance*) – объект некоторого класса с заданными значениями атрибутов;
 - атрибут (*attribute*) – свойство, которым обладает объект;
 - объект может являться абстракцией:
 - реального физического объекта;
 - события, действия или процесса (заказ, услуга, транзакция).

Атрибуты

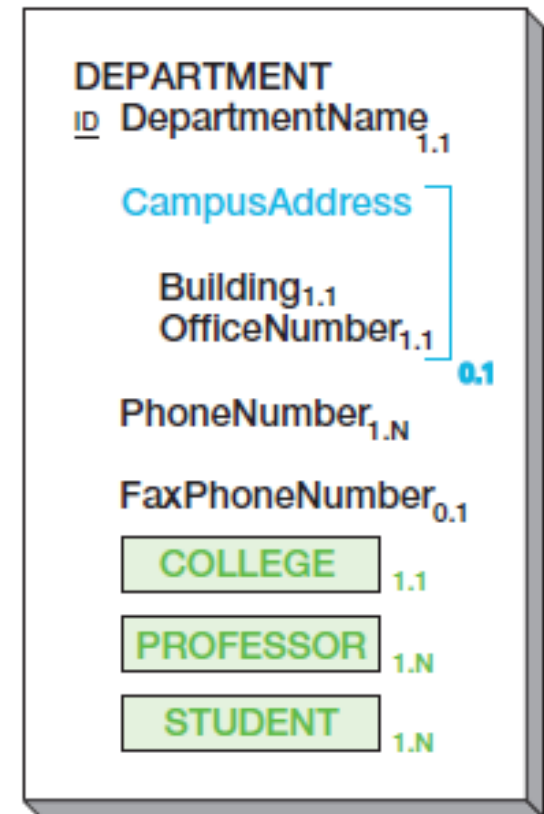
- *атрибу́т (attribute)* – свойство, которым обладает объект;
- все объекты определенного класса обладают одинаковым набором атрибутов, но различаются значениями данных атрибутов (исчерпывающее описание, *sufficient description*);
- в общем случае, атрибуты могут быть:
 - простыми (*simple*): атрибут со значением элементарного типа;
 - групповыми, или составными (*group*): атрибут состоит из набора простых атрибутов;
 - объектными (*object*): атрибут является семантическим объектом (очевидно, атрибуты данного типа являются <зеркальными> для двух соответствующих объектов).



(a) DEPARTMENT Object

Кардинальные числа атрибутов

- *кардинальное число (cardinality, мощность)* указывает количество значений, которые может принимать атрибут в корректном (valid) экземпляре объекта;
- *максимальное кардинальное число (maximum cardinality)* - максимальное количество значений, которые может принимать атрибут – как правило, 1 или N;
- *минимальное кардинальное число (minimum cardinality)* - минимальное количество значений, которые должен принимать атрибут – как правило, 0 или 1.
- дополнительные термины для описания атрибутов:
 - однозначный атрибут (*single-value attribute*) – кардинальность ..1;
 - многозначный атрибут (*multivalued attribute*) – кардинальность ..N;
 - необъектный атрибут (*non-object attribute*) – простой или групповой.



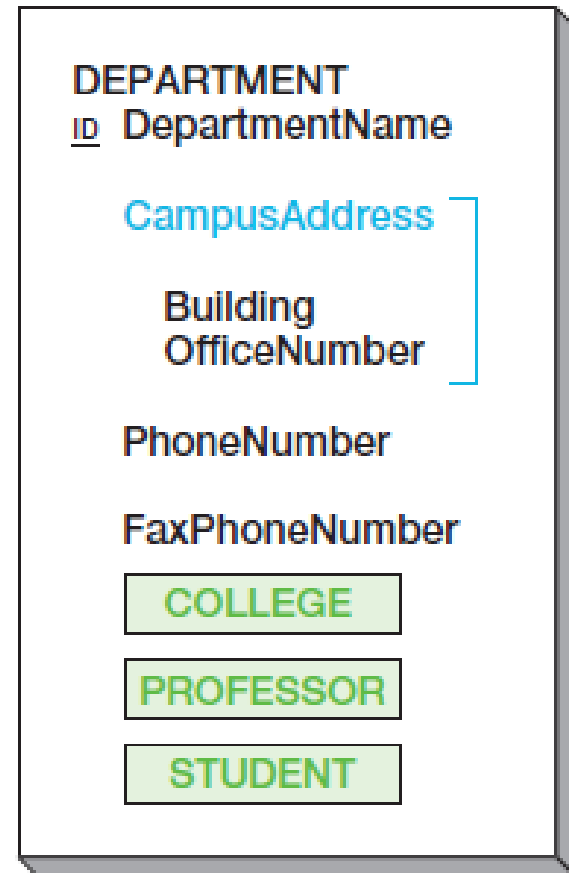
(b) DEPARTMENT Object with Cardinalities

Домены атрибутов

- домен (*domain*) – множество допустимых значений атрибута;
- понятие домена имеет различный смысл для атрибутов различных типов:
 - для простых атрибутов:
 - физическое описание: тип данных и дополнительные наложенные ограничения;
 - семантическое (логическое) описание: описывает назначение данного атрибута в модели;
 - для составных атрибутов:
 - физическое описание: список и порядок входящих в него простых атрибутов;
 - семантическое описание: описывает назначение данного атрибута в модели;
 - для объектных атрибутов: множество всех экземпляров объектов рассматриваемого класса.

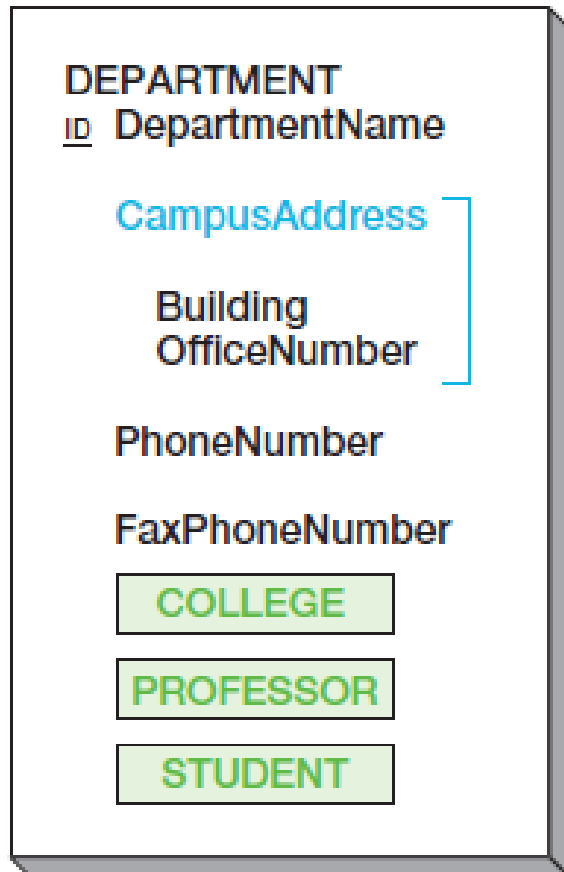
Идентификаторы

- идентификатор объекта (*object identifier*) – набор атрибутов, которые определяют, именуют сущность;
- идентификаторы могут быть составными, т.е. состоять более, чем из одного атрибута;
- обозначаются «ID» или «ID» в случае уникальности;

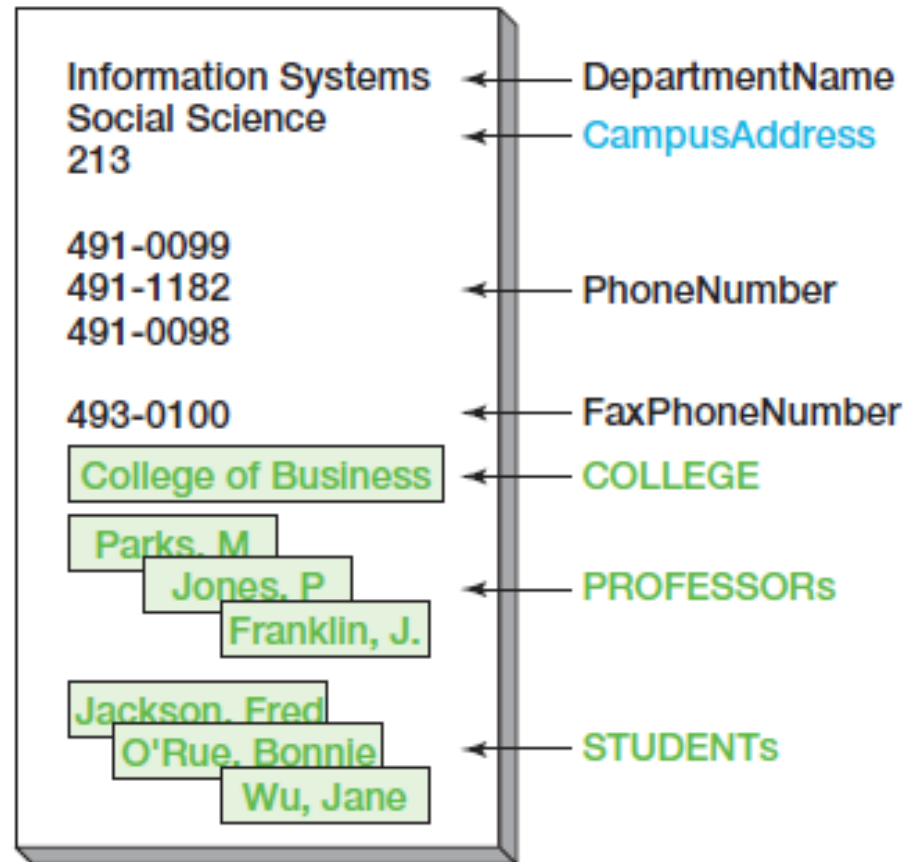


(a) DEPARTMENT Object

Экземпляры семантических объектов



(a) DEPARTMENT Object



Типы объектов: простой объект

- простой объект (*simple object*) – объект, содержащий только однозначные простые или групповые атрибуты.

EQUIPMENT TAG:

EquipmentNumber: 100 Description: Desk
AcquisitionDate: 2/27/2008 PurchaseCost: \$350.00

EQUIPMENT TAG:

EquipmentNumber: 200 Description: Lamp
AcquisitionDate: 3/1/2008 PurchaseCost: \$39.95

(a) Reports Based on a Simple Object

EQUIPMENT

ID EquipmentNumber
 Description
 AcquisitionDate
 PurchaseCost

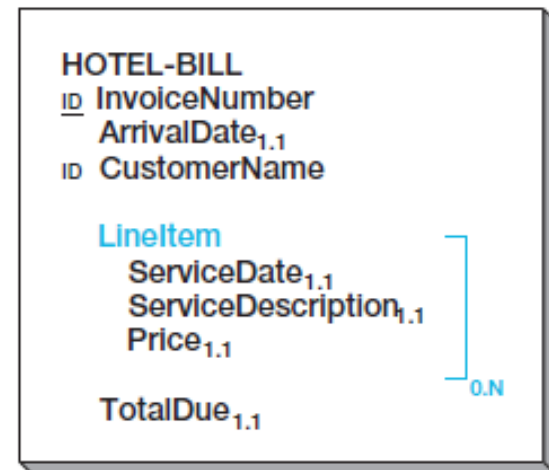
(b) EQUIPMENT
Simple Object

Типы объектов: составной объект

- составной объект (*composite object*) – объект, содержащий один или более многозначный простой или групповой атрибуты.

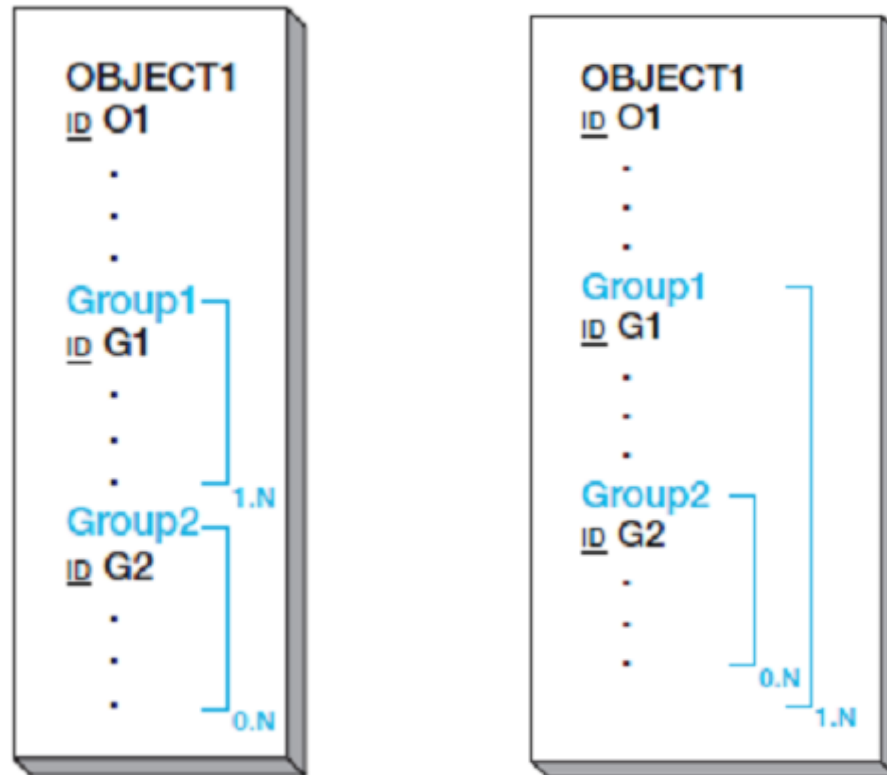
GRANDVIEW HOTEL <u>Sea Bluffs, California</u>		
Invoice Number: 1234		Arrival Date: 10/12/2008
Customer Name: Mary Jones		
10/12/2001	Room	\$ 99.00
10/12/2001	Food	\$ 37.55
10/12/2001	Phone	\$ 2.50
10/12/2001	Tax	\$ 15.00
10/13/2001	Room	\$ 99.00
10/13/2001	Food	\$ 47.90
10/13/2001	Tax	\$ 15.00
Total Due		\$ 315.95

(a) Reports Based on a Composite Object



(b) HOTEL-BILL Composite Object

Составной объект



- с независимыми группами;
- с вложенными группами.

Типы объектов: сложный объект

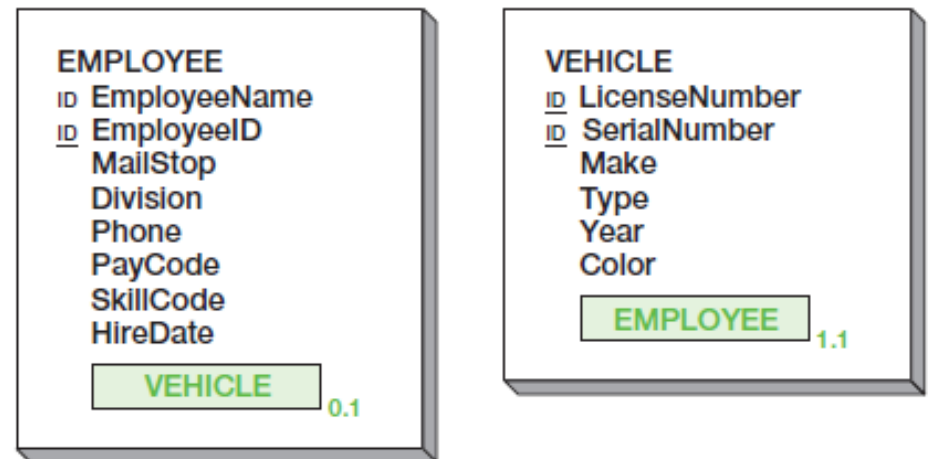
- сложный объект (*compound object*) – объект, содержащий один или более объектный атрибут.

VEHICLE DATA			
License number	Serial number		
Make	Type	Year	Color
Employee assignment			

EMPLOYEE WORK DATA			
Employee name		Employee ID	
MailStop		Division	Phone
Pay code	Skill code	Hire date	Auto assigned

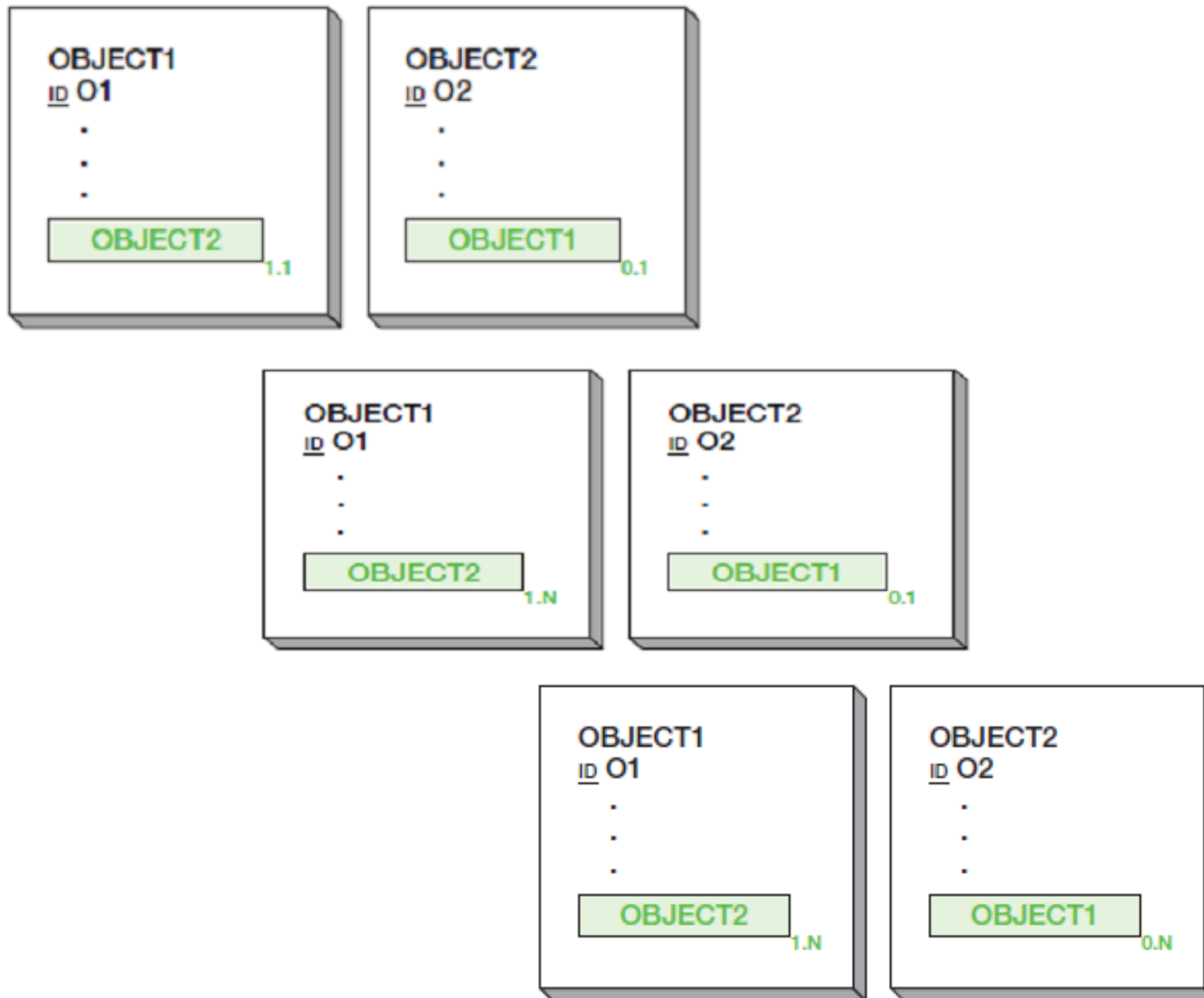
(a) Example Vehicle and Employee Data Entry Forms

Object1 Can Contain			
Object2 Can Contain		One	Many
	One	1:1	1:N
	Many	M:1	M:N



(b) EMPLOYEE and VEHICLE Compound Objects

Сложный объект (1:1; 1:N; M:N)

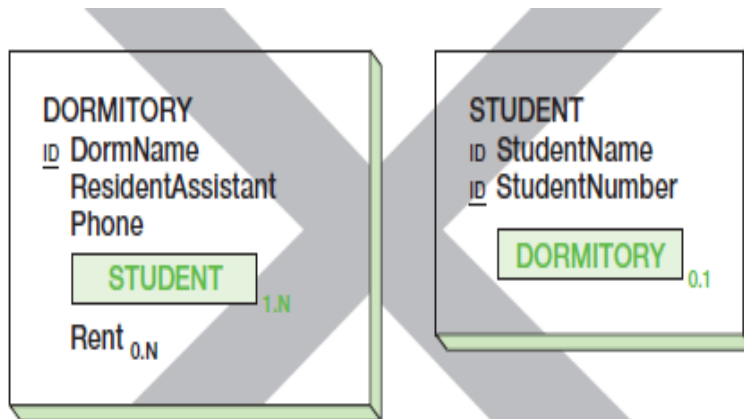


Типы объектов: гибридный объект

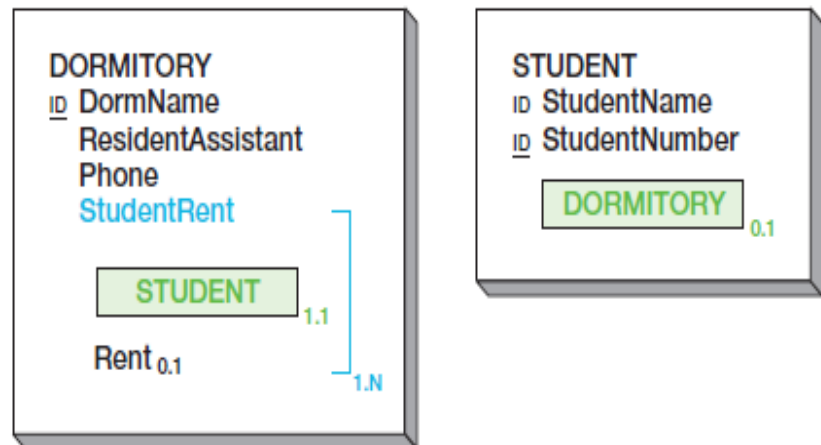
- гибридный объект (*hybrid object*) – объект, содержащий один или более групповой атрибут, содержащий в себе объектный атрибут

DORMITORY OCCUPANCY REPORT		
<u>Dormitory</u>	<u>Resident Assistant</u>	<u>Phone</u>
Ingersoll	Sarah and Allen French	3-5567
<u>Student name</u>	<u>Student Number</u>	<u>Rent</u>
Adams, Elizabeth	710	\$175.00
Baker, Rex	104	\$225.00
Baker, Brydie	744	\$175.00
Charles, Stewart	319	\$135.00
Scott, Sally	447	\$225.00
Taylor, Lynne	810	\$175.00

(a) Dormitory Report with Rent Property

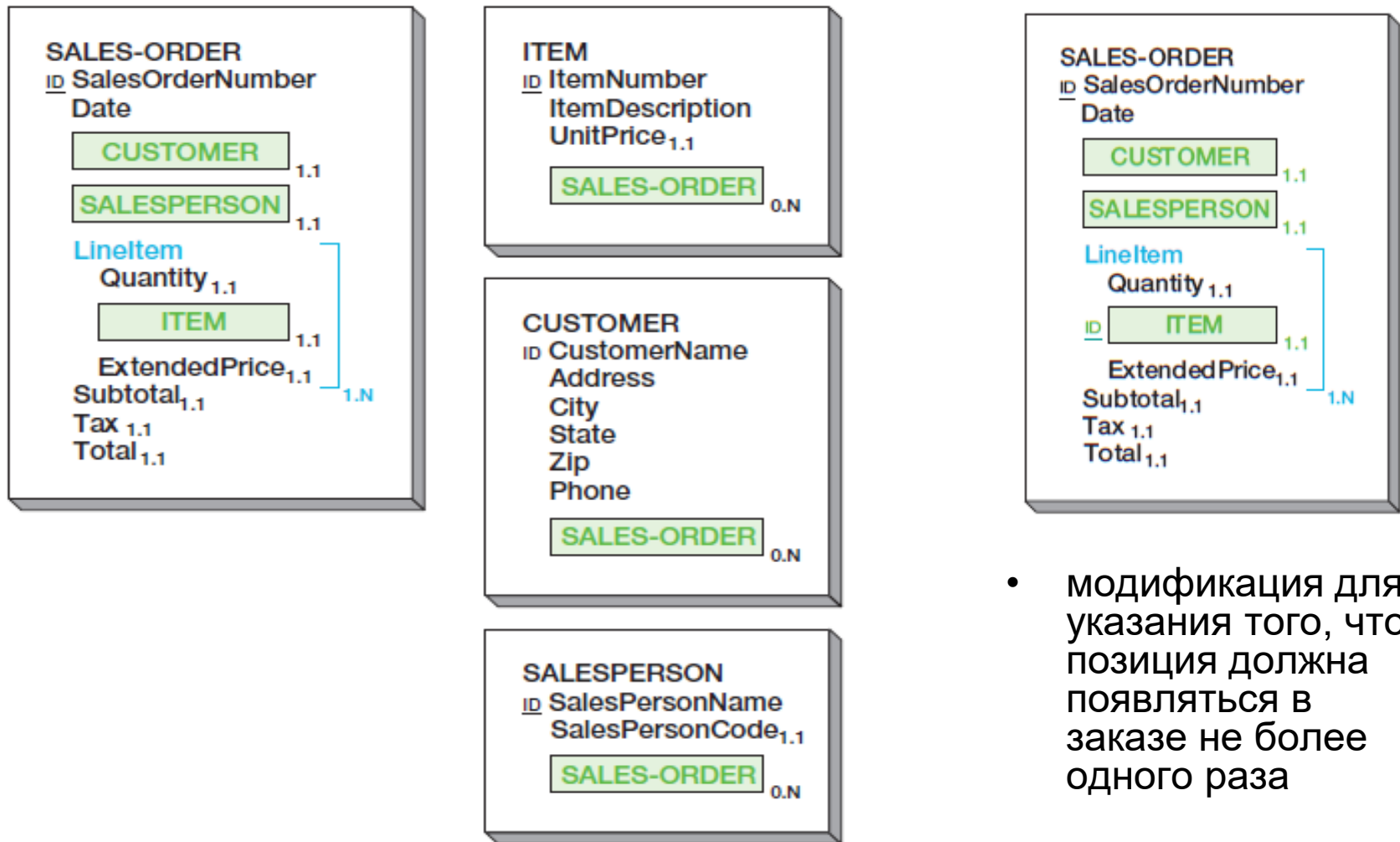


(c) Incorrect DORMITORY and STUDENT Objects



(b) Correct DORMITORY and STUDENT Objects

Гибридный объект: пример



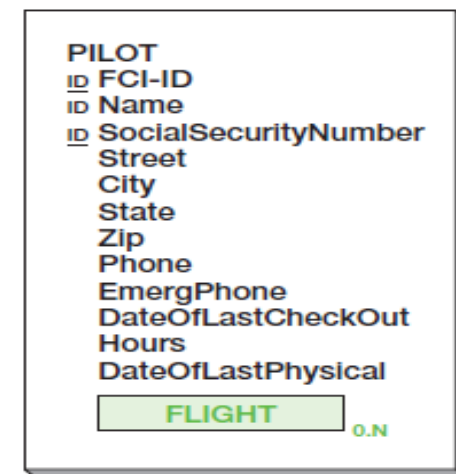
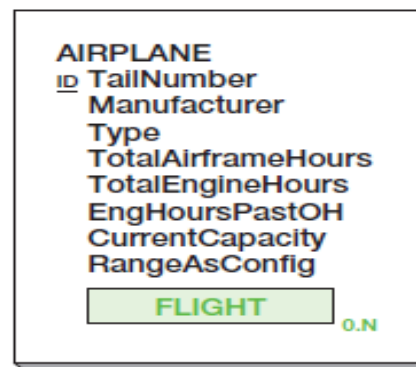
- модификация для указания того, что позиция должна появляться в заказе не более одного раза

(b) Objects to Model Sales Order Form

Типы объектов: ассоциативный объект

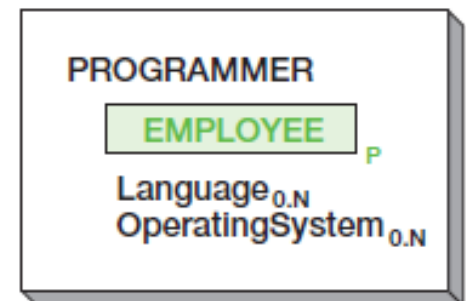
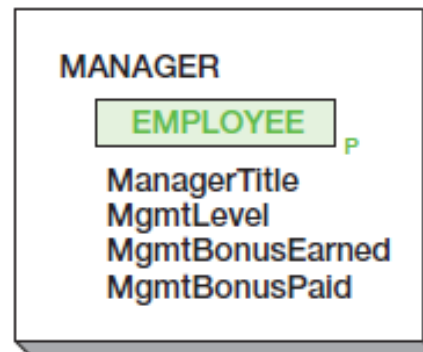
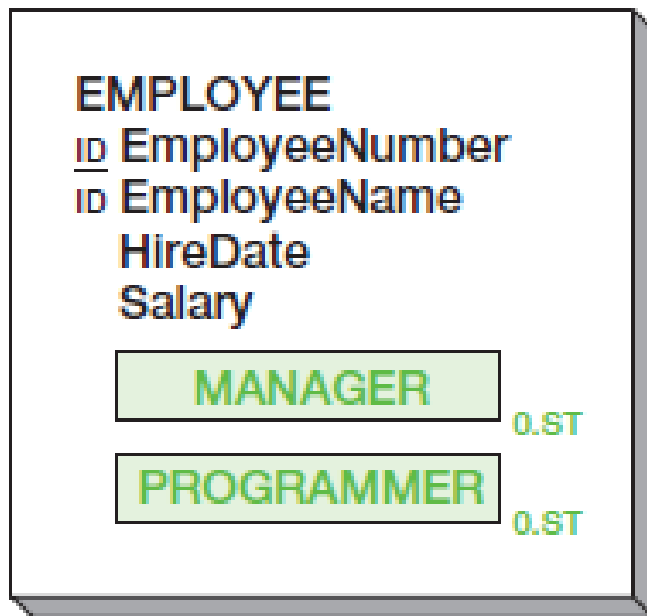
- ассоциативный объект (*association object*) – объект, сопоставляющий два или более объектов, и содержащий дополнительную информацию о таком сопоставлении

FLY CHEAP INTERNATIONAL Flight Planning Data Report		
FLIGHT NUMBER	FC-17	DATE 7/30/2008
ORIGINATING CITY	Seattle	DESTINATION Hong Kong
FUEL ON TAKEOFF		
WEIGHT ON TAKEOFF		
AIRPLANE		
Tail Number	N1234FI	
Type	747-SP	
Capacity	148	
PILOT		
Name	Michael Nilson	
FI-ID	32887	
Flight Hours	18,348	

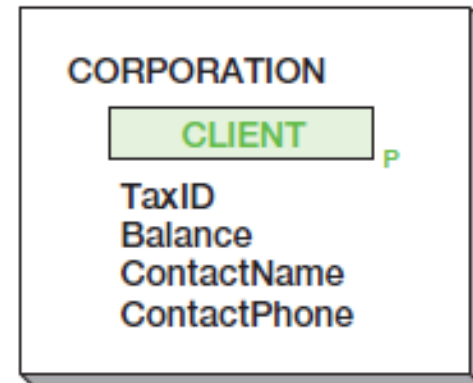
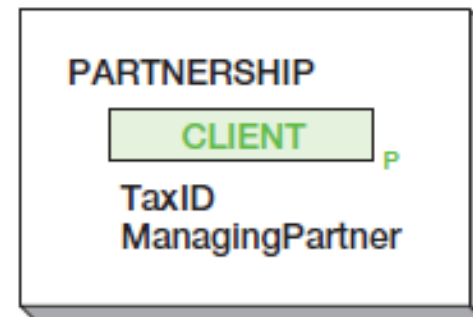
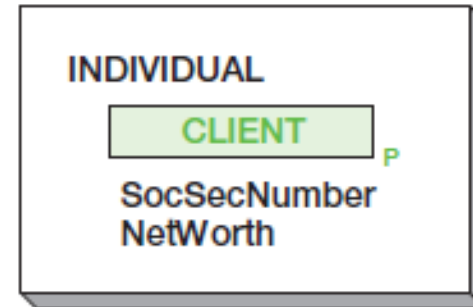
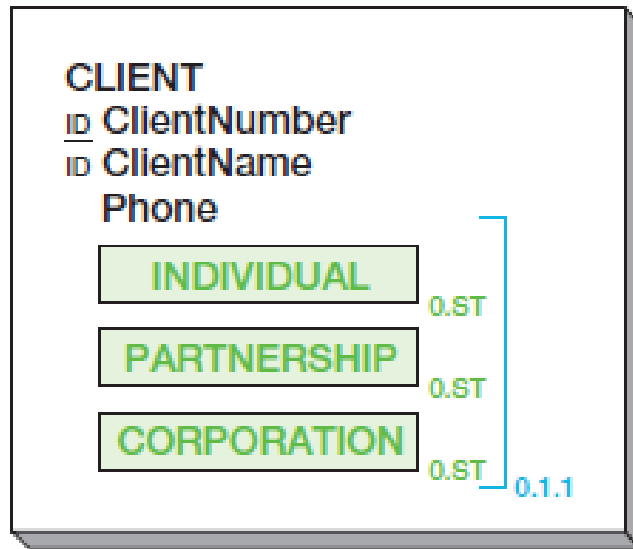


Типы объектов: родитель / подтип

- родительский объект (*parent / supertype object*) содержит атрибуты, общие для нескольких объектов-подтипов (кардинальное число определяет обязательность объекта-подтипа);
- объект-подтип (*subtype object*) расширяет набор атрибутов родительского объекта.

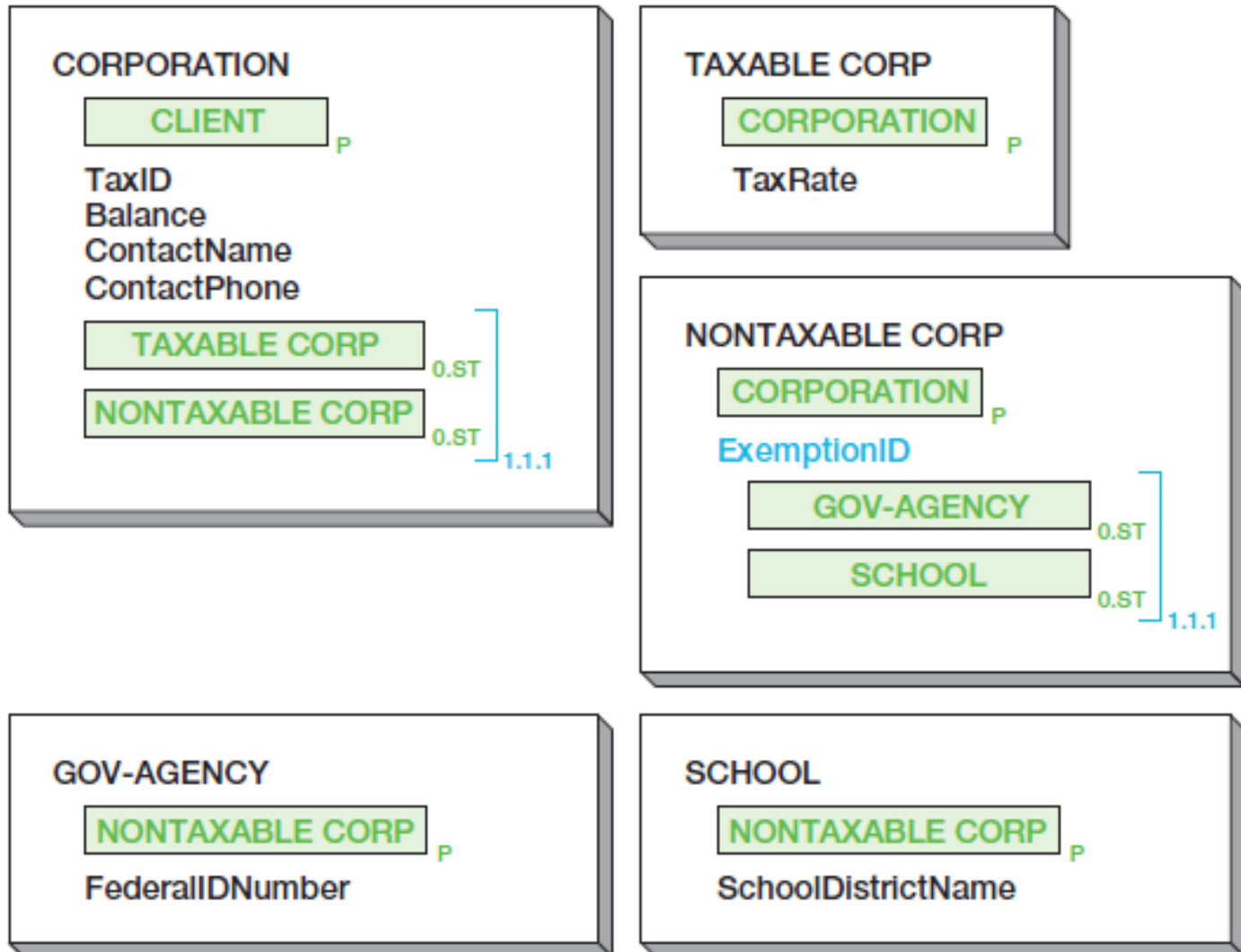


Родитель / подтип: взаимоисключающие подтипы



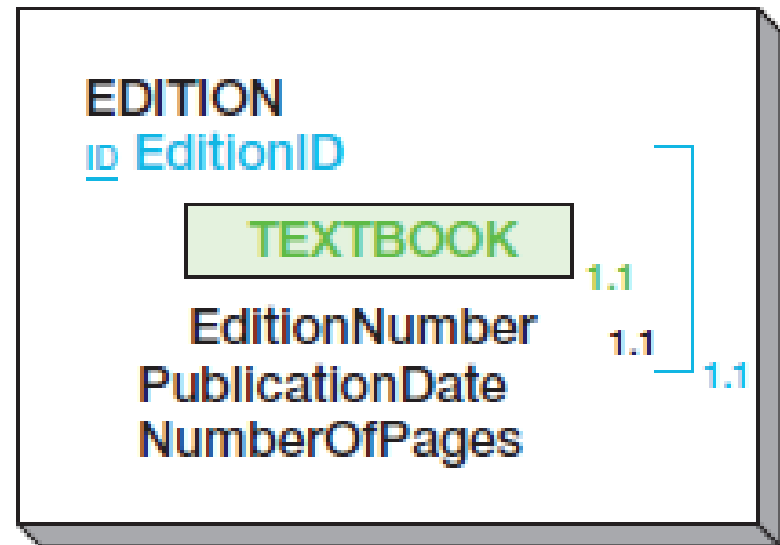
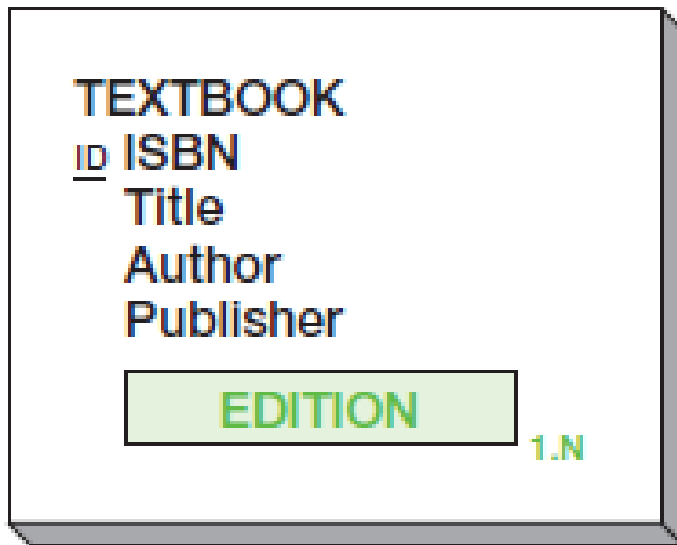
- X.Y.Z: для взаимоисключающих типов
 - X (0/1) – минимальное кардинальное число;
 - Y – минимальное количество атрибутов со значениями;
 - Z – максимальное количество атрибутов со значениями.

Родитель / подтип: вложенные подтипы

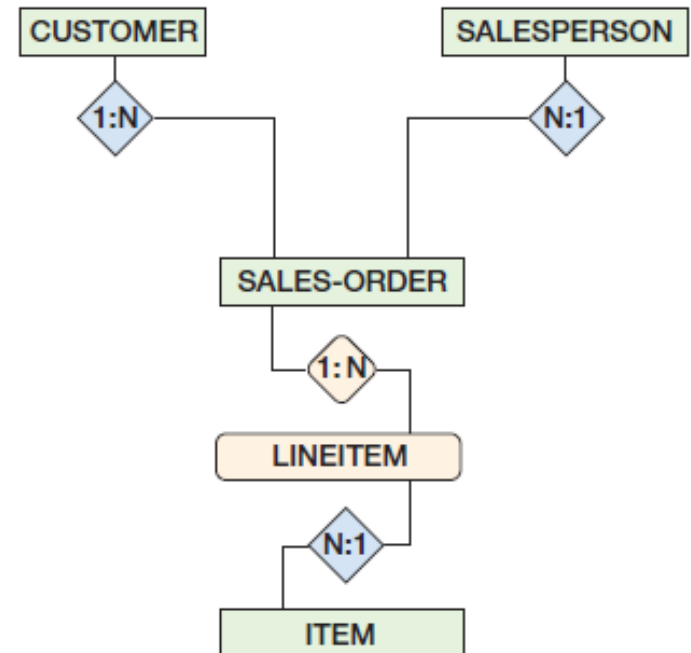
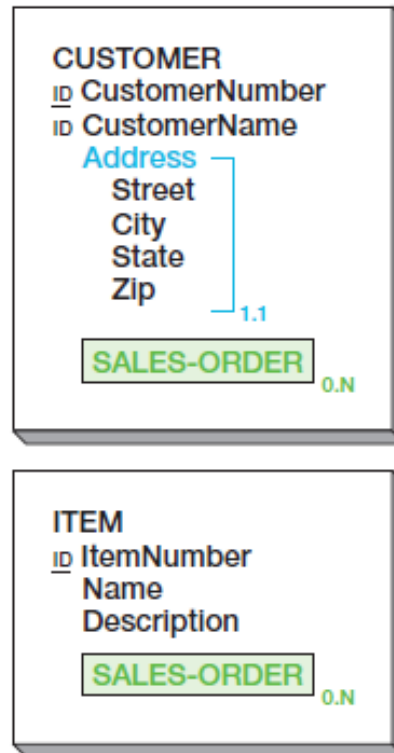
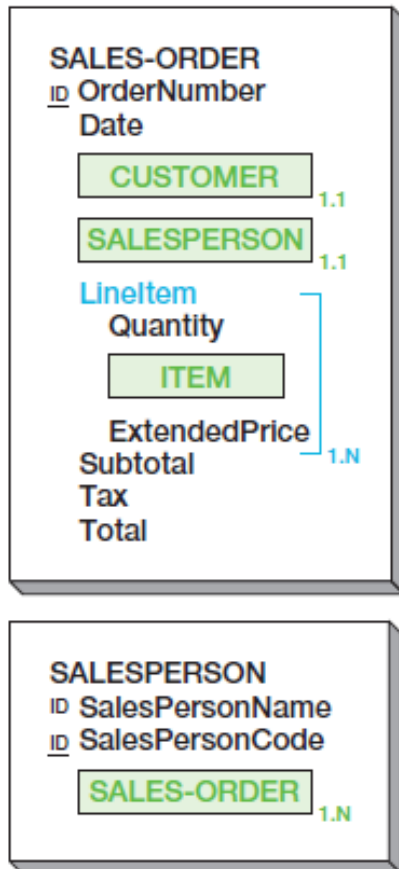


Типы объектов: прототип / экземпляр

- объект-прототип (*archetype object*) является некоторым абстрактным объектом, обладающим набором версий;
- объект-версия (*version object*) описывает различные реализации объекта-прототипа.



Semantic objects vs E-R



Вопросы к экзамену

- Процесс разработки баз данных и уровни моделирования информационных систем. Моделирование данных. Модель семантических объектов и ее основные элементы (объекты, атрибуты).
- Модель семантических объектов и ее основные элементы (объекты, атрибуты). Кардинальные числа атрибутов. Идентификаторы.
- Типы семантических объектов. Простой объект. Составной объект. Составной объект с независимыми и вложенными группами.
- Типы семантических объектов. Сложный объект. Гибридный объект. Ассоциативный объект.
- Типы семантических объектов. Схема «родитель-подтип». Взаимоисключающие и вложенные подтипы. Схема «прототип-экземпляр».

Лабораторная работа №2.

Моделирование данных с использованием модели семантических объектов

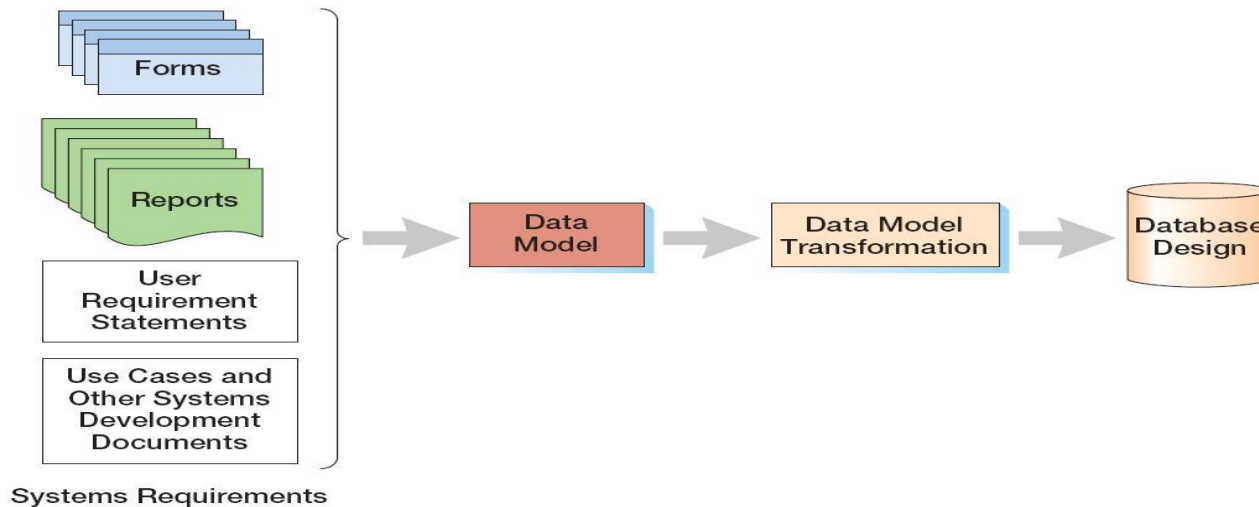
1. Создать модель семантических объектов для предметной области, выбранной в лабораторной работе №1.
2. Обосновать выбор кардинальных чисел атрибутов и типов объектов.

Базы данных

Тема 3

Реляционная модель и нормализация

Три уровня моделей информационных систем



- внешние модели (*external schema*): представление пользователей о системе (user view);
- концептуальная модель (*conceptual schema*): абстрактное представление системы (данных);
 - модель «сущность-связь» (*entity-relationship model*);
 - модель семантических объектов (*semantic object model*);
- внутренние модели (*internal schema*).

Реляционная модель

- реляционная модель (*relational database model*)
 - введена в 1970г. Э.Ф.Коддом (*E.F.Codd, «A Relational Model of Data for Large Shared Databanks», Communications of the ACM, 06/1970, p. 377-387*);
 - основана на применении концепции реляционной алгебры (*relational algebra*) к проблеме хранения больших объемов данных;
 - определяет способ хранения данных в виде таблиц со строками и столбцами, при этом минимизируется дублирование и исключаются определенные типы ошибок обработки;
 - дает стандартный способ структурирования и обработки данных;
 - фактически является промышленным стандартом хранения и обработки информации в базах данных.

Понятие нормализации

- нормализация (*normalization*) – последовательность преобразований, обеспечивающих выполнение определенного набора требований;

EmpNum	EmpName	DeptNum	DeptName
100	Jones	10	Accounting
150	Lau	20	Marketing
200	McCauley	10	Accounting
300	Griffin	10	Accounting

(a) One-Table Design

DeptNum	DeptName
10	Accounting
20	Marketing

OR?

EmpNum	EmpName	DeptNum
100	Jones	10
150	Lau	20
200	McCauley	10
300	Griffin	10

(b) Two-Table Design



How the customer explained it



How the Project Leader understood it



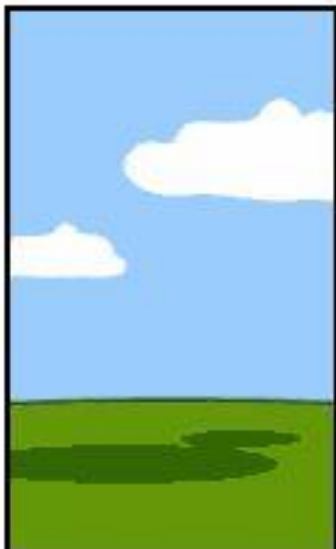
How the Analyst designed it



How the Programmer wrote it



How the Business Consultant described it



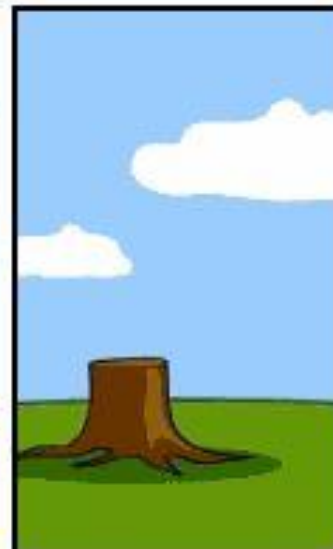
How the project was documented



What operations installed



How the customer was billed



How it was supported



What the customer really needed

Отношение

- отношение (*relation*) – двумерная таблица, обладающая следующими свойствами:
 - каждая строка содержит данные, описывающие некоторую сущность или ее часть;
 - каждый столбец представляет один из атрибутов сущности;
 - каждая ячейка содержит одиночное значение;
 - все значения в одном столбце имеют один и тот же тип;
 - каждый столбец имеет уникальное имя;
 - не важен порядок строк;
 - не важен порядок столбцов;
 - не может существовать двух идентичных строк.

Отношение: пример

EmployeeNumber	FirstName	LastName	Department	Email	Phone
100	Jerry	Johnson	Accounting	JJ@somewhere.com	834-1101
200	Mary	Abernathy	Finance	MA@somewhere.com	834-2101
300	Liz	Smathers	Finance	LS@somewhere.com	834-2102
400	Tom	Caruthers	Accounting	TC@somewhere.com	834-1102
500	Tom	Jackson	Production	TJ@somewhere.com	834-4101
600	Eleanore	Caldera	Legal	EC@somewhere.com	834-3101
700	Richard	Bandalone	Legal	RB@somewhere.com	834-3102

Таблица, не являющаяся отношением: пример (1)

EmployeeNumber	FirstName	LastName	Department	Email	Phone
100	Jerry	Johnson	Accounting	JJ@somewhere.com	834-1101
200	Mary	Abernathy	Finance	MA@somewhere.com	834-2101
300	Liz	Smathers	Finance	LS@somewhere.com	834-2102
400	Tom	Caruthers	Accounting	TC@somewhere.com	834-1102, 834-1191, 834-1192
500	Tom	Jackson	Production	TJ@somewhere.com	834-4101
600	Eleanore	Caldera	Legal	EC@somewhere.com	834-3101
700	Richard	Bandalone	Legal	RB@somewhere.com	834-3102, 834-3191

Таблица, не являющаяся отношением: пример (2)

EmployeeNumber	FirstName	LastName	Department	Email	Phone
100	Jerry	Johnson	Accounting	JJ@somewhere.com	834-1101
200	Mary	Abernathy	Finance	MA@somewhere.com	834-2101
300	Liz	Smathers	Finance	LS@somewhere.com	834-2102
400	Tom	Caruthers	Accounting	TC@somewhere.com	834-1102
				Fax:	834-9911
				Home:	723-8795
500	Tom	Jackson	Production	TJ@somewhere.com	834-4101
600	Eleanore	Caldera	Legal	EC@somewhere.com	834-3101
				Fax:	834-9912
				Home:	723-7654
700	Richard	Bandalone	Legal	RB@somewhere.com	834-3102

Альтернативная терминология

отношение (<i>relation</i>)	атрибут (<i>attribute</i>)	кортеж (<i>tuple</i>)
таблица (<i>table</i>)	столбец (<i>column</i>)	строка (<i>row</i>)
файл (<i>file</i>)	поле (<i>field</i>)	запись (<i>record</i>)

Ключи

- ключ (*key*) - один или несколько атрибутов, идентифицирующих строку отношения;
- уникальность ключа:
 - уникальный ключ (*unique key*) – ключ, каждое из значений которого может присутствовать в отношении не более одного раза;
 - неуникальный ключ (*non-unique key*) – ключ, каждое из значений которого может присутствовать в отношении более одного раза;
- составной (композиционный) ключ (*composite key*) – ключ, состоящий из более чем одного атрибута.

Первичный ключ

- первичный ключ (*primary key*) – один из уникальных ключей отношения, выбранный в качестве основного для идентификации записей отношения;
 - идеальным является короткий, неизменяемый ключ числового типа;
- ключ-кандидат (*candidate/alternate key*) – любой из уникальных ключей отношения, не являющийся первичным;
- обозначение отношения:
ИМЯ_ОТНОШЕНИЯ(ПЕРВИЧНЫЙ КЛЮЧ,

список неключевых атрибутов)

Суррогатный ключ

- суррогатный ключ (*surrogate key*) – искусственно созданный уникальный ключ, используемый в качестве первичного ключа отношения;
 - как правило, создаются средствами СУБД;
 - значения суррогатных ключей бессмысленны для пользователей, соответственно, они не должны отображаться в формах и отчетах;

RENTAL_PROPERTY (Street, City,
State/Province, Zip/PostalCode, Country, Rental_Rate)

RENTAL_PROPERTY (PropertyID, Street, City,
State/Province, Zip/PostalCode, Country, Rental_Rate)

Выбор ключа

- уникальность определяется из требований, и не может быть определена на основании фактических значений

ACTIVITY (SID, Activity, Fee)

Key: SID

Sample Data

SID	Activity	Fee
100	Skiing	200
150	Swimming	50
175	Squash	50
200	Swimming	50

SID	Activity	Fee
100	Skiing	200
100	Golf	65
150	Swimming	50
175	Squash	50
175	Swimming	50
200	Swimming	50
200	Golf	65

Внешний ключ

- внешний ключ (*foreign key*) – первичный ключ одного отношения, помещенный в качестве атрибута в другое отношение для формирования связи между этими отношениями;
- очевидно, что внешний ключ, как и первичный, может быть составным;
- внешние ключи также позволяют формировать ограничения ссылочной целостности (*referential integrity constraints*), т.е. условие того, что внешний ключ может принимать только существующие значения соответствующего первичного ключа.

DEPARTMENT (DepartmentName, BudgetCode, ManagerName)

EMPLOYEE (EmployeeNumber, EmployeeName, DepartmentName)

Функциональная зависимость

- функциональная зависимость (*functional dependency*) имеет место, когда один из атрибутов (или несколько атрибутов) определяет значения другого атрибута (нескольких атрибутов):

$$A \rightarrow B$$

$$A \rightarrow (B, C) \Rightarrow A \rightarrow B, A \rightarrow C$$

$$(A, B) \rightarrow C \not\Rightarrow A \rightarrow C, B \rightarrow C$$

- левая часть функциональной зависимости называется детерминантом (*determinant*);

Функциональная зависимость: пример

- определение функциональных зависимостей (факта уникальности детерминанта) также производится на основе требований, а не фактических значений атрибутов

Object Color	Weight	Shape
Red	5	Ball
Blue	3	Cube
Yellow	7	Cube

ObjectColor → Weight

ObjectColor → Shape

ObjectColor → (Weight, Shape)

Функциональная зависимость: пример (2)

	OrderNumber	SKU	Quantity	Price	ExtendedPrice
1	1000	201000	1	300.00	300.00
2	1000	202000	1	130.00	130.00
3	2000	101100	4	50.00	200.00
4	2000	101200	2	50.00	100.00
5	3000	100200	1	300.00	300.00
6	3000	101100	2	50.00	100.00
7	3000	101200	1	50.00	50.00

(OrderNumber, SKU) → (Quantity, Price, ExtendedPrice)

(Quantity, Price) → (ExtendedPrice)

Аномалии модификации

- аномалии модификации (*modification anomalies*) – нежелательные побочные эффекты, возникающие при модификации записей отношения:
 - аномалия удаления (*deletion anomaly*): удаление единственной строки отношения приводит к удалению информации о двух сущностях;
 - аномалия вставки (*insertion anomaly*): невозможность вставки информации об одной сущности до тех пор, пока не будет внесен некоторый факт о другой сущности;
 - аномалия обновления (*update anomaly*): некорректные результаты операции обновления.

Аномалии модификации: пример

- результаты некорректной модификации отношения EQUIPMENT_REPAIR (аномалия обновления)

	ItemNumber	Type	AcquisitionCost	RepairNumber	RepairDate	RepairCost
1	100	Drill Press	3500.00	2000	2009-05-05	375.00
2	200	Lathe	4750.00	2100	2009-05-07	255.00
3	100	Drill Press	3500.00	2200	2009-06-19	178.00
4	300	Mill	27300.00	2300	2009-06-19	1875.00
5	100	Drill Press	3500.00	2400	2009-07-05	0.00
6	100	Drill Press	3500.00	2500	2009-08-17	275.00

	ItemNumber	Type	AcquisitionCost	RepairNumber	RepairDate	RepairCost
1	100	Drill Press	3500.00	2000	2009-05-05	375.00
2	200	Lathe	4750.00	2100	2009-05-07	255.00
3	100	Drill Press	3500.00	2200	2009-06-19	178.00
4	300	Mill	27300.00	2300	2009-06-19	1875.00
5	100	Drill Press	3500.00	2400	2009-07-05	0.00
6	100	Drill Press	5500.00	2500	2009-08-17	275.00

Нормализация

- нормализация (*normalization*) – процесс классификации и преобразования отношений с целью исключения аномалий модификации

Источник аномалии	Нормальные формы	Принципы проектирования
Функциональные зависимости	1НФ, 2НФ, 3НФ, БКНФ	БКНФ: каждый детерминант должен являться ключом-кандидатом
Многозначные зависимости	4НФ	4НФ: каждая многозначная зависимость должна формировать отдельное отношение
Ограничения	5НФ, ДКНФ	ДКНФ: каждое ограничение должно являться следствием ключа-кандидата или домена

Иерархия нормальных форм

- 1NF – first NF
- 2NF – second NF
- 3NF – third NF
- EKNF – Elementary Key NF
- **BCNF** – Boyce / Codd NF
- 4NF – fourth NF
- **ETNF** – Essential Tuple NF
- RFNF (KCNF) – Redundancy Free (Key Complete) NF
- SKNF – SuperKey NF
- **5NF** – fifth NF
- **6NF** – sixth NF

Первая нормальная форма (1НФ)

- любая таблица, удовлетворяющая определению отношения, находится в первой нормальной форме (*first normal form, 1NF*).

ACTIVITY (SID, Activity, Fee)

Key: SID

Sample Data

SID	Activity	Fee
100	Skiing	200
150	Swimming	50
175	Squash	50
200	Swimming	50

Первая нормальная форма (пример)

<u>bug_id</u>	<u>tag1</u>	<u>tag2</u>	<u>tag3</u>
1234	crash		
3456	printing	crash	
5678	report	crash	data

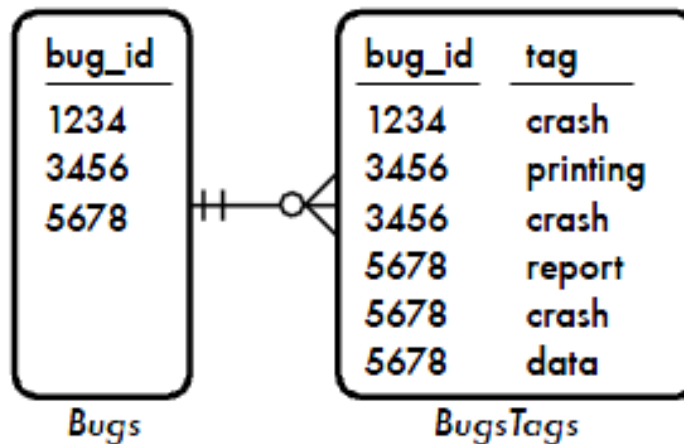
*Multicolumn
Attributes*

<u>bug_id</u>	<u>tags</u>
1234	crash
3456	printing, crash
5678	report, crash, data

Jaywalking

BugsTags

*First
Normal
Form*



Вторая нормальная форма (2НФ)

- отношение находится во второй нормальной форме (*second normal form, 2NF*), если каждый из неключевых атрибутов зависит от всего первичного ключа;
- необходимо:
 - выделить атрибуты функциональной зависимости, нарушающей требования 2НФ, в отдельное отношение (с детерминантом качестве первичного ключа);
 - оставить детерминант в качестве внешнего ключа в исходном отношении.

SID	Activity	Fee
100	Skiing	200
100	Golf	65
150	Swimming	50
175	Squash	50
175	Swimming	50
200	Swimming	50
200	Golf	65

STU-ACT (SID, Activity)
Key: SID

SID	Activity
100	Skiing
150	Swimming
175	Squash
200	Swimming

ACT-COST (Activity, Fee)
Key: Activity

Activity	Fee
Skiing	200
Swimming	50
Squash	50

Вторая нормальная форма (пример)

PK(bug_id, tag)

(bug_id,tag)→
tagger

tag → coiner

<u>bug_id</u>	<u>tag</u>	tagger	coiner
1234	crash	Larry	Shemp
3456	printing	Larry	Shemp
3456	crash	Moe	Shemp
5678	report	Moe	Shemp
5678	crash	Larry	Shemp
5678	data	Moe	Shemp

Redundancy

<u>bug_id</u>	<u>tag</u>	tagger	coiner
1234	crash	Larry	Shemp
3456	printing	Larry	Shemp
3456	crash	Moe	Shemp
5678	report	Moe	Shemp
5678	crash	Larry	Curly
5678	data	Moe	Shemp

Anomaly

BugsTags

*Second
Normal
Form*

<u>bug_id</u>	<u>tag</u>	tagger
1234	crash	Larry
3456	printing	Larry
3456	crash	Moe
5678	report	Moe
5678	crash	Larry
5678	data	Moe

BugsTags

<u>tag</u>	<u>coiner</u>
crash	Shemp
printing	Shemp
report	Shemp
data	Shemp

Tags

Третья нормальная форма (3НФ)

- отношение находится в третьей нормальной форме (*third normal form, 3NF*), если оно находится во второй нормальной форме и не имеет транзитивных зависимостей (*transitive dependency*);
- необходимо: выделить функциональную зависимость в отдельное отношение (аналогично 2НФ)

HOUSING (SID, Building, Fee)

Key: SID

Functional

dependencies: Building $\rightarrow$ Fee

SID $\rightarrow$ Building $\rightarrow$ Fee

SID	Building	Fee
100	Randolph	1200
150	Ingersoll	1100
200	Randolph	1200
250	Pitkin	1100
300	Randolph	1200

(a)

STU-HOUSING (SID, Building)

Key: SID

SID	Building
100	Randolph
150	Ingersoll
200	Randolph
250	Pitkin
300	Randolph

BLDG-FEE (Building, Fee)

Key: Building

Building	Fee
Randolph	1200
Ingersoll	1100
Pitkin	1100

(b)

Третья нормальная форма (пример)

<u>bug_id</u>	<u>assigned_to</u>	<u>assigned_email</u>
1234	Larry	larry@example.com
3456	Moe	moe@example.com
5678	Moe	moe@example.com

Redundancy

<u>bug_id</u>	<u>assigned_to</u>	<u>assigned_email</u>
1234	Larry	larry@example.com
3456	Moe	moe@example.com
5678	Moe	curly@example.com

Anomaly

Bugs

*Third
Normal
Form*

<u>bug_id</u>	<u>assigned_to</u>
1234	Larry
3456	Moe
5678	Moe

Bugs

<u>account_id</u>	<u>email</u>
Larry	larry@example.com
Moe	moe@example.com

Accounts

Нормальная форма Бойса-Кодда (БКНФ)

- отношение находится в нормальной форме Бойса-Кодда (*Boyce-Codd normal form, BC/NF*), если каждый детерминант является ключом-кандидатом.

ADVISER (SID, Major, Fname)

Key (primary): (SID, Major)

Key (candidate): (SID, Fname)

Functional dependencies: Fname → Major

SID	Major	Fname
100	Math	Cauchy
150	Psychology	Jung
200	Math	Riemann
250	Math	Cauchy
300	Psychology	Perls
300	Math	Riemann

(a)

STU-ADV (SID, Fname)
Key: SID, Fname

SID	Fname
100	Cauchy
150	Jung
200	Riemann
250	Cauchy
300	Perls
300	Riemann

(b)

ADV-SUBJ (Fname, Subject)
Key: Fname

Fname	Subject
Cauchy	Math
Jung	Psychology
Riemann	Math
Perls	Psychology

"I swear to construct my tables so that all non-key columns are dependent on the key, the whole key and nothing but the key, so help me Codd."

Нормальная форма Бойса-Кодда (пример)

PK(bug_id, tag)

AK(bug_id, tag_type)

(tag) → tag_type

Аномалии: вставка,
изменение,
удаление

bug_id	tag	tag_type
1234	crash	impact
3456	printing	subsystem
3456	crash	impact
5678	report	subsystem
5678	crash	impact
5678	data	fix

*Multiple
Candidate
Keys*

bug_id	tag	tag_type
1234	crash	impact
3456	printing	subsystem
3456	crash	impact
5678	report	subsystem
5678	crash	subsystem
5678	data	fix

Anomaly

BugsTags

*Boyce-Codd
Normal
Form*

bug_id	tag
1234	crash
3456	printing
3456	crash
5678	report
5678	crash
5678	data

BugsTags

tag	tag_type
crash	impact
printing	subsystem
report	subsystem
data	fix

Tags

БКНФ: процедура непосредственного приведения

1. выделить все функциональные зависимости;
2. определить все ключи-кандидаты;
3. если существует функциональная зависимость, детерминант которой не является ключом-кандидатом:
 - необходимо выделить атрибуты, участвующие в данной функциональной зависимости в новое отношение;
 - в качестве первичного ключа нового отношения будет выступать детерминант функциональной зависимости;
 - детерминант функциональной зависимости также должен остаться в исходном отношении в качестве внешнего ключа (с заданием ограничения ссылочной целостности);
4. шаги могут повторяться многократно до тех пор, пока для каждого из отношений не выполняются требования БКНФ.

БКНФ: пример – исходное отношение

	ItemNumber	Type	AcquisitionCost	RepairNumber	RepairDate	RepairCost
1	100	Drill Press	3500.00	2000	2009-05-05 ...	375.00
2	200	Lathe	4750.00	2100	2009-05-07 ...	255.00
3	100	Drill Press	3500.00	2200	2009-06-19 ...	178.00
4	300	Mill	27300.00	2300	2009-06-19 ...	1875.00
5	100	Drill Press	3500.00	2400	2009-07-05 ...	0.00
6	100	Drill Press	3500.00	2500	2009-08-17 ...	275.00

EQUIPMENT_REPAIR (ItemNumber, Type, AcquisitionCost,
 RepairNumber, RepairDate, RepairAmount)

ItemNumber → (Type, AcquisitionCost)

RepairNumber → (ItemNumber, Type, AcquisitionCost,
 RepairDate, RepairCost)

БКНФ: пример – отношения в БКНФ

ITEM

	ItemNumber	Type	AcquisitionCost
1	100	Drill Press	3500.00
2	200	Lathe	4750.00
3	300	Mill	27300.00

REPAIR

	RepairNumber	ItemNumber	RepairDate	RepairCost
1	2000	100	2009-05-05	375.00
2	2100	200	2009-05-07	255.00
3	2200	100	2009-06-19	178.00
4	2300	300	2009-06-19	1875.00
5	2400	100	2009-07-05	0.00
6	2500	100	2009-07-05	275.00

ITEM (ItemNumber, Type, AcquisitionCost)

REPAIR (ItemNumber, RepairNumber, RepairDate, RepairAmount)

Where REPAIR.ItemNumber must exist in
ITEM.ItemNumber

Многозначные зависимости

- многозначная зависимость (*multivalued dependency*) имеет место, когда детерминанту соответствует некоторый набор значений;
- общая схема:

$R(A,B,C)$

$A \twoheadrightarrow B$

$A \twoheadrightarrow C$

В и С независимы

- возникает аномалия модификации — необходимо внести все комбинации для многозначно-зависимых атрибутов;
- очевидно, что детерминант многозначной зависимости никогда не может быть первичным ключом.

PARTKIT_PART

	PartKit	Part
1	Bike Repair	Screwdriver
2	Bike Repair	Tube Fix
3	Bike Repair	Wrench
4	First Aid	Aspirin
5	First Aid	Band-aids
6	First Aid	Elastic Band
7	First Aid	Ibuprofen
8	Vice	Extension Screw
9	Vice	Handle
10	Vice	Vice Jaw

Многозначные зависимости: пример

```
-----
| EMPLOYEE | SKILL | LANGUAGE |
=====
```

EMPLOYEE	SKILL	LANGUAGE
Smith	cook	
Smith	type	
Smith		French
Smith		German
Smith		Greek

EMPLOYEE	SKILL	LANGUAGE
Smith	cook	French
Smith	cook	German
Smith	cook	Greek
Smith	type	French
Smith	type	German
Smith	type	Greek

EMPLOYEE	SKILL	LANGUAGE
Smith	cook	French
Smith	type	German
Smith	type	Greek

EMPLOYEE	SKILL	LANGUAGE
Smith	cook	French
Smith	type	German
Smith		Greek

EMPLOYEE	SKILL	LANGUAGE
Smith	cook	French
Smith	type	
Smith		German
Smith	type	Greek

```
-----
| EMPLOYEE | SKILL |
=====
| EMPLOYEE | LANGUAGE |
=====
```


Четвертая нормальная форма (4НФ)

- отношение находится в четвертой нормальной форме (*fourth normal form, 4NF*), если оно находится в нормальной форме Бойса-Кодда и не имеет нетривиальных многозначных зависимостей

STU-MAJOR (SID, Major)
Key: (SID, Major)

SID	Major
100	Music
100	Accounting
150	Math

STU-ACT (SID, Activity)
Key: (SID, Activity)

SID	Activity
100	Skiing
100	Swimming
100	Tennis
150	Jogging

Четвертая нормальная форма (4НФ)

- отношение находится в четвертой нормальной форме (*fourth normal form, 4NF*), если оно находится в нормальной форме Бойса-Кодда и не имеет нетривиальных многозначных зависимостей (все нетривиальные многозначные зависимости фактически являются функциональными зависимостями от его ключей).
- или... : отношение находится в четвёртой нормальной форме тогда и только тогда, когда в случае существования таких подмножеств A и B атрибутов этого отношения, для которых выполняется нетривиальная многозначная зависимость $A \twoheadrightarrow B$, все атрибуты переменной отношения R также функционально зависят от A .

STU-MAJOR (SID, Major)
Key: (SID, Major)

SID	Major
100	Music
100	Accounting
150	Math

STU-ACT (SID, Activity)
Key: (SID, Activity)

SID	Activity
100	Skiing
100	Swimming
100	Tennis
150	Jogging

Четвертая нормальная форма (пример)

– проблема возникает, если:

- существуют атрибуты, не участвующие в многозначной зависимости;
- существуют несколько *независимых* многозначных зависимостей.

<u>bug_id</u>	<u>reported_by</u>	<u>assigned_to</u>	<u>verified_by</u>
1234	Zeppo	NULL	NULL
3456	Chico	Groucho	Harpo
3456	Chico	Spalding	Harpo
5678	Chico	Groucho	NULL
5678	Zeppo	Groucho	NULL
5678	Gummo	Groucho	NULL

*Redundancy,
NULLs,
No Primary Key*

BugsAccounts

*Fourth
Normal
Form*

<u>bug_id</u>	<u>reported_by</u>
1234	Zeppo
3456	Chico
5678	Chico
5678	Zeppo
5678	Gummo

BugsReported

<u>bug_id</u>	<u>assigned_to</u>
3456	Groucho
3456	Spalding
5678	Groucho

BugsAssigned

<u>bug_id</u>	<u>verified_by</u>
3456	Harpo

BugsVerified

Пятая нормальная форма

- Fifth Normal Form (5NF) / Project-Join Normal Form (PJ/NF);
- рассматривает случаи, когда информация может быть восстановлена из отдельных составляющих, обладающих меньшей избыточностью;
- в случае отсутствия дополнительных семантических правил четвертая и пятая нормальные формы эквивалентны;
- при условии отсутствия составного первичного ключа и выполнении условий БКНФ, отношение автоматически находится в 5НФ.

Пятая нормальная форма: пример

AGENT	COMPANY	PRODUCT
Smith	Ford	car
Smith	Ford	truck
Smith	GM	car
Smith	GM	truck
Jones	Ford	car
Jones	Ford	truck
Brown	Ford	car
Brown	GM	car
Brown	Totota	car
Brown	Totota	bus

AGENT	COMPANY
Smith	Ford
Smith	GM
Jones	Ford
Brown	Ford
Brown	GM
Brown	Toyota

COMPANY	PRODUCT
Ford	car
Ford	truck
GM	car
GM	truck
Toyota	car
Toyota	bus

AGENT	PRODUCT
Smith	car
Smith	truck
Jones	car
Jones	truck
Brown	car
Brown	bus

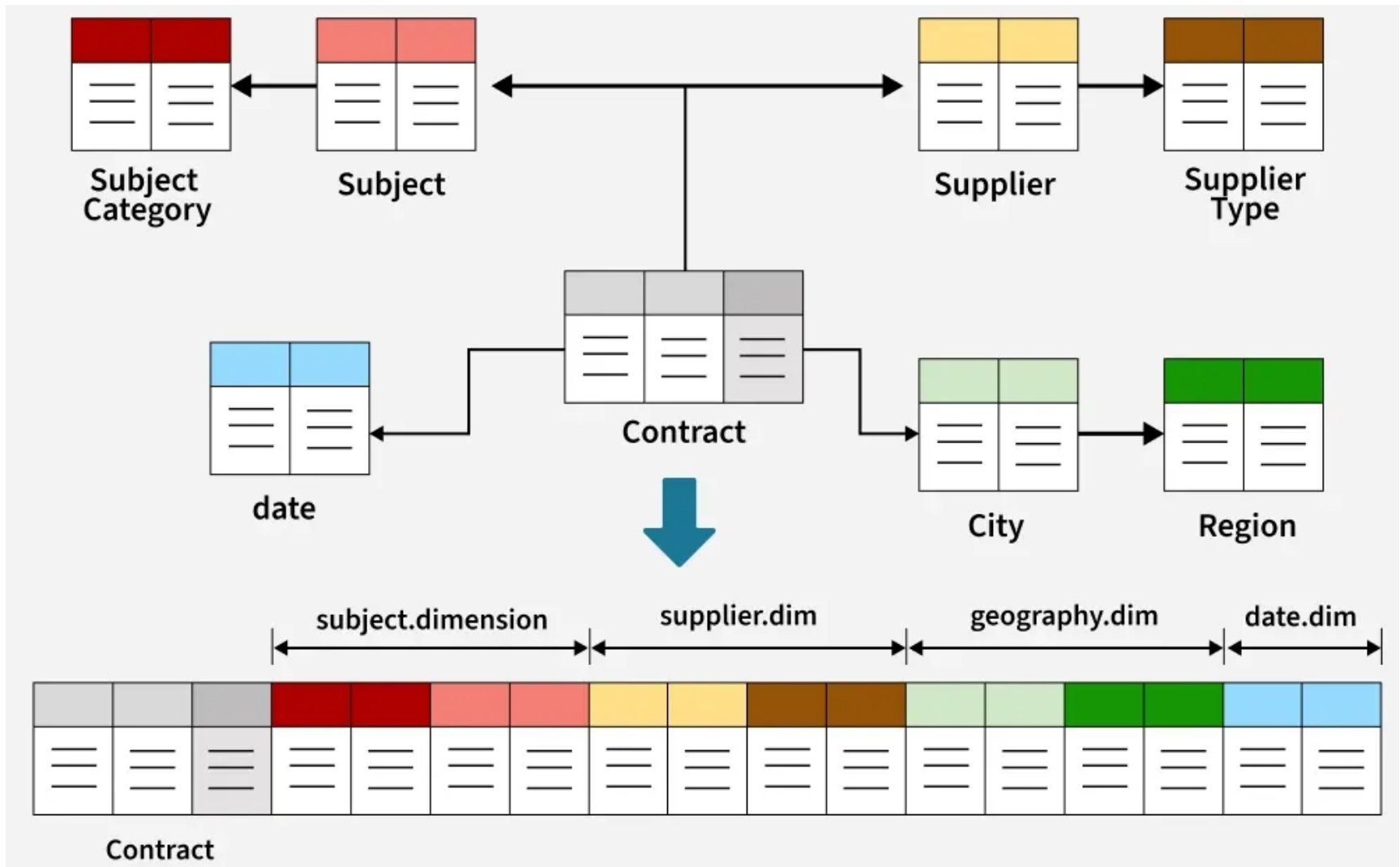
Fifth
Normal
Form

- правило: если агент представляет компанию и продает продукты определенного типа, то он продает продукты данного типа всех компаний, которые он представляет

Нормализация и денормализация

- преимущества нормализации:
 - отсутствие аномалий модификации;
 - минимизация избыточности данных:
 - более простое поддержание целостности данных;
 - меньшие накладные расходы на хранение;
- недостатки нормализации:
 - более сложные запросы для извлечения данных;
 - следовательно, большие накладные расходы на их выполнение, что может снижать производительность;
- тип базы данных:
 - обновляемая (транзакционная, «updateable», operational): OLTP (Online Transaction Processing);
 - преимущественно не обновляемая (аналитическая, «read mostly», non-operational): OLAP (Online Analytic Processing)

Нормализация и денормализация: пример



Дополнительные аспекты проектирования

- многозначные зависимости, выраженные в нескольких атрибутах (*multicolumn values*)
EMPLOYEE (EmpNumber, Name, Email, Auto1_LicenseNumber, Auto2_LicenseNumber, Auto3_LicenseNumber)
- противоречивость значений (*inconsistent values*) - наибольшая проблема для первичных и внешних ключей:
 - различное написание (ошибки ввода);
 - различный порядок ввода;
- отсутствующие значения (*missing values, null values*):
 - неприменимое значение (*inappropriate value*);
 - неизвестное значение (*unknown value*);
 - не введенное значение.

Вопросы к экзамену

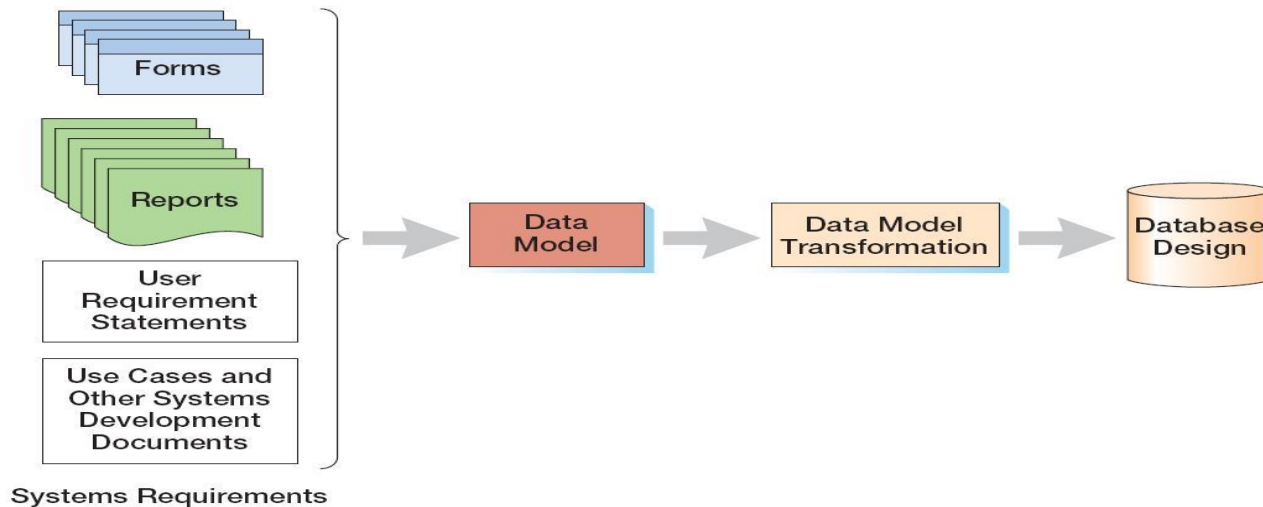
- Понятие реляционной модели. Отношения. Ключи, их свойства и типы. Функциональная зависимость. Многозначная зависимость.
- Реляционная модель: аномалии модификации. Функциональная зависимость. Нормализация. Первая, вторая, третья нормальные формы. Нормальная форма Бойса-Кодда.
- Реляционная модель: аномалии модификации. Многозначная зависимость. Нормализация. Четвертая нормальная форма. Пятая нормальная форма. Нормализация и денормализация.

Базы данных

Тема 4

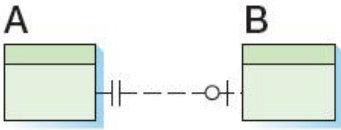
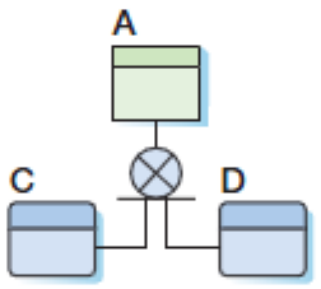
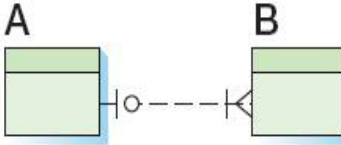
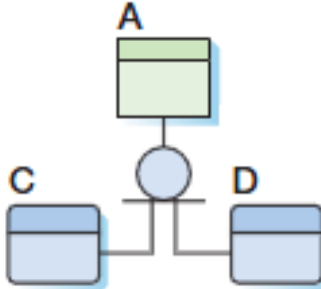
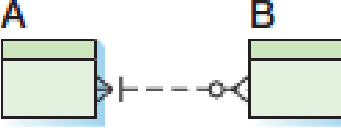
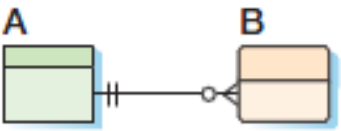
Преобразование модели «сущность-связь»
в реляционную модель

Три уровня моделей информационных систем



- внешние модели (*external schema*): представление пользователей о системе (user view);
- концептуальная модель (*conceptual schema*): абстрактное представление системы (данных);
 - модель «сущность-связь» (entity-relationship model);
 - модель семантических объектов (semantic object model);
- внутренние модели (*internal schema*).

Резюме по нотации

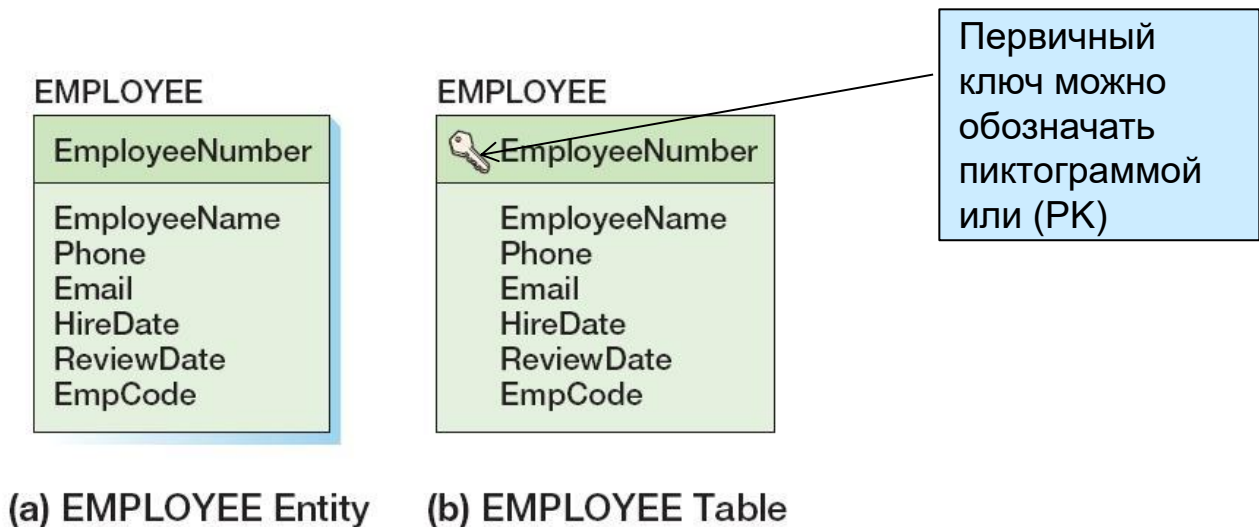
DEPARTMENT DepartmentName BudgetCode OfficeNumber		сущность DEPARTMENT; DepartmentName – идентификатор; BudgetCode, OfficeNumber – атрибуты; Идентификационно-зависимые сущности изображаются прямоугольниками со скругленными углами	
	неидентифицирующая связь 1:1 сущности A соответствует 0 или 1 сущности B, сущности B соответствует ровно одна сущность A		сущность A является супертипом, C и D являются взаимоисключающими подтипами, дискриминатор не указан
	неидентифицирующая связь 1:N сущности A соответствует 1..N сущностей B, сущности B соответствует 0 или одна сущность A		сущность A является супертипом, C и D являются невязаимоисключающими подтипами
	неидентифицирующая связь M:N сущности A соответствует 0..N сущностей B, сущности B соответствует 0..M сущностей A		
	идентифицирующая связь 1:N сущности A соответствует 0..N сущностей B, сущности B соответствует ровно одна сущность A		

Основные шаги

1. создание таблицы для каждой сущности:
 - определение первичного ключа (возможно, суррогатного);
 - определение ключей-кандидатов;
 - определение свойств каждого столбца: типа данных (data type), возможности неопределенного значения (null value), значения по умолчанию (default value), ограничений на значения (data constraints);
 - проверка нормализации (как правило, нормализация производится до БКНФ/4НФ);
2. создание связей с помощью внешних ключей:
 - между сильными сущностями (1:1, 1:N, N:M);
 - для идентификационно-зависимых сущностей;
 - для слабых сущностей;
 - для сущностей тип-подтип;
 - рекурсивных;
3. обеспечение условий минимальной кардинальности

Создание отношения для сущности и выбор первичного ключа

EMPLOYEE (EmployeeNumber, EmployeeName, Phone,
Email, HireDate, ReviewDate, EmpCode)



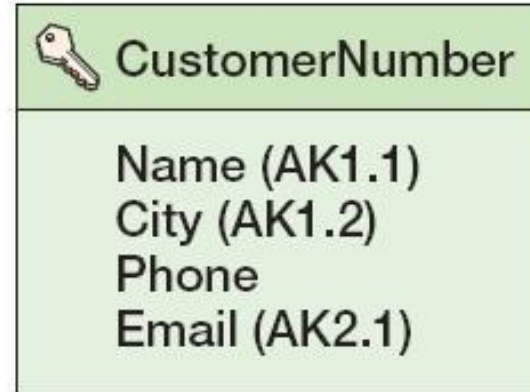
- идеальным является короткий, неизменяемый ключ числового типа;
- суррогатный ключ (*surrogate key*) – искусственно созданный уникальный ключ, используемый в качестве первичного ключа (*primary key*).

Определение ключей-кандидатов

EMPLOYEE



CUSTOMER



- ключ-кандидат (*candidate/alternate key*) – любой из уникальных ключей отношения, не являющийся первичным;
- для обозначения можно использовать обозначения ERwin:
AK n . m
 - n – номер ключа-кандидата;
 - m – номер атрибута в соответствующем ключе-кандидате.

Возможность неопределенных значений

EMPLOYEE

	EmployeeNumber: NOT NULL
	EmployeeName: NOT NULL
	Phone: NULL
	Email: NULL (AK1.1)
	HireDate: NOT NULL
	ReviewDate: NULL
	EmpCode: NULL

- отсутствующие значения (*missing values, null values*):
 - неприменимое значение (*inappropriate value*);
 - неизвестное значение (*unknown value*);
 - не введенное значение.

Определение типа данных

EMPLOYEE



EmployeeNumber: int

EmployeeName: char(50)

Phone: char(15)

Email: char(50) (AK1.1)

HireDate: datetime

ReviewDate: datetime

EmpCode: char(18)

Data Type	Description
Binary	Binary, length 0 to 8,000 bytes.
Char	Character, length 0 to 8,000 bytes.
Datetime	8-byte datetime. Range from January 1, 1753, through December 31, 9999, with an accuracy of three-hundredths of a second.
Image	Variable length binary data. Maximum length 2,147,483,647 bytes.
Integer	4-byte integer. Value range from -2,147,483,648 through 2,147,483,647.
Money	8-byte money. Range from -922,337,203,685,477.5808 through +922,337,203,685,477.5807, with accuracy to a ten-thousandth of a monetary unit.
Numeric	Decimal – can set precision and scale. Range $-10^{38} + 1$ through $10^{38} - 1$.
Smalldatetime	4-byte datetime. Range from January 1, 1900, through June 6, 2079, with an accuracy of one minute.
Smallint	2-byte integer. Range from -32,768 through 32,767.
Smallmoney	4-byte money. Range from 214,748.3648 through +214,748.3647, with accuracy to a ten-thousandth of a monetary unit.
Text	Variable length text, maximum length 2,147,483,647 characters.
Tinyint	1-byte integer. Range from 0 through 255.
Varchar	Variable-length character, length 0 to 8,000 bytes.

Типы данных (SQL Server 2025)

- ТОЧНЫЕ ЧИСЛОВЫЕ ТИПЫ:
 - tinyint, smallint, int, bigint;
 - bit;
 - decimal, numeric;
 - money, smallmoney;
- ВЕЩЕСТВЕННЫЕ ТИПЫ:
 - float, real;
- ДАТА И ВРЕМЯ:
 - date, time, datetime, datetime2, smalldatetime, datetimeoffset;
- СТРОКОВЫЕ ТИПЫ:
 - char(n), varchar(n), text;
 - Unicode: nchar(n), nvarchar(n), ntext;
- ДВОИЧНЫЕ СТРОКИ:
 - binary(n), varbinary(n), image;
- СПЕЦИАЛЬНЫЕ ТИПЫ:
 - geography, geometry - пространственные типы (spatial types);
 - hierarchyid;
 - json, xml;
 - vector;
 - rowversion;
 - sql_variant;
 - cursor;
 - table;
 - uniqueidentifier.

Типы данных (PostgreSQL 17)

- ТОЧНЫЕ ЧИСЛОВЫЕ ТИПЫ:
 - smallint, integer, bigint;
 - boolean;
 - decimal, numeric;
 - money;
- ВЕЩЕСТВЕННЫЕ ТИПЫ:
 - real, double precision;
- ДАТА И ВРЕМЯ:
 - timestamp(p), date, time(p), interval (p);
- СТРОКОВЫЕ ТИПЫ:
 - character varying(n), varchar(n);
 - character(n), char(n), bpchar(n);
 - bpchar, text;
- ДВОИЧНЫЕ СТРОКИ:
 - bytea;
- СПЕЦИАЛЬНЫЕ ТИПЫ:
 - smallserial, serial, bigserial;
 - перечисления (enum), массивы, составные типы (структуры)
 - int4range, int8range, numrange, tsrange, tstzrange, daterange диапазоны (range types);
 - point, line, lseg, box, path, polygon, circle - геометрические типы;
 - cidr, inet, macaddr, macaddr8 типы для представления сетевых адресов;
 - bit(n), bit varying(n) – битовые строки;
 - tsvector, tsquery;
 - json, jsonb, xml;
 - UUID.

Определение значений по умолчанию

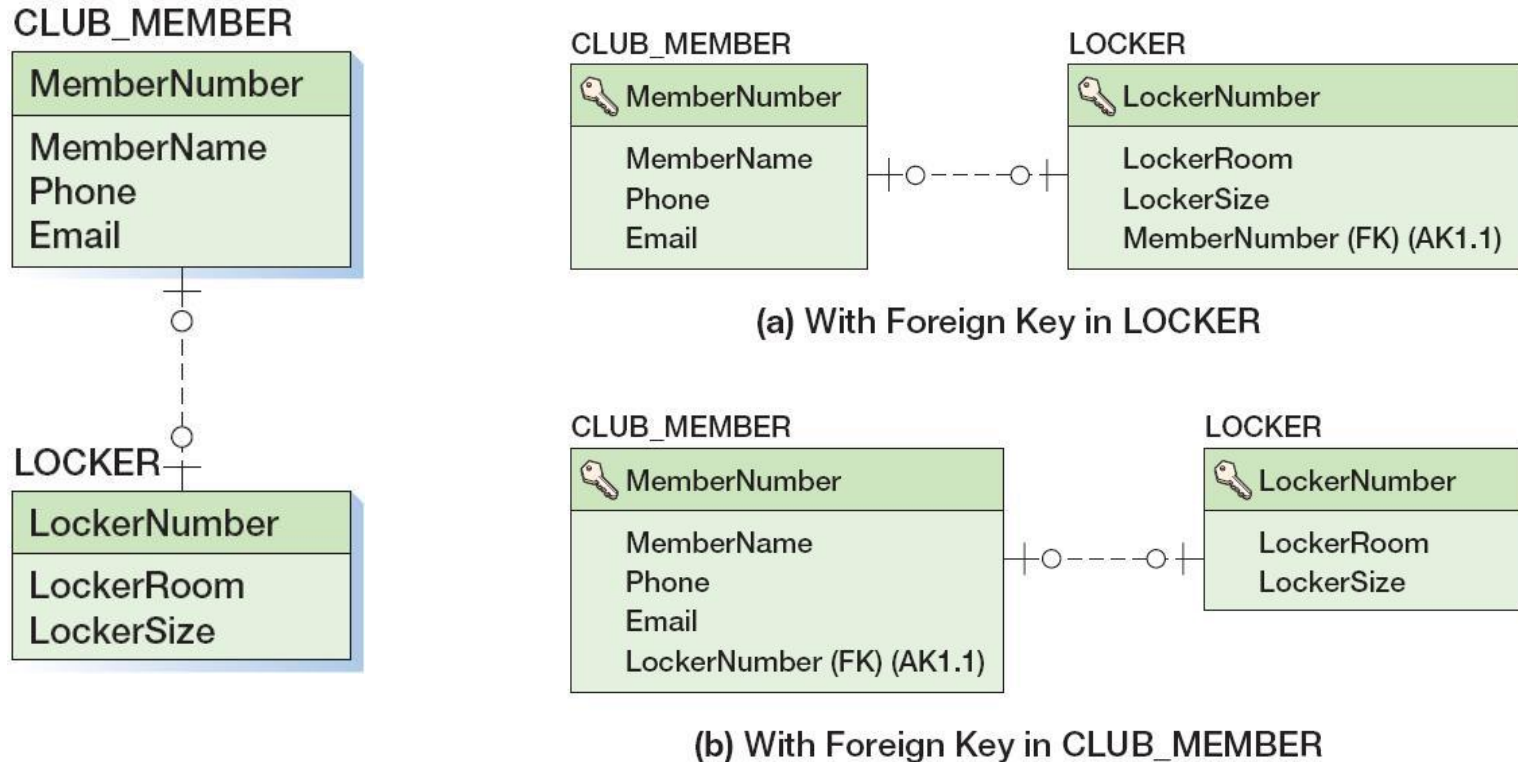
Table	Column	Default Value
ITEM	ItemNumber	Surrogate key
ITEM	Category	None
ITEM	ItemPrefix	If Category = 'Perishable' then 'P' If Category = 'Imported' then 'I' If Category = 'One-off' then 'O' Otherwise = 'N'
ITEM	ApprovingDept	If ItemPrefix = 'I' then 'SHIPPING/PURCHASING' Otherwise = 'PURCHASING'
ITEM	ShippingMethod	If ItemPrefix = 'P' then 'Next Day' Otherwise = 'Ground'

- значение по умолчанию (default value) – автоматически создаваемое СУБД значение при вставке новой записи, если оно не задано явно.

Определение ограничений

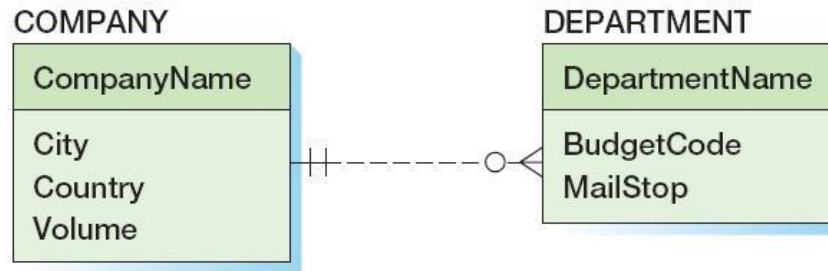
- категории целостности данных:
 - *сущностная*: определяет строку как уникальную сущность в конкретной таблице, поддерживается через введение ключа;
 - *доменная*: определяет достоверность записей в конкретном столбце;
 - *ссылочная*: сохраняет определенные связи между таблицами при вводе или удалении записей;
 - *пользовательская* целостность;
- ограничения:
 - *домена* (domain constraint): значения атрибута должны быть из определенного множества;
 - *интервальные* (range constraint): значения атрибута должны быть в определенном интервале;
 - *внутренние* (intrarelation constraint): значения атрибута должны удовлетворять определенным условиям сравнения с другими атрибутами того же отношения;
 - *внешние* (interrelation constraint): значения атрибута должны удовлетворять определенным условиям сравнения с атрибутами другого отношения – ограничения ссылочной целостности (referential integrity constraint) посредством внешних ключей.

Создание связей: сильные сущности 1:1

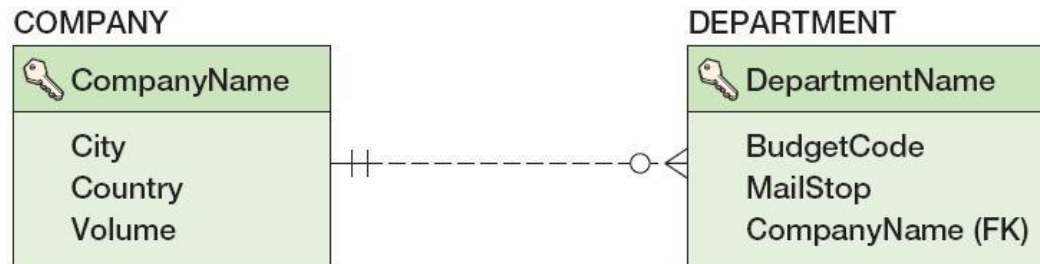


- первичный ключ отношения, соответствующего одной из сущностей, помещается в качестве внешнего ключа в отношение, представляющее вторую сущность.

Создание связей: сильные сущности 1:N



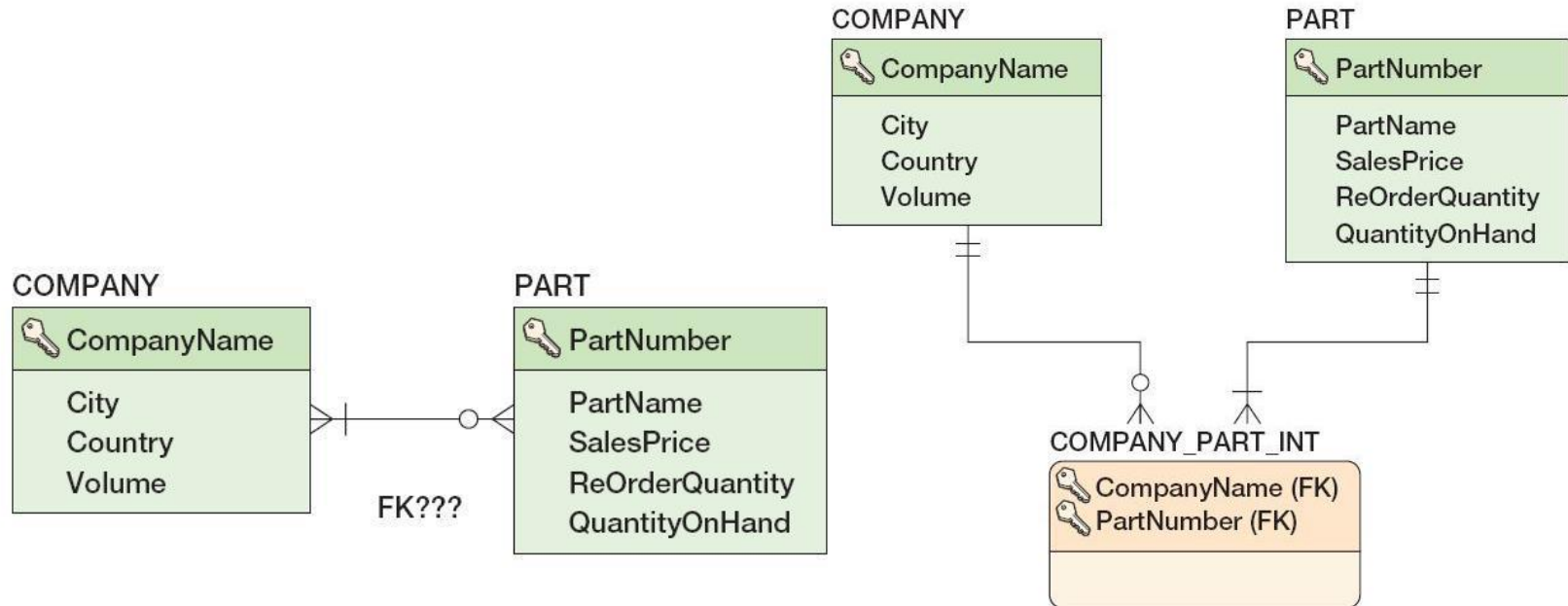
(a) 1:N Relationship Between Strong Entities



(b) Placing the Primary Key of the Parent in the Child as a Foreign Key

- первичный ключ отношения, соответствующего родительской сущности, помещается в качестве внешнего ключа в отношение, представляющее дочернюю сущность.

Создание связей: сильные сущности M:N

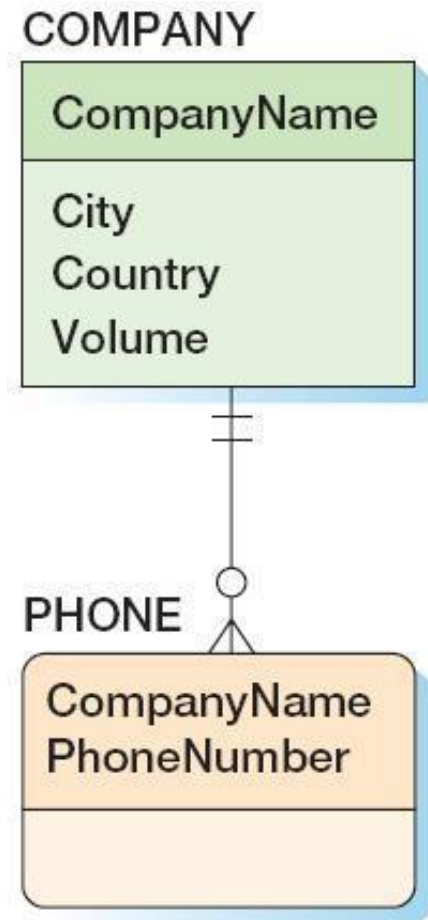


COMPANY_PART_INT (CompanyName, PartNumber)

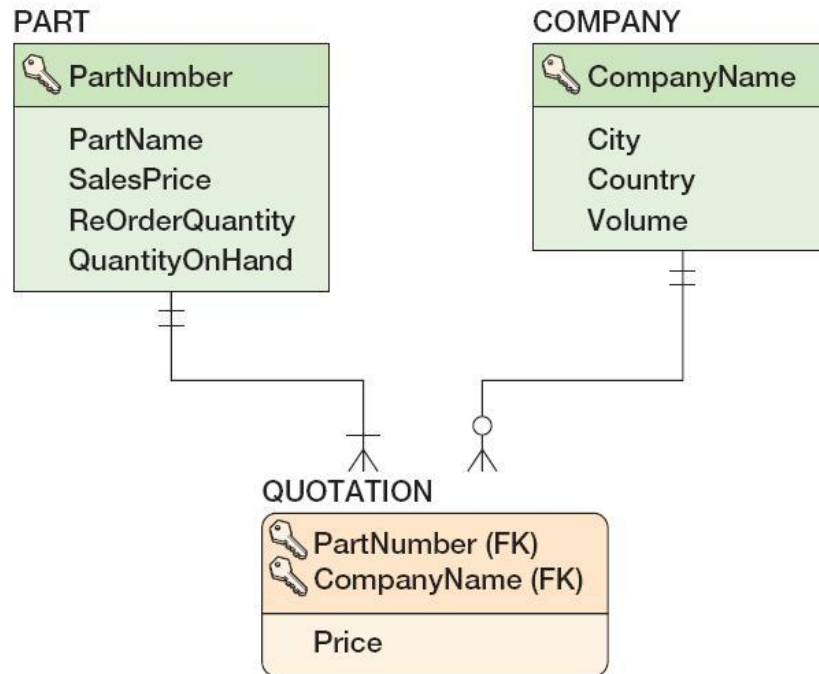
- создается промежуточное отношение (*intersection table*):
 - содержит в качестве внешних первичные ключи обоих отношений;
 - оба внешних ключа формируют составной первичный ключ нового отношения;
 - другие поля отсутствуют.

Создание связей: идентификационно-зависимые сущности

- *идентификационно-зависимой (ID-dependent)* называется такая дочерняя сущность, идентификатор которой содержит идентификатор родительской сущности (дочерняя сущность является логическим расширением или составной частью родительской);
- варианты использования идентификационно-зависимых сущностей:
 - шаблон «сопряжение» (association pattern);
 - многозначные атрибуты (multivalued attributes);
 - шаблон «прототип-экземпляр» (archetype/instance pattern);



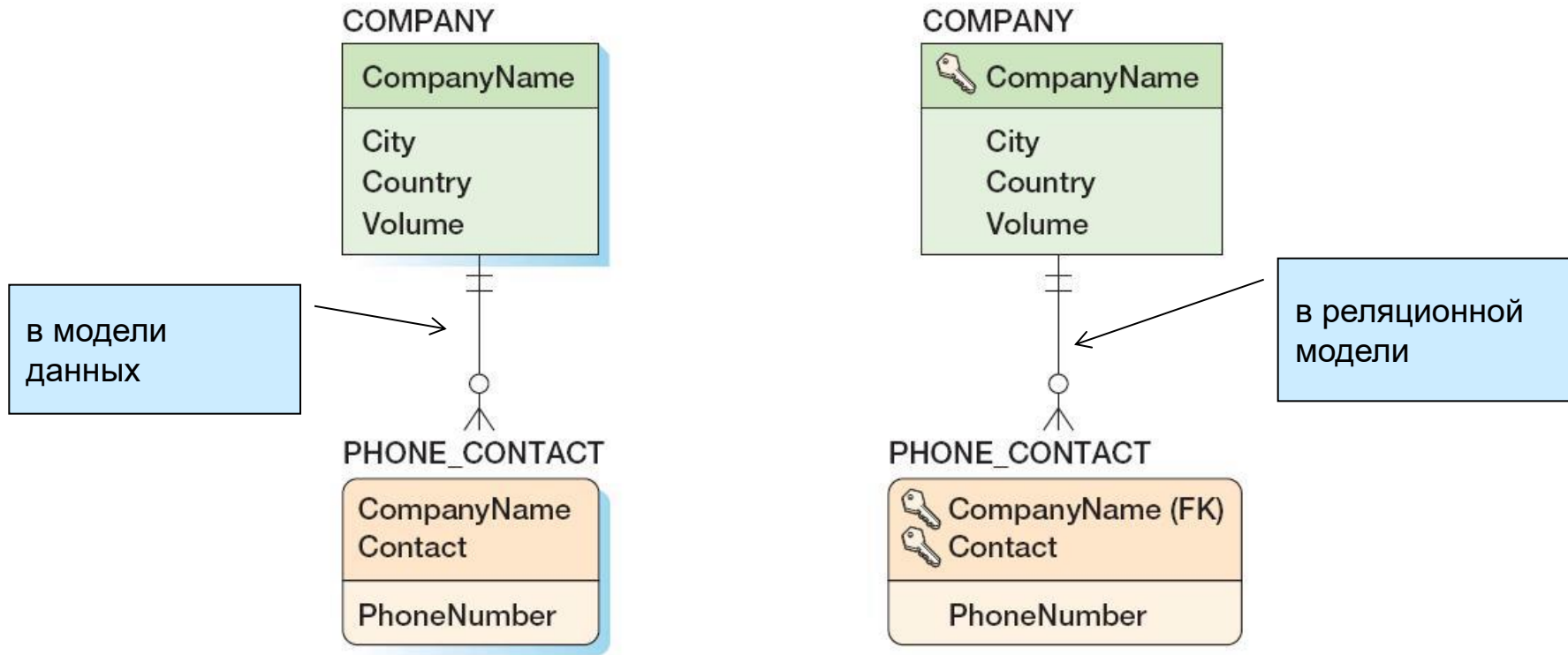
Создание связей: шаблон «сопряжение»



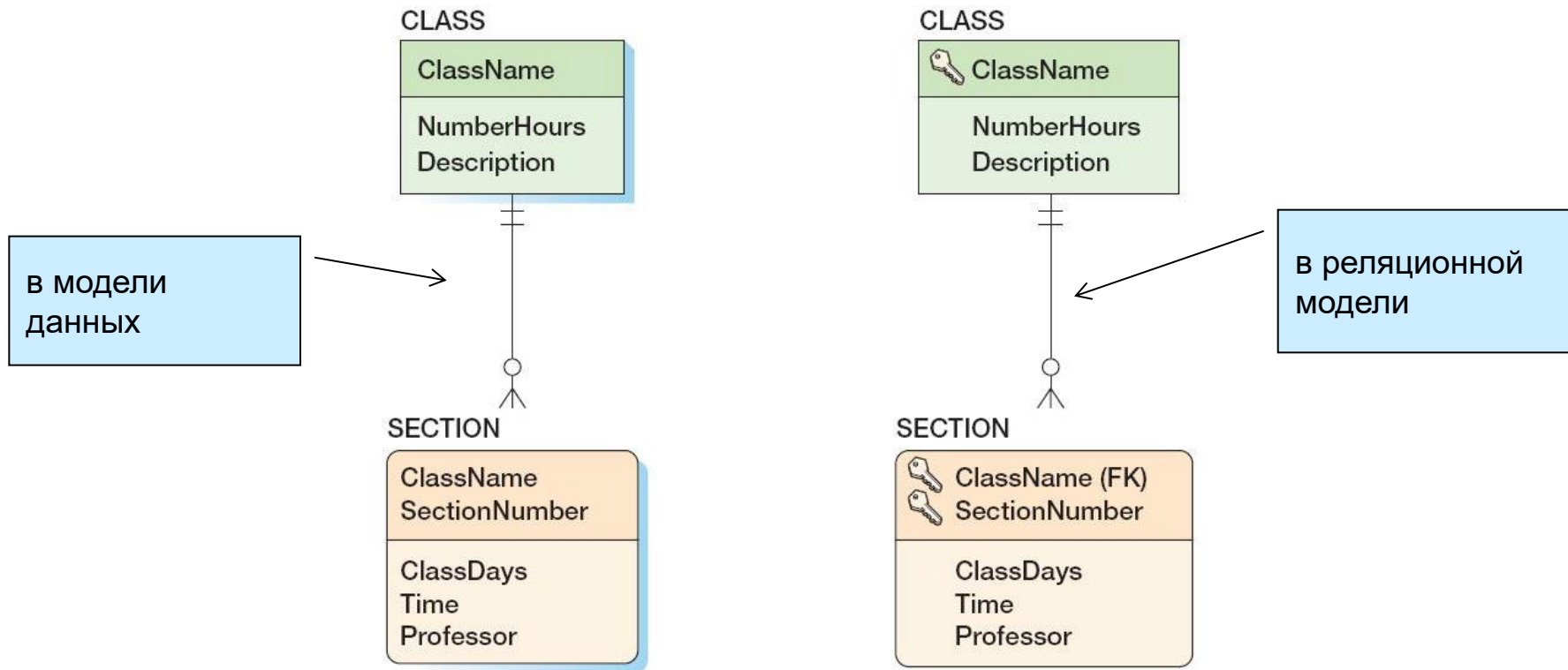
QUOTATION (CompanyName, PartNumber, Price)

- создается промежуточное отношение (*association table*):
 - содержит в качестве внешних первичные ключи обоих отношений;
 - оба внешних ключа формируют составной первичный ключ нового отношения;
 - присутствуют дополнительные атрибуты.

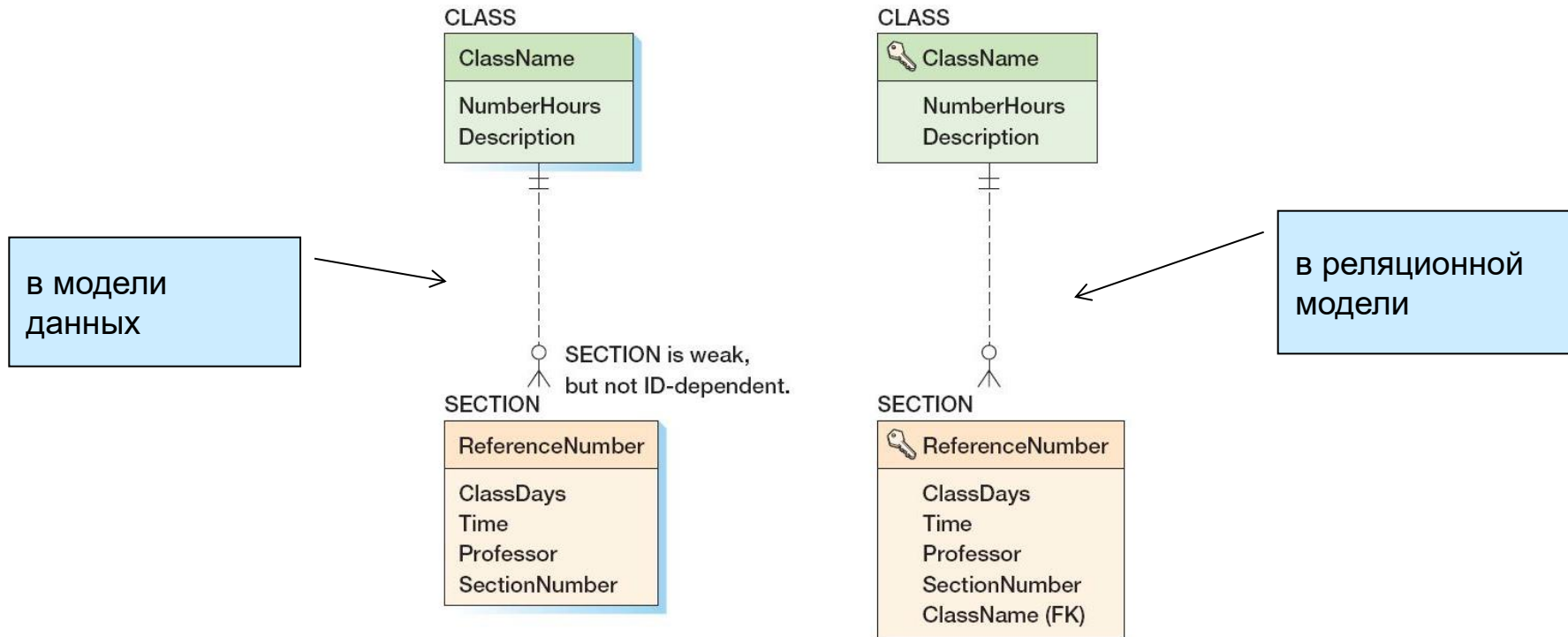
Создание связей: многозначные атрибуты



Создание связей: шаблон «прототип-экземпляр»

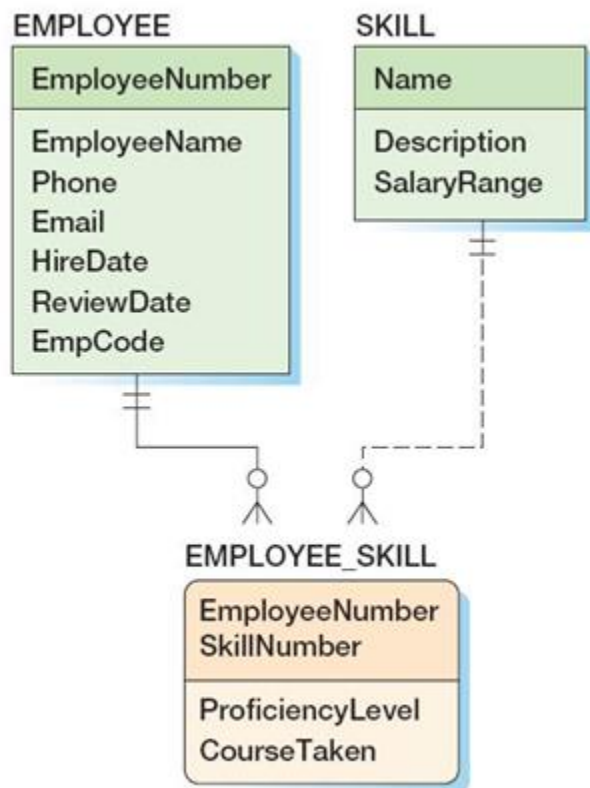


Создание связей: шаблон «прототип-экземпляр» - слабые сущности

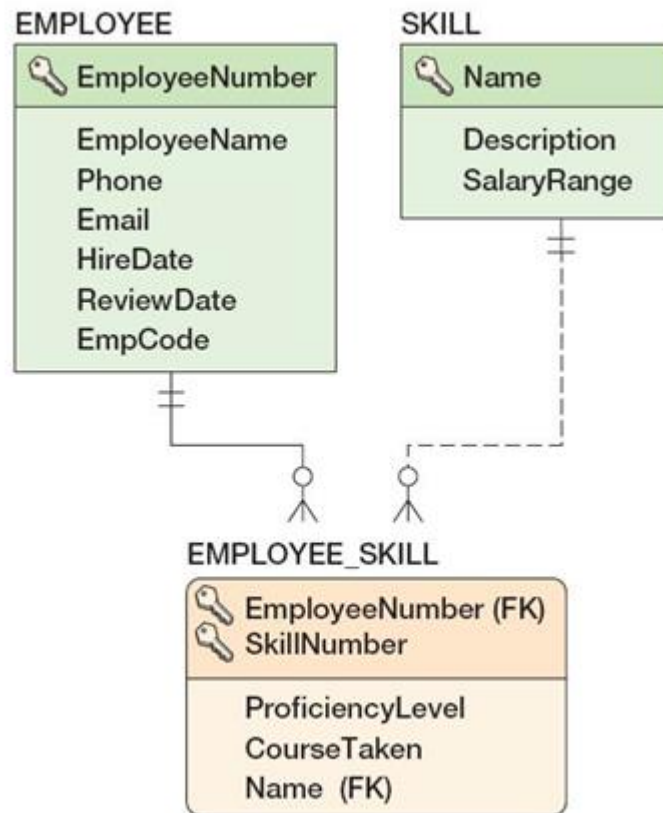


- *слабой сущностью (weak entity)* называется сущность, существование которой зависит от наличия родительской сущности;
- все идентификационно-зависимые сущности являются слабыми;
- возможно наличие в модели слабых сущностей, которые при этом не являются идентификационно-зависимыми;

Создание связей: сложный пример (1)

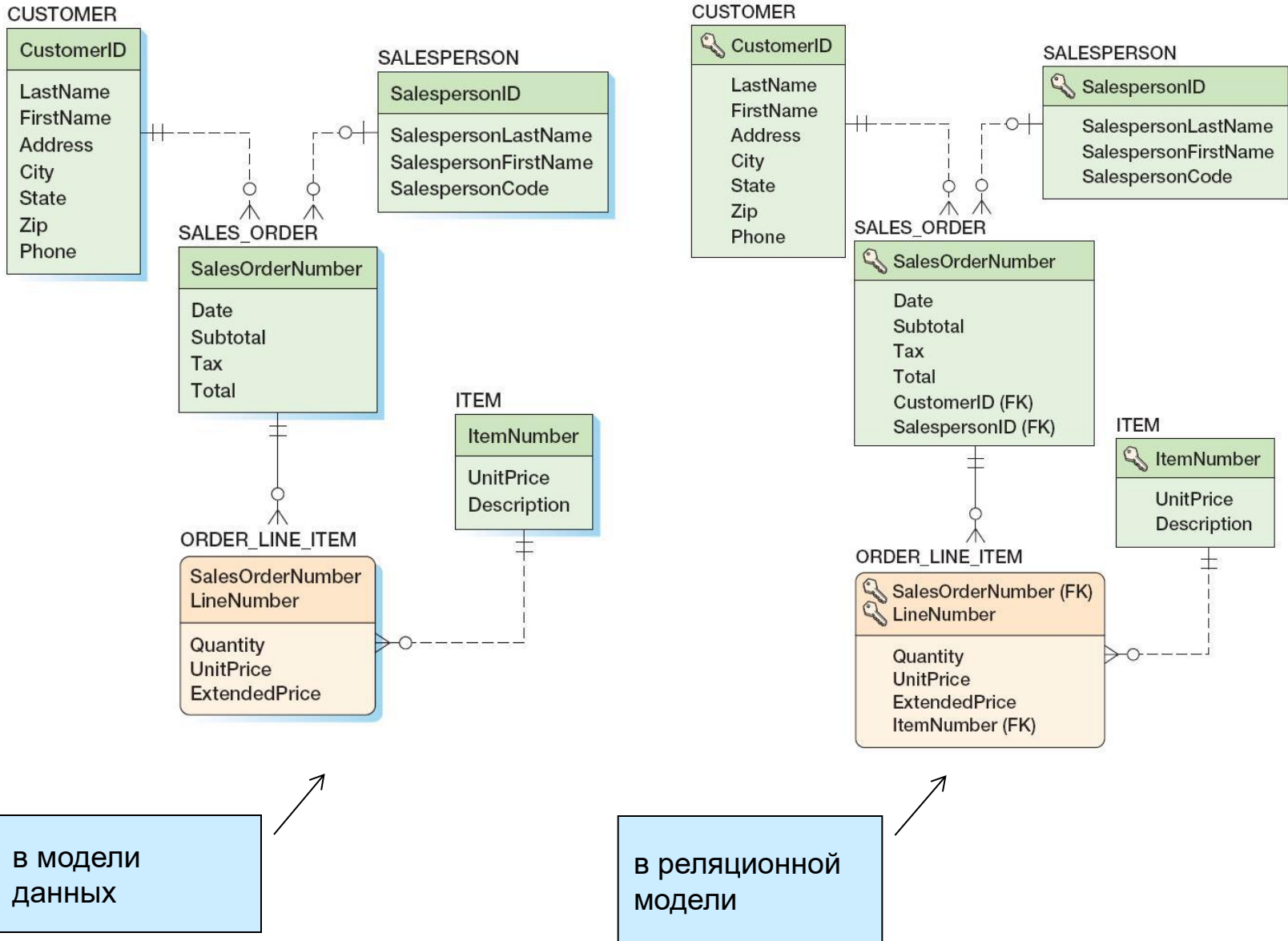


В модели
данных

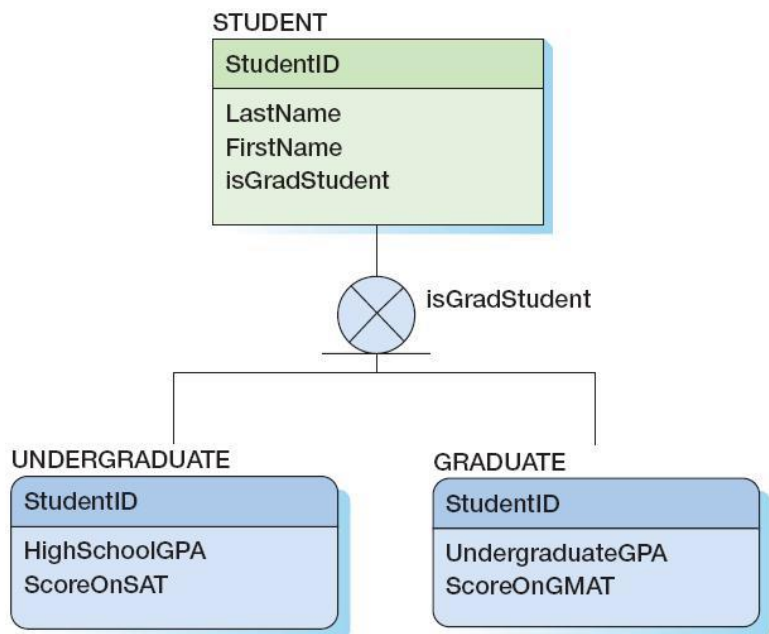


в реляционной
модели

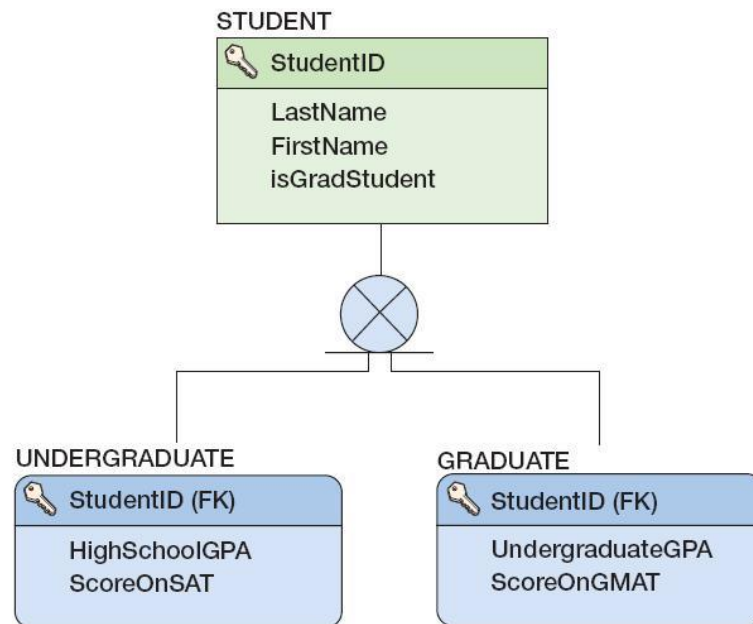
Создание связей: сложный пример (2)



Создание связей: подтипы

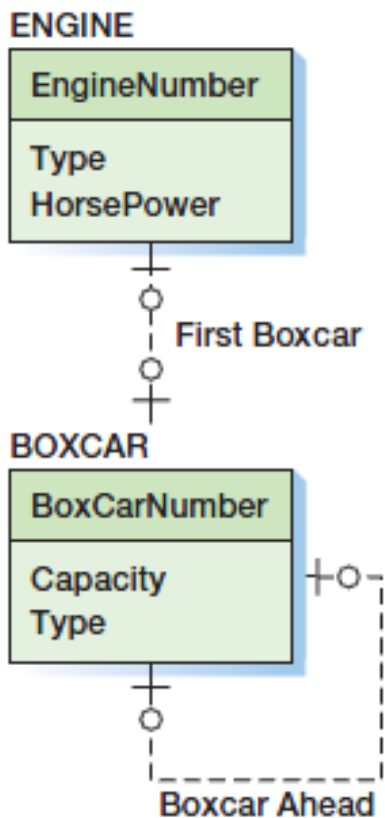


в модели
данных

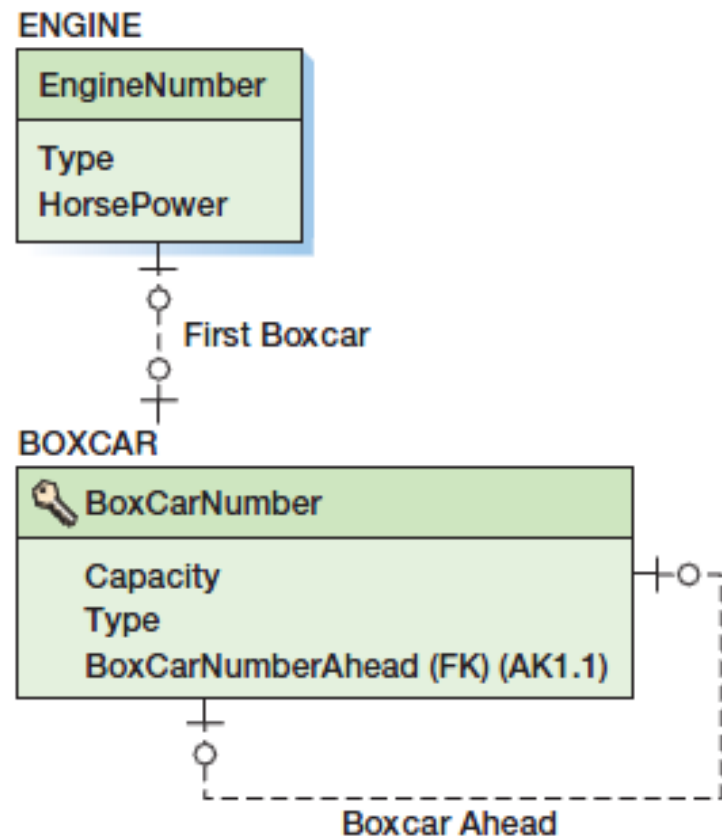


в реляционной
модели

Создание связей: рекурсия 1:1

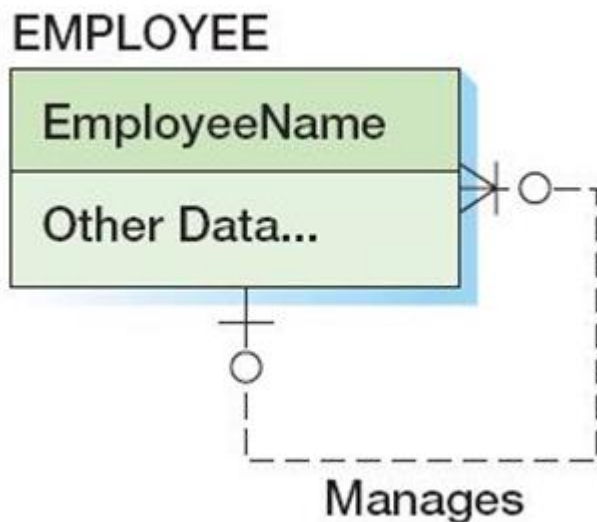


в модели
данных

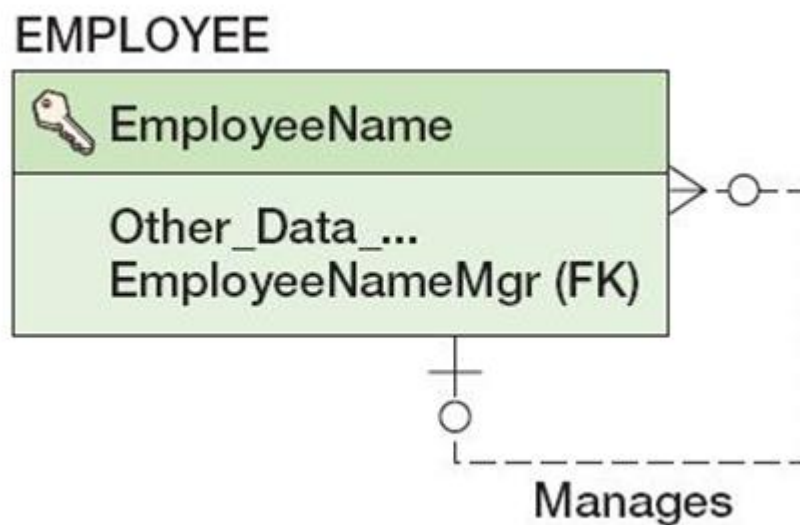


в реляционной
модели

Создание связей: рекурсия 1:N

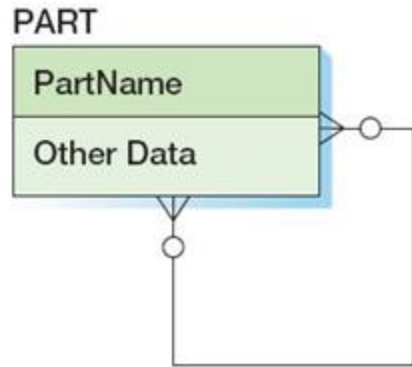


в модели
данных

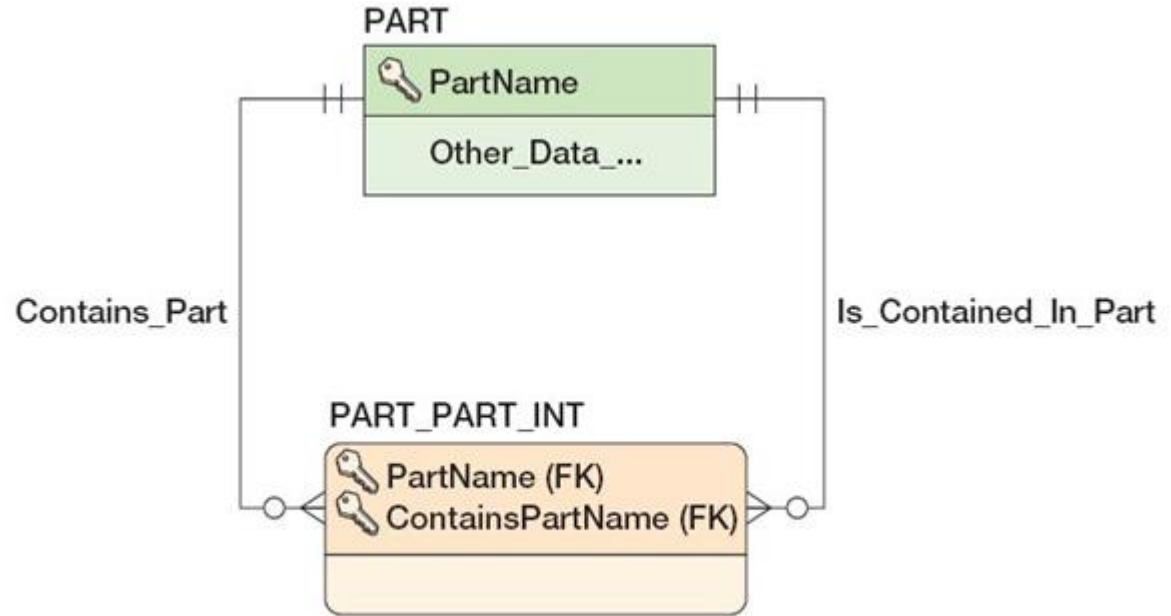


в реляционной
модели

Создание связей: рекурсия M:N



в модели
данных



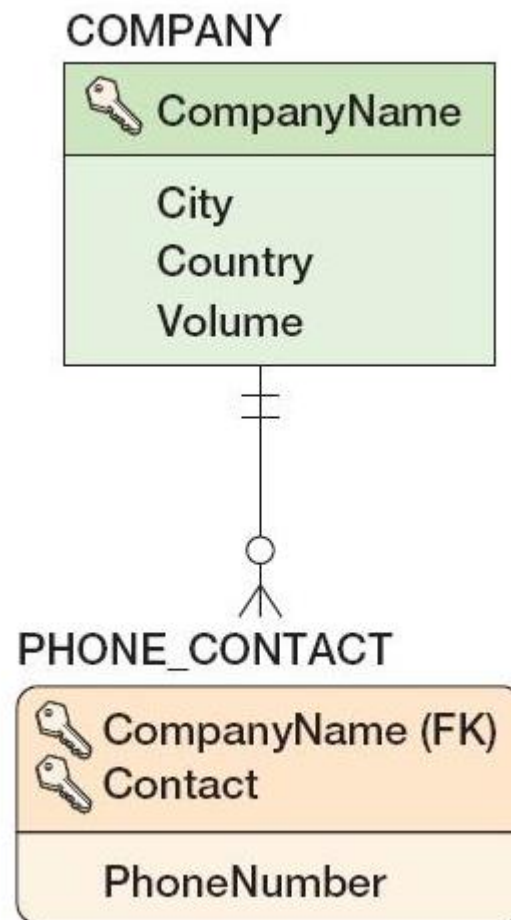
в реляционной
модели

Ограничения минимальной кардинальности

- связи могут иметь следующие варианты минимальной кардинальности:
 - **O-O** (*optional-optional*): родительская и дочерняя сущности являются необязательными;
 - **M-O** (*mandatory-optional*): родительская сущность является обязательной, дочерняя сущность является необязательной;
 - **O-M** (*optional-mandatory*): родительская сущность является необязательной, дочерняя сущность является обязательной;
 - **M-M** (*mandatory-mandatory*): родительская и дочерняя сущности являются обязательными;
- часто для обозначения процедуры обеспечения ограничений минимальной кардинальности используют термин **действие** (*action*);
- рассматриваемые операции над записями:
 - вставка (*insert*);
 - изменение (*update*) первичного или внешнего ключа;
 - удаление (*delete*);
- очевидно, что обеспечение ограничения **O-O** не требует никаких действий.

Каскадные ограничения ссылочной целостности

- каскадное обновление (*cascade update*) имеет место, когда необходимо изменить значение(я) внешнего ключа (в дочерней таблице) при изменении первичного ключа родительской:
 - заметим, что суррогатные ключи не изменяются, следовательно, для них не нужно каскадное обновление;
- каскадное удаление (*cascade delete*) имеет место, когда необходимо удалить строки в дочерней таблице при удалении записи в родительской таблице:
 - для сильных сущностей: как правило, не используется;
 - для слабых сущностей: как правило, используется.



Действия, необходимые для связей с минимальной кардинальностью М-О

Операция	Действие над родительской таблицей	Действие над дочерней таблицей
Вставка	---	• подбор новой родительской записи; • запрет.
Изменение	• каскадное обновление; • запрет.	• допустимо, если новое значение внешнего ключа соответствует некоторому первичному в родительской таблице; • запрет.
Удаление	• каскадное удаление; • запрет.	---

Действия, необходимые для связей с минимальной кардинальностью О-М

Операция	Действие над родительской таблицей	Действие над дочерней таблицей
Вставка	• подбор новой дочерней записи; • запрет.	---
Изменение	• обновление внешнего ключа хотя бы одной дочерней записи; • запрет.	• если есть другие дочерние записи у соответствующей родительской, доп. действия не нужны; • иначе, запрет или поиск замены.
Удаление	---	• если есть другие дочерние записи у соответствующей родительской, доп. действия не нужны; • иначе, запрет или поиск замены.

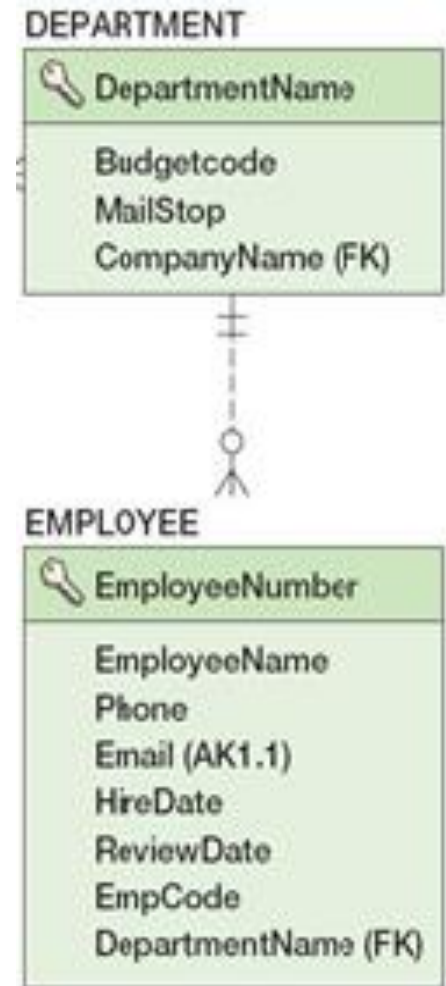
Действия, необходимые для обеспечения ограничений минимальной кардинальности

O-O	---	---
M-O	Действия, необходимые для связей с минимальной кардинальностью M-O	обеспечивается средствами СУБД: • определение ограничений ссылочной целостности (PK+FK); • задание условия NOT NULL для внешнего ключа.
O-M	Действия, необходимые для связей с минимальной кардинальностью O-M	• обеспечивается средствами СУБД путем создания триггеров или использования других средств программирования; • обеспечивается внешними по отношению к СУБД средствами.
M-M	Оба набора действий (M-x + x-M)	• обеспечивается средствами СУБД или внешними по отношению к СУБД средствами; • обеспечение ограничений минимальной кардинальности является в данном случае крайне сложной задачей (особенно для сильных сущностей).

- триггер – некоторый хранимый программный код, который выполняется при наступлении некоторого события

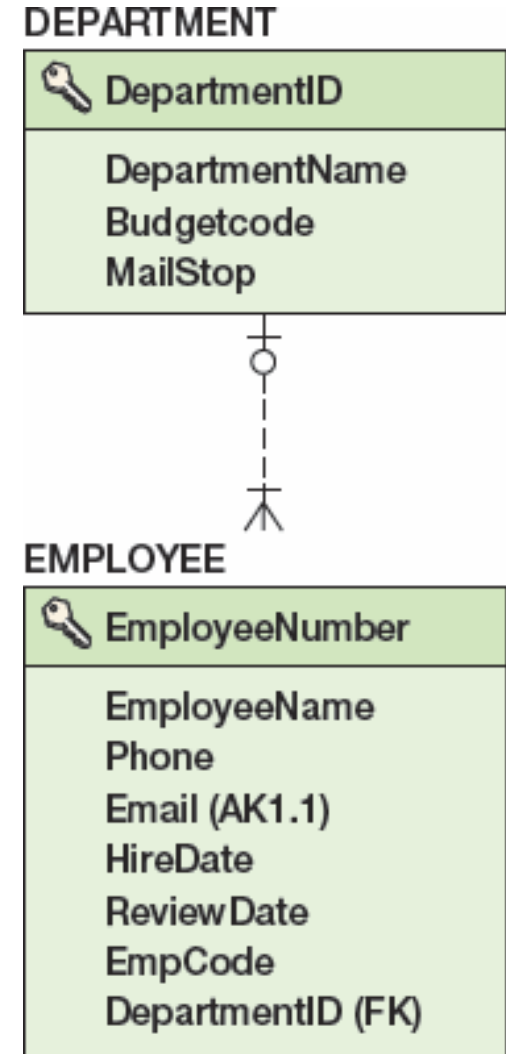
Действия для связей М-О: пример

- действия для DEPARTMENT:
 - вставка: без ограничений;
 - изменение первичного ключа: каскадное обновление;
 - удаление:
 - каскадное удаление;
 - запрет на удаление;
- действия для EMPLOYEE (подразумевается определение ограничений ссылочной целостности (РК+FK) и задание условия NOT NULL для внешнего ключа):
 - вставка: возможна только при корректном значении внешнего ключа;
 - изменение внешнего ключа: возможно только при корректном новом значении;
 - удаление: без ограничений.

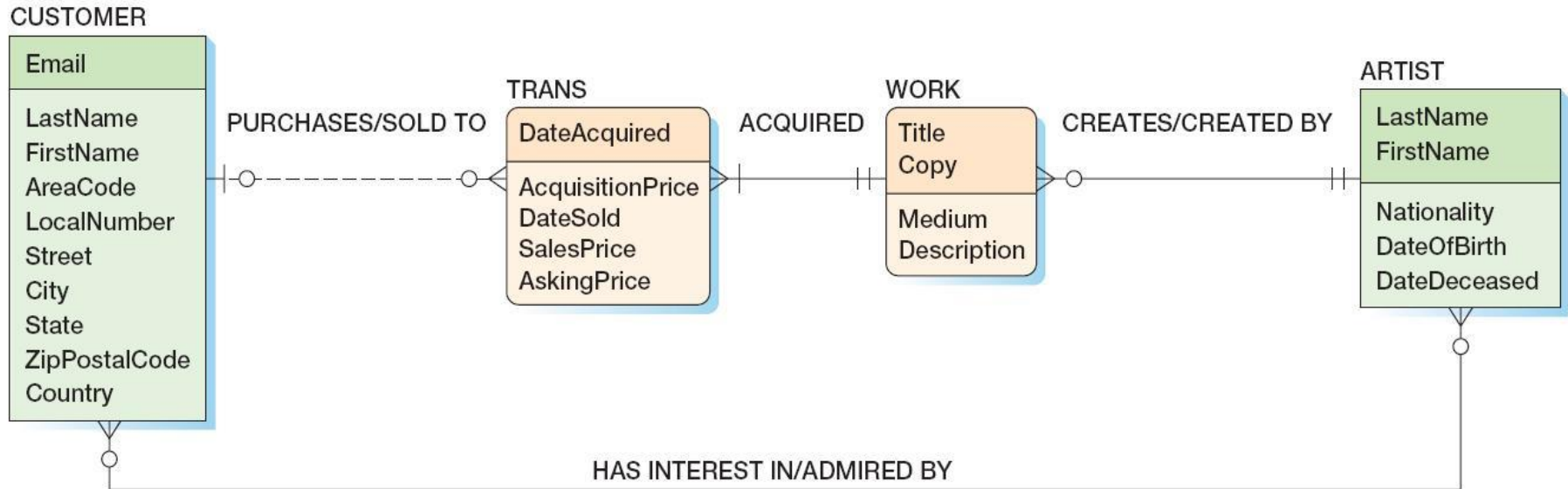


Действия для связей О-М: пример

- действия для DEPARTMENT:
 - вставка: операция возможна только при одновременном создании или выборе записи в дочерней таблице: необходим триггер;
 - изменение первичного ключа: операция возможна только при одновременном выборе записи (изменении внешнего ключа в этой записи) в дочерней таблице: необходим триггер;
 - удаление: без ограничений;
- действия для EMPLOYEE:
 - вставка: без ограничений;
 - изменение внешнего ключа: операция возможна, только если к родительской записи относится более одной дочерней записи;
 - удаление: операция возможна, только если к родительской записи относится более одной дочерней записи.

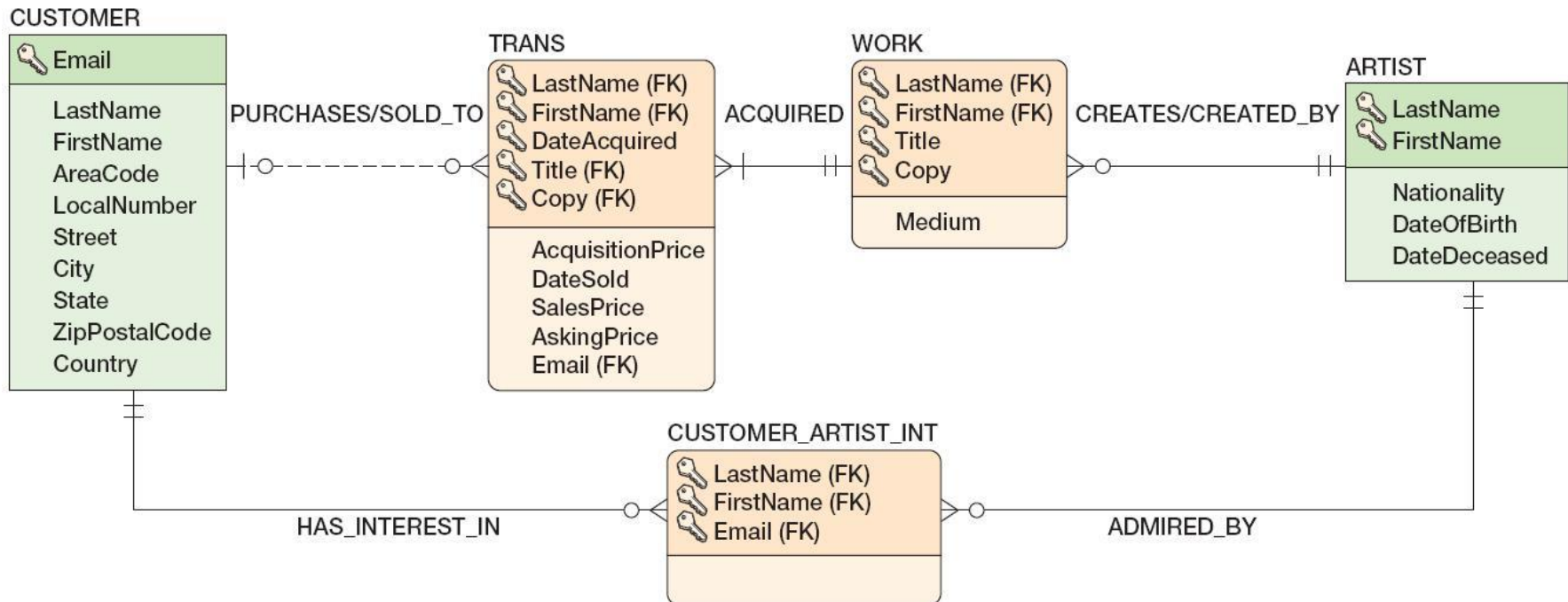
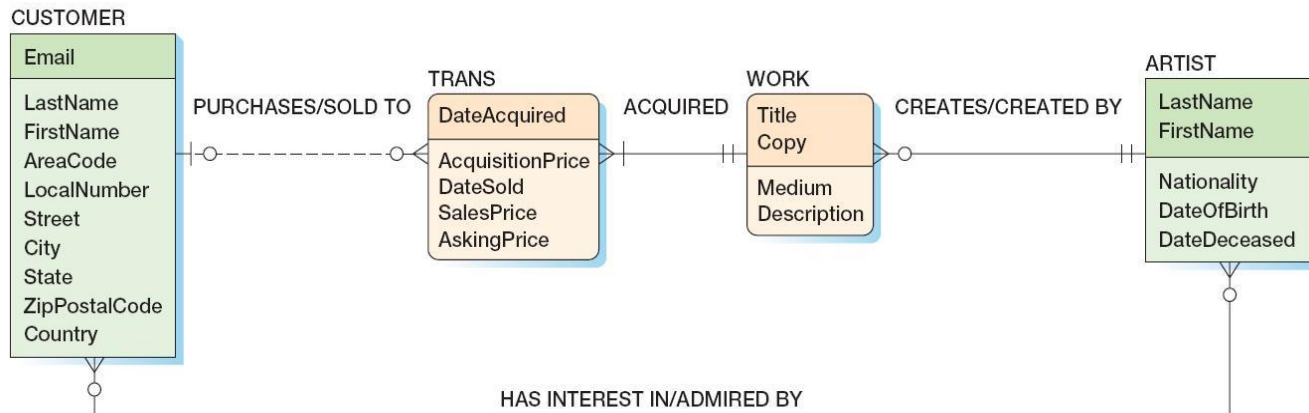


Пример: галерея современного искусства

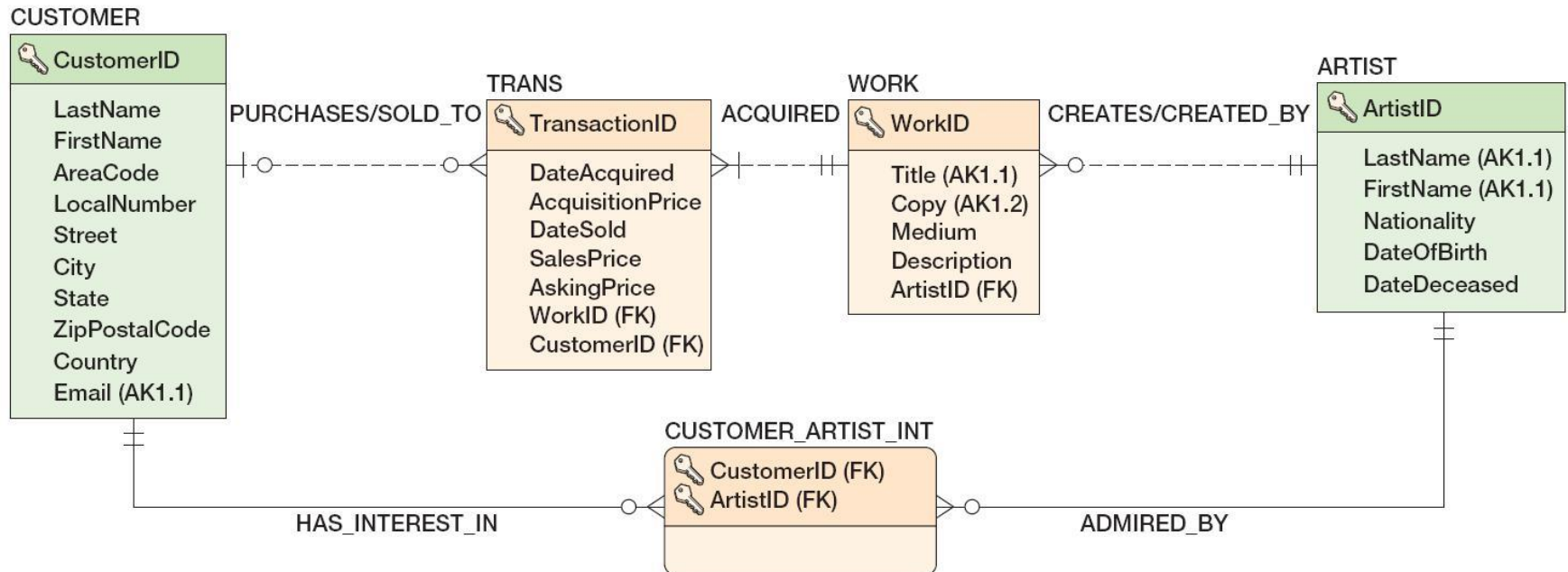
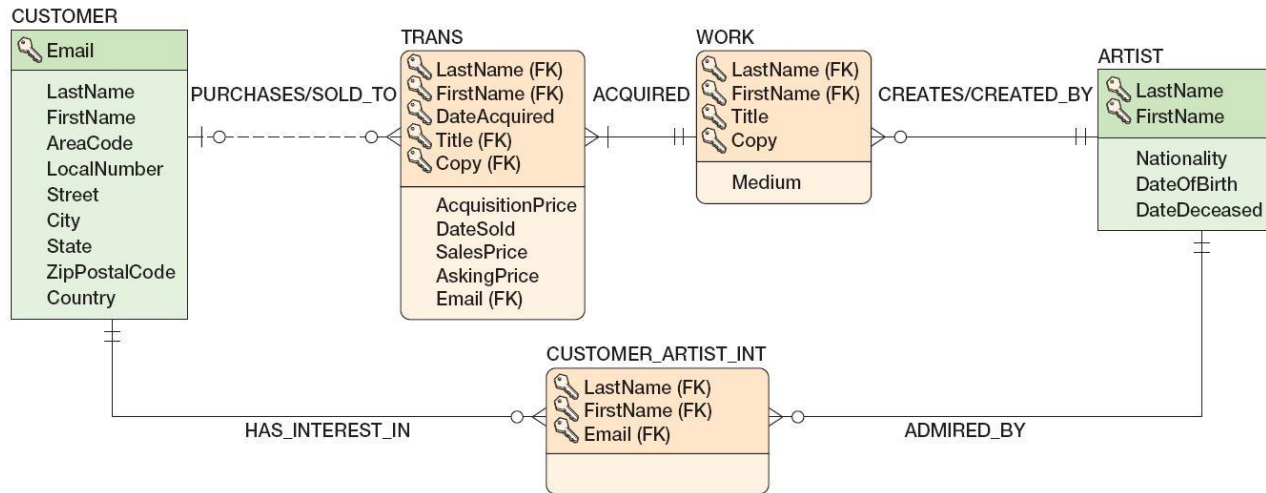


- приложение должно обеспечивать:
 - хранение информации:
 - о клиентах и их интересах;
 - о покупках работ галереей и их продажах клиентам галереи;
 - отображение информации:
 - о художниках и их работах, которые присутствовали в галерее;
 - о периоде между покупкой и продажей работ художника и полученной прибыли;
 - отображение информации об имеющихся на текущий момент в распоряжении галереи работах на веб-странице.

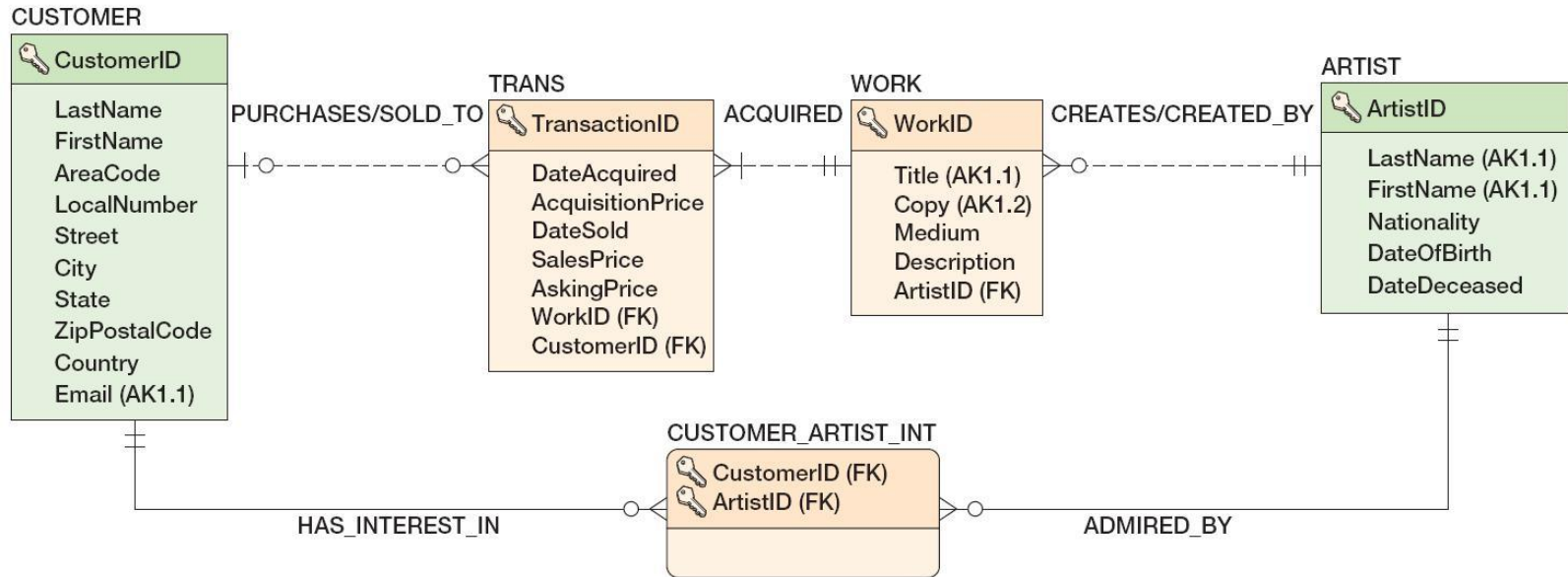
Пример: первая итерация проектирования



Пример: вторая итерация проектирования

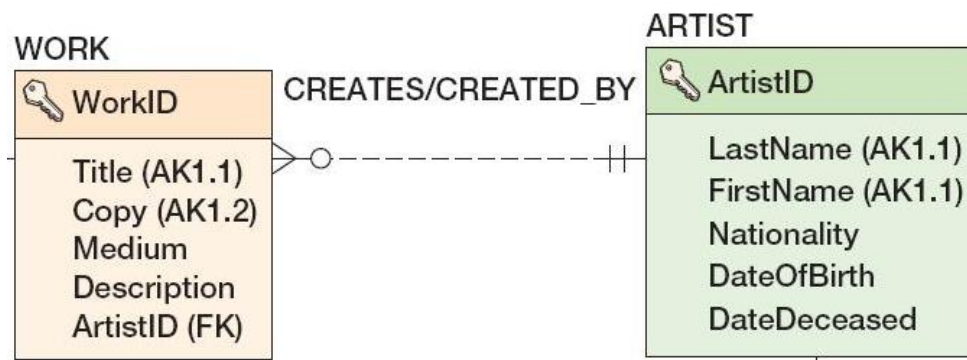


Пример: ограничения минимальной кардинальности



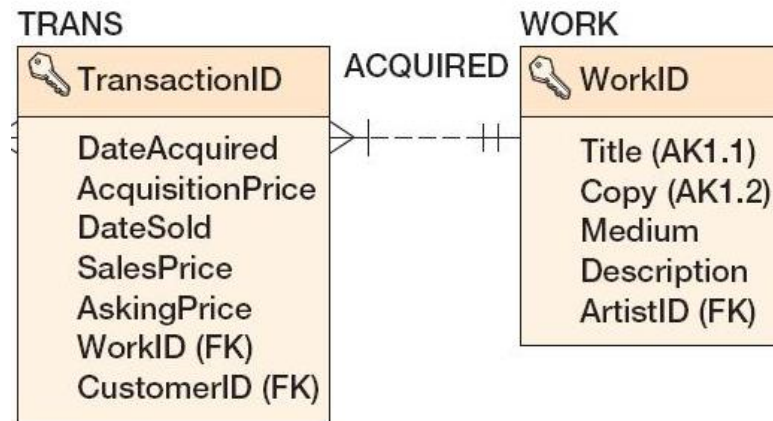
Relationship		Cardinality		
Parent	Child	Type	MAX	MIN
ARTIST	WORK	Nonidentifying	1:N	M-O
WORK	TRANS	Nonidentifying	1:N	M-M
CUSTOMER	TRANS	Nonidentifying	1:N	O-O
CUSTOMER	CUSTOMER_ARTIST_INT	Identifying	1:N	M-O
ARTIST	CUSTOMER_ARTIST_INT	Identifying	1:N	M-O

Пример: ограничения минимальной кардинальности, ARTIST-to-WORK, M-O



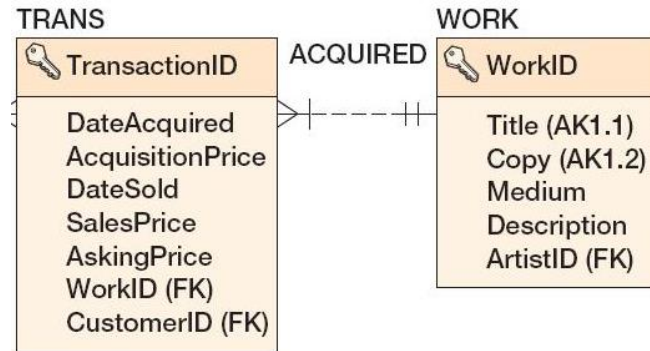
ARTIST Is Required Parent	Action on ARTIST (Parent)	Action on WORK (Child)
Insert	None.	Get a parent.
Modify key or foreign key	Prohibit—ARTIST uses a surrogate key.	Prohibit— ARTIST uses a surrogate key.
Delete	Prohibit if WORK exists— data related to a transaction is never deleted (business rule). Allow if no WORK exists (business rule).	None.

Пример: ограничения минимальной кардинальности, WORK-to-TRANS, M-O



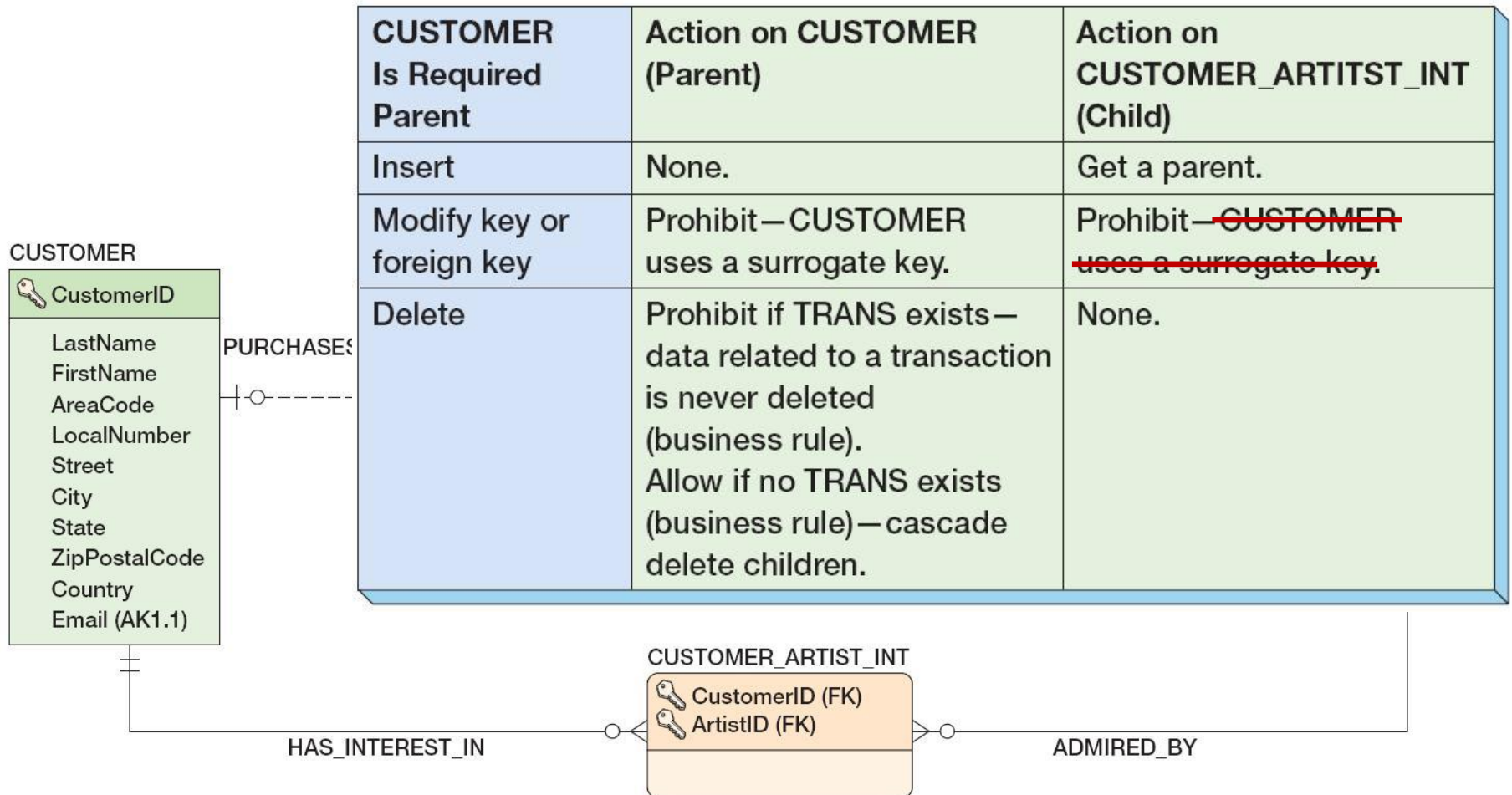
WORK Is Required Parent	Action on WORK (Parent)	Action on TRANS (Child)
Insert	None.	Get a parent.
Modify key or foreign key	Prohibit—WORK uses a surrogate key.	Prohibit— WORK uses a surrogate key.
Delete	Prohibit—data related to a transaction is never deleted (business rule).	None.

Пример: ограничения минимальной кардинальности, WORK-to-TRANS, O-M

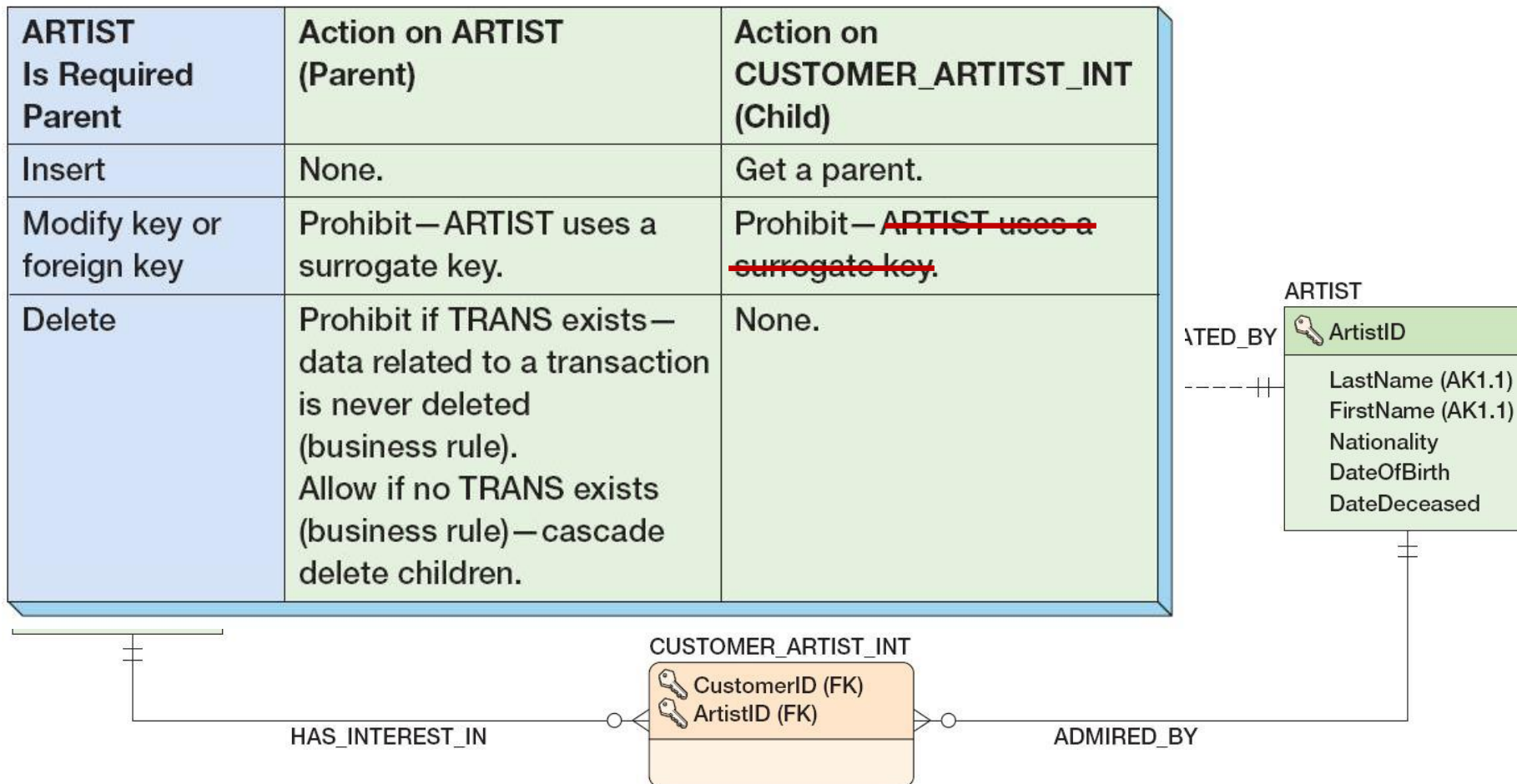


TRANS Is Required Parent	Action on WORK (Parent)	Action on TRANS (Child)
Insert	INSERT trigger on WORK to create row in TRANS. TRANS will be given data for DateAcquired and AcquisitionPrice. Other columns will be null.	Will be created by INSERT trigger on WORK.
Modify key or foreign key	Prohibit—surrogate key.	Prohibit—TRANS must always refer to the WORK associated with it.
Delete	Prohibit—data related to a transaction is never deleted (business rule).	Prohibit—data related to a transaction is never deleted (business rule).

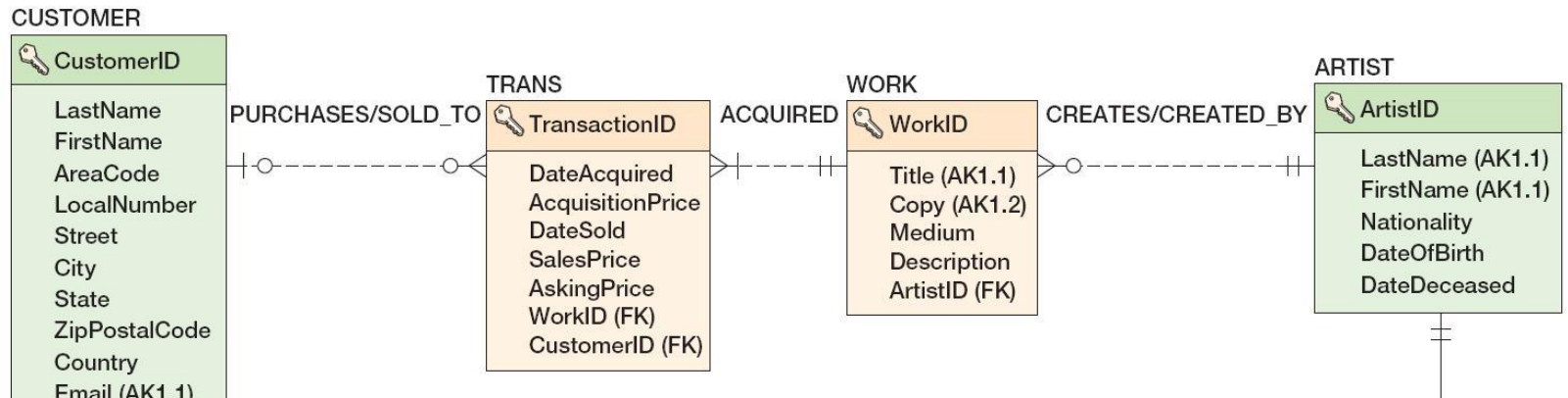
Пример: ограничения минимальной кардинальности, CUSTOMER-to-INT, M-O



Пример: ограничения минимальной кардинальности, ARTIST-to-INT, M-O

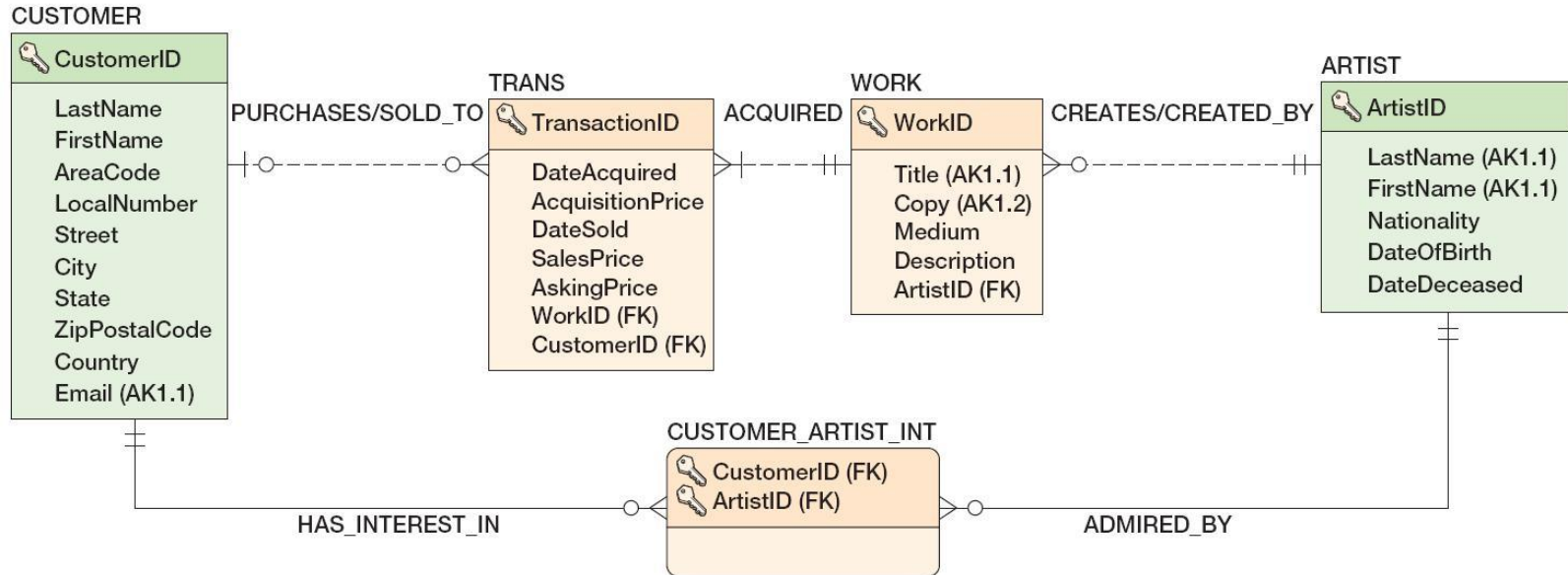


Пример: таблица ARTIST



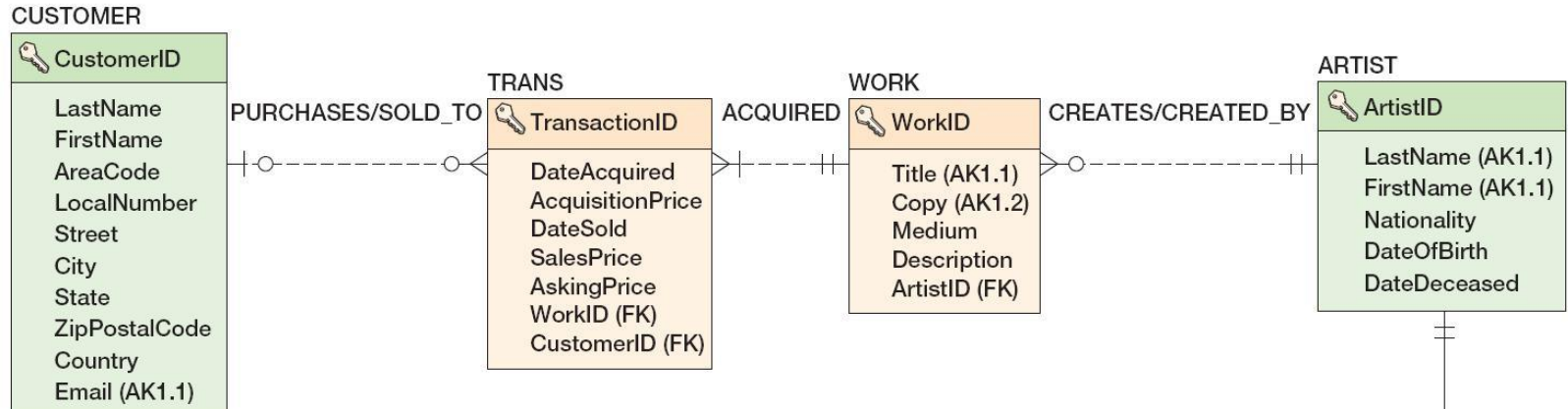
Column Name	Type	Key	NULL Status	Remarks
ArtistID	Int	Primary Key	NOT NULL	Surrogate Key IDENTITY (1,1)
LastName	Char (25)	Alternate Key	NOT NULL	Unique (AK1.1)
FirstName	Char (25)	Alternate Key	NOT NULL	Unique (AK1.2)
Nationality	Char (30)	No	NULL	IN ('Canadian', 'English', 'French', 'German', 'Mexican', 'Russian', 'Spanish', 'United States')
DateOfBirth	Numeric (4)	No	NULL	(DateOfBirth < DateDeceased) (BETWEEN 1900 and 1999)
DateDeceased	Numeric (4)	No	NULL	(BETWEEN 1900 and 1999)

Пример: таблица WORK



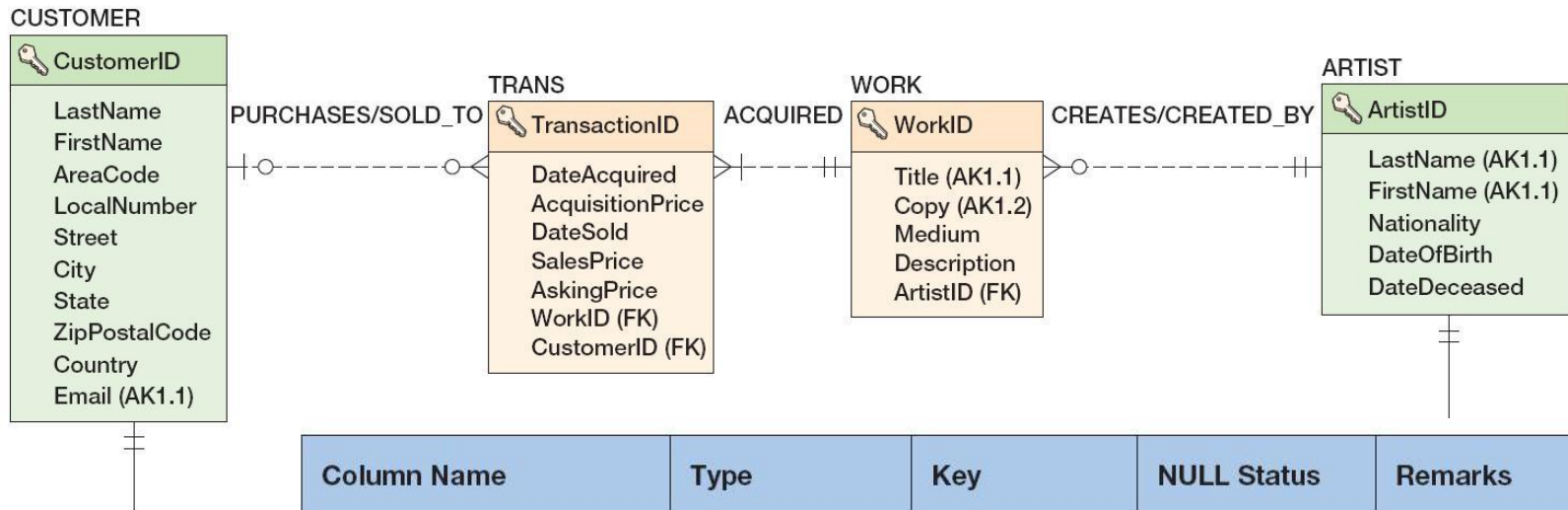
Column Name	Type	Key	NULL Status	Remarks
WorkID	Int	Primary Key	NOT NULL	Surrogate Key IDENTITY (500,1)
Title	Char (35)	Alternate Key	NOT NULL	Unique (AK1.1)
Copy	Char (12)	Alternate Key	NOT NULL	Unique (AK1.2)
Medium	Char (35)	No	NULL	
Description	Varchar (1000)	No	NULL	DEFAULT value = 'Unknown provenance'
ArtistID	Int	Foreign Key	NOT NULL	

Пример: таблица TRANS



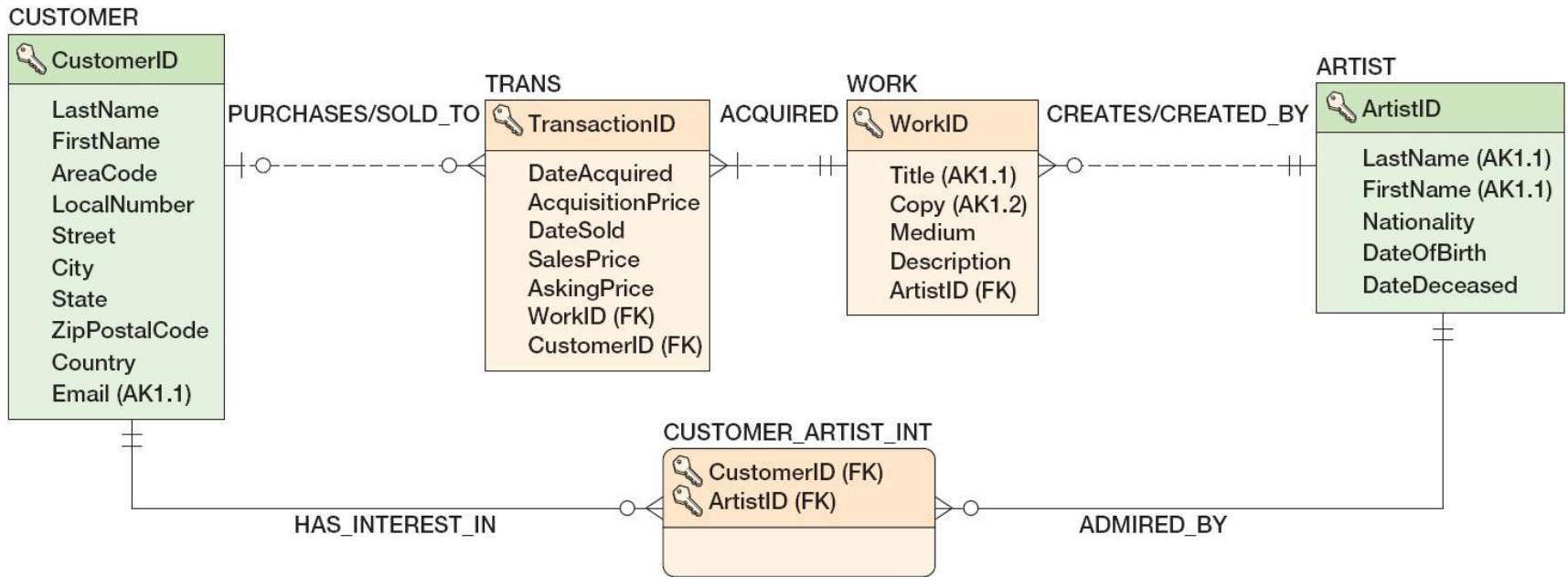
Column Name	Type	Key	NULL Status	Remarks
TransactionID	Int	Primary Key	NOT NULL	Surrogate Key IDENTITY (100,1)
DateAcquired	Date	No	NOT NULL	
AcquisitionPrice	Numeric (8,2)	No	NOT NULL	
AskingPrice	Numeric (8,2)	No	NULL	
DateSold	Date	No	NULL	(DateAcquired <= DateSold)
SalesPrice	Numeric (8,2)	No	NULL	(SalesPrice > 0) AND (SalesPrice <=500000)
CustomerID	Int	Foreign Key	NULL	
WorkID	Int	Foreign Key	NOT NULL	

Пример: таблица CUSTOMER



Column Name	Type	Key	NULL Status	Remarks
CustomerID	Int	Primary Key	NOT NULL	Surrogate Key IDENTITY (100,1)
LastName	Char (25)	No	NOT NULL	
FirstName	Char (25)	No	NOT NULL	
Street	Char (30)	No	NULL	
City	Char (35)	No	NULL	
State	Char (2)	No	NULL	
ZipPostalCode	Char (9)	No	NULL	
Country	Char (50)	No	NULL	
AreaCode	Char (3)	No	NULL	
PhoneNumber	Char (8)	No	NULL	
Email	Varchar (100)	Alternate Key	NULL	Unique (AK 1.1)

Пример: таблица CUSTOMER_ARTIST_INT



Column Name	Type	Key	NULL Status	Remarks
ArtistID	Int	Primary Key, Foreign Key	NOT NULL	
CustomerID	Int	Primary Key, Foreign Key	NOT NULL	

Вопросы к экзамену

- Преобразование модели «сущность-связь»: основные этапы. Создание таблиц для каждой из сущностей. Определение свойств столбцов таблицы. Нормализация.
- Преобразование модели «сущность-связь»: основные этапы. Создание связей: сильные сущности, идентификационно-зависимые сущности, шаблоны «сопряжение» и «прототип-экземпляр».
- Преобразование модели «сущность-связь»: основные этапы. Создание связей: шаблон «категория-подтип», представление многозначных атрибутов. Рекурсивные связи.
- Преобразование модели «сущность-связь»: основные этапы. Создание связей: ограничения минимальной кардинальности связей.

Лабораторная работа №3.

Преобразование модели «сущность-связь» в реляционную модель

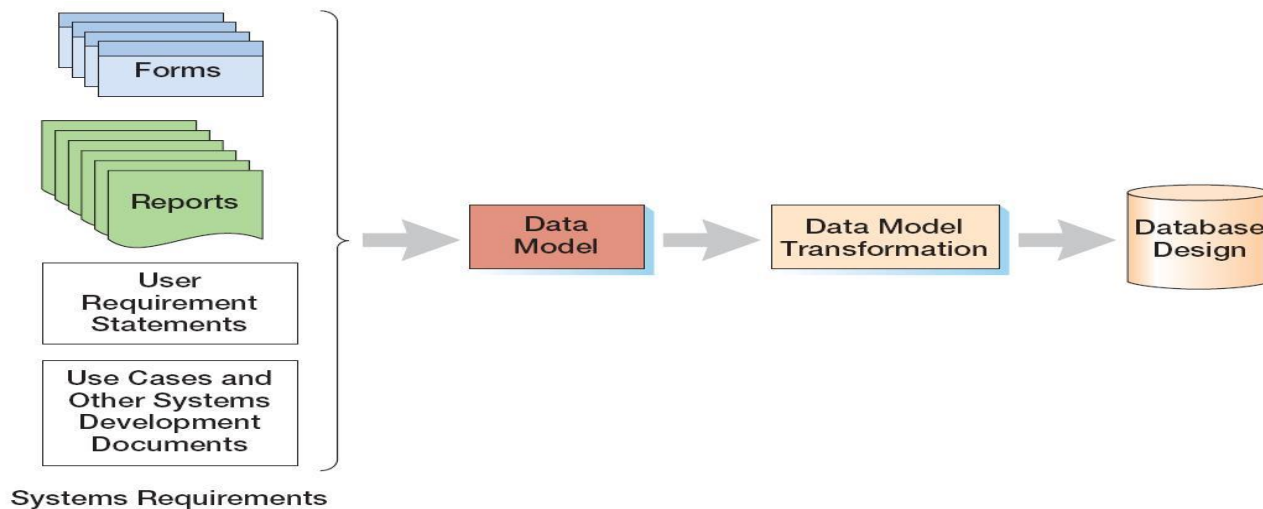
1. Преобразовать модель «сущность-связь», созданную в лабораторной работе №1, в реляционную модель согласно процедуре преобразования.
2. Обосновать выбор типов данных, ключей, правил обеспечения ограничений минимальной кардинальности.

Базы данных

Тема 4

Преобразование модели семантических
объектов в реляционную модель

Три уровня моделей информационных систем



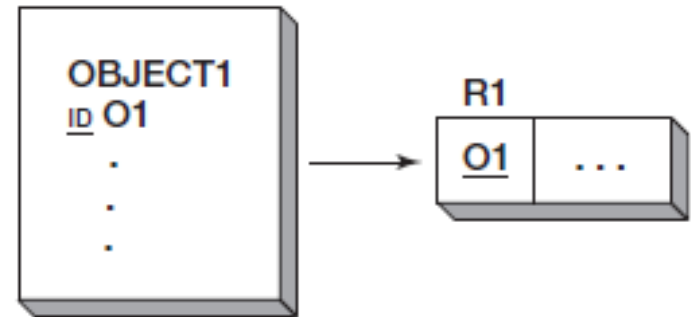
- внешние модели (*external schema*): представление пользователей о системе (user view);
- концептуальная модель (*conceptual schema*): абстрактное представление системы (данных);
 - модель «сущность-связь» (entity-relationship model);
 - модель семантических объектов (semantic object model);
- внутренние модели (internal schema).

Типы семантических объектов

- простой объект;
- составной объект:
 - с независимыми группами;
 - с вложенными группами;
- сложный объект:
 - 1:1;
 - 1:N;
 - M:N;
- гибридный объект;
- ассоциативный объект;
- родитель / подтип;
- прототип / экземпляр.

Простой объект

- простой объект (simple object) – объект, содержащий только однозначные простые или групповые атрибуты



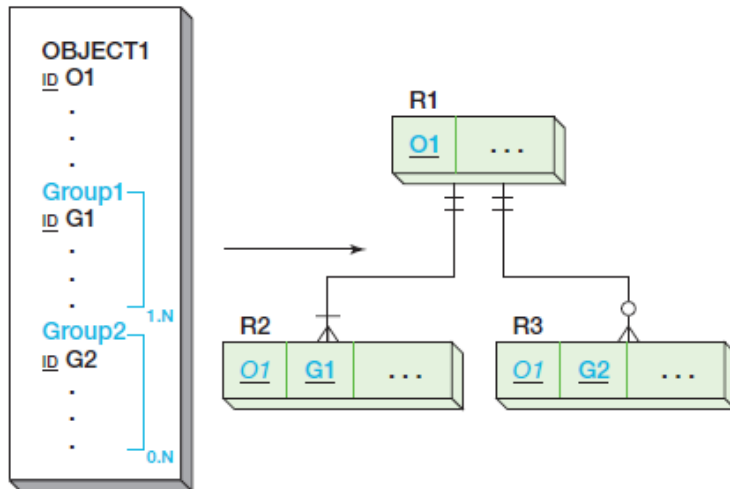
(a)

EQUIPMENT (EquipmentNumber, Description, AcquisitionDate, PurchaseCost)

(b)

Составной объект с независимыми группами

- составной объект (composite object) – объект, содержащий один или более многозначный простой или групповой атрибуты

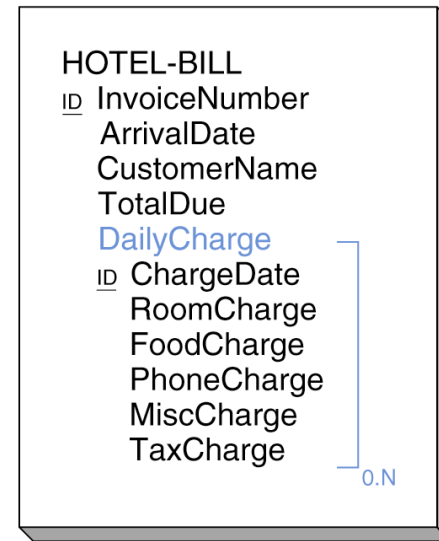


Referential integrity constraints:

O1 in R2 must exist in O1 in R1

O1 in R3 must exist in O1 in R1

(a) Composite Objects with Separate Groups



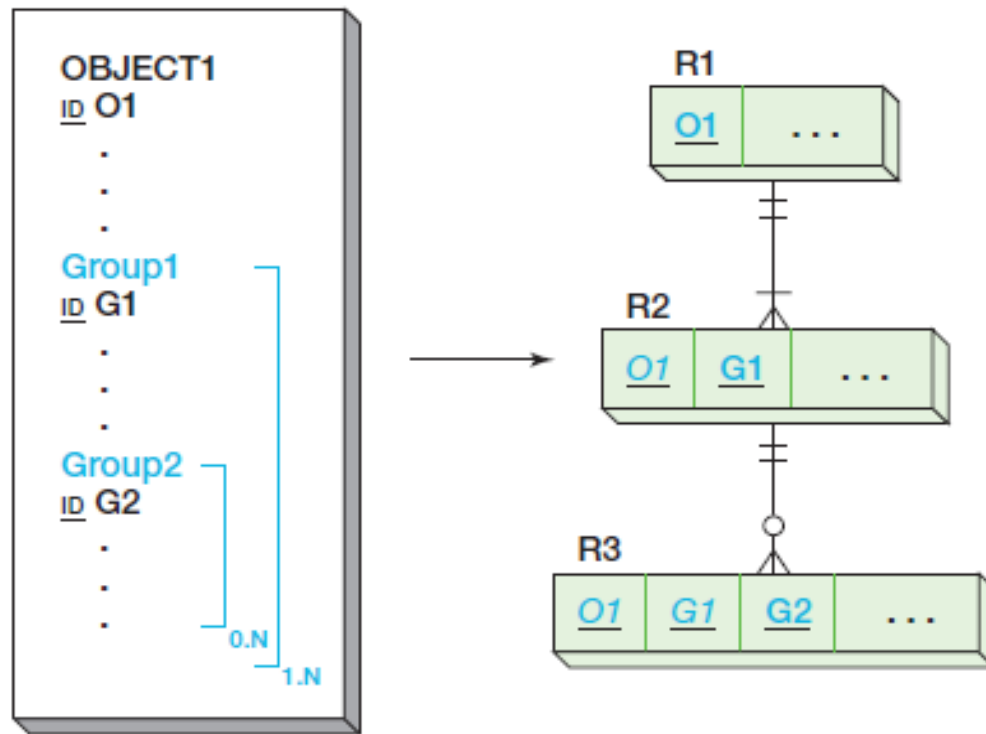
(a)

HOTEL-BILL (InvoiceNumber, ArrivalDate, CustomerName, TotalDue)

DAILY-CHARGE (InvoiceNumber, ChargeDate, RoomCharge, FoodCharge, PhoneCharge, MiscCharge, TaxCharge)

(b)

Составной объект с вложенными группами

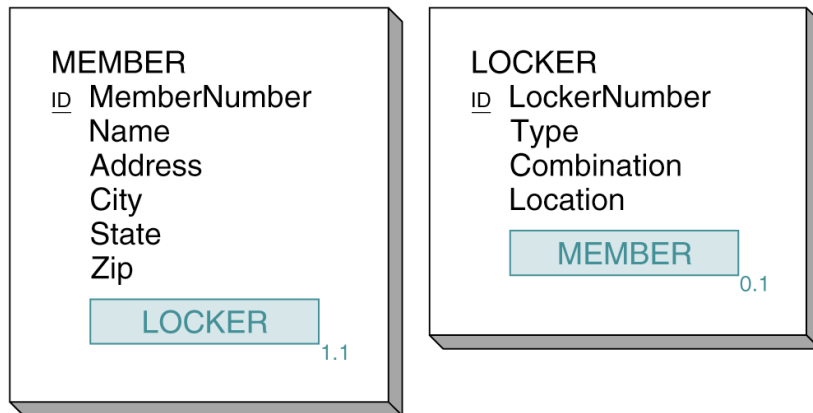


Referential integrity constraints:

- O1 in R2 must exist in O1 in R1
 - (O1, G1) in R3 must exist in (O1, G1) in R2
- (b) Composite Objects with Nested Groups

Сложный объект (1:1)

- сложный объект (compound object) – объект, содержащий один или более объектных атрибутов

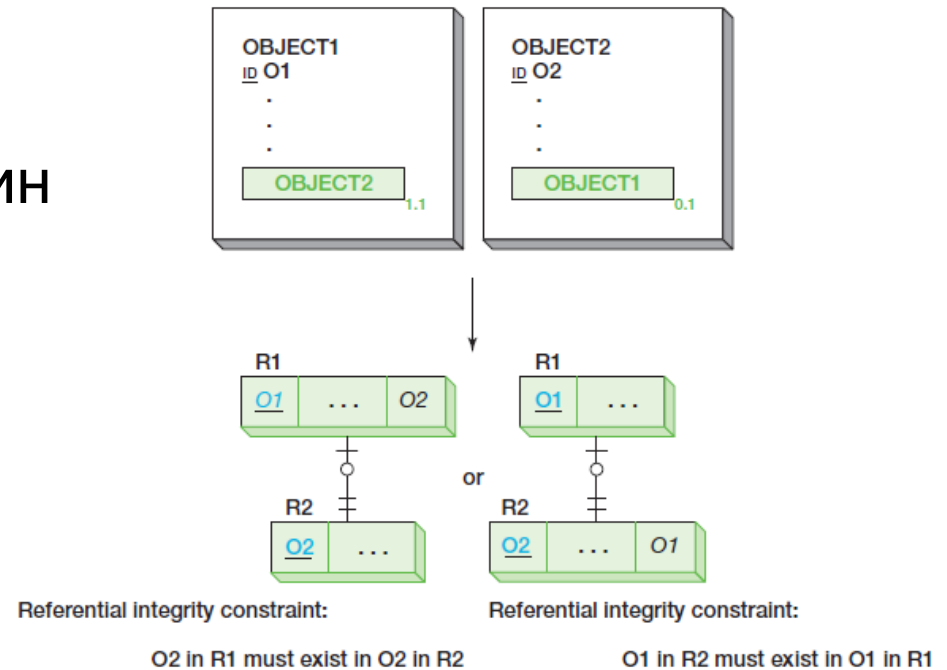


(a)

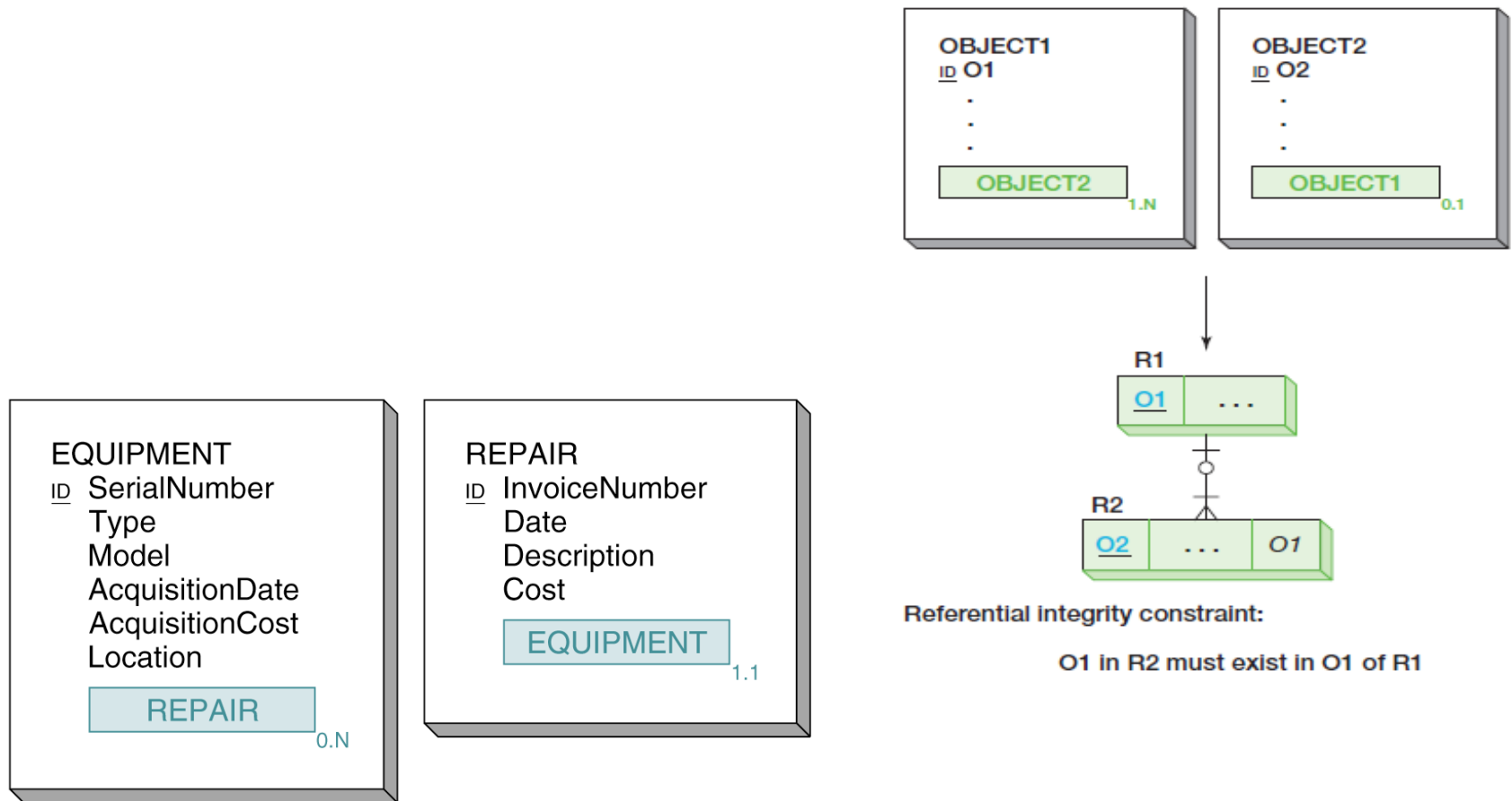
MEMBER (MemberNumber, Name, Address, City, State, Zip, LockerNumber)

LOCKER (LockerNumber, Type, Combination, Location)

(b)



Сложный объект (1:N)



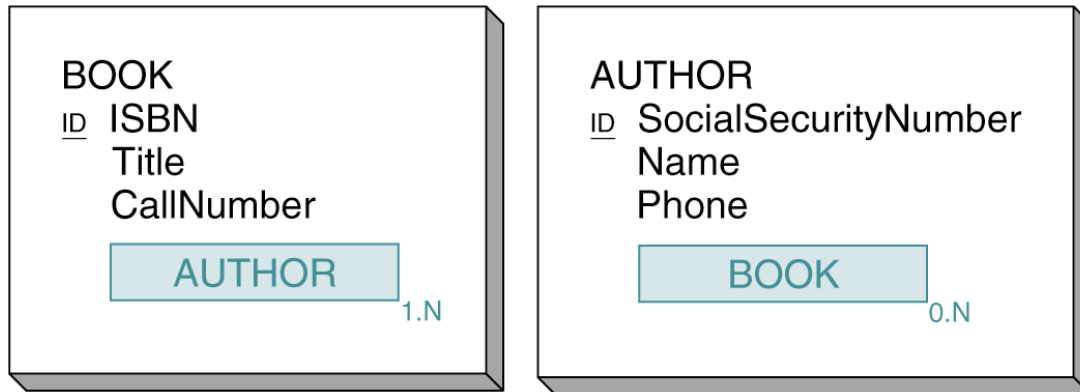
(a)

EQUIPMENT (SerialNumber, Type, Model, AcquisitionDate, AcquisitionCost, Location)

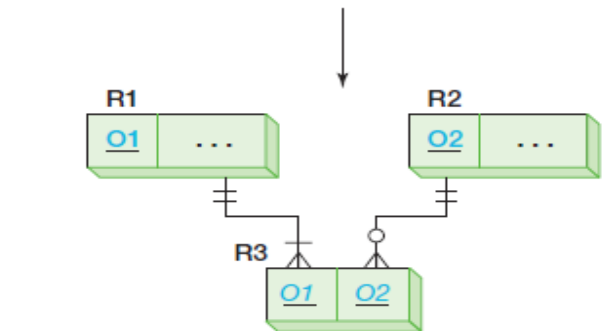
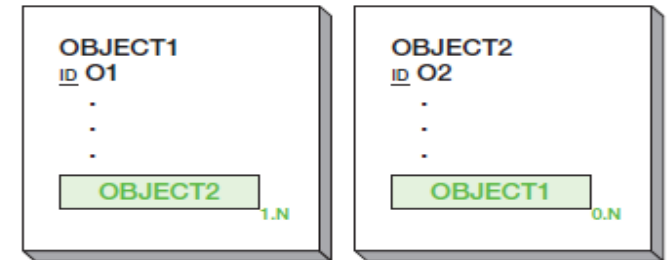
REPAIR (InvoiceNumber, Date, Description, Cost, SerialNumber)

(b)

Сложный объект (M:N)



(a)



Referential integrity constraints:

O1 in R3 must exist in O1 in R1
O2 in R3 must exist in O2 in R2

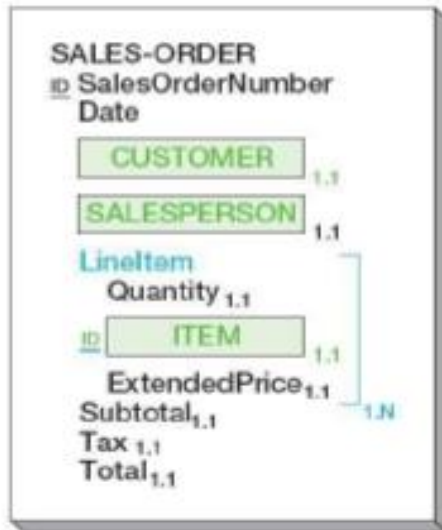
BOOK (ISBN, Title, CallNumber)

AUTHOR (SocialSecurityNumber, Name, Phone)

BOOK-AUTHOR-INT (ISBN, SocialSecurityNumber)

(b)

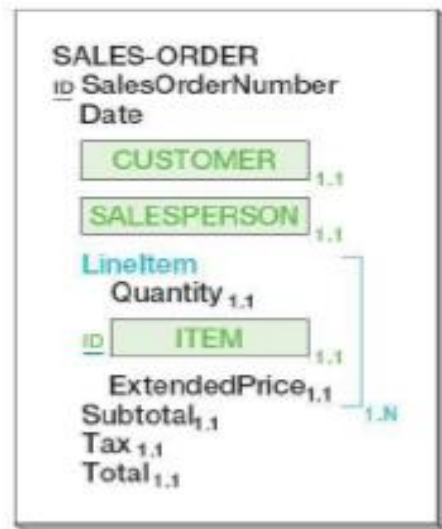
Гибридный объект: варианты кардинальности



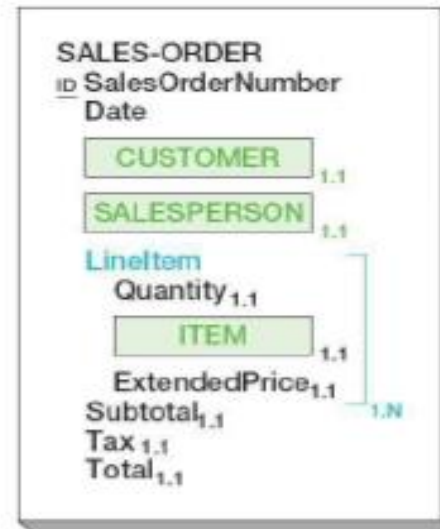
(a) ITEM in One ORDER



(b) ITEM in (Possibly) Many LineItems of One ORDER

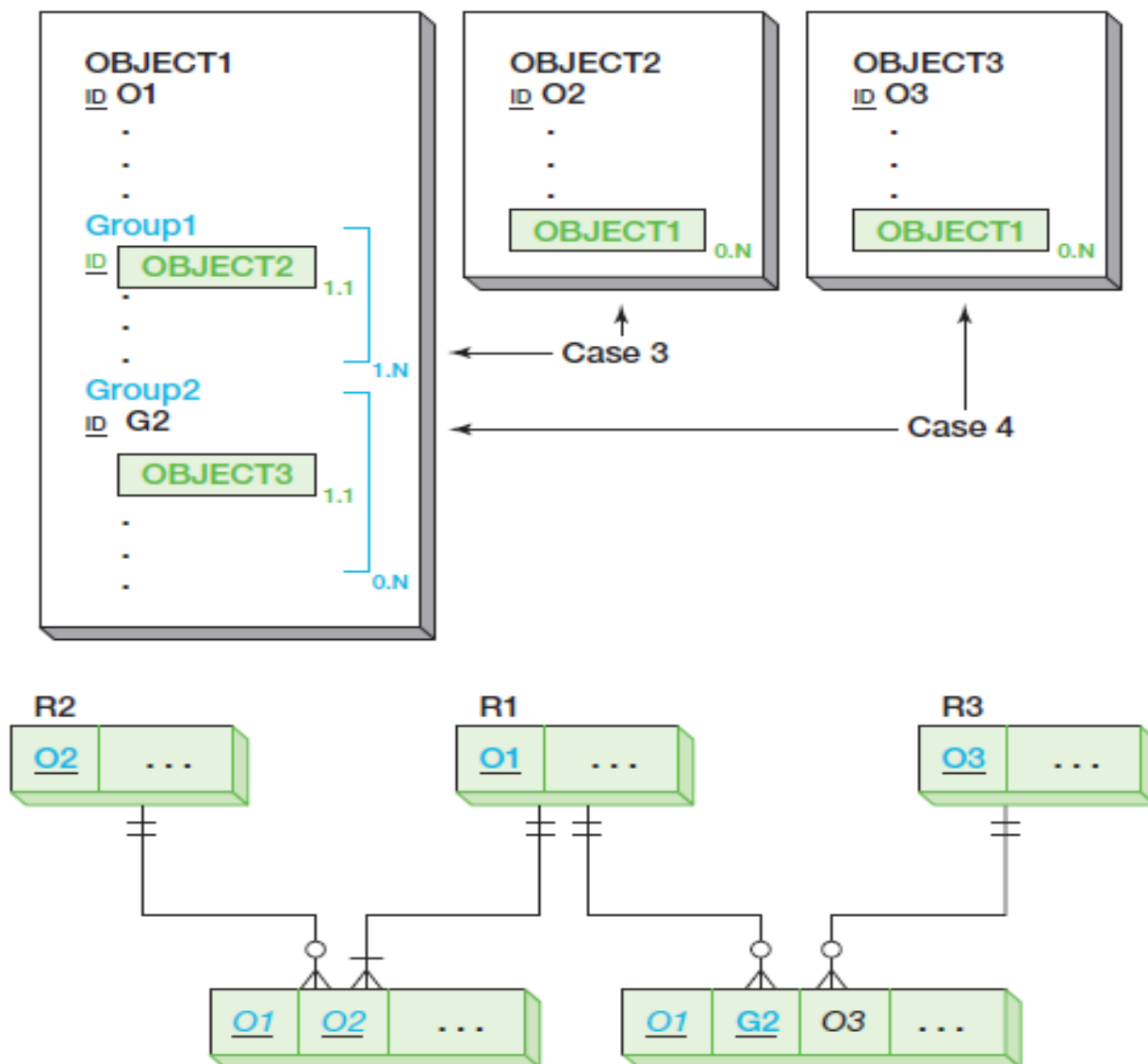


(c) ITEM in One LineItem of (Possibly) Many ORDERS



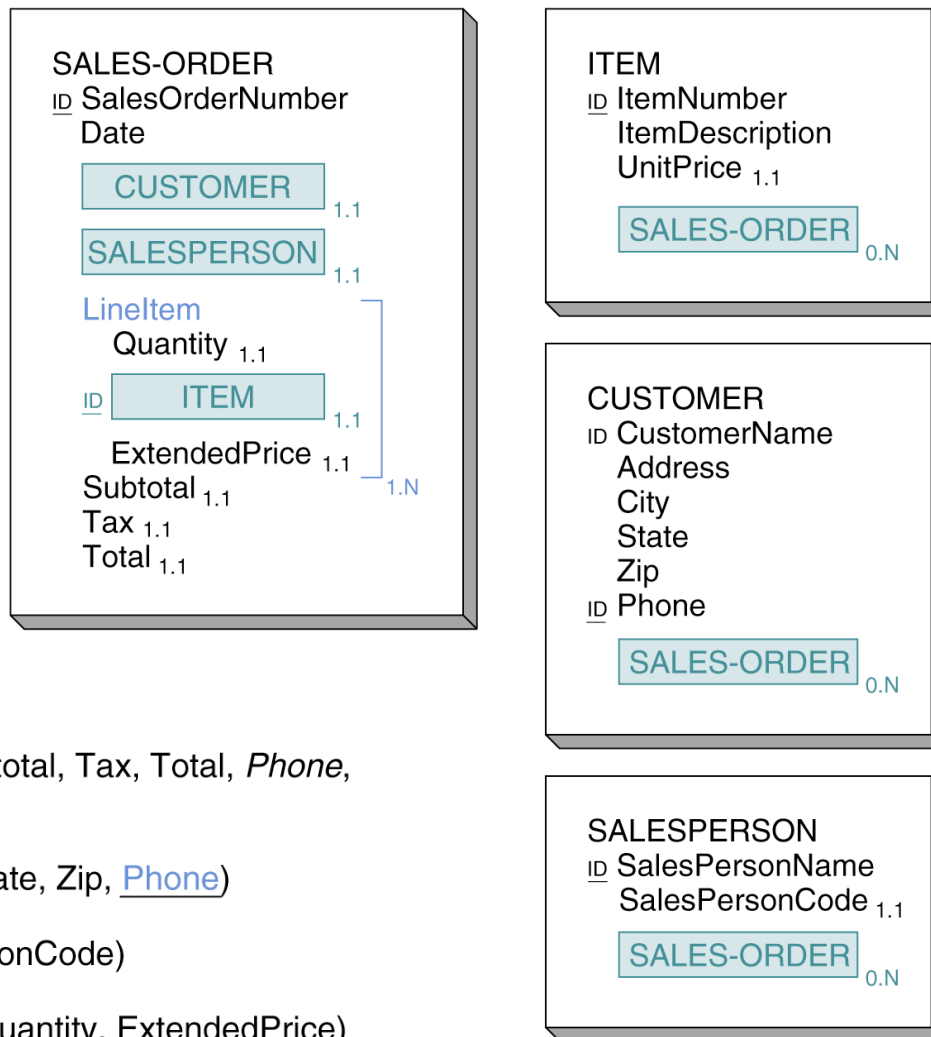
(d) ITEM in (Possibly) Many LineItems of (Possibly) Many ORDERS

Гибридный объект: схема преобразования



Гибридный объект: пример

- гибридный объект (hybrid object) – объект, содержащий один или более групповой атрибут, содержащий в себе объектный атрибут



SALES-ORDER (SalesOrderNumber, Date, Subtotal, Tax, Total, *Phone*,
SalespersonName)

CUSTOMER (CustomerName, Address, City, State, Zip, Phone)

SALESPERSON (SalesPersonName, SalesPersonCode)

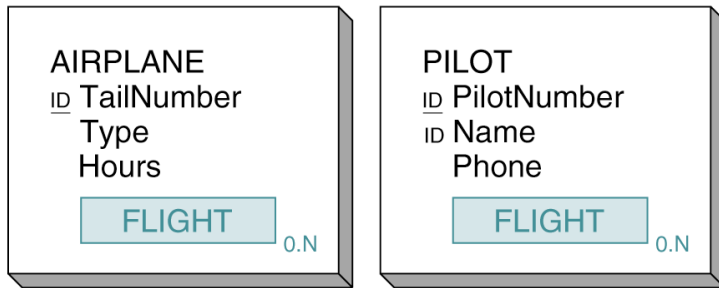
LINE-ITEM (*SalesOrderNumber*, *ItemNumber*, Quantity, ExtendedPrice)

ITEM (ItemNumber, ItemDescription, UnitPrice)

(a)

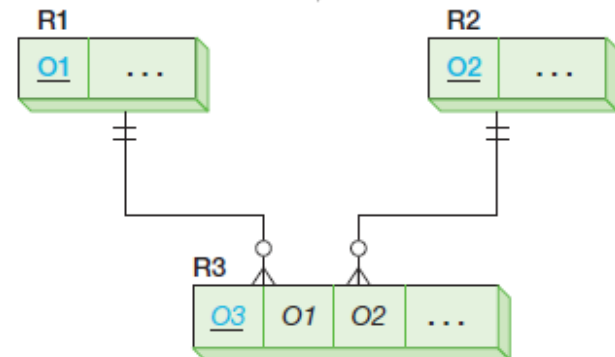
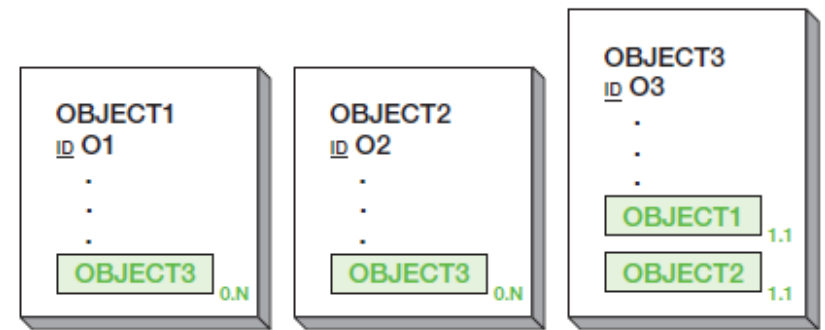
(b)

Ассоциативный объект



(a)

- ассоциативный объект (association object) – объект, сопоставляющий два или более объектов, и содержащий дополнительную информацию о таком сопоставлении



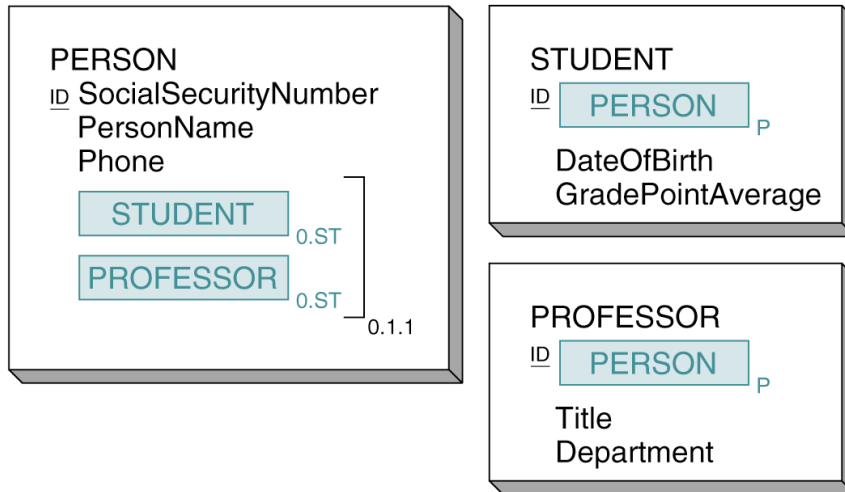
AIRPLANE (TailNumber, TypeHours)

PILOT (PilotNumber, Name, Phone)

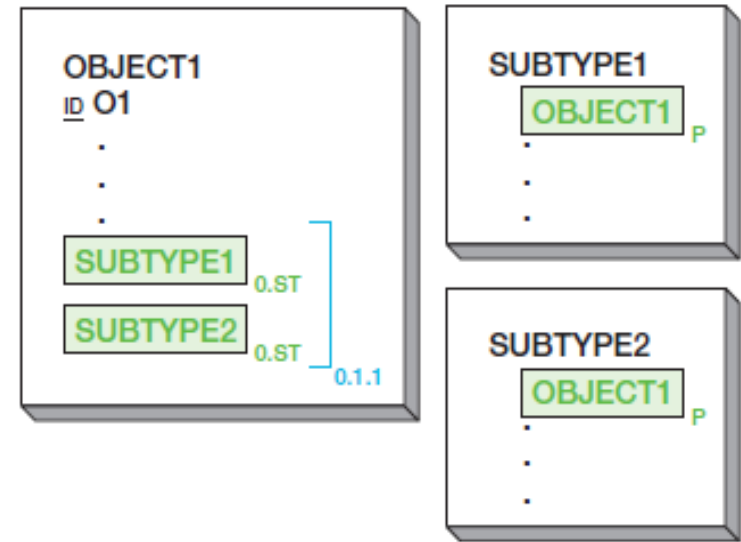
FLIGHT (FlightNumber, Date, OriginationCity, DestinationCity,
TailNumber, PilotNumber)

(b)

Родитель / подтип

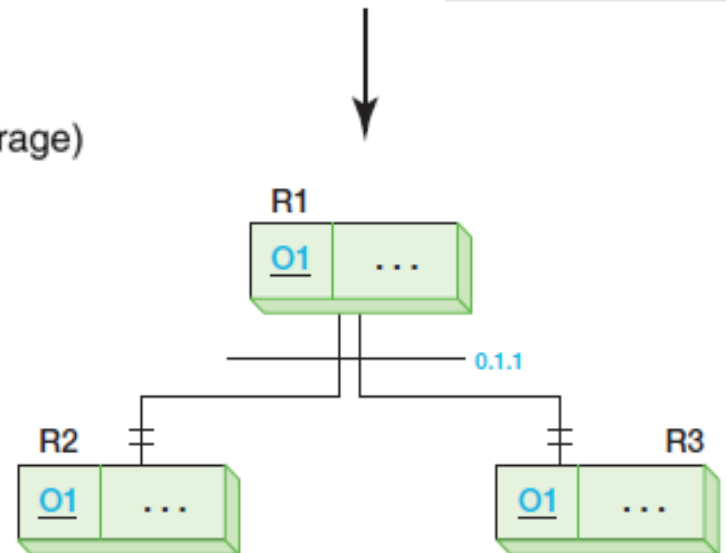


(a)

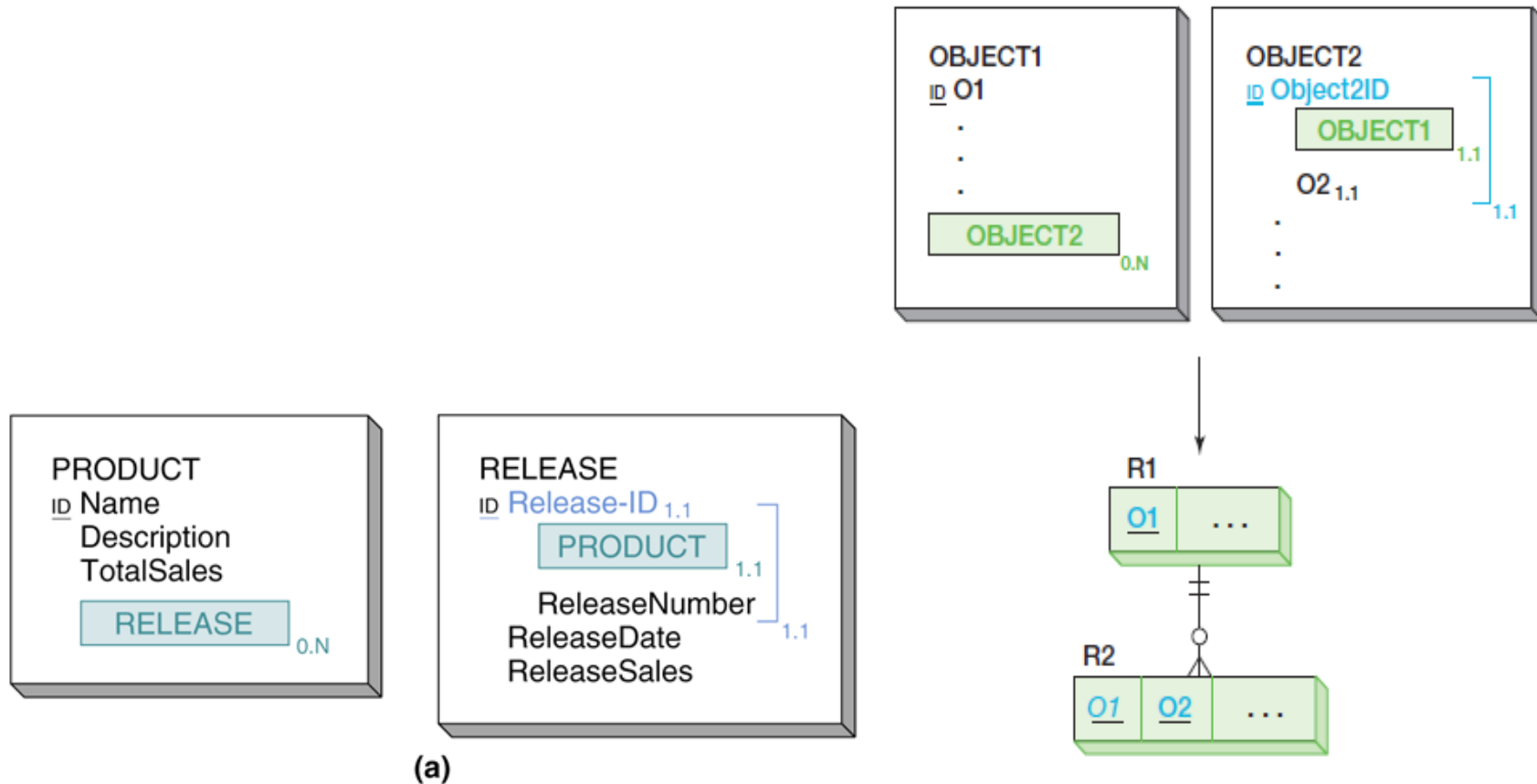


PERSON (SocialSecurityNumber, PersonName, Phone)
 STUDENT (SocialSecurityNumber, DateOfBirth, GradePointAverage)
 PROFESSOR (SocialSecurityNumber, Title, Department)

(b)



Прототип / экземпляр



PRODUCT (Name, Description, TotalSales)

RELEASE (Name, ReleaseNumber, ReleaseDate, ReleaseSales)

(b)

Вопросы к экзамену

- Преобразование модели семантических объектов.
Типы объектов: простой, составной, сложный.
- Преобразование модели семантических объектов.
Типы объектов: гибридный, ассоциативный,
«родитель-подтип», «прототип-экземпляр».

Лабораторная работа №4.

Преобразование модели семантических объектов в реляционную модель

1. Преобразовать модель семантических объектов, созданную в лабораторной работе №2, в реляционную модель согласно процедуре преобразования.
2. Сопоставить результаты проектирования с использованием модели «сущность-связь» и модели семантических объектов (лабораторные работы №№3,4).
3. Обосновать различия результатов, выявить и исправить ошибки проектирования.

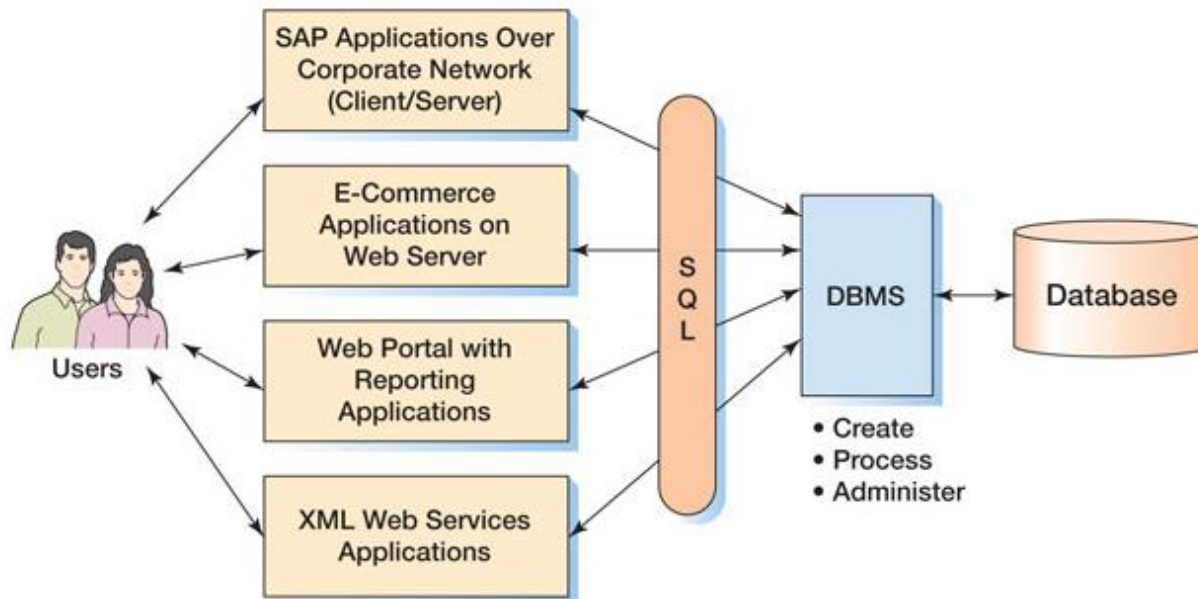
Базы данных

Тема 5

Язык SQL: Операторы DML

Язык SQL

- язык SQL (*Structured Query Language* - «язык структурированных запросов») – язык для создания, модификации и управления данными в реляционной СУБД;
- является декларативным языком (с процедурными расширениями);
- основан на реляционной алгебре и реляционном исчислении, при этом, однако, не полностью им соответствует.



Стандарты языка SQL

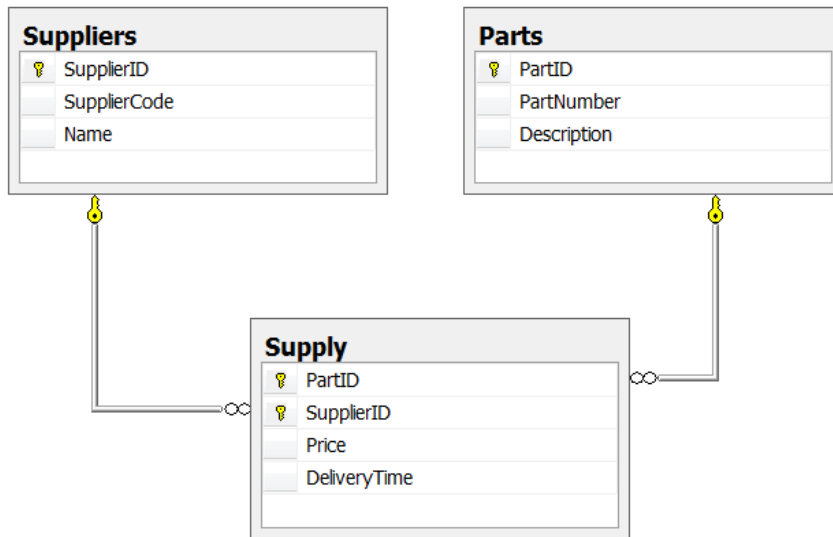
- язык был разработан в конце 1970-х в IBM Corporation;
- был принят в качестве национального стандарта (American National Standards Institute, ANSI) в 1986 году;
- третья версия стандарта (1992) известна как **SQL-92**;
- новые версии стандарта включают в себя элементы поддержки XML, JSON, объектно-ориентированных концепций и графовых запросов;
- все версии стандарта: SQL:1986 (1989, 1992, 1996, 1999, 2003, 2006, 2008, 2011, 2016), SQL:2023 (ISO/IEC 9075:2023).

Категории операторов SQL

- операторы определения данных (*Data Definition Language, DDL*):
 - CREATE;
 - ALTER;
 - DROP;
- операторы манипуляции данными (*Data Manipulation Language, DML*):
 - SELECT;
 - INSERT;
 - UPDATE;
 - DELETE;
- операторы определения доступа к данным (*Data Control Language, DCL*):
 - GRANT;
 - REVOKE;
 - DENY;
- операторы управления транзакциями (*Transaction Control Language, TCL*):
 - COMMIT;
 - ROLLBACK;
 - SAVEPOINT.

СУБД SQL Server 2025

Пример базы данных



SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
2	AVTORIF	NULL
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

PartID	SupplierID	Price	DeliveryTime
2	3	55	2
4	2	200	35
4	3	100	5
4	5	200	60

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

Выборка данных: **SELECT** (**SELECT statement**)

- FROM – источник выборки;
- WHERE – условие выборки;
- HAVING – условие группировки;
- GROUP BY – список группировки;
- ORDER BY – сортировка результатов;
- UNION, EXCEPT, INTERCEPT –
комбинирование результатов нескольких
выборок.

SELECT: список выбора и указание источника

- список выбора:
 - список всех полей;
 - отдельные поля;
 - определение порядка отдельных полей;
 - псевдонимы для полей;
 - использование выражений и выражений с псевдонимами;
- источник (FROM...):
 - единственная таблица;
 - несколько таблиц;
 - псевдонимы для таблиц;
- фильтрация уникальных строк: DISTINCT.

SELECT: указание списка полей

```
SELECT [PartID],[PartNumber],[Description]  
FROM [Parts]
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

SELECT: указание списка полей (2)

```
SELECT [PartNumber],[Description]  
FROM [Parts]
```

PartNumber	Description
MZ311785	Фильтр воздушный
ME013343	Фильтр масляный
1230A046	Фильтр масляный
ME215002	Фильтр масляный
MB220900	Фильтр топливный
XB220900	Фильтр топливный
MB891677	Амортизатор задний
MR389546	Колодки передние
MZ690170	Колодки задние

SELECT: указание списка полей (3)

```
SELECT [Description], [PartID],[PartNumber]  
FROM [Parts]
```

Description	PartID	PartNumber
Фильтр воздушный	1	MZ311785
Фильтр масляный	2	ME013343
Фильтр масляный	3	1230A046
Фильтр масляный	4	ME215002
Фильтр топливный	6	MB220900
Фильтр топливный	7	XB220900
Амортизатор задний	8	MB891677
Колодки передние	9	MR389546
Колодки задние	10	MZ690170

SELECT: указание списка полей (4)

```
SELECT *  
FROM [Parts]
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

SELECT: псевдонимы полей

```
SELECT [PartID] as [Identity], [PartNumber],[Description]  
FROM [Parts]
```

Identity	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

SELECT: псевдонимы таблиц

```
SELECT a.[PartNumber], a.[Description], c.[SupplierCode], b.[Price],  
       b.[DeliveryTime]  
FROM [Parts] as a, [Supply] as b, [Suppliers] as c  
WHERE a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID]
```

PartNumber	Description	SupplierCode	Price	DeliveryTime
ME013343	Фильтр масляный	UAE	55	2
ME215002	Фильтр масляный	AVTORIF	200	35
ME215002	Фильтр масляный	UAE	100	5
ME215002	Фильтр масляный	GER	200	60

SELECT: уникальность строк в выборке

```
SELECT a.[PartNumber], a.[Description]
FROM [Parts] as a, [Supply] as b, [Suppliers] as c
WHERE a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID]
```

```
SELECT DISTINCT a.[PartNumber], a.[Description]
FROM [Parts] as a, [Supply] as b, [Suppliers] as c
WHERE a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID]
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

SELECT: условие выбора записей и сортировка

- условие выбора записей (WHERE...) – задается с помощью логического выражения, формируемого с помощью:
 - операторов сравнения (>, <, =, <>, <=, >=);
 - логических операторов (AND, OR, NOT);
 - задания диапазона (*value* BETWEEN *min* AND *max*);
 - условия принадлежности множеству значений (*value* IN (*list*), список может формироваться с помощью вложенных запросов);
 - условия непустого множества строк, возвращаемого в результате выполнения подзапроса (EXISTS (*subquery*));
 - задания шаблона для строковых значений (*value* LIKE '...')
 - символ % является шаблоном подстроки;
 - символ _ является шаблоном единственного символа;
 - [],[^] позволяют указать/ограничить допустимый набор/диапазон символов;
 - определения наличия неопределенного значения (IS NULL | IS NOT NULL)
- сортировка (ORDER BY):
 - ASC – по возрастанию (по умолчанию);
 - DESC – по убыванию.

SELECT: сортировка строк в выборке

```
SELECT a.[PartNumber],  
       a.[Description]  
FROM [Parts] as a
```

PartNumber	Description
MZ311785	Фильтр воздушный
ME013343	Фильтр масляный
1230A046	Фильтр масляный
ME215002	Фильтр масляный
MB220900	Фильтр топливный
XB220900	Фильтр топливный
MB891677	Амортизатор задний
MR389546	Колодки передние
MZ690170	Колодки задние

```
SELECT a.[PartNumber],  
       a.[Description]  
FROM [Parts] as a  
ORDER BY a.[PartNumber]
```

PartNumber	Description
1230A046	Фильтр масляный
MB220900	Фильтр топливный
MB891677	Амортизатор задний
ME013343	Фильтр масляный
ME215002	Фильтр масляный
MR389546	Колодки передние
MZ311785	Фильтр воздушный
MZ690170	Колодки задние
XB220900	Фильтр топливный

SELECT: порядок сортировки

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a  
ORDER BY a.[PartNumber] asc
```

PartNumber	Description
1230A046	Фильтр масляный
MB220900	Фильтр топливный
MB891677	Амортизатор задний
ME013343	Фильтр масляный
ME215002	Фильтр масляный
MR389546	Колодки передние
MZ311785	Фильтр воздушный
MZ690170	Колодки задние
XB220900	Фильтр топливный

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a  
ORDER BY a.[PartNumber] desc
```

PartNumber	Description
XB220900	Фильтр топливный
MZ690170	Колодки задние
MZ311785	Фильтр воздушный
MR389546	Колодки передние
ME215002	Фильтр масляный
ME013343	Фильтр масляный
MB891677	Амортизатор задний
MB220900	Фильтр топливный
1230A046	Фильтр масляный

SELECT: условия выбора строк (between)

```
SELECT a.[PartNumber], a.[Description], c.[SupplierCode], b.[Price], b.[DeliveryTime]
FROM [Parts] as a, [Supply] as b, [Suppliers] as c
WHERE (a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID])
```

```
SELECT a.[PartNumber], a.[Description], c.[SupplierCode], b.[Price], b.[DeliveryTime]
FROM [Parts] as a, [Supply] as b, [Suppliers] as c
WHERE (a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID])
and b.Price between 12 and 100
```

PartNumber	Description	SupplierCode	Price	DeliveryTime
ME013343	Фильтр масляный	UAE	55	2
ME215002	Фильтр масляный	AVTORIF	200	35
ME215002	Фильтр масляный	UAE	100	5
ME215002	Фильтр масляный	GER	200	60

PartNumber	Description	SupplierCode	Price	DeliveryTime
ME013343	Фильтр масляный	UAE	55	2
ME215002	Фильтр масляный	UAE	100	5

SELECT: условия выбора строк (in)

```
SELECT a.[PartID], a.[PartNumber],  
       a.[Description]  
FROM [Parts] as a
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

```
SELECT * FROM [Supply]
```

PartID	SupplierID	Price	DeliveryTime
2	3	55	2
4	2	200	35
4	3	100	5
4	5	200	60

```
SELECT a.[PartNumber],  
       a.[Description]  
FROM [Parts] as a  
WHERE a.[PartID] in (select PartID  
                     from [Supply])
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

SELECT: условия выбора строк (exists)

```
SELECT a.[PartID], a.[PartNumber],  
       a.[Description]  
FROM [Parts] as a
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

```
SELECT * FROM [Supply]
```

PartID	SupplierID	Price	DeliveryTime
2	3	55	2
4	2	200	35
4	3	100	5
4	5	200	60

```
SELECT a.[PartNumber],  
       a.[Description]  
FROM [Parts] as a  
WHERE exists (select PartID from  
          [Supply] as b where a.[PartID] =  
          b.[PartID])
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

SELECT: подзапрос со сложным условием

```
SELECT a.[PartID], a.[PartNumber],  
       a.[Description]  
FROM [Parts] as a
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a  
WHERE exists (  
    select PartID  
    from [Parts] as b  
    where (  
        (a.[PartID] <> b.[PartID]) and  
        (a.[Description] = b.[Description])  
    ))
```

PartNumber	Description
ME013343	Фильтр масляный
1230A046	Фильтр масляный
ME215002	Фильтр масляный
MB220900	Фильтр топливный
XB220900	Фильтр топливный

SELECT: условия выбора строк (like)

```
SELECT a.[PartID],  
       a.[PartNumber], a.[Description]  
FROM [Parts] as a
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

```
SELECT a.[PartNumber],  
       a.[Description]  
FROM [Parts] as a  
WHERE  
       a.[PartNumber] like '%0%'
```

PartNumber	Description
ME013343	Фильтр масляный
1230A046	Фильтр масляный
ME215002	Фильтр масляный
MB220900	Фильтр топливный
XB220900	Фильтр топливный
MZ690170	Колодки задние

SELECT: условия выбора строк (функции)

```
SELECT * FROM [Suppliers]
```

```
SELECT * FROM [Suppliers]  
WHERE LEN(SupplierCode) = 3
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
2	AVTORIF	NULL
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

SupplierID	SupplierCode	Name
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

SELECT: неопределенные значения (NULL)

SELECT *	SupplierID	SupplierCode	Name
FROM suppliers	1	EXIST	ООО Эксист
	2	AVTORIF	NULL
	3	UAE	Эмираты
	5	GER	Германия
	6	JAP	Япония

set ansi_nulls off	SupplierID	SupplierCode	Name
SELECT * from suppliers	2	AVTORIF	NULL

where Name = null	SupplierID	SupplierCode	Name
SELECT * from suppliers	1	EXIST	ООО Эксист
where Name <> null	3	UAE	Эмираты
	5	GER	Германия
	6	JAP	Япония

set ansi_nulls on	SupplierID	SupplierCode	Name
SELECT * from suppliers			
where Name = null			

SELECT * from suppliers	SupplierID	SupplierCode	Name
where Name <> null			

Значения NULL и трехзначная логика (3VL, three-valued logic)

p	q	$p \text{ OR } q$	$p \text{ AND } q$	$p = q$
True	True	True	True	True
True	False	True	False	False
True	Unknown	True	Unknown	Unknown
False	True	True	False	False
False	False	False	False	True
False	Unknown	Unknown	False	Unknown
Unknown	True	True	Unknown	Unknown
Unknown	False	Unknown	False	Unknown
Unknown	Unknown	Unknown	Unknown	Unknown

p	NOT p
True	False
False	True
Unknown	Unknown

SELECT: неопределенные значения (NULL)

SELECT *	SupplierID	SupplierCode	Name
FROM suppliers	1	EXIST	ООО Эксист
	2	AVTORIF	NULL
	3	UAE	Эмираты
	5	GER	Германия
	6	JAP	Япония

SELECT *	SupplierID	SupplierCode	Name
FROM suppliers	2	AVTORIF	NULL
WHERE Name IS NULL			

SELECT *	SupplierID	SupplierCode	Name
FROM suppliers	1	EXIST	ООО Эксист
WHERE Name IS NOT NULL	3	UAE	Эмираты
	5	GER	Германия
	6	JAP	Япония

SELECT: группировка, условия группировки и функции агрегирования

- список полей, по которым осуществляется группировка:

GROUP BY ...

- HAVING позволяет задавать условия с функциями агрегирования - данное условие выполняется после отбора записей по условию WHERE;
- функции агрегирования: COUNT, MIN, MAX, SUM, AVG
 - все поля из списка выборки, не задействованные в функциях агрегирования, должны присутствовать в списке GROUP BY;
 - с помощью COUNT можно подсчитывать количество строк в таблице (COUNT(*)) или значений в столбце

SELECT: подсчет строк и значений (count)

```
select *  
from parts
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

```
select count(*)  
from parts
```

(No column name)
9

```
select count(PartID) as cnt  
from parts
```

cnt
9

SELECT: подсчет строк и значений (2)

select * from suppliers

SupplierID

SupplierCode

Name

1

EXIST

ООО Эксист

2

AVTORIF

NULL

3

UAE

Эмираты

5

GER

Германия

6

JAP

Япония

select count(*)

(No column name)

from suppliers

5

select count(Name)

(No column name)

from suppliers

4

SELECT: подсчет уникальных значений

```
select *  
from parts
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

```
select  
    count(Description)  
from parts
```

(No column name)
9

```
select  
    count(distinct  
        Description)  
from parts
```

(No column name)
6

SELECT: группировка (group by)

```
select * from parts
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

```
select Description, count(*)  
      as cnt
```

```
from parts
```

```
GROUP BY Description
```

Description	cnt
Амортизатор задний	1
Колодки задние	1
Колодки передние	1
Фильтр воздушный	1
Фильтр масляный	3
Фильтр топливный	2

SELECT: функции агрегирования

```
select * from supply
```

```
select partID, MIN(Price) as minPrice  
from supply  
group by partID
```

PartID	SupplierID	Price	DeliveryTime
2	3	55	2
4	2	200	35
4	3	100	5
4	5	200	60

partID	minPrice
2	55
4	100

SELECT: условия на группы (having)

PartID	SupplierID	Price	DeliveryTime
2	3	55	2
4	2	200	35
4	3	100	5
4	5	200	60

```
select partID, Min(Price)
from supply
group by partID
HAVING count(price) > 1
```

partId	(No column name)
4	100

```
select partID, Min(Price)
from supply
where price < 200
group by partid
having count(price) > 1
```

partId	(No column name)
--------	------------------

SELECT: комбинация результатов нескольких выборок

- UNION – объединение результатов с исключением повторяющихся записей;
- UNION ALL - объединение результатов без исключения повторяющихся записей;
- EXCEPT – исключение результатов второй выборки из результатов первой;
- INTERSECT – получение пересечения результатов выборок.

Объединение результатов выборок без фильтрации повторяющихся записей (UNION ALL)

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a, [Supply] as b  
where a.[PartID] = b.[PartID]
```

UNION ALL

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a, [Supply] as b  
where a.[PartID] = b.[PartID]
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный
ME013343	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный

Объединение результатов выборок с фильтрацией повторяющихся записей: UNION

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a, [Supply] as b  
where a.[PartID] = b.[PartID]
```

UNION

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a, [Supply] as b  
where a.[PartID] = b.[PartID]
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

Исключение результатов выборки: EXCEPT

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a, [Supply] as b  
where a.[PartID] = b.[PartID]
```

EXCEPT

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a, [Supply] as b  
where a.[PartID] = b.[PartID]
```

PartNumber	Description
------------	-------------

Пересечение результатов выборок: INTERSECT

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a, [Supply] as b  
where a.[PartID] = b.[PartID]
```

INTERSECT

```
SELECT a.[PartNumber], a.[Description]  
FROM [Parts] as a, [Supply] as b  
where a.[PartID] = b.[PartID]
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

SELECT: соединения (JOINS)

- внутреннее соединение:
 - INNER JOIN (JOIN)
- внешние соединения:
 - LEFT OUTER JOIN (LEFT JOIN)
 - RIGHT OUTER JOIN (RIGHT JOIN)
 - FULL OUTER JOIN (FULL JOIN)

Внутреннее соединение: INNER JOIN

```
select a.partid, a.partnumber, a.Description,  
b.Price, b.DeliveryTime  
from parts as a, supply as b  
where a.partId = b.partid
```

```
select a.partid, a.partnumber, a.Description,  
b.Price, b.DeliveryTime  
from parts as a JOIN supply as b  
ON a.partId = b.partid
```

partid	partnumber	Description	Price	DeliveryTime
2	ME013343	Фильтр масляный	55	2
4	ME215002	Фильтр масляный	200	35
4	ME215002	Фильтр масляный	100	5
4	ME215002	Фильтр масляный	200	60

partid	partnumber	Description	Price	DeliveryTime
2	ME013343	Фильтр масляный	55	2
4	ME215002	Фильтр масляный	200	35
4	ME215002	Фильтр масляный	100	5
4	ME215002	Фильтр масляный	200	60

Внутреннее соединение: INNER JOIN (2)

```
select a.partid, a.Price, a.DeliveryTime,  
b.partid, b.Price, b.DeliveryTime  
from supply as a INNER JOIN supply as b  
ON a.partid = b.partid
```

partid	Price	DeliveryTime	partid	Price	DeliveryTime
2	55	2	2	55	2
4	100	5	4	200	35
4	200	35	4	200	35
4	200	60	4	200	35
4	200	35	4	100	5
4	100	5	4	100	5
4	200	60	4	100	5
4	200	35	4	200	60
4	100	5	4	200	60
4	200	60	4	200	60

Левое соединение: LEFT OUTER JOIN

```
select a.partid, a.partnumber, a.Description,  
b.Price, b.DeliveryTime  
from parts as a JOIN supply as b ON a.partId = b.partid
```

```
select a.partid, a.partnumber, a.Description,  
b.Price, b.DeliveryTime  
from parts as a LEFT JOIN supply as b ON a.partId = b.partid
```

partid	partnumber	Description	Price	DeliveryTime
2	ME013343	Фильтр масляный	55	2
4	ME215002	Фильтр масляный	200	35
4	ME215002	Фильтр масляный	100	5
4	ME215002	Фильтр масляный	200	60

partid	partnumber	Description	Price	DeliveryTime
1	MZ311785	Фильтр воздушный	NULL	NULL
2	ME013343	Фильтр масляный	55	2
3	1230A046	Фильтр масляный	NULL	NULL
4	ME215002	Фильтр масляный	200	35
4	ME215002	Фильтр масляный	100	5
4	ME215002	Фильтр масляный	200	60
6	MB220900	Фильтр топливный	NULL	NULL
7	XB220900	Фильтр топливный	NULL	NULL
8	MB891677	Амортизатор задний	NULL	NULL
9	MR389546	Колодки передние	NULL	NULL
10	MZ690170	Колодки задние	NULL	NULL

Левое соединение (2)

```
select a.*, b.SupplierID
from parts as a left join supply as b on a.partId = b.partid
```

```
select a.partnumber, count(b.price) as cnt          -- count(distinct b.price) as cnt
from parts as a left join supply as b on a.partId = b.partid
group by a.partnumber
```

PartID	PartNumber	Description	SupplierID	Partnumber	cnt
1	MZ311785	Фильтр воздушный	NULL		
2	ME013343	Фильтр масляный	3	1230A046	0
3	1230A046	Фильтр масляный	NULL	MB220900	0
4	ME215002	Фильтр масляный	2	MB891677	0
4	ME215002	Фильтр масляный	3	ME013343	1
4	ME215002	Фильтр масляный	5	ME215002	3
6	MB220900	Фильтр топливный	NULL	MR389546	0
7	XB220900	Фильтр топливный	NULL	MZ311785	0
8	MB891677	Амортизатор задний	NULL	MZ690170	0
9	MR389546	Колодки передние	NULL	XB220900	0
10	MZ690170	Колодки задние	NULL		

Правое соединение: RIGHT OUTER JOIN

```
select a.partid, a.partnumber, b.Price, b.DeliveryTime, b.SupplierID  
from parts as a JOIN supply as b ON a.partId = b.partid
```

```
select b.PartID, b.Price, b.DeliveryTime, c.SupplierCode  
from supply as b RIGHT JOIN suppliers as c ON b.supplierId = c.supplierid
```

partid	partnumber	Price	DeliveryTime	SupplierID
2	ME013343	55	2	3
4	ME215002	200	35	2
4	ME215002	100	5	3
4	ME215002	200	60	5

PartID	Price	DeliveryTime	SupplierCode
NULL	NULL	NULL	EXIST
4	200	35	AVTORIF
2	55	2	UAE
4	100	5	UAE
4	200	60	GER
NULL	NULL	NULL	JAP

Вставка строк: INSERT (INSERT statement)

- вставка единственной строки:
 - с указанием всех значений;
 - с указанием некоторых значений;
 - с указанием значений в порядке, отличающемся от порядка полей таблицы;
- вставка нескольких строк:
 - с явным указанием значений;
 - с использованием результатов выборки из другой таблицы.

INSERT: вставка с указанием всех значений

```
INSERT INTO Production.UnitMeasure  
VALUES (N'FT', N'Feet', '20080414');  
GO
```

```
INSERT INTO Production.UnitMeasure  
VALUES (N'FT2', N'Square Feet ', '20080923'),  
        (N'Y', N'Yards', '20080923'),  
        (N'Y3', N'Cubic Yards', '20080923');  
GO
```

```
INSERT INTO Production.UnitMeasure (Name, UnitMeasureCode,  
    ModifiedDate)  
VALUES (N'Square Yards', N'Y2', GETDATE())  
GO
```

INSERT: вставка в таблицу со значениями по умолчанию

```
IF OBJECT_ID ('dbo.T1', 'U') IS NOT NULL  
DROP TABLE dbo.T1;  
GO
```

```
CREATE TABLE dbo.T1 (  
    column_1 AS 'Computed column ' + column_2,  
    column_2 varchar(30) CONSTRAINT default_name DEFAULT ('my column default'),  
    column_3 rowversion,  
    column_4 varchar(40) NULL  
);  
GO
```

```
INSERT INTO dbo.T1 (column_4) VALUES ('Explicit value');  
INSERT INTO dbo.T1 (column_2, column_4) VALUES ('Explicit value', 'Explicit value');  
INSERT INTO dbo.T1 (column_2) VALUES ('Explicit value');  
INSERT INTO T1 DEFAULT VALUES;  
GO
```

INSERT: вставка в таблицу с автоинкрементным полем (IDENTITY)

```
IF OBJECT_ID ('dbo.T1', 'U') IS NOT NULL  
DROP TABLE dbo.T1;  
GO
```

```
CREATE TABLE dbo.T1 (  
    column_1 int IDENTITY,  
    column_2 VARCHAR(30)  
);  
GO
```

```
INSERT T1 VALUES ('Row #1');  
INSERT T1 (column_2) VALUES ('Row #2');  
GO
```

```
SET IDENTITY_INSERT T1 ON;  
GO  
INSERT INTO T1 (column_1, column_2) VALUES (-99, 'Explicit identity value');  
GO
```

INSERT: вставка результатов выборки (SELECT)

```
IF OBJECT_ID ('dbo.EmployeeSales', 'U') IS NOT NULL
DROP TABLE dbo.EmployeeSales;
GO
```

```
CREATE TABLE dbo.EmployeeSales (
    DataSource varchar(20) NOT NULL,
    BusinessEntityID varchar(11) NOT NULL,
    LastName varchar(40) NOT NULL,
    SalesDollars money NOT NULL
);
GO
```

```
INSERT INTO dbo.EmployeeSales
SELECT 'SELECT', sp.BusinessEntityID, c.LastName, sp.SalesYTD
FROM Sales.SalesPerson AS sp
    INNER JOIN Person.Person AS c ON sp.BusinessEntityID = c.BusinessEntityID
WHERE sp.BusinessEntityID LIKE '2%'
ORDER BY sp.BusinessEntityID, c.LastName;
GO
```

Изменение строк: UPDATE (UPDATE statement)

```
UPDATE Person.Address  
SET ModifiedDate = GETDATE();           -- SET ModifiedDate = DEFAULT;  
GO
```

```
UPDATE Sales.SalesPerson  
SET Bonus = 6000, CommissionPct = .10, SalesQuota = NULL;  
GO
```

```
UPDATE Production.Product  
SET Color = N'Metallic Red'  
WHERE Name LIKE N'Road-250%' AND Color = N'Red';  
GO
```

```
UPDATE Production.Product  
SET ListPrice = ListPrice * 2;  
GO
```

```
DECLARE @NewPrice int = 10;  
UPDATE Production.Product  
SET ListPrice += @NewPrice  
GO
```

Изменение строк: UPDATE (2)

```
UPDATE TOP (10) HumanResources.Employee  
SET VacationHours = VacationHours * 1.25 ;  
GO
```

```
UPDATE HumanResources.Employee  
SET VacationHours = VacationHours + 8  
    FROM (SELECT TOP 10 BusinessEntityID  
          FROM HumanResources.Employee  
          ORDER BY HireDate ASC) AS th  
    WHERE HumanResources.Employee.BusinessEntityID = th.BusinessEntityID;  
GO
```

```
UPDATE Sales.SalesPerson  
SET SalesYTD = SalesYTD +  
    (SELECT SUM(so.SubTotal)  
     FROM Sales.SalesOrderHeader AS so  
     WHERE so.OrderDate = (SELECT MAX(OrderDate)  
                           FROM Sales.SalesOrderHeader AS so2  
                           WHERE so2.SalesPersonID = so.SalesPersonID)  
     AND Sales.SalesPerson.BusinessEntityID = so.SalesPersonID  
     GROUP BY so.SalesPersonID);  
GO
```

Удаление строк: DELETE (DELETE statement)

```
DELETE FROM Sales.SalesPersonQuotaHistory;  
GO  
-- TRUNCATE TABLE Sales.SalesPersonQuotaHistory;  
-- GO
```

```
DELETE FROM Production.ProductCostHistory  
WHERE StandardCost > 1000.00;  
GO
```

```
DELETE FROM Sales.SalesPersonQuotaHistory  
WHERE BusinessEntityID IN  
    (SELECT BusinessEntityID FROM Sales.SalesPerson WHERE SalesYTD > 2500000.00);  
GO
```

```
DELETE FROM Sales.SalesPersonQuotaHistory  
FROM Sales.SalesPersonQuotaHistory AS spqh  
    INNER JOIN Sales.SalesPerson AS sp ON spqh.BusinessEntityID = sp.BusinessEntityID  
    WHERE sp.SalesYTD > 2500000.00;  
GO
```


Вопросы к экзамену

- Ядро СУБД. Язык SQL: стандарты, категории операторов.
- Выборка данных: структура оператора SELECT. Указание списка выбора и источника. Использование псевдонимов.
- Выборка данных: условие отбора записей и сортировка. Значение NULL и трехзначная логика.
- Выборка данных: группировка, условия группировки и функции агрегирования.
- Выборка данных: комбинация результатов нескольких выборок и их сортировка.
- Выборка данных: соединения и их типы.
- Вставка данных: оператор INSERT.
- Обновление данных: оператор UPDATE. Условие отбора записей. Значение NULL и трехзначная логика.
- Удаление данных: оператор DELETE. Условие отбора записей. Значение NULL и трехзначная логика.

Базы данных

Тема 6

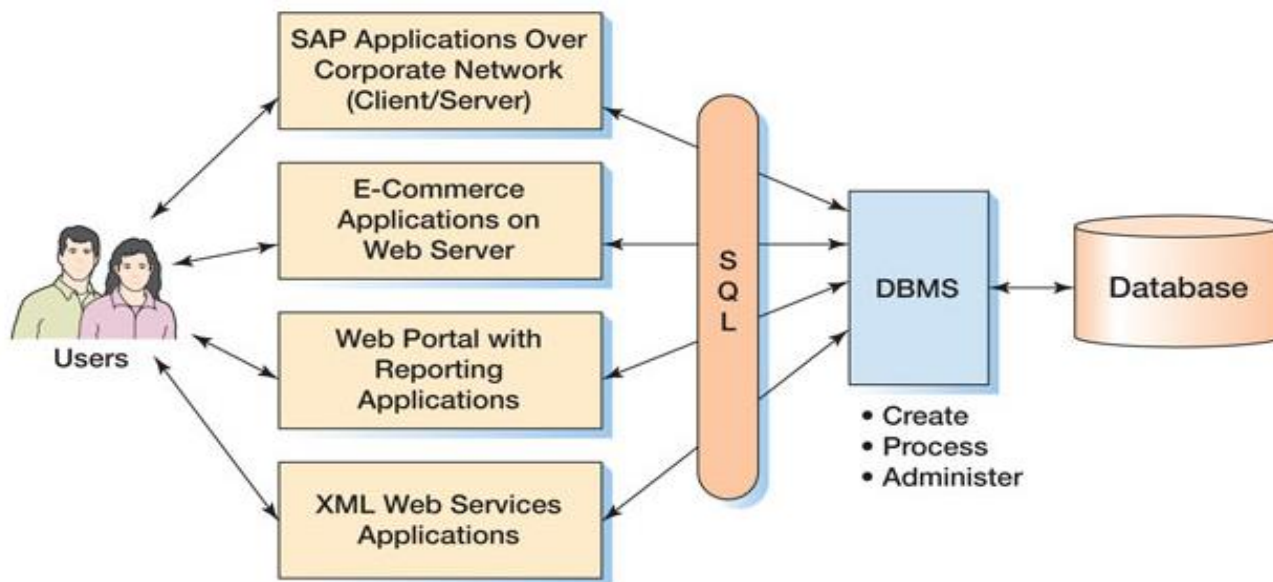
СУБД: экземпляры, организация БД.

База данных

- база данных – самодокументированный набор интегрированных записей;
- самодокументированность: база данных содержит не только данные, но и их описание – метаданные (metadata):
 - возможность определить структуру и содержимое базы данных путем обращения к самой базе данных;
 - изменение структуры в первую очередь затрагивает метаданные, и, следовательно, ограничивает количество программ, на которые влияют эти изменения;
- интегрированность:
 - байты → поля → записи / объекты → файлы;
 - метаданные;
 - вспомогательные структуры данных (например, индексы);
 - метаданные приложения (дополнительные данные для построения форм и отчетов);

СУБД

- синонимичные термины:
 - система обработки баз данных (*database processing system*);
 - СУБД - система управления базами данных (*DBMS, database management system*);
- СУБД - является посредником между приложением и данными:
 - изоляция от физической организации хранения данных: реализация операций CRUD (*create, read, update, delete*) посредством языка запросов (*query language*) или программного интерфейса (*API, application programming interface*);
 - данные интегрированы;
 - уменьшение дублирования данных;
 - независимость программ от форматов файлов;
 - возможность гибкого выбора формы представления данных;



Компоненты современных СУБД

- ядро СУБД (*database engine*) обеспечивает хранение, обработку и защиту данных:
 - преобразование запросов в действия над данными;
 - управление совместным доступом к данным (транзакциями);
 - разграничение доступа к данным;
 - резервное копирование и восстановление;
- средства интерактивной аналитической обработки (*OLAP – online analytic processing*) и средства интеллектуального анализа данных;
- средства, обеспечивающие интеграцию инструментов и алгоритмов машинного обучения;
- средства интеграции данных (*ETL – extract-transform-load*), обеспечивающие экспорт и импорт данных;
- средства создания и публикации форм и отчетов;
- средства репликации, обеспечивающие копирование и распространение данных и объектов баз данных из одной базы данных в другую с возможностью последующей синхронизации между базами данных для поддержания согласованности данных для поддержания согласованности);
- средства выполнения полнотекстовых запросов к неструктурированным текстовым данным;
- ...

Экземпляр СУБД

- экземпляр СУБД (*instance*) – совокупность ресурсов операционной системы (процессов и памяти), обеспечивающих выполнение функций соответствующей СУБД;
- в общем случае, экземпляр СУБД может быть реализован как:
 - компонент (библиотека), встраиваемый в пользовательское приложение;
 - отдельное серверное приложение;
 - как совокупность серверных приложений, функционирующих на одном или нескольких вычислительных узлах.
- один экземпляр может обслуживать несколько баз данных, и наоборот – несколько экземпляров могут обслуживать одну и ту же базу данных.

Хранение данных

- организация физического хранения данных (как основных, так и служебных) может существенно отличаться в различных СУБД:
 - СУБД в памяти (*in-memory database*):
 - хранение базы данных в оперативной памяти;
 - внешняя память: исключительно для обеспечения отказоустойчивости и долговременного хранения;
 - дисковые СУБД:
 - хранение данных на внешних устройствах (локальных и сетевых дисках и дисковых массивах);
 - подгружаются в оперативную память данные, непосредственно необходимые для выполнения текущих операций;
- каждой базе данных, как правило, соответствует некоторый набор файлов на диске;
- степень изоляции одной базы данных от другой в рамках экземпляра (будет ли организовано хранение разных баз данных в отдельных наборах файлов, или в одних и тех же файлах будет содержаться информация, относящаяся к разным базам) зависит от конкретной СУБД.

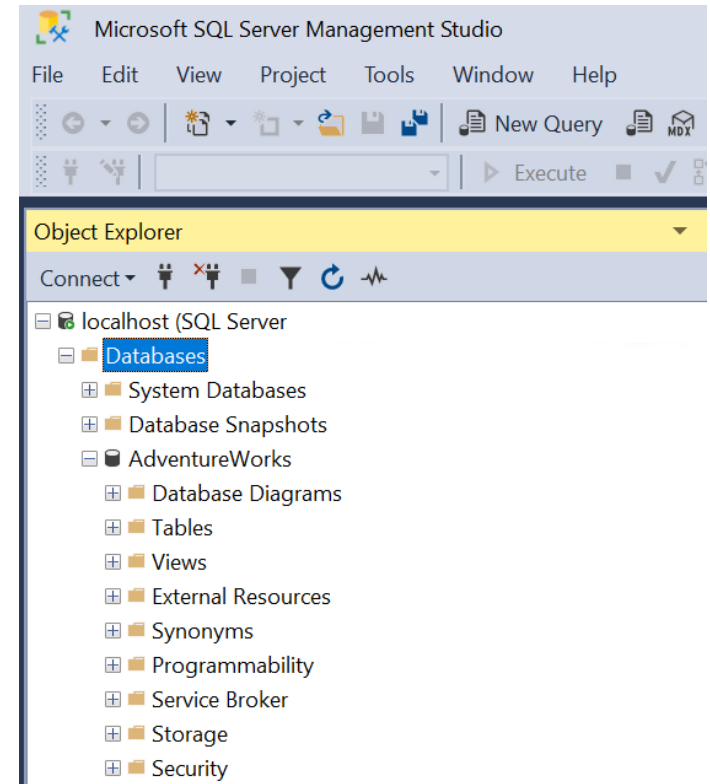
СУБД SQL Server 2025 –
Организация БД: файлы,
файловые группы, схемы.

Компоненты SQL Server 2025

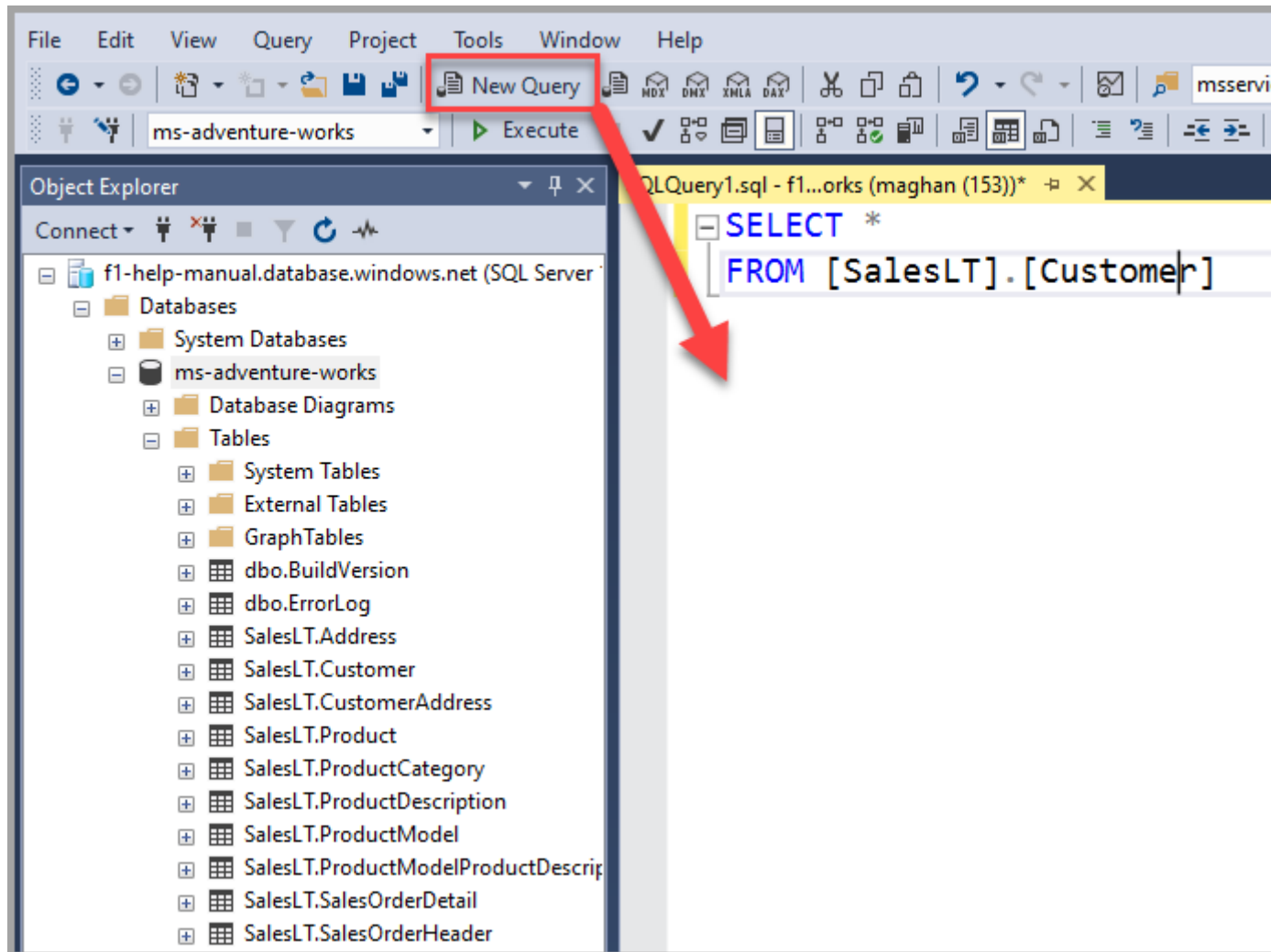
Database Engine	основная служба, обеспечивающая хранение, обработку и защиту данных
Machine Learning Services (MLS)	интеграция средств машинного обучения с использованием языков программирования R и Python
Integration Services (SSIS)	платформа для создания высокопроизводительных решений в области интеграции данных
Analysis Services (SSAS)	средства интерактивной аналитической обработки (OLAP) и средства интеллектуального анализа данных
Reporting Services (SSRS)	серверное решение для создания отчетов, объединяющих сведения из разнообразных реляционных и многомерных источников данных, публикации отчетов
Replication	набор технологий копирования и распространения данных и объектов баз данных из одной базы данных в другую, а также последующей синхронизации между базами данных для поддержания согласованности

Среда SQL Server Management Studio (SSMS)

- SQL Server Management Studio - это набор административных инструментальных средств для управления компонентами, входящими в состав SQL Server;
- интегрированная среда позволяет пользователям выполнять разнообразные задачи разработки и администрирования, например резервное копирование данных, редактирование запросов и автоматизацию общих функций в одном интерфейсе.



Среда SQL Server Management Studio



Метаданные в реляционной базе данных

- хранение метаданных осуществляется в системных таблицах (*system tables*), которые могут располагаться в специализированной системной базе данных

USER_TABLES Table

TableName	NumberColumns	PrimaryKey
STUDENT	3	StudentNumber
CLASS	4	ClassNumber
GRADE	3	(StudentNumber, ClassNumber)

USER_COLUMNS Table

ColumnName	TableName	DataType	Length (bytes)
StudentNumber	STUDENT	Integer	4
StudentName	STUDENT	Text	50
EmailAddress	STUDENT	Text	50
ClassNumber	CLASS	Integer	4
Name	CLASS	Text	50
Term	CLASS	Text	5
Section	CLASS	SmallInteger	2
StudentNumber	GRADE	Integer	4
ClassNumber	GRADE	Integer	4
Grade	GRADE	Decimal	(3, 2)

Системные базы данных

база данных master	главная база данных, содержит всю системную информацию СУБД SQL Server (общие для всего экземпляра метаданные, такие как сведения об учетных записях, связанных серверах, и параметры конфигурации системы), а также информацию обо всех остальных базах данных;
база данных Resource	доступна только для чтения, содержит все входящие в SQL Server системные объекты (такие как sys.objects), которые физически расположены в базе данных Resource , а логически отображаются для каждой базы данных в схеме sys ;
база данных msdb	используется агентом SQL Server (SQL Server Agent) для создания расписания предупреждений и заданий, а также другими компонентами (SQL Server Management Studio, Service Broker and Database Mail);
база данных model	используется в качестве шаблона для всех баз данных, созданных для экземпляра SQL Server (в том числе tempdb);
база данных tempdb	хранит все служебные и временные объекты (например, временные таблицы, временные хранимые процедуры и результаты промежуточных вычислений, создаваемые как пользователями, так и службами SQL Server; повторно создается при каждом запуске SQL Server, временные объекты удаляются автоматически при отключении.

Объекты базы данных

- база данных в SQL Server 2025 включает:
 - коллекции таблиц (пользовательских и системных), в которых хранится набор структурированных данных;
 - элементы управления (для таблиц):
 - ограничения;
 - триггеры;
 - значения по умолчанию;
 - пользовательские типы данных;
 - ограничения ссылочной целостности (ключи);
 - индексы;
 - хранимые процедуры;
 - определяемые пользователем функции, которые используют программный код Transact-SQL или .NET Framework для выполнения операций над данными в базе данных;
 - представления;
 -

Идентификаторы

- идентификатор – имя сервера, базы данных и ее объектов (таблиц, представлений, столбцов, индексов, триггеров, процедур, ограничений и правил, и т.п.), создаваемое при определении объекта и используемое для обращения к нему;
- классы идентификаторов:
 - обычные: соответствуют правилам форматирования идентификаторов
 - первый символ – буква (Unicode Standard 3.2) или символы `_` , `@` или `#` ;
 - последующие символы – буквы, десятичные цифры или символы `_` , `@` , `#` или `$` ;
 - не должны совпадать с зарезервированными словами (независимо от регистра);
 - не должны содержать пробелы и спецсимволы;
 - расширенные (идентификаторы с разделителем):
 - могут содержать любые символы;
 - заключаются в квадратные скобки или двойные кавычки: `[..]`, `".."`
 - длина идентификаторов - до 128 символов.

Идентификаторы с разделителями

```
SET QUOTED_IDENTIFIER OFF      -- default
GO
```

```
CREATE TABLE [table] (
    tablename char(128) NOT NULL,
    [USER] char(128) NOT NULL,
    [SELECT] char(128) NOT NULL,
    [INSERT] char(128) NOT NULL,
    [UPDATE] char(128) NOT NULL,
    [DELETE] char(128) NOT NULL
);
GO
```

```
SET QUOTED_IDENTIFIER ON;
GO
SELECT * FROM "table";
GO
```

```
SET QUOTED_IDENTIFIER OFF;
GO
SELECT * FROM [table];
GO
```


Виды идентификаторов

- имена объектов:

`[[[server.][database].][schema_name].]object_name[.column_name]`

`server.database.schema_name.object_name`

`server.database..object_name`

`server..schema_name.object_name`

`server...object_name`

`database.schema_name.object_name`

`database..object_name`

`schema_name.object_name`

`database_name.schema_name.object_name.column_name`

`database_name..object_name.column_name`

`schema_name.object_name.column_name`

`object_name.column_name`

- локальные переменные и параметры:

`@.....`

- локальные временные таблицы и процедуры:

`#.....`

- глобальные временные объекты (таблицы и процедуры):

`##.....`

Видимость объектов и правила квалификации

- при создании объекта или обращении к объекту Microsoft SQL Server 2025 использует следующие значения по умолчанию для неуказанных частей имени

Неуказанная часть	Значение по умолчанию
<i>Сервер</i>	сервером по умолчанию является локальный сервер
<i>База данных</i>	базой данных по умолчанию является текущая база данных
<i>Имя схемы</i>	именем схемы по умолчанию является имя пользователя в указанной базе данных, связанное с идентификатором имени входа текущего соединения. Если этот пользователь не владеет объектом с указанным именем, SQL Server ищет объект с указанным именем, принадлежащий владельцу базы данных (dbo)

Схема

- схема - это коллекция объектов базы данных, формирующая единое пространство имен (набор, в котором у каждого элемента есть свое уникальное имя);
- дополнительно схема позволяет администрировать права доступа к содержащимся в ней объектам.

```
CREATE SCHEMA Sprockets  
  CREATE TABLE NineProngs (  
    source int,  
    cost   int,  
    partnumber int) ;
```

```
ALTER SCHEMA HumanResources TRANSFER Person.Address;
```

```
DROP TABLE Sprockets.NineProngs;
```

```
DROP SCHEMA Sprockets;
```

Физическая структура базы данных: файлы данных

первичный (* .mdf)	содержит сведения, необходимые для запуска базы данных, и ссылки на другие файлы в базе данных; данные и объекты пользователя могут храниться в данном файле или во вторичном файле данных; в каждой базе данных имеется один первичный файл данных;
вторичный (.ndf)	не являются обязательными; могут быть использованы для распределения данных на несколько дисков, в этом случае каждый файл записывается на отдельный диск;
журнал транзакций (.ldf)	содержат сведения, используемые для восстановления базы данных; для каждой базы данных должен существовать хотя бы один файл журнала.

Физическая структура базы данных: файловые группы

- файлы данных могут быть объединены в файловые группы для удобства распределения и администрирования.

первичная	первичная файловая группа, содержащая первичный файл все системные таблицы размещены в первичной файловой группе;
пользовательская	любая файловая группа, созданная пользователем при создании или изменении базы данных;
файловые группы по умолчанию	если в базе данных создаются объекты без указания файловой группы, к которой они относятся, они назначаются файловой группе по умолчанию; только одна файловая группа создается как файловая группа по умолчанию; первичная файловая группа является файловой группой по умолчанию.

Пример физической структуры базы данных

База данных (*MyDB*)

Файл журнала (*LogOne*)
C:\data\MyLog.ldf

Файл журнала (*LogTwo*)
C:\log\MyLog.ldf

Главная группа файлов

Главный файл данных
(*MyDB_PrimaryData*)
C:\data\MyDB.mdf

Дополнительный файл данных
(*MyDB_SecondaryData*)
C:\data\MyDB1.ndf

Определяемая пользователем
группа файлов (*LargeFileGroup*)

Дополнительный файл данных
(*MyDB_LargeData1*)
D:\data\MyDB2.ndf

Дополнительный файл данных
(*MyDB_LargeData2*)
E:\data\MyDB2.ndf

Операции с базой данных

создание	CREATE DATABASE
удаление	DROP DATABASE
добавление / изменение файловых групп	CREATE DATABASE ALTER DATABASE
установка параметров базы данных, переименование	ALTER DATABASE + SET
присоединение	sp_attach_db
отсоединение	sp_detach_db
изменение владельца	sp_changedbowner
создание резервной копии	BACKUP DATABASE
восстановление из резервной копии	RESTORE DATABASE

Создание и удаление базы данных

```
USE master;  
GO
```

```
IF DB_ID (N'Sales') IS NOT NULL  
DROP DATABASE Sales;  
GO
```

```
CREATE DATABASE Sales  
    ON ( NAME = Sales_dat, FILENAME = 'C:\data\salesdat.mdf',  
        SIZE = 10, MAXSIZE = UNLIMITED, FILEGROWTH = 5% )  
    LOG ON ( NAME = Sales_log, FILENAME = 'C:\data\salelog.ldf',  
            SIZE = 5MB, MAXSIZE = 25MB, FILEGROWTH = 5MB );  
GO
```

```
USE Sales;  
GO
```


Задание файлов и файловых групп

```
CREATE DATABASE MyDB
ON PRIMARY
    ( NAME = MyDB_PrimaryData,
      FILENAME = 'c:\data\mydb.mdf' ),
    ( NAME = MyDB_SecondaryData,
      FILENAME = 'c:\data\mydb1.ndf' ),
FILEGROUP LargeFileGroup
    ( NAME = MyDB_LargeData1,
      FILENAME = 'd:\data\mydb2.ndf' ),
    ( NAME = MyDB_LargeData2,
      FILENAME = 'e:' )
LOG ON
    ( NAME = LogOne, FILENAME = 'c:\data\mylog.ldf' ),
    ( NAME = LogTwo, FILENAME = 'c:\log\mylog.ldf' )
GO
```

Создание и восстановление резервной копии

```
BACKUP DATABASE AdventureWorks
    TO DISK = 'C:\MSSQL\BACKUP\AdventureWorks.Bak'
    WITH FORMAT,
    NAME = 'Full Backup of AdventureWorks'
GO
```

```
RESTORE DATABASE MyDB
    FROM DISK = 'c:\backup\MyDBBackup.dat'
GO
```

```
RESTORE DATABASE MyDB
    FROM DISK = 'c:\backup\MyDBBackup.dat'
    WITH
        MOVE 'MyDB_SecondaryData' TO 'c:\data\mydb.ndf',
        MOVE 'LogTwo' TO 'c:\data\mylog2.ldf'
GO
```

Вопросы к экзамену

- Системные базы данных: состав и назначение. Именованное объекты базы данных.Схемы.
- Физическая структура базы данных: файлы и файловые группы.

Лабораторная работа №5.

Операции с базой данных, файлами, схемами

1. Создать базу данных (CREATE DATABASE..., определение настроек размеров файлов).
2. Создать произвольную таблицу (CREATE TABLE...).
3. Добавить файловую группу и файл данных (ALTER DATABASE...).
4. Сделать созданную файловую группу файловой группой по умолчанию.
5. (*) Создать еще одну произвольную таблицу.
6. (*) Удалить созданную вручную файловую группу.
7. Создать схему, переместить в нее одну из таблиц, удалить схему.

Базы данных

Тема 7

СУБД: Создание таблиц, категории целостности
данных

Целостность данных

- целостность данных (*data integrity*) – это гарантия точности, полноты и согласованности данных организации на любом этапе их жизненного цикла;
- цель обеспечения целостности данных:
 - гарантировать, что аналитика данных основана на достоверной информации;
 - гарантировать соответствие отраслевым стандартам и законодательным ограничениям;
- комплексный подход, затрагивающий технологическую инфраструктуру организации, политики и лиц, работающих с данными:
 - проверка на наличие ошибок;
 - процедуры валидации;
 - защита данных организации от потерь, утечек и искажений (шифрование, контроль доступа, резервное копирование);
- в базах данных обеспечивается, в том числе, ограничениями целостности данных (*integrity constraints*).

Категории целостности данных

- *сущностная целостность (entity integrity)*: определяет уникальность сущности в рамках хранимого набора данных;
- *доменная целостность (domain integrity)*: определяет достоверность значений в конкретном элементе данных (количество, формат и допустимые для ввода значения);
- *ссылочная целостность (referential integrity)*: обеспечивает наличие соответствующих взаимосвязей в связанных элементах данных, предотвращая появление потерянных (*orphaned*) данных при добавлении, изменении или удалении;
- *пользовательская целостность (user-defined integrity)*: определяет правила и ограничения для данных в соответствии с требованиями предметной области (бизнес-правила);
- *физическая целостность (physical integrity)*: обеспечивает целостность данных при их хранении и извлечении.

Сущностная целостность: реляционные базы данных

- определяет уникальность сущности в рамках хранимого набора данных, т.е. уникальность строки в конкретной таблице;
- предполагает обеспечение целостности столбцов идентификаторов или первичного ключа таблицы с помощью:
 - ограничений PRIMARY KEY;
 - ограничений UNIQUE;
 - индексов;
 - других средств, специфичных для конкретной СУБД.

Доменная целостность: реляционные базы данных

- определяет достоверность записей в конкретном столбце (формат и допустимые для ввода значения);
- доменная целостность обеспечивается:
 - определением типа данных: с любым объектом, содержащим данные (столбцы таблиц и представлений, переменные, параметры и результаты функций), связан тип, определяющий виды данных, которые могут храниться в этом объекте;
 - определением формата при помощи ограничений CHECK;
 - определением набора или диапазона возможных значений при помощи ограничений FOREIGN KEY, CHECK, DEFAULT, NOT NULL.

Ссылочная целостность: реляционные базы данных

- обеспечивает согласованность записей в связанных таблицах – предотвращаются следующие действия:
 - добавление или изменение записей в связанной таблице, если в первичной таблице нет соответствующей записи;
 - изменение значений в первичной таблице, которое приводит к появлению потерянных записей в связанной таблице;
 - удаление записей из первичной таблицы, если имеются совпадающие ключи в дочерних таблицах;
- ссылочная целостность, как правило, основана на связи первичных и внешних ключей и обеспечивается с помощью ограничений FOREIGN KEY.

Пользовательская целостность: реляционные базы данных

- определяет правила и ограничения для данных в соответствии с требованиями предметной области (бизнес-правила), дополняющие ограничения других категорий целостности;
- пользовательская целостность обеспечивается:
 - ограничениями уровня столбца и уровня таблицы (в инструкции CREATE TABLE);
 - средствами программирования, специфичными для конкретной СУБД (триггерами, функциями, хранимыми процедурами и т.п.).

СУБД SQL Server 2025 – Создание таблиц: типы данных, ограничения.

Выражения

- выражение - это сочетание идентификаторов, значений и операторов, которое SQL Server 2025 может вычислить для получения результата;
- выражения могут использоваться в различных контекстах во время доступа к данным или их изменения, например:
 - как часть данных, получаемых в запросе;
 - в качестве условий поиска данных, отвечающих определенному набору критериев;
- выражение может быть:
 - константой;
 - функцией;
 - именем столбца;
 - переменной;
 - вложенным запросом;
 - функцией CASE, NULLIF или COALESCE;
 - комбинацией перечисленных сущностей, соединенных операторами.

Категории операторов

Операции	Категория оператора
Сравнение значения с другим значением или выражением.	Операторы сравнения
Проверка истинности условия, такого как AND, OR, NOT, LIKE, ANY, ALL, IN.	Логические
Сложение, вычитание, умножение, деление и взятие остатка от деления.	Арифметические операторы
Выполнение действия над одним операндом, например положительным или отрицательным, или над дополнением.	Унарные
Временное обращение регулярных числовых значений в целочисленные и выполнение побитовой арифметической операции.	Битовые операторы
Постоянное или временное объединение двух строк (символьных или двоичных) в одну строку.	Оператор сцепления строк
Присваивание значения переменной или связывание столбца результирующего набора с псевдонимом.	Присвоение

Функции

- функции можно использовать и включать в следующие конструкции:
 - список выбора инструкции SELECT;
 - предложение WHERE инструкций SELECT, INSERT, DELETE или UPDATE для ограничения количества строк, удовлетворяющих условиям запроса;
 - условие поиска в предложении WHERE представления, которое должно динамически соответствовать пользователю или среде во время выполнения;
 - любое выражение;
 - ограничение CHECK или триггер для поиска указанных значений при вставке данных;
 - ограничение DEFAULT или триггер для вставки значений по умолчанию;

Категории функций

Категория функции	Описание
Статистические функции	Выполняют операции, объединяющие несколько значений в одно.
Функции настройки	Скалярные функции, возвращающие сведения о параметрах конфигурации.
Криптографические функции	Поддерживают шифрование, дешифрование, цифровые подписи и их проверку.
Функции для работы с курсорами	Возвращают сведения о состоянии курсора.
Функции даты и времени	Изменяют значения даты и времени.
Математические функции	Служат для выполнения тригонометрических, геометрических и других математических операций.
Функции метаданных	Возвращают сведения об атрибутах баз данных и объектов баз данных.
Ранжирующая функция	Недетерминированные функции, возвращающие ранжирующее значение для каждой строки секции.
Функции наборов строк	Возвращают результирующие наборы, которые можно использовать вместо ссылок на таблицу в инструкции Transact-SQL.
Функции безопасности	Возвращают сведения о пользователях и ролях.
Строковые функции	Изменяют значения char , varchar , nchar , nvarchar , binary и varbinary .
Системные функции	Работают или возвращают отчет о различных параметрах и объектах уровня системы.
Системные статистические функции	Возвращают сведения о производительности SQL Server.
Функции обработки текста и изображений	Изменяют значения text и image .

Категории функций: примеры

Категория функции	Описание
Статистические функции	AVG, MIN, CHECKSUM, SUM, CHECKSUM_AGG, STDEV, COUNT, STDEVP, COUNT_BIG, VAR, GROUPING, VARP, MAX
Функции конфигурации	@@OPTIONS, @@SERVERNAME, @@LANGUAGE, @@SERVICENAME, @@LOCK_TIMEOUT @@MAX_CONNECTIONS @@MAX_PRECISION @@VERSION @@NESTLEVEL,
Криптографические функции	SignByAsymKey, VerifySignedByAsmKey, SignByCert, ...
Функции для работы с курсорами	@@CURSOR_ROWS, CURSOR_STATUS, @@FETCH_STATUS
Функции даты и времени	DATEADD, DATEDIFF, DATENAME, DATEPART, DAY, GETDATE, GETUTCDATE, MONTH, YEAR
Математические функции	ABS, RAND, COS, EXP, ROUND, FLOOR, SIGN, ATAN, LOG, SIN
Функции метаданных	DB_NAME, OBJECT_NAME, DATABASEPROPERTY, FILEPROPERTY, ...
Ранжирующая функция	RANK, NTILE, DENSE_RANK, ROW_NUMBER
Функции наборов строк	CONTAINSTABLE, OPENQUERY, FREETEXTTABLE, OPENROWSET, OPENDATASOURCE, OPENXML
Функции безопасности	CURRENT_USER, SCHEMA_ID, USER_NAME, SUSER_ID, IS_MEMBER
Строковые функции	SOUNDEX, STR, REPLACE, LEFT, SUBSTRING, LEN, LOWER, LTRIM, ...
Системные функции	ISDATE, ISNULL, ROWCOUNT, NEWID, HOST_ID, CAST, CONVERT, ...
Системные статистические функции	@@CONNECTIONS, @@TOTAL_WRITE, @@CPU_BUSY, @@PACK_SENT, @@TIMETICKS, @@TOTAL_ERRORS, @@IO_BUSY, @@TOTAL_READ, ...
Функции обработки текста и изображений	PATINDEX TEXTVALID TEXTPTR

Функция CASE

- функция CASE - специальное выражение Transact-SQL, которое позволяет формировать альтернативные значения в зависимости от условия;
- функция CASE состоит из:
 - ключевого слова CASE;
 - имени преобразуемого столбца;
 - предложений WHEN, которые задают выражения для поиска, и предложений THEN, которые задают выражения для замены;
 - ключевого слова END;
 - необязательного предложения AS, задающего псевдоним функции CASE

```
SELECT ProductNumber, Name, 'Price Range' = CASE
WHEN ListPrice = 0 THEN 'Mfg item - not for resale'
WHEN ListPrice < 50 THEN 'Under $50'
WHEN ListPrice >= 50 and ListPrice < 250
                                THEN 'Under $250'
WHEN ListPrice >= 250 and ListPrice < 1000
                                THEN 'Under $1000'
ELSE 'Over $1000'
END [AS...]
FROM Production.Product
ORDER BY ProductNumber;
```

Типы данных

- с объектом, содержащим данные, связан тип, определяющий виды данных, которые могут храниться в объекте;
- типы данных имеют следующие объекты:
 - столбцы таблиц и представлений;
 - параметры хранимых процедур / функций;
 - переменные;
 - функции Transact-SQL, возвращающие одно или несколько значений;
 - хранимые процедуры, возвращающие значение (всегда имеет тип **integer**);
- можно создавать два вида пользовательских типов данных:
 - типы данных псевдонима - создаются на основе базовых типов данных (позволяют назначить типу данных имя, лучше характеризующее типы значений, которые будут храниться в объекте):

```
CREATE TYPE SSN FROM VARCHAR(11) NOT NULL;
```
 - пользовательские типы данных CLR основаны на типах данных, созданных в управляемом коде и помещенных в сборку SQL Server.

Атрибуты типов данных

- при назначении типа данных объекту определяются четыре атрибута объекта:
 - вид данных, содержащихся в объекте;
 - размер или длина хранимого объектом значения: количество байт, используемых для хранения числа;
 - точность: количество десятичных знаков;
 - масштаб числа: количество десятичных знаков справа от десятичного разделителя;
- точность (*precision*) и масштаб (*scale*) числовых типов данных, кроме **decimal** и **numeric**, фиксированы.

Встроенные типы данных

- ТОЧНЫЕ ЧИСЛОВЫЕ ТИПЫ:
 - tinyint, smallint, int, bigint;
 - bit;
 - decimal, numeric;
 - money, smallmoney;
- ВЕЩЕСТВЕННЫЕ ТИПЫ:
 - float, real;
- ДАТА И ВРЕМЯ:
 - date, time, datetime, datetime2, smalldatetime, datetimeoffset;
- СТРОКОВЫЕ ТИПЫ:
 - char(n), varchar(n), text;
 - Unicode: nchar(n), nvarchar(n), ntext;
- ДВОИЧНЫЕ СТРОКИ:
 - binary(n), varbinary(n), image;
- СПЕЦИАЛЬНЫЕ ТИПЫ:
 - geography, geometry - пространственные типы (spatial types);
 - hierarchyid;
 - json, xml;
 - vector;
 - rowversion;
 - sql_variant;
 - cursor;
 - table;
 - uniqueidentifier.

Числовые типы данных

Тип данных	Диапазон	Размер	
bit	0 или 1	кратно 1 байту	
bigint	-9 223 372 036 854 775 808 ... 9 223 372 036 854 775 807	8 байт	
int	-2 147 483 648 ... 2 147 483 647	4 байта	
smallint	-32 768 ... 32 767	2 байта	
tinyint	0 ... 255	1 байт	
decimal [(p[,s])] numeric [(p[,s])]	$-10^{38} + 1 \dots 10^{38} - 1$ точность $p=1\dots38$, по умолчанию $p=18$ масштаб $0 \leq s \leq p$, по умолчанию $s=0$	$p=1\dots9$	5 байт
		$p=10\dots19$	9 байт
		$p=20\dots28$	13 байт
		$p=29\dots38$	17 байт

битовые константы (bit)	0 или 1
целочисленные константы (int)	1894 -2 +15
константы decimal	1894.1204 2.0

Вещественные типы данных

Тип данных	Диапазон	Размер		
float [(<i>n</i>)]	$-1.79\text{E}+308 \dots -2.23\text{E}-308, 0,$ $2.23\text{E}-308 \dots 1.79\text{E}+308$ <i>n</i> =1...53 – количество битов, используемых для хранения мантиссы, фактически, <i>n</i> =24 или <i>n</i> =53 (по умолчанию)	<i>n</i> =1...24	7 разрядов	4 байта
		<i>n</i> =25...53	15 разрядов	8 байт
real	$-3.40\text{E}+38 \dots -1.18\text{E}-38, 0,$ $-1.18\text{E}-38 \dots 3.40\text{E}+38$	4 байта		

вещественные константы (float и real)	-12.34 101.5E5 0.5E-2
--	-----------------------------

Строковые (символьные) типы данных

Тип данных	Диапазон	Размер
char (<i>n</i>)	строковые данные фиксированного размера ($n = 1..8000$), значения дополняются пробелами, UTF-8 / кодовая страница по умолчанию	n байт $\leq n$ символов
varchar (<i>n max</i>)	строковые данные переменного размера ($n = 1..8000$; <i>max</i> – до 2 ГБ), UTF-8 / кодовая страница по умолчанию	$(n+2)$ байт $\leq n$ символов
nchar (<i>n</i>)	строковые данные фиксированного размера ($n = 1..4000$), значения дополняются пробелами	$2*n$ байт $\leq n$ символов
nvarchar (<i>n max</i>)	строковые данные переменного размера ($n = 1..8000$; <i>max</i> – до 2 ГБ)	$(n+2)$ байт $\leq n$ символов

символьные строки (кодовая страница по умолчанию)	'O'Brien' 'The level for job_id: %d should be between %d and %d.'
строки в Юникоде (UCS-2 / UTF-16)	N'Michél'

- по умолчанию $n = 1$ или $n = 30$ (функции CAST и CONVERT);
- правила сопоставления (collation) определяют кодировку (SBCS/MBCS, UCS-2, UTF-8, UTF-16) и правила сравнения символьных данных.

Двоичные типы данных

Тип данных	Диапазон	Размер
binary [(<i>n</i>)]	двоичные данные фиксированного размера ($n = 1..8000$), значения дополняются нулями	<i>n</i> байт
varbinary [(<i>n</i> max)]	двоичные данные переменного размера ($n = 1..8000$; max – до 2 ГБ)	(<i>n</i> +2) байт

константы двоичных строк	0x12Ef 0x69048AEFDD010E 0x
--------------------------	----------------------------------

- по умолчанию $n = 1$ или $n = 30$ (функции CAST и CONVERT).

Типы данных `uniqueidentifier`

Тип данных	Диапазон	Размер
<code>uniqueidentifier</code>	128-битовый глобально-уникальный идентификатор	16 байт

константы <code>uniqueidentifier</code>	'6F9619FF-8B86-D011-B42D-00C04FC964FF' 0xff19966f868b11d0b42d00c04fc964ff
---	--

- GUID (*globally unique identifier*), UUID (*universally unique identifier*) – 128-битный идентификатор, статистическая уникальность которого обеспечивается алгоритмом его вычисления (стандарты ITU-T Rec. X.667; ISO/IEC 9834-8; IETF RFC4122 / RFC 9562);
- вычисление нового значения в SQL Server 2025:
 - NEWID() – соответствует RFC4122;
 - NEWSEQUENTIALID().

Типы данных для даты и времени

Тип данных	Диапазон	Размер
date	0001-01-01 ... 9999-12-31	3 байта
time[(f)]	00:00:00.0000000 ... 23:59:59.9999999 <i>f=0..7 (по умолчанию f=7)</i>	3-5 байт
datetime2[(f)]	0001-01-01 ... 9999-12-31 00:00:00.0000000 ... 23:59:59.9999999 <i>f=0..7 (по умолчанию f=7)</i>	6-8 байт
smalldatetime	1900-01-01 ... 2079-06-06 00:00:00 ... 23:59:59 секунды округляются до минут	4 байта
datetimeoffset[(f)]	0001-01-01 ... 9999-12-31 00:00:00 ... 23:59:59.9999999 -14:00 ... +14:00 <i>f=0..7 (по умолчанию f=7)</i>	8-10 байт
datetime	1753-01-01 ... 9999-12-31 00:00:00 ... 23:59:59.997	8 байт

Константы для даты и времени

- формат ISO 8601 (не зависит от настроек языка / формата даты):
 - 'уууу-ММ-ддТНН:мм:сс[.nnnnnnnn][{+|-}hh:mm]'
 - 'уууу-ММ-ддТНН:мм:сс[.nnnnnnnn]Z'
 - '2004-05-23T14:25:10'
 - '2004-05-23T14:25:10.487'
 - '2007-05-08 12:35:29.1234567 +12:15'
- форматы, зависящие от настроек языка и/или формата даты/времени (SET LANGUAGE / SET DATEFORMAT):
 - алфавитный формат:
 - '20 Apr 2005'
 - '1996 APR[IL] [15]'
 - числовой формат (используются разделители '/', '-', ' или '.') :
 - '20/4/2005'
 - '04/15/98'
 - '04:24 PM'
 - '14:30:24'
 - строковый формат без разделителей:
 - '20050420'
 - '960420'

Преобразование типов данных

- тип выражения определяется типом входящих в него операндов;
- если, оператор соединяет два выражения разных типов, то производится приведение значений левой и правой частей к типу с высшим приоритетом согласно правилам предшествования типов (*data type precedence*) – по мере убывания приоритета: пользовательские типы, **json**, **sql_variant**, **xml**, **datetimeoffset**, **datetime2**, **datetime**, **smalldatetime**, **date**, **time**, **float**, **real**, **decimal**, **money**, **smallmoney**, **bigint**, **int**, **smallint**, **tinyint**, **bit**, **ntext**, **text**, **image**, **timestamp**, **uniqueidentifier**, **nvarchar** (включая **nvarchar(max)**), **nchar**, **varchar** (включая **varchar(max)**), **char**, **varbinary** (включая **varbinary(max)**), **binary**;
- если неявное приведение типов невозможно, выдается сообщение об ошибке;
- явное приведение типов:
 - `CAST ( expression AS data_type [ ( length ) ] )`
 - `CONVERT ( data_type [ ( length ) ] , expression [ , style ] )`
- очевидно, что в ряде случаев может быть невозможно как неявное, так и явное приведение типов.

Создание, изменение и удаление таблиц

- минимальный синтаксис инструкции CREATE TABLE требует указания имени новой таблицы, а также имен и типов данных каждого столбца (имена таблиц и столбцов должны соответствовать синтаксису идентификаторов)

```
CREATE TABLE products (  
    product_no integer,  
    name varchar(max),  
    price numeric  
) ON [PRIMARY];
```

```
ALTER TABLE doc_exc  
    ADD column_b VARCHAR(20) NOT NULL  
        CONSTRAINT exb_unique UNIQUE;
```

```
DROP TABLE Orders
```

Ограничение NOT NULL

- ограничение NOT NULL для столбца указывает, что столбец не может принимать неопределённое значение (NULL);
- данное ограничение явно задаётся в определении столбца, при этом также можно присвоить ему имя.

```
CREATE TABLE products (  
    product_no integer NOT NULL,  
    name nvarchar(50) NOT NULL,  
    price numeric(5,2)  
);
```

```
CREATE TABLE products (  
    product_no integer NOT NULL,  
    name nvarchar(50) CONSTRAINT products_name_not_null NOT NULL,  
    price numeric(5,2)  
);
```

Значения по умолчанию

- значение по умолчанию – значение, которое будет автоматически присвоено столбцу, если оно не будет явно указано при добавлении новой записи;
- если значение по умолчанию не объявлено явно, им считается значение NULL;
- значение по умолчанию может быть выражением, которое в этом случае вычисляется в момент присваивания значения по умолчанию.

```
CREATE TABLE products (  
    product_no integer,  
    name nvarchar(255),  
    price numeric DEFAULT 9.99,  
    added_at datetime2 DEFAULT getdate()  
);
```


Вычисляемые столбцы

- вычисляемый столбец – столбец, вычисляемый из других столбцов таблицы (не допускаются подзапросы);
- хранимый вычисляемый столбец вычисляется при записи (добавлении или изменении) и занимает место в таблице так же, как и обычный столбец.
- произвести запись непосредственно в генерируемый столбец нельзя.

```
CREATE TABLE dbo.tableX
(
    low INT,
    high INT,
    average AS (low + high) / 2,
    delta AS (low + high) PERSISTED
);
```

Ограничение CHECK

- ограничение CHECK – некоторое логическое выражение, вычисляемое при вставке или обновлении записи таблицы для проверки соответствия данных рассматриваемому ограничению;
- ограничение CHECK может быть определено:
 - на уровне отдельного столбца;
 - на уровне таблицы.

```
CREATE TABLE products (  
    product_no integer,  
    name nvarchar (255),  
    price numeric CHECK (price > 0),  
    discounted_price numeric,  
    CHECK (discounted_price > 0),  
    CONSTRAINT valid_discount CHECK (price > discounted_price)  
);
```

Ограничение первичного ключа

- ограничение первичного ключа подразумевает, что образующий его столбец или набор столбцов может быть уникальным идентификатором строк в таблице;
- значения входящих в первичный ключ столбцов должны быть уникальными в совокупности и каждое из них должно быть отлично от NULL.

```
CREATE TABLE dbo.Employee  
(  
    EmployeeID INT PRIMARY KEY CLUSTERED  
);
```

```
CREATE TABLE [dbo].[Supply](  
    [PartID] [int] NOT NULL,  
    [SupplierID] [int] NOT NULL,  
    [Price] [float] NOT NULL,  
    [DeliveryTime] [int] NOT NULL,  
    CONSTRAINT [PK_Supply] PRIMARY KEY CLUSTERED ([PartID],[SupplierID])  
);
```

Ограничение UNIQUE

- ограничение уникальности гарантирует, что данные в определённом столбце или группе столбцов уникальны среди всех строк таблицы;
- допускается наличие неопределённых значений (NULL) в столбцах, входящих в ограничение;
- допускается наличие нескольких ограничений UNIQUE в таблице.

```
CREATE TABLE Production.TransactionHistoryArchive4  
(  
    TransactionID int NOT NULL,  
    CONSTRAINT AK_TransactionID UNIQUE(TransactionID)  
);
```

```
ALTER TABLE Person.Password  
ADD CONSTRAINT AK_Password UNIQUE (PasswordHash, PasswordSalt);
```

```
ALTER TABLE dbo.DocExc  
DROP CONSTRAINT UNQ_ColumnB_DocExc;
```

Ограничение внешнего ключа

- ограничение внешнего ключа указывает, что значения столбца (или группы столбцов) должны соответствовать значениям в некоторой строке другой таблицы (ссылочная целостность);
- порядок, число и типы столбцов в ограничении должны соответствовать порядку, числу и типам целевых столбцов (целевые столбцы должны входить в ограничения PRIMARY KEY, UNIQUE или формировать уникальный индекс);
- таблица может содержать несколько ограничений внешнего ключа.

```
CREATE TABLE Sales.TempSalesReason (  
    TempID INT NOT NULL,  
    Name NVARCHAR(50),  
    CONSTRAINT PK_TempSales  
        PRIMARY KEY NONCLUSTERED (TempID),  
    CONSTRAINT FK_TempSales_SalesReason FOREIGN KEY (TempID)  
        REFERENCES Sales.SalesReason(SalesReasonID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

Действия для ограничений внешнего ключа

- предложения REFERENCES инструкций CREATE TABLE и ALTER TABLE поддерживают предложения ON DELETE и ON UPDATE:
 - [ON DELETE { NO ACTION | CASCADE | SET NULL | SET DEFAULT }]
 - [ON UPDATE { NO ACTION | CASCADE | SET NULL | SET DEFAULT }]
- по умолчанию подразумевается действие NO ACTION (указывает, что при попытке удалить или изменить строку с ключом, на которую ссылаются внешние ключи в строках других таблиц, нужно сообщить об ошибке, а для соответствующей инструкции DELETE / UPDATE выполнить откат);
- действия CASCADE, SET NULL и SET DEFAULT позволяют удалять и изменять значения ключей в таблицах, ссылающихся на таблицу, в которую вносятся изменения:
 - CASCADE: каскадное изменение ссылающихся таблиц;
 - SET NULL: установка NULL для ссылающихся внешних ключей;
 - SET DEFAULT: установка значений по умолчанию для ссылающихся внешних ключей.

Ограничения множественных каскадных действий

- последовательности каскадных ссылочных действий, запускаемые отдельными инструкциями DELETE или UPDATE, должны образовывать дерево без циклических ссылок;
- никакая таблица не должна появляться больше одного раза в списке всех каскадных ссылочных действий, вызванных инструкциями DELETE или UPDATE;
- кроме того, в дереве каскадных ссылочных действий к любой из задействованных таблиц должен быть только один путь;
- любая ветвь в дереве прерывается, как только встречается таблица, для которой указано действие NO ACTION или вообще не указано действие.

Создание столбцов идентификаторов: свойство **IDENTITY**

- при вставке значений в таблицу со столбцом идентификатора (столбцом, помеченным свойством **IDENTITY**), автоматически формируется следующее значение данного столбца (путём добавления значение шага приращения к текущему значению);
- использование свойства **IDENTITY**:
 - уникальность свойства **IDENTITY** гарантирована только в рамках таблицы, в которой оно использовано;
 - таблица может содержать только один столбец со свойством **IDENTITY**;
 - столбец должен иметь тип данных **decimal(p,0)**, **int**, **numeric**, **smallint**, **bigint** или **tinyint**;
 - можно указать начальное значение и шаг (по умолчанию, (1,1));
 - столбец идентификатора не должен допускать значений **NULL** и содержать определений или объектов по умолчанию.

Использование столбцов идентификаторов

- необходимо дополнительно определять ограничение уникальности с помощью PRIMARY KEY / UNIQUE;
- сгенерированные значения могут иметь пропуски (в результате отката транзакций или восстановления после сбоев сервера)
- сослаться на столбец в списке выборки можно создать с помощью указания ключевого слова \$IDENTITY, также сослаться на столбец можно при помощи имени;
- значение столбца идентификатора не может быть задано явно или как значение по умолчанию.

```
CREATE TABLE TZ (  
    Z_id INT IDENTITY(1,1) PRIMARY KEY,  
    Z_name VARCHAR(20) NOT NULL  
);
```

```
SELECT SCOPE_IDENTITY() AS [SCOPE_IDENTITY];
```

```
SELECT @@IDENTITY AS [@@IDENTITY];
```

```
SELECT IDENT_CURRENT('TZ') AS [IDENT_CURRENT];
```

Глобальные уникальные идентификаторы

- если в приложении нужно сформировать столбец идентификатора, уникальный для всей базы данных или всех баз данных во всех компьютерных сетях, необходимо использовать свойство ROWGUIDCOL, тип данных **uniqueidentifier** и функцию NEWID;
- использование свойства ROWGUIDCOL для определения столбца идентификаторов GUID:
 - в таблице может содержаться только один столбец ROWGUIDCOL, который должен иметь тип данных **uniqueidentifier**;
 - не обеспечивается уникальность значений, хранимых в столбце (необходимо использовать PRIMARY KEY / UNIQUE);
 - значение столбца не формируется автоматически – для вставки глобального уникального значения необходимо:
 - использовать для столбца определение DEFAULT с вызовом функций NEWID() / NEWSEQUENTIALID();
 - использовать вызов функций NEWID() / NEWSEQUENTIALID() в инструкции INSERT;
 - неизменность значения не контролируется;
 - ссылку на столбец в списке выборки, помимо использования имени, можно создать с помощью указания ключевого слова \$ROWGUID.

Использование столбцов глобальных идентификаторов

```
CREATE TABLE [dbo].[PurchaseOrderDetail]
(
    [LineNumber] [smallint] NOT NULL,
    [ProductID] [int] NULL REFERENCES Production.Product(ProductID),
    [UnitPrice] [money] NULL,
    [rowguid] [uniqueidentifier] ROWGUIDCOL NOT NULL
        CONSTRAINT [DF_PurchaseOrderDetail_rowguid] DEFAULT (newid()),
) ON [PRIMARY];
```

```
CREATE TABLE MyUniqueTable
(
    UniqueColumn UNIQUEIDENTIFIER DEFAULT NEWID(),
    Characters VARCHAR(10)
)
GO;
```

```
INSERT INTO MyUniqueTable(Character) VALUES ('abc')
INSERT INTO MyUniqueTable VALUES (NEWID(), 'def')
```

Последовательности

```
CREATE SEQUENCE [schema_name . ] sequence_name
  [ AS [ built_in_integer_type | user-
    defined_integer_type ] ]
  [ START WITH <constant> ]
  [ INCREMENT BY <constant> ]
  [ { MINVALUE [ <constant> ] } | { NO MINVALUE } ]
  [ { MAXVALUE [ <constant> ] } | { NO MAXVALUE } ]
  [ CYCLE | { NO CYCLE } ]
  [ { CACHE [ <constant> ] } | { NO CACHE } ]
  [ ; ]
```

- в отличие от полей IDENTITY:
 - очередное значение последовательности можно получить с помощью функции NEXT VALUE FOR;
 - последовательность не привязана к конкретной таблице, следовательно, возможно обеспечить уникальность значений более чем в одной таблице;
 - для полей, сформированных с помощью последовательностей, не контролируются неизменность и уникальность значений.

Использование последовательностей

```
CREATE TABLE Test.Orders
```

```
(  
    OrderID int PRIMARY KEY,  
    Name varchar(20) NOT NULL,  
    Qty int NOT NULL  
);
```

```
GO
```

```
CREATE SEQUENCE Test.CountByOne
```

```
    START WITH 1
```

```
    INCREMENT BY 1;
```

```
GO
```

```
-- Insert records
```

```
INSERT Test.Orders (OrderID, Name, Qty)
```

```
VALUES (NEXT VALUE FOR Test.CountByOne, 'Tire', 2);
```

```
INSERT test.Orders (OrderID, Name, Qty)
```

```
VALUES (NEXT VALUE FOR Test.CountByOne, 'Seat', 1);
```

Вопросы к экзамену

- Основные элементы синтаксиса Transact-SQL: константы, встроенные функции, выражения.
- Типы данных: атрибуты, категории, применение. Приведение типов.
- Категории целостности данных.
- Создание, изменение и удаление таблиц: реализация сущностной целостности, вычисляемые столбцы.
- Создание, изменение и удаление таблиц: формирование значений суррогатных ключей.
- Создание, изменение и удаление таблиц: реализация доменной целостности, значения по умолчанию.
- Создание, изменение и удаление таблиц: реализация ссылочной целостности, действия для ограничений внешнего ключа.

Лабораторная работа №6.

Создание таблиц: ключи, ограничения, значения по умолчанию

1. Создать таблицу с автоинкрементным первичным ключом. Изучить функции, предназначенные для получения сгенерированного значения IDENTITY.
2. Добавить поля, для которых используются ограничения (CHECK), значения по умолчанию (DEFAULT), также использовать встроенные функции для вычисления значений.
3. Создать таблицу с первичным ключом на основе глобального уникального идентификатора.
4. Создать таблицу с первичным ключом на основе последовательности.
5. Создать две связанные таблицы, и протестировать на них различные варианты действий для ограничений ссылочной целостности (NO ACTION | CASCADE | SET NULL | SET DEFAULT).

Базы данных

Тема 8

Представления. Индексы

Представления

- представление (*view*) - это виртуальная таблица, состоящая из совокупности именованных столбцов и строк данных, содержимое которой определяется запросом:
 - определяющий представление запрос может быть инициирован в одной или нескольких базовых таблицах или представлениях;
 - пока представление не будет материализовано / проиндексировано, оно не существует в базе данных как хранимая совокупность значений: строки и столбцы динамически формируются при обращении к представлению на основе данных из таблиц (и других представлений), указанных в определяющем представлении запросе – т.е., фактически представление является сохранённым именованным запросом.

Функции представлений

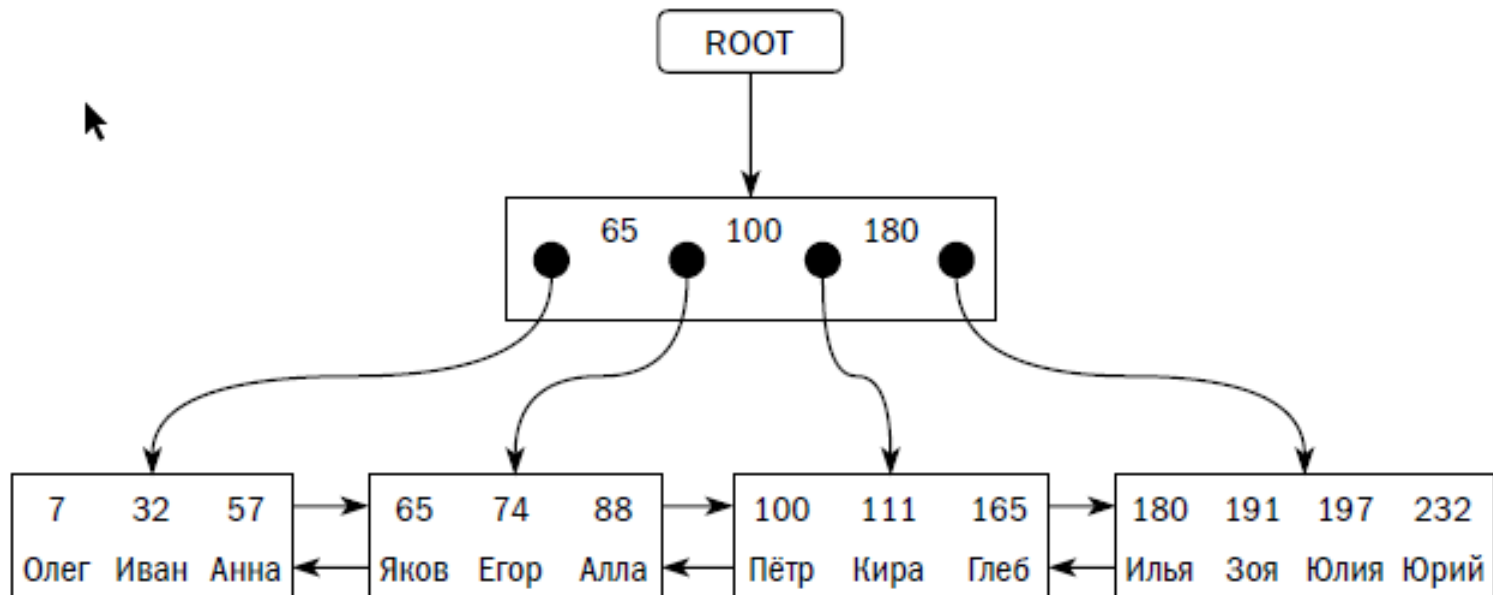
- упрощение и настройка восприятия каждым пользователем информации базы данных;
- реализация механизмов безопасности, обеспечивающих возможность обращения пользователей к данным без предоставления им разрешений на непосредственный доступ к базовым таблицам;
- обеспечение интерфейса обратной совместимости, моделирующего таблицу, которая существует, но схема которой изменилась;
- определение представлений с данными из нескольких гетерогенных источников.

Индексы

- индекс является структурой на диске, которая связана с таблицей или представлением и может способствовать сокращению времени выполнения запроса за счет сокращения количества дисковых операций ввода-вывода и уменьшения потребления системных ресурсов;
- индекс содержит ключи, построенные на основе одного или нескольких столбцов в таблице или представлении;
- индексы могут быть уникальными (т.е. никакие две строки не имеют одинаковое значение для ключа индекса) или неуникальными;
- уникальные индексы создаются автоматически при определении ограничений PRIMARY KEY или UNIQUE на основе столбцов таблицы;
- основной индексной структурой, используемой для организации индексов, является B-дерево.

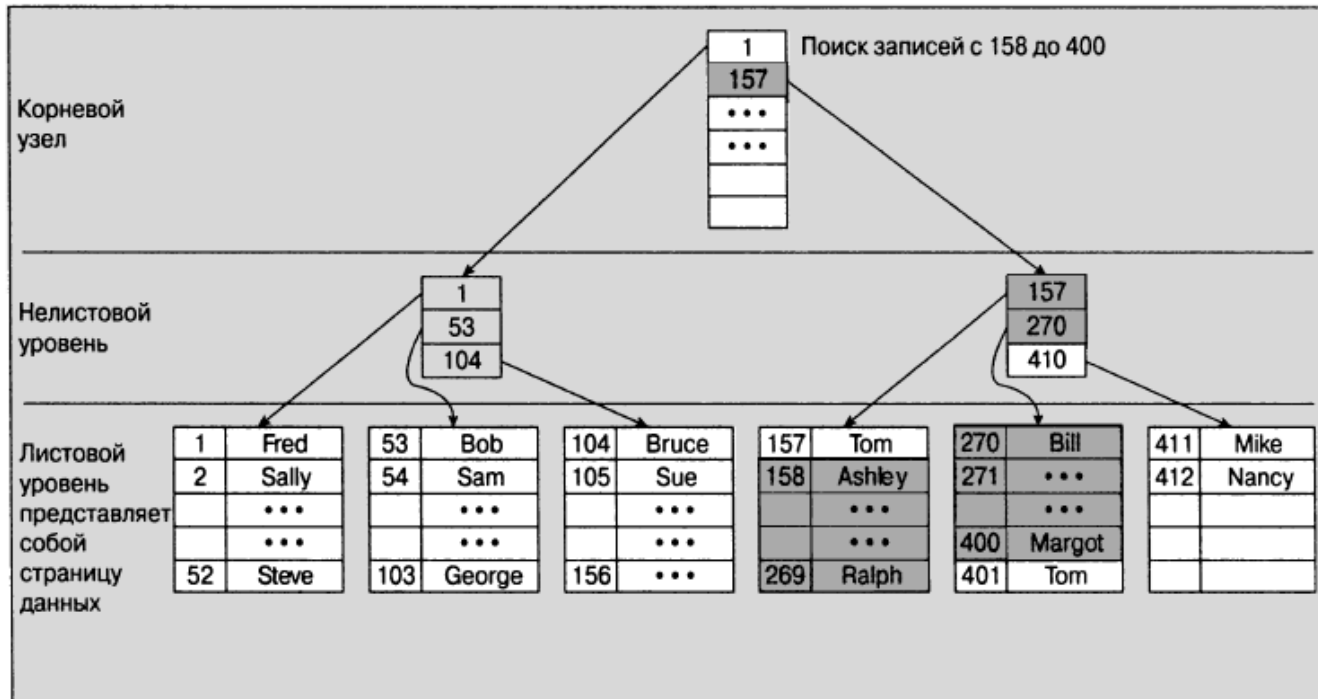
В-дерево

- преимущества:
 - эффективная вставка (количество изменяемых блоков не превышает глубины дерева);
 - эффективный поиск (глубина дерева – $\log N / \log f$);
 - поддерживает поиск по значению (равенство) и по диапазону (неравенства);
 - результаты последовательного чтения индекса упорядочены по значению ключа индекса.



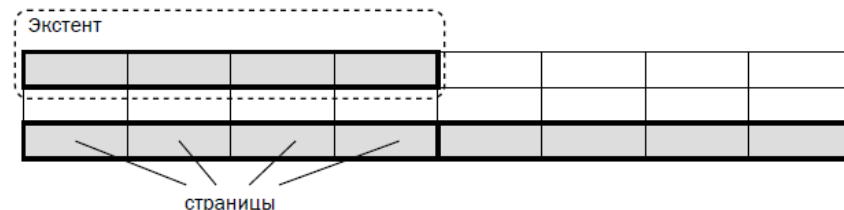
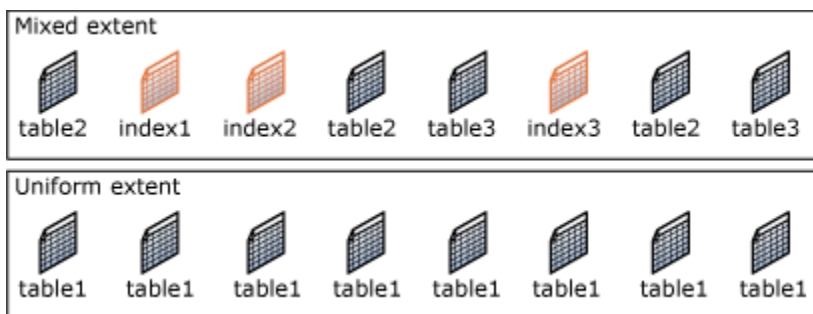
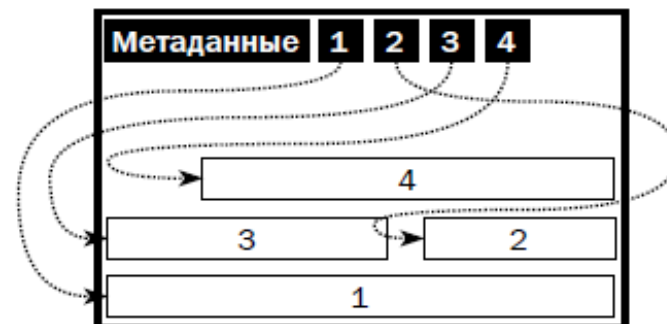
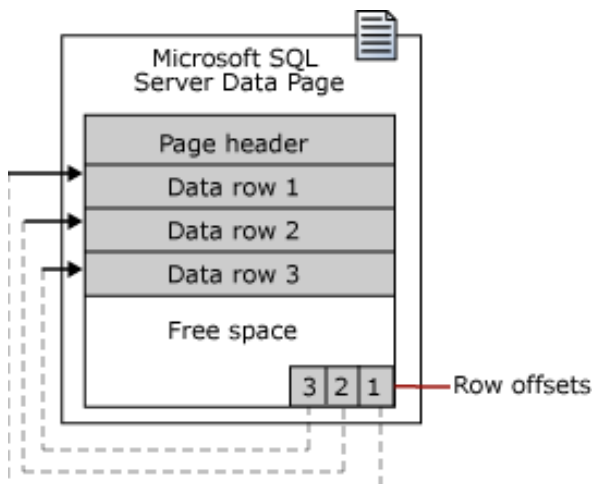
В-дерево: варианты доступа

- индексы на основе В-дерева эффективно поддерживают следующие варианты доступа:
 - поиск уникального ключа;
 - поиск множества ключей (неуникальный индекс, поиск по части ключа, поиск о диапазоне ключей) – результат отсортирован по значению ключа;
 - полный просмотр – результат отсортирован по значению ключа;
- также может быть эффективным быстрый просмотр индекса (просмотр всех страниц в случайном порядке как альтернатива полному сканированию таблицы).



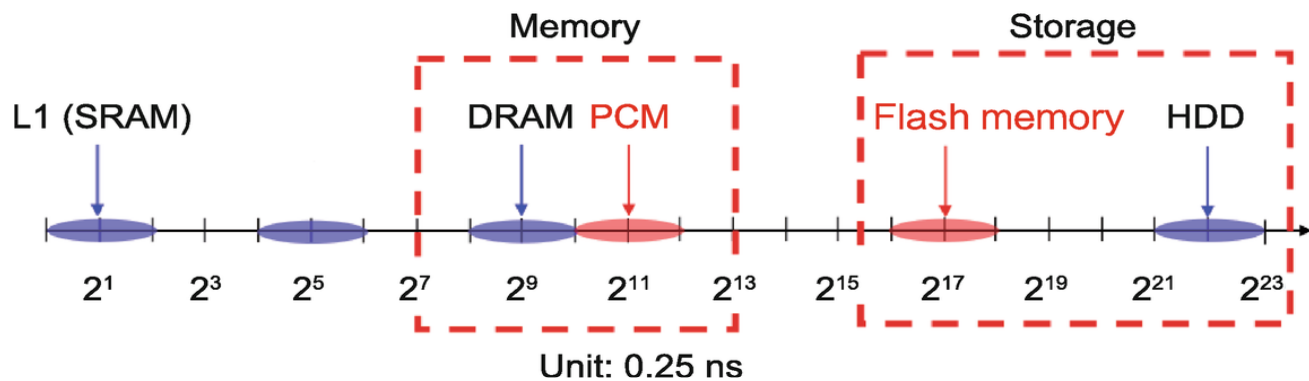
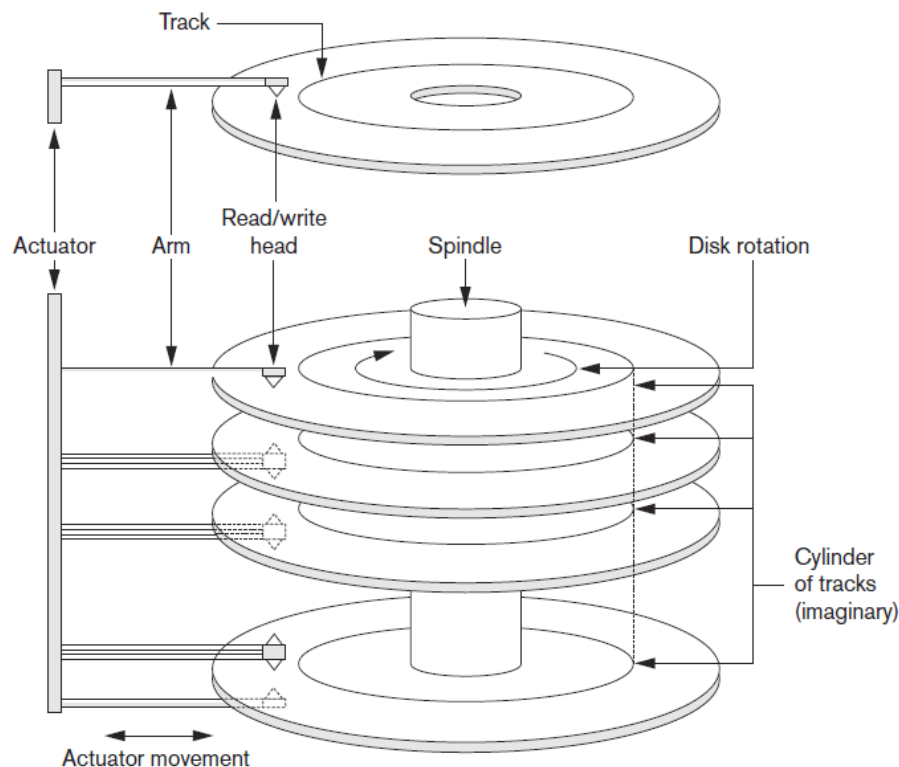
Хранение данных (построчное)

- варианты организации построчного хранения табличных данных в дисковых СУБД:
 - неупорядоченная структура, которая называется кучей (*heap, heap-organized table*);
 - кластеризованная таблица: строки данных хранятся упорядоченно в кластеризованном индексе (индексе, листовые узлы которого хранят всю строку таблицы).



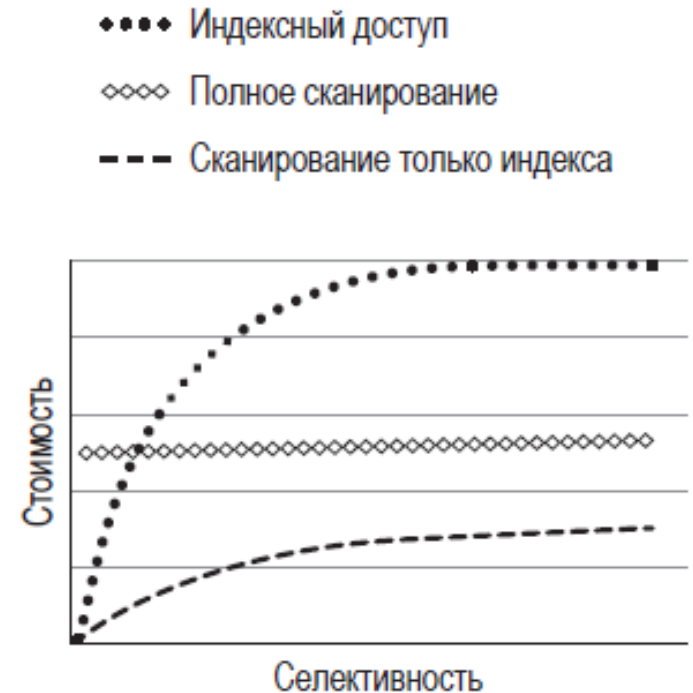
Предпосылки использования индексов

- индексы в СУБД предназначены для:
 - поддержания уникальности значений;
 - ускорения поиска и получения данных по определённым критериям.



Использование индексов

- необходимость использования индексов, их набор и характеристики определяются исходя из набора запросов;
- преимущества: время отклика;
- недостатки:
 - накладные расходы на хранение;
 - накладные расходы на обновление;
 - накладные расходы в процессе использования (конкуренция за ресурсы);
- индексы используются, если:
 - низкая селективность индекса (высокая кардинальность значений в таблице);
 - покрывающий / составной индекс;
- индексы не используются, если:
 - для выполнения запроса нужна большая часть строк таблицы
 - небольшая таблица полностью считывается в оперативную память.



СУБД SQL Server 2025 – Представления, индексы.

Типы представлений

- стандартные: сочетание данных из одной или нескольких таблиц;
- индексируемые: материализованное представление (вычисленное и сохраненное в базе данных), изменения данных в базовых таблицах, отражаются в данных, хранимых в индексируемом представлении;
- секционированные: объединение горизонтально секционированных данных набора базовых таблиц, находящихся на одном или нескольких серверах (локальное / распределенное секционированное представление).

Создание представлений

- определяющий представление запрос может быть инициирован в одной или нескольких базовых таблицах или представлениях;
- столбцу обязательно должно быть присвоено имя, если значение столбца представления формируется выражением и имя не было назначено в самом запросе;
- проверка схемы базовых объектов производится только при обращении к представлению.

```
CREATE VIEW hiredate_view
AS
SELECT p.FirstName, p.LastName, e.BusinessEntityID, e.HireDate
FROM HumanResources.Employee AS e JOIN Person.Person AS p
      ON e.BusinessEntityID = p.BusinessEntityID;
```

```
CREATE VIEW Sales.SalesPersonPerform
AS
SELECT TOP (100) SalesPersonID, SUM(TotalDue) AS TotalSales
FROM Sales.SalesOrderHeader
WHERE OrderDate > CONVERT(DATETIME, '20001231', 101)
GROUP BY SalesPersonID;
```

Создание представлений: привязка к схеме

- при указании SCHEMABINDING базовые таблицы не могут быть изменены каким-либо образом, затрагивающим определение представления – при необходимости таких изменений представление должно быть создано повторно;
- все базовые объекты представления (таблицы, представления и функции) должны располагаться в одной базе данных;
- запрос, определяющий представление, должен использовать полные имена объектов (*схема.объект*).

```
CREATE OR ALTER VIEW Taxi.[Kamaz_cabs]  
WITH SCHEMABINDING  
AS  
SELECT CabID, CarNumber  
FROM Taxi.Cab  
WHERE Brand = 'Kamaz'
```

Обновляемые представления

- любые изменения, в том числе инструкции UPDATE, INSERT и DELETE, должны ссылаться на столбцы только одной базовой таблицы;
- изменяемые в представлении столбцы должны прямо ссылаться на данные в столбце базовой таблицы - они не могут быть получены другим способом, например:
 - статистическими функциями (AVG, COUNT, SUM, MIN, MAX, GROUPING, STDEV, STDEVP, VAR или VARP);
 - вычислением на основе других столбцов (столбцы, полученные при помощи операторов UNION, UNION ALL, CROSSJOIN, EXCEPT и INTERSECT, также основаны на вычислениях и поэтому недоступны для изменения);
- изменяемые столбцы не могут указываться в предложениях GROUP BY, HAVING и DISTINCT.

Обновление представлений: CHECK OPTION

- при указании CHECK OPTION все изменения данных через представление должны удовлетворять ограничениям запроса, определяющего представление (таким образом, изменения данных не должны влиять на их видимость через представление).

```
CREATE VIEW dbo.SeattleOnly
AS
SELECT p.LastName, p.FirstName, e.JobTitle, a.City,
       sp.StateProvinceCode
FROM HumanResources.Employee e INNER JOIN Person.Person p
    ON p.BusinessEntityID = e.BusinessEntityID
    INNER JOIN Person.BusinessEntityAddress bea
        ON bea.BusinessEntityID = e.BusinessEntityID
    INNER JOIN Person.Address a
        ON a.AddressID = bea.AddressID
    INNER JOIN Person.StateProvince sp
        ON sp.StateProvinceID = a.StateProvinceID
WHERE a.City = 'Seattle'
WITH CHECK OPTION;
```

Изменение и удаление представлений

```
CREATE VIEW HumanResources.EmployeeHireDate
AS
SELECT p.FirstName, p.LastName, e.HireDate
FROM HumanResources.Employee AS e JOIN Person.Person AS p
      ON e.BusinessEntityID = p.BusinessEntityID ;
GO;
```

```
ALTER VIEW HumanResources.EmployeeHireDate
AS
SELECT p.FirstName, p.LastName, e.HireDate
FROM HumanResources.Employee AS e JOIN Person.Person AS p
      ON e.BusinessEntityID = p.BusinessEntityID
WHERE HireDate < CONVERT(DATETIME,'20020101',101) ;
GO
```

```
DROP VIEW IF EXISTS dbo.Reorder;
```

Индексированные представления

- индексировать представление можно, создав для него уникальный кластеризованный индекс:
 - индексированные представления значительно повышают производительность некоторых типов запросов (например, группирующих множество строк);
 - индексированные представления не очень хорошо подходят для часто обновляющихся базовых наборов данных;
- запрос, формирующий индексированное представление:
 - должен ссылаться на единственную таблицу или соединение нескольких таблиц;
 - может выполнять агрегирование данных (GROUP BY) с ограничениями;
 - ключевые столбцы и условия WHERE и GROUP BY могут содержать только точные детерминированные выражения.

Индексированные представления: пример

```
SET NUMERIC_ROUNDABORT OFF;  
SET ANSI_PADDING, ANSI_WARNINGS, CONCAT_NULL_YIELDS_NULL, ARITHABORT, QUOTED_IDENTIFIER,  
ANSI_NULLS ON;
```

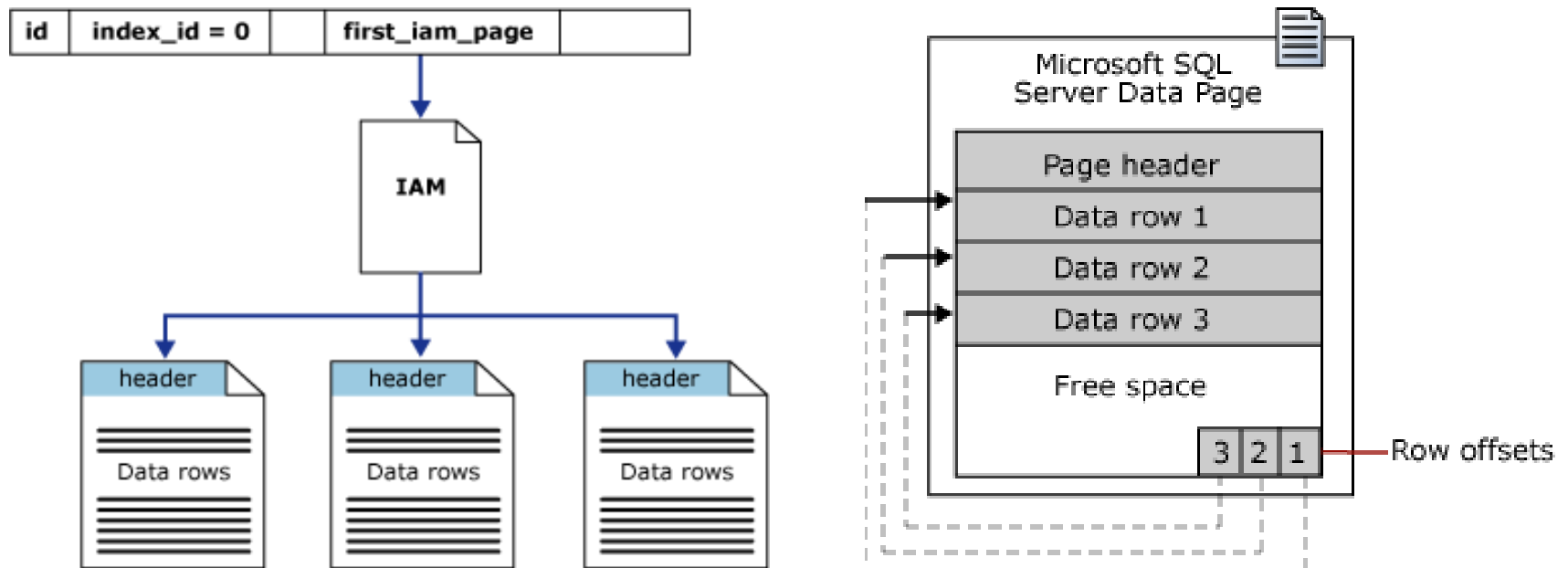
```
CREATE VIEW Sales.vOrders WITH SCHEMABINDING  
AS  
SELECT SUM(UnitPrice * OrderQty * (1.00 - UnitPriceDiscount)) AS Revenue,  
       OrderDate, ProductID, COUNT_BIG(*) AS COUNT  
FROM Sales.SalesOrderDetail AS od, Sales.SalesOrderHeader AS o  
WHERE od.SalesOrderID = o.SalesOrderID  
GROUP BY OrderDate, ProductID;  
GO
```

```
CREATE UNIQUE CLUSTERED INDEX IDX_V1 ON Sales.vOrders (  
    OrderDate, ProductID  
);  
GO
```

```
SELECT OrderDate, SUM(UnitPrice * OrderQty * (1.00 - UnitPriceDiscount)) AS Rev  
FROM Sales.SalesOrderDetail AS od INNER JOIN Sales.SalesOrderHeader AS o  
    ON od.SalesOrderID = o.SalesOrderID  
    AND o.OrderDate >= CONVERT(DATETIME, '03/01/2012', 101)  
    AND o.OrderDate < CONVERT(DATETIME, '04/01/2012', 101)  
GROUP BY OrderDate  
ORDER BY OrderDate ASC;
```

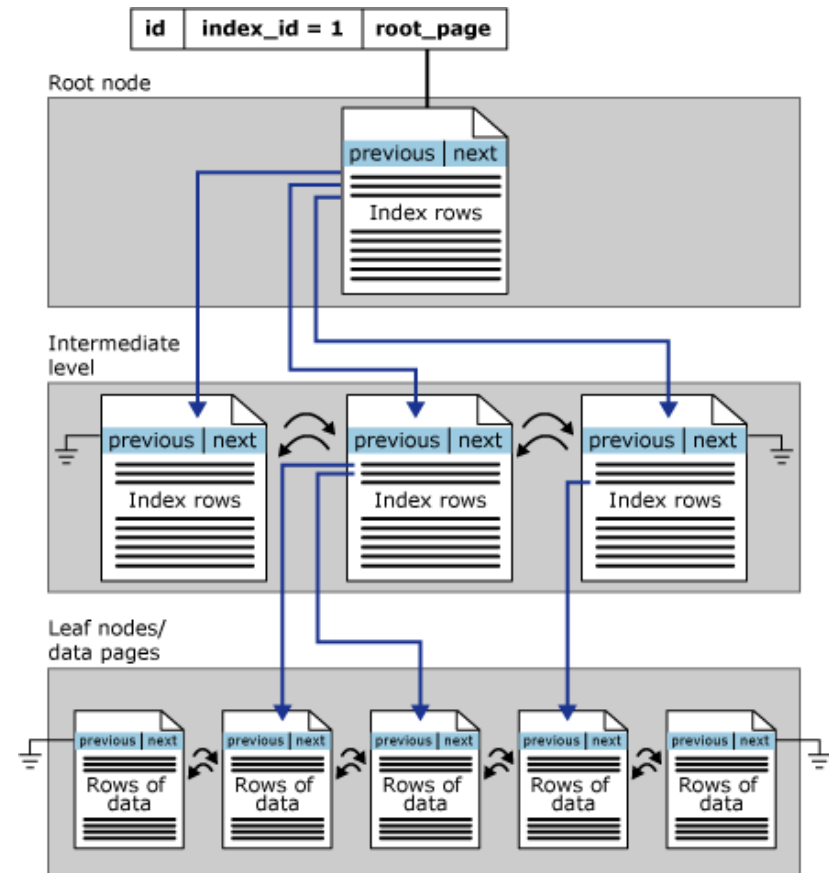
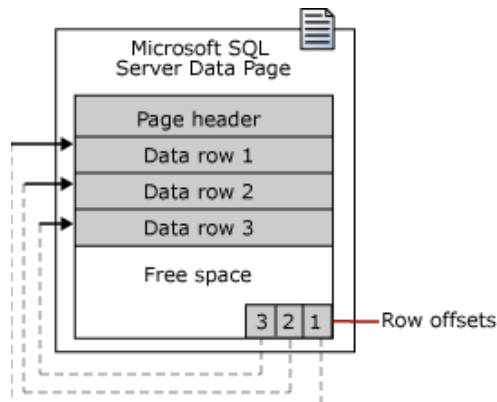
Хранение данных: куча

- если у таблицы нет кластеризованного индекса, то строки данных хранятся в неупорядоченной структуре (куче, *heap*);
- положение строки определяется комбинацией номера страницы и номера строки в странице.



Хранение данных: кластеризованная таблица

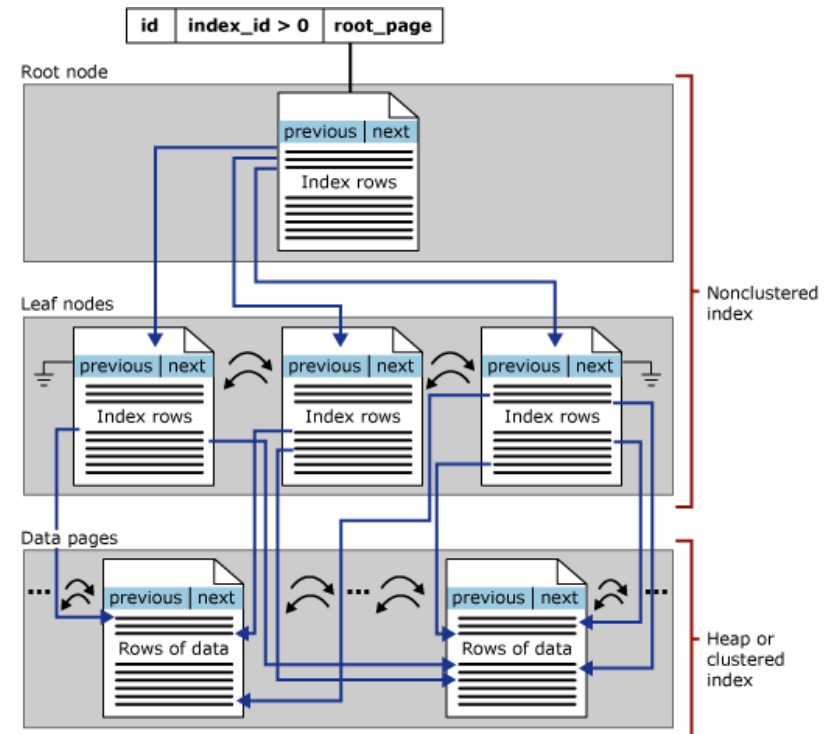
- строки данных хранятся упорядоченно в кластеризованном индексе (*clustered index*):
 - листовые узлы индекса хранят всю строку таблицы (представления), следовательно, строки отсортированы по значению ключа индекса;
 - может быть создан только один кластеризованный индекс для каждой таблицы (представления);
 - положение строки определяется значением ключа.



- ❖ по умолчанию, уникальный кластеризованный индекс создаётся для первичного ключа таблицы

Некластеризованные индексы

- некластеризованные (*non-clustered*) индексы имеют структуру, отдельную от строк данных;
- в некластеризованном индексе содержатся значения его ключа, и каждая запись ключа в листовой странице содержит указатель строки (*row locator*) на соответствующую строку данных:
 - для кучи указатель строки является идентификатором строки (RID, *row identifier*);
 - для кластеризованной таблицы указатель строки является ключом кластеризованного индексе;
- добавление дополнительных неключевых столбцов на конечный уровень некластеризованного индекса позволяет выполнять полностью индексированные (покрывающие, *covering*) запросы.



Проектирование индексов

- при проектировании и создании индексов учитываются следующие аспекты:
 - набор ключевых полей индекса;
 - порядок сортировки индекса;
 - уникальность индекса;
 - добавление неключевых столбцов к конечному уровню некластеризованного индекса;
 - количество, размер и типы столбцов в ключе индекса (до 32 столбцов; общий размер ограничен 900 байтами для кластеризованного индекса и 1700 – для некластеризованного; не допускается использование типов `ntext`, `text`, `varchar(max)`, `nvarchar(max)`, `varbinary(max)`, `xml`, `image`, `json`, `vector`);
 - размещение индексов в файловых группах.

Создание и удаление индексов

- уникальный кластеризованный индекс автоматически создаётся при определении ограничения первичного ключа (PRIMARY KEY);
- уникальный некластеризованный индекс автоматически создаётся при определении ограничения UNIQUE (значение NULL также считается уникальным и может встречаться не более одного раза).

```
CREATE CLUSTERED INDEX IX_TestT_TestC ON dbo.TestTable (TestCol1);
```

```
CREATE NONCLUSTERED INDEX IX_ProductVendor_VendorID  
ON Purchasing.ProductVendor (BusinessEntityID);
```

```
IF EXISTS (SELECT name from sys.indexes  
           WHERE name = N'AK_UnitMeasure_Name'  
           AND object_id = OBJECT_ID(N'Production.UnitMeasure', N'U'))  
  DROP INDEX AK_UnitMeasure_Name ON Production.UnitMeasure;  
GO
```

```
CREATE UNIQUE INDEX AK_UnitMeasure_Name ON Production.UnitMeasure (Name);
```

Покрывающие индексы

- покрывающие (*covering*) индексы могут существенно повысить производительность запросов за счёт того, что все столбцы, необходимые для выполнения запроса, присутствуют в индексе (следовательно, не требуются обращения к основной таблице)
 - составные индексы;
 - индексы с дополнительными неключевыми полями.

```
CREATE INDEX IX_VendorID  
ON dbo.ProductVendor (VendorID DESC, Name ASC, Address DESC);
```

```
CREATE NONCLUSTERED INDEX IX_Address_PostalCode  
ON Person.Address (PostalCode)  
INCLUDE (AddressLine1, AddressLine2, City, StateProvinceID);  
GO
```

```
SELECT AddressLine1, AddressLine2, City, StateProvinceID, PostalCode  
FROM Person.Address  
WHERE PostalCode BETWEEN N'98000' and N'99999';  
GO
```

Частичные индексы

- частичные (*filtered*) индексы позволяют проиндексировать только некоторое подмножество строк таблицы (представляющих наибольший интерес с точки зрения бизнес-логики приложения, использующего данные).

```
DROP INDEX IF EXISTS FIBillOfMaterialsWithEndDate  
    ON Production.BillOfMaterials  
GO
```

```
CREATE NONCLUSTERED INDEX FIBillOfMaterialsWithEndDate  
    ON Production.BillOfMaterials (ComponentID, StartDate)  
    WHERE EndDate IS NOT NULL ;  
GO
```

```
SELECT ProductAssemblyID, ComponentID, StartDate  
FROM Production.BillOfMaterials  
WHERE EndDate IS NOT NULL  
    AND ComponentID = 5  
    AND StartDate > '01/01/2008';
```


Изменение и перестроение индексов

```
CREATE INDEX IX_FF ON dbo.FactFinance (FinanceKey ASC, DateKey ASC);
```

```
CREATE INDEX IX_FF ON dbo.FactFinance (FinanceKey, DateKey, OrganizationKey DESC)  
    WITH (DROP_EXISTING = ON, FILLFACTOR = 80);
```

```
-----  
ALTER INDEX PK_Employee_EmployeeID ON HumanResources.Employee REBUILD;
```

```
-----  
ALTER INDEX ALL ON Production.Product REBUILD WITH (FILLFACTOR = 80, SORT_IN_TEMPDB = ON,  
STATISTICS_NORECOMPUTE = ON);
```

```
-----  
ALTER INDEX IX_Employee_ManagerID ON HumanResources.Employee DISABLE;
```

```
-----  
ALTER INDEX PK_Department_DepartmentID ON HumanResources.Department DISABLE;
```

```
ALTER INDEX PK_Department_DepartmentID ON HumanResources.Department REBUILD;
```

```
ALTER TABLE HumanResources.EmployeeDepartmentHistory  
CHECK CONSTRAINT FK_EmployeeDepartmentHistory_Department_DepartmentID;
```

```
-----  
ALTER INDEX test_idx on test_table REBUILD WITH (ONLINE = ON, MAXDOP = 1, RESUMABLE = ON);
```

Вопросы к экзамену

- Представления: назначение, типы. Привязка к схеме. Обновление данных через представления.
- Хранение данных: куча, кластеризованная таблица. Индексы на основе В-дерева. Некластеризованный индекс. Индексированное представление.
- Проектирование и использование индексов. Уникальные, покрывающие и частичные индексы.

Лабораторная работа №7.

Представления и индексы

1. Создать представление на основе одной из таблиц задания 6.
2. Создать представление на основе полей обеих связанных таблиц задания 6.
3. Создать индекс для одной из таблиц задания 6, включив в него дополнительные неключевые поля; привести пример запроса к данным, выполнение которого может быть ускорено при использовании созданного индекса.
4. Создать индексированное представление.

Базы данных

Тема 9

СУБД SQL Server 2025 – Хранимые процедуры,
курсоры, пользовательские функции

Комментарии

- комментарии представляют собой неисполняемые текстовые строки в программном коде;
- комментарий должен целиком содержаться внутри пакета;
- СУБД SQL Server поддерживает два типа символов выделения комментариев:

1. -- (двойной дефис)

- символы комментария могут находиться в той же строке, что и исполняемый программный код, или в отдельных строках;
- все, что расположено после двойного дефиса и до конца строки, является частью комментария;
- в многострочных комментариях двойной дефис необходимо указывать в начале каждой строки комментария;

2. /* ... */

- символы комментария могут находиться в той же строке, что и исполняемый программный код, или в отдельных строках, или даже внутри исполняемого кода;
- все, что расположено после комбинации символов, открывающей комментарий (/*), и до комбинации символов, закрывающей комментарий (*/), рассматривается как часть комментария.

Пакеты

- пакет - это группа из одной или более инструкций языка Transact-SQL, отправленных единовременно от приложения к SQL Server 2025 для выполнения. SQL Server компилирует инструкции пакета в единый исполняемый модуль, называемый планом выполнения, который затем последовательно выполняется;
- ошибка компиляции, например синтаксическая ошибка, прекращает компиляцию плана выполнения, поэтому никакие инструкции в пакете не выполняются;
- ошибка выполнения, например арифметическое переполнение или нарушение ограничения, приведет к одному из двух результатов:
 - большинство ошибок времени выполнения останавливают текущую инструкцию и инструкции, следующие за ней в пакете;
 - некоторые ошибки времени выполнения, например нарушения ограничений, останавливает только текущую инструкцию, а все остальные инструкции в пакете будут выполнены;
- ошибка выполнения не окажет влияния на инструкции, выполненные до инструкции, которая столкнулась с ошибкой времени выполнения. (единственное исключение - если пакет находится в транзакции, и из-за ошибки будет выполнен откат транзакции, - в этом случае будет выполнен откат любых незафиксированных изменений данных, сделанных перед ошибкой времени выполнения).

Задание пакетов

- наряду с хранимыми процедурами, триггерами и сценариями, пакеты являются механизмом реализации процессов, которые не могут быть реализованы с использованием одиночной инструкции Transact-SQL;
- пакеты реализованы как часть API базы данных:
 - в ADO пакет представляет собой строку инструкций Transact-SQL, заключенную в свойстве CommandText объекта Command

```
Dim Cmd As New ADODB.Command
Set Cmd.ActiveConnection = Cn
Cmd.CommandText = "SELECT * FROM Purchasing.Vendor; SELECT
                    * FROM Production.Product"

Cmd.CommandType = adCmdText
Cmd.Execute
```
 - В OLE DB пакет - это строка инструкций Transact-SQL, заключенная в строку, которая используется для задания текста команды;
 - В ODBC пакет представляет собой строку инструкций Transact-SQL, заключенную в вызове SQLPrepare или SQLExecDirect;
- SQL Server Management Studio и другие программы используют команду GO для обозначения конца пакета. GO - это не инструкция Transact-SQL; она говорит программам, сколько инструкций SQL должно быть включено в пакет.

Объявление переменных

- инструкция DECLARE инициализирует переменную Transact-SQL следующим образом:
 - задает имя, начинающееся с одного знака @ ;
 - задает тип данных, определенный системой или пользователем, и длину; для числовых переменных указывается масштаб и точность;
 - присваивает созданной переменной значение NULL;
- инструкция DECLARE позволяет объявить несколько переменных одинакового или разного типов через запятую;
- областью видимости переменной называют диапазон инструкций Transact-SQL, которые могут к ней обращаться;
- областью видимости переменной являются все инструкции между ее объявлением и концом пакета или хранимой процедуры, где она объявлена;
- для присвоения значения переменной можно:
 - применить инструкцию SET (предпочтительный способ);
 - указать переменную в списке выбора инструкции SELECT;

```
DECLARE @FirstNameVariable nvarchar(20),  
        @RegionVariable nvarchar(30)
```

```
SET @FirstNameVariable = N'Anne'
```


Управление выполнением

- язык Transact-SQL содержит специальные ключевые слова, известные как язык управления выполнением, которые управляют порядком выполнения инструкций на языке Transact-SQL, блоками инструкций и хранимыми процедурами;
- инструкции управления выполнением не могут быть распределены по разным пакетам или хранимым процедурам;
- существуют следующие ключевые слова, управляющие выполнением:
 - BEGIN...END
 - BREAK
 - GOTO
 - CONTINUE
 - IF...ELSE
 - WHILE
 - RETURN
 - WAITFOR

Составной оператор (BEGIN...END)

- инструкции BEGIN и END используются для группирования нескольких инструкций Transact-SQL в одном логическом блоке;
- инструкции BEGIN и END используются, когда:
 - в цикл WHILE необходимо включить блок инструкций;
 - в элемент функции CASE необходимо включить блок инструкций;
 - в предложение IF или ELSE необходимо включить блок инструкций

```
IF (@@ERROR <> 0)
```

```
BEGIN
```

```
    SET @ErrorSaveVariable = @@ERROR
```

```
    PRINT 'Error encountered, ' +
```

```
        CAST(@ErrorSaveVariable AS
```

```
    VARCHAR(10))
```

```
END
```

Ветвления и переходы

- ветвление (проверка выполнения условия):
 IF *expression*
 statement
 ELSE
 statement
- безусловное завершение запроса, хранимой процедуры или пакета:
 RETURN [*@ReturnCode*]
- безусловный переход к метке:
 GOTO *Label*
 ...
 Label_name:
 ...

Организация цикла

- ЦИКЛ:

`WHILE expression`
`statement`

- управление циклом:

- завершение цикла `BREAK`
- переход к очередной итерации `CONTINUE`

```
DECLARE abc CURSOR FOR
SELECT * FROM Purchasing.ShipMethod;
OPEN abc;
FETCH NEXT FROM abc
WHILE (@@FETCH_STATUS = 0)
    FETCH NEXT FROM abc;
CLOSE abc;
DEALLOCATE abc;
GO
```

Инструкция WAITFOR

- инструкция WAITFOR приостанавливает выполнение пакета, хранимой процедуры или транзакции до тех пор, пока:
 - не пройдет заданный интервал времени:
`DELAY(time_to_pass)`
 - не наступит заданное время:
`TIME(time_to_execute)`
 - инструкция RECEIVE не изменит или не возвратит, по крайней мере одну строку в очередь компонента Service Broker:
`RECEIVE / TIMEOUT`

```
WAITFOR DELAY '00:00:02'
```

```
SELECT EmployeeID FROM MyDB.HumanResources.Employee
```

Обработка ошибок

- ошибки, вызываемые компонентом Database Engine, можно обрабатывать на двух уровнях:
 - в компоненте Database Engine, вводя код обработки ошибок в пакеты Transact-SQL, хранимые процедуры, триггеры или в пользовательские функции. Механизмы обработки ошибок Transact-SQL включают:
 - конструкцию TRY...CATCH...END CATCH (получение информации об ошибке с помощью функций ERROR_LINE, ERROR_MESSAGE, ERROR_NUMBER, ERROR_PROCEDURE, ERROR_SEVERITY и ERROR_STATE)
 - инструкции RAISERROR / THROW / PRINT (возврат приложению сообщения в формате системных ошибок или предупреждений / в формате символьной строки)
 - функцию @@ERROR (получение номера ошибки, созданной предыдущей инструкцией языка Transact-SQL)
 - на уровне вызывающего приложения: каждый из API, используемых приложением для доступа к компоненту Database Engine, наделен механизмами для возвращения сведений об ошибках соответствующему приложению.

Хранимые процедуры

- хранимые процедуры в Microsoft SQL Server аналогичны процедурам в других языках программирования:
 - обрабатывают входные аргументы и возвращают вызывающей процедуре или пакету значения в виде выходных аргументов (в том числе через параметры типа cursor);
 - содержат программные инструкции, которые выполняют операции в базе данных, в том числе вызывающие другие процедуры;
 - возвращают значение состояния (код результата) вызывающей процедуре или пакету, таким образом передавая сведения об успешном или неуспешном завершении;
- по сравнению с программами Transact-SQL, которые хранятся локально на клиентских компьютерах, хранимые процедуры SQL Server имеют следующие преимущества:
 - регистрируются на сервере;
 - могут иметь атрибуты безопасности;
 - позволяют сделать защиту приложений более надежной;
 - поддерживают модульное программирование;
 - представляют собой именованный код, дающий возможность отсроченного связывания;
 - позволяют уменьшить сетевой трафик.

Типы хранимых процедур

- пользовательские хранимые процедуры:
 - Transact-SQL: сохраненная коллекция инструкций языка Transact-SQL, которая может принимать и возвращать параметры;
 - CLR: ссылка на метод общезыковой среды выполнения (CRL) платформы Microsoft .NET Framework, который может принимать и возвращать пользователю параметры;
- расширенные хранимые процедуры (устарели и не будут поддерживаться в следующих выпусках): библиотеки DLL, которые могут динамически загружаться и выполняться экземпляром Microsoft SQL Server;
- системные хранимые процедуры: специальные процедуры, позволяющие осуществлять административные действия в SQL Server:
 - физически системные хранимые процедуры хранятся в базе данных ресурсов и имеют префикс `sp_...`;
 - логически же они отображаются в любой системной или пользовательской базе данных в схеме `sys`);
 - SQL Server поддерживает расширенные системные хранимые процедуры (имеют префикс `xp_`), обеспечивающие интерфейс между SQL Server и внешними программами для выполнения различных действий по обслуживанию системы.

Создание и удаление хранимых процедур

```
CREATE PROCEDURE procedure_name [ ; number ]  
    [ { @parameter data_type }  
      [ VARYING ] [ = default ] [ OUTPUT ]  
    ] [ ,...n ]  
AS  
    sql_statement [ ...n ]
```

```
IF OBJECT_ID ('HumanResources.usp_GetAllEmployees', 'P' ) IS NOT NULL  
DROP PROCEDURE HumanResources.usp_GetAllEmployees;  
GO
```

```
CREATE PROCEDURE HumanResources.usp_GetAllEmployees AS  
    SELECT LastName, FirstName, JobTitle, Department  
    FROM HumanResources.vEmployeeDepartment;  
GO
```

```
EXECUTE HumanResources.usp_GetAllEmployees;  
GO  
-- Or  
EXEC HumanResources.usp_GetAllEmployees;  
GO  
-- Or, if this procedure is the first statement within a batch:  
HumanResources.usp_GetAllEmployees;
```

Хранимые процедуры с подстановочными параметрами

```
IF OBJECT_ID ( 'HumanResources.usp_GetEmployees2', 'P') IS NOT NULL
    DROP PROCEDURE HumanResources.usp_GetEmployees2;
```

GO

```
CREATE PROCEDURE HumanResources.usp_GetEmployees2
    @lastname varchar(40) = 'D%',
    @firstname varchar(20) = '%'
```

AS

```
    SELECT LastName, FirstName, JobTitle, Department
    FROM HumanResources.vEmployeeDepartment
    WHERE FirstName
    LIKE @firstname AND LastName LIKE @lastname;
```

GO

```
EXECUTE HumanResources.usp_GetEmployees2 'Wi%';
-- Or
EXECUTE HumanResources.usp_GetEmployees2 @firstname = '%';
-- Or
EXECUTE HumanResources.usp_GetEmployees2 '[CK]ars[OE]n';
-- Or
EXECUTE HumanResources.usp_GetEmployees2 'Hesse', 'Stefen'
```

Возвращение данных с использованием параметров OUTPUT и кода возврата

- если при выполнении хранимой процедуры указано OUTPUT для параметра, а параметр не указан при помощи OUTPUT в хранимой процедуре, выдается сообщение об ошибке;
- можно выполнять хранимую процедуру с параметрами OUTPUT и не указывать OUTPUT при выполнении хранимой процедуры: сообщение об ошибке не будет выдаваться, но нельзя будет использовать выходное значение в вызывающей программе.

```
CREATE PROCEDURE Sales.usp_GetEmployeeSalesYTD
    @SalesPerson nvarchar(50),
    @SalesYTD money OUTPUT
AS
    SELECT @SalesYTD = SalesYTD
    FROM Sales.SalesPerson AS sp
    JOIN HumanResources.vEmployee AS e ON e.EmployeeID = sp.SalesPersonID
    WHERE LastName = @SalesPerson;
RETURN
GO

DECLARE @SalesYTDBySalesPerson money;
EXECUTE @result = Sales.usp_GetEmployeeSalesYTD
    N'Blythe', @SalesYTD = @SalesYTDBySalesPerson OUTPUT;
PRINT 'YTD sales' + convert(varchar(10),@SalesYTDBySalesPerson);
GO
```

Курсоры

- операции в реляционной базе данных выполняются над множеством строк. Приложения, особенно интерактивные, не всегда эффективно работают с результирующим набором (набором строк, возвращаемым инструкцией). *Курсоры* являются расширением результирующих наборов, которые предоставляют механизм, позволяющий обрабатывать одну строку или небольшое их число за один раз;
- курсоры обеспечивают возможность:
 - позиционирования на отдельные строки результирующего набора;
 - получения одной или нескольких строк от текущей позиции в результирующем наборе;
 - изменения данных в строках в текущей позиции результирующего набора.

Типы курсоров

- статические (*static*):
 - результирующий набор фиксируется при открытии курсора (не отражает изменения, влияющие на входение в результирующий набор, изменения значений, а также изменения в результате удаления строк в базе данных);
 - в Microsoft SQL Server доступны только для чтения;
- основанные на потенциальном ключе (*keyset*):
 - набор записей фиксируется при открытии: членство и порядок строк являются фиксированными при открытии курсора;
 - курсоры управляются с помощью набора уникальных идентификаторов – ключей, которые создаются из набора столбцов, уникально идентифицирующего строки результирующего набора;
 - изменения в значениях столбцов, не входящих в ключевой набор (внесенные владельцем курсора или другими пользователями), становятся видны при прокрутке курсора;
 - вставка в базу данных, выполненная вне курсора, видна только после его повторного открытия;
 - при попытке запросить состояние строки, удаленной после открытия курсора, функция @@FETCH_STATUS возвращает состояние "строка отсутствует";
 - обновление ключевого столбца действует аналогично удалению старого значения ключа с последующей вставкой нового значения (если обновление не было выполнено с помощью курсора, то новое ключевое значение не будет видимо);
- динамические:
 - набор записей не фиксируется при открытии курсора (отражаются все изменения строк в результирующем наборе при прокрутке курсора: значения типа данных, порядок и членство строк в результирующем наборе могут меняться для каждой выборки).

Обработка курсоров

- курсоры Transact-SQL и API-курсоры имеют различный синтаксис, но для всех курсоров SQL Server используется одинаковый цикл обработки:
 1. связать курсор с результирующим набором инструкции Transact-SQL и задать его характеристики (например, возможность обновления строк);
 2. выполнить инструкцию Transact-SQL для заполнения курсора;
 3. получить в курсор необходимые строки:
 1. операция получения в курсор одной и более строк называется выборкой;
 2. выполнение серии выборок для получения строк в прямом или обратном направлении называется прокруткой;
 4. при необходимости выполнить операции изменения (обновления или удаления) строки в текущей позиции курсора;
 5. закрыть курсор.
- *однонаправленный курсор* не поддерживает прокрутку, а только последовательную выборку строк от начала курсора до его конца:
 - строки не извлекаются из базы данных, пока они не будут выбраны;
 - результаты всех инструкций INSERT, UPDATE и DELETE, которые влияют на строки результирующего набора и выполненные текущим пользователем или зафиксированы другими пользователями, отображаются в процессе выборки строк из курсора.

Объявление курсора (Transact-SQL) и использование переменной типа *cursor*

```
DECLARE cursor_name CURSOR
    [ LOCAL | GLOBAL ]
    [ FORWARD_ONLY | SCROLL ]
    [ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
    [ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
    [ TYPE_WARNING ]
    FOR select_statement
    [ FOR UPDATE [ OF column_name [ ,...n ] ] ]
```

```
DECLARE @cursor_variable_name CURSOR
```

```
SET @cursor_variable_name = CURSOR
    [ FORWARD_ONLY | SCROLL ]
    [ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
    [ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
    [ TYPE_WARNING ]
    FOR select_statement
    [ FOR UPDATE [ OF column_name [ ,...n ] ] ]
```

Объявление курсора: примеры

```
/* Use DECLARE @local_variable, DECLARE CURSOR and SET*/  
DECLARE @MyVariable CURSOR  
DECLARE MyCursor CURSOR FOR  
SELECT LastName FROM AdventureWorks.Person.Contact  
SET @MyVariable = MyCursor  
GO
```

```
/* Use DECLARE @local_variable and SET */  
DECLARE @MyVariable CURSOR  
SET @MyVariable = CURSOR SCROLL KEYSET FOR  
SELECT LastName FROM AdventureWorks.Person.Contact;  
.....  
DEALLOCATE MyCursor;
```


Работа с курсором

- открытие курсора:

```
OPEN { { [ GLOBAL ] cursor_name }  
      | @cursor_variable_name }
```

- позиционирование курсора:

```
FETCH  
      [ [ NEXT | PRIOR | FIRST | LAST  
        | ABSOLUTE { n | @nvar }  
        | RELATIVE { n | @nvar } ]  
      FROM ]  
      { { [ GLOBAL ] cursor_name } |  
        @cursor_variable_name }  
      [ INTO @variable_name [ ,...n ] ]
```

- заккрытие курсора:

```
CLOSE { { [ GLOBAL ] cursor_name }  
       | @cursor_variable_name }
```

- удаление курсора:

```
DEALLOCATE { { [ GLOBAL ] cursor_name }  
            | @cursor_variable_name }
```

Пример использования курсора

```
DECLARE @name varchar(40)
```

```
DECLARE authors_cursor CURSOR  
    FOR SELECT au_lname FROM authors
```

```
OPEN authors_cursor
```

```
FETCH NEXT FROM authors_cursor INTO @name
```

```
WHILE @@FETCH_STATUS = 0  
BEGIN  
    FETCH NEXT FROM authors_cursor INTO @name  
END
```

```
CLOSE authors_cursor
```

```
DEALLOCATE authors_cursor
```

Изменение данных с использованием курсора

```
DECLARE complex_cursor CURSOR FOR
    SELECT a.BusinessEntityID
    FROM HumanResources.EmployeePayHistory AS a
    WHERE RateChangeDate <>
        (SELECT MAX(RateChangeDate) FROM
         HumanResources.EmployeePayHistory AS b
         WHERE a.BusinessEntityID = b.BusinessEntityID);

OPEN complex_cursor;
FETCH FROM complex_cursor;

DELETE FROM HumanResources.EmployeePayHistory
WHERE CURRENT OF complex_cursor;

CLOSE complex_cursor;
DEALLOCATE complex_cursor;
```

Использование типа данных **cursor** в параметре **OUTPUT**

- хранимые процедуры языка Transact-SQL могут использовать тип данных **cursor** только для параметров **OUTPUT**:
 - если тип данных **cursor** указан для параметра, должны быть также указаны оба параметра **VARYING** и **OUTPUT**;
 - если для параметра указано ключевое слово **VARYING**, тип данных должен быть **cursor** и должно быть указано ключевое слово **OUTPUT**;
- следующие правила относятся к выходным параметрам типа **cursor** при выполнении процедуры:
 - для однонаправленного курсора – в результирующий набор курсора будут возвращены только строки с текущей позиции курсора до конца курсора; текущая позиция курсора определяется при окончании выполнения хранимой процедуры;
 - для прокручиваемого курсора – в результирующий набор курсора будут возвращены все строки; текущая позиция курсора остается в позиции последней выборки, выполненной в процедуре;
 - для любого типа курсора – если курсор закрыт, то будет возвращено значение **NULL**, то же произойдет в случае, если курсор присвоен параметру, но этот курсор никогда не открывался.

Пример использования курсора

```
CREATE PROCEDURE dbo.currency_cursor
    @currency_cursor CURSOR VARYING OUTPUT
AS
    SET @currency_cursor = CURSOR
        FORWARD_ONLY STATIC FOR
        SELECT CurrencyCode, Name
        FROM Sales.Currency;
OPEN @currency_cursor;
GO

DECLARE @MyCursor CURSOR;
EXEC dbo.currency_cursor @currency_cursor = @MyCursor OUTPUT;
WHILE (@@FETCH_STATUS = 0)
BEGIN;
    FETCH NEXT FROM @MyCursor;
END;
CLOSE @MyCursor;
DEALLOCATE @MyCursor;
GO
```

Определяемые пользователем функции

- преимущества использования:
 - возможность модульного программирования;
 - ускорение выполнения за счет кэширования и повторного использования планов выполнения;
 - уменьшение сетевого трафика;
- языки реализации:
 - функции CLR (предоставляют значительный выигрыш в производительности для вычислительных задач, работы со строками и бизнес-логикой, позволяют реализовать статистические функции);
 - функции Transact-SQL (лучше приспособлены для логики доступа к данным);
- ограничение использования функций: инструкции внутри функции могут изменять только локальные по отношению к этой функции объекты, например, локальные курсоры или переменные.

Компоненты пользовательской функции

- функция принимает ноль и более параметров и возвращает:
 - скалярное значение;
 - таблицу;
- любая пользовательская функция состоит из двух частей:
 - заголовка, содержащего:
 - имя функции с необязательным именем схемы или владельца;
 - имя и тип данных входного параметра;
 - тип данных возвращаемого значения и необязательное имя;
 - тела, состоящего из:
 - одной или нескольких инструкций Transact-SQL, реализующих логику функции;
 - ссылки на сборку .NET .

Определяемые пользователем функции: пример

```
CREATE FUNCTION dbo.GetWeekDay (@Date datetime)
    RETURNS int
    AS
    BEGIN
        RETURN DATEPART (weekday, @Date)
    END;
GO
```

```
SELECT dbo.GetWeekDay(CONVERT(DATETIME, '20020201', 101)) AS
    DayOfWeek;
```


Виды определяемых пользователем функций

- скалярные функции:
 - возвращают единственное значение скалярного типа;
 - могут использоваться везде, где допустимы выражения возвращаемого типа;
- табличные функции:
 - возвращают значение типа table;
 - могут использоваться везде, где допустимы табличные выражения (в том числе, как альтернатива представлений и хранимых процедур, возвращающих один результирующий набор).

Свойства функций

- свойства пользовательских функций определяют способность ядра СУБД индексировать результаты функции как с помощью индексов вычисляемых столбцов, вызывающих функцию, так и с помощью индексированных представлений, которые на нее ссылаются:
 - детерминизм: способность возвращать один и тот же результат, если предоставлять им один и тот же набор входных значений и использовать одно и то же состояние базы данных;
 - точность: отсутствие операций над числами с плавающей запятой;
 - доступ к данным: осуществление доступа к локальному серверу баз данных с помощью внутрипроцессного управляемого поставщика SQL Server;
 - доступ к системным данным: осуществление доступа к системным метаданным на локальном сервере баз данных с помощью внутрипроцессного управляемого поставщика SQL Server
 - свойство IsSystemVerified: определяет, может ли ядро СУБД проверить детерминизм и точность функции.

Скалярные функции: синтаксис и пример

```
CREATE FUNCTION function_name
    ( [ { @parameter_name [AS]
        scalar_parameter_data_type } [ ,...n ] ] )
    RETURNS scalar_return_data_type
    [ AS ]
    BEGIN
        function_body
        RETURN scalar_expression
    END
```

```
CREATE FUNCTION dbo.ufnGetStock(@ProductID int)
    RETURNS int
    AS
    BEGIN
        DECLARE @ret int;
        SELECT @ret = SUM(p.Quantity)
            FROM Production.ProductInventory p
            WHERE p.ProductID = @ProductID AND p.LocationID = '6';
        IF (@ret IS NULL)
            SET @ret = 0
        RETURN @ret
    END;
GO
```

Табличные функции: синтаксис

```
CREATE FUNCTION function_name
    ( [ { @parameter_name [AS]
        scalar_parameter_data_type } [ ,...n ] ] )
    RETURNS @return_variable TABLE < table_type_definition >
    [ AS ]
    BEGIN
        function_body
    RETURN
    END
```

----- inline

```
CREATE FUNCTION function_name
    ( [ { @parameter_name [AS]
        scalar_parameter_data_type } [ ,...n ] ] )
    RETURNS TABLE
    [ AS ]
    RETURN [ ( ] select-statement [ ) ]
```

Табличные функции: пример встроенной (inline) функции

```
CREATE FUNCTION Sales.fn_SalesByStore (@storeid int)
RETURNS TABLE
AS
RETURN
(
    SELECT P.ProductID, P.Name, SUM(SD.LineTotal) AS 'YTD Total'
    FROM Production.Product AS P
        JOIN Sales.SalesOrderDetail AS SD
            ON SD.ProductID = P.ProductID
        JOIN Sales.SalesOrderHeader AS SH
            ON SH.SalesOrderID = SD.SalesOrderID
    WHERE SH.CustomerID = @storeid
    GROUP BY P.ProductID, P.Name
);
GO
```

Табличные функции: пример функции, состоящего из нескольких инструкций

```
CREATE FUNCTION dbo.fn_FindReports (@InEmpID INTEGER)
RETURNS @retFindReports TABLE
(
    EmployeeID int primary key NOT NULL,
    Name nvarchar(255) NOT NULL,
    Title nvarchar(50) NOT NULL,
    EmployeeLevel int NOT NULL,
    Sort nvarchar (255) NOT NULL )
AS
BEGIN
    WITH DirectReports(Name, Title, EmployeeID, EmployeeLevel, Sort) AS
        (SELECT CONVERT(Varchar(255), c.FirstName + ' ' + c.LastName), e.Title,
            e.EmployeeID, 1, CONVERT(Varchar(255), c.FirstName + ' ' + c.LastName)
            FROM HumanResources.Employee AS e
            JOIN Person.Contact AS c ON e.ContactID = c.ContactID
            WHERE e.EmployeeID = @InEmpID
            UNION ALL
            SELECT CONVERT(Varchar(255), REPLICATE ('| ' , EmployeeLevel) + c.FirstName + ' ' + c.LastName), e.Title, e.EmployeeID, EmployeeLevel + 1, CONVERT (Varchar(255),
            RTRIM(Sort) + '| ' + FirstName + ' ' + LastName) FROM HumanResources.Employee as e
            JOIN Person.Contact AS c ON e.ContactID = c.ContactID
            JOIN DirectReports AS d ON e.ManagerID = d.EmployeeID
        )
    INSERT @retFindReports
    SELECT EmployeeID, Name, Title, EmployeeLevel, Sort
    FROM DirectReports
    RETURN
END;
GO
```

```
SELECT EmployeeID, Name, Title, EmployeeLevel FROM dbo.fn_FindReports(109) ORDER BY Sort;
```

Вопросы к экзамену

- Основные элементы синтаксиса Transact-SQL: управление выполнением, обработка ошибок
- Хранимые процедуры: назначение, типы
- Курсоры: назначение, типы, обработка
- Определяемые пользователем функции: скалярные функции. Свойства функций и их влияние на эффективность использования функций
- Определяемые пользователем функции: табличные функции. Сравнение функций и хранимых процедур

Лабораторная работа №8.

Хранимые процедуры, курсоры и пользовательские функции

1. Создать хранимую процедуру, производящую выборку из некоторой таблицы и возвращающую результат выборки в виде курсора.
2. Модифицировать хранимую процедуру п.1. таким образом, чтобы выборка осуществлялась с формированием столбца, значение которого формируется пользовательской функцией.
3. Создать хранимую процедуру, вызывающую процедуру п.1., осуществляющую прокрутку возвращаемого курсора и выводящую сообщения, сформированные из записей при выполнении условия, заданного еще одной пользовательской функцией.
4. Модифицировать хранимую процедуру п.2. таким образом, чтобы выборка формировалась с помощью табличной функции.

Базы данных

Тема 10

СУБД SQL Server 2025 – Триггеры

Триггеры

- *триггером* называют хранимую процедуру особого типа, которая автоматически выполняется при возникновении языкового события;
- триггеры являются одним из двух механизмов реализации бизнес-правил и целостности данных (наряду с ограничениями);
- типы триггеров:
 - DML-триггер: действие, которое выполняется при наступлении события языка DML (инструкции UPDATE, INSERT и DELETE, выполняемые в таблице или представлении) на сервере базы данных - используются для расширения бизнес-правил при изменении данных, а также для расширения логики проверки целостности ограничений, правил и значений по умолчанию;
 - DDL-триггер: действие, которое выполняется при возникновении событий языка определения данных (DDL) на сервере баз данных;
- триггер и инструкция, при выполнении которой он срабатывает, считаются одной транзакцией, которую можно откатить назад внутри триггера;
- варианты реализации:
 - Transact-SQL,
 - CLR.

Триггеры DML: варианты использования

- для каскадных изменений в связанных таблицах базы данных; (данные изменения можно более эффективно выполнить при помощи каскадных ограничений ссылочной целостности);
- для предотвращения случайных или некорректных операций INSERT, UPDATE и DELETE;
- для реализации других более сложных ограничений, чем определенных при помощи ограничения CHECK (в отличие от ограничений CHECK, DML-триггеры могут ссылаться на столбцы других таблиц);
- для оценки состояния и анализа таблицы до и после изменения данных и выполнения действия на основе этого различия;
- несколько DML-триггеров одинакового типа (INSERT, UPDATE или DELETE) для таблицы позволяют предпринять несколько различных действий в ответ на одну инструкцию изменения данных;
- для реализации более сложных методов управления ошибками (по сравнению со стандартными системными сообщениями);
- для определения необходимых действий по обновлению представления (расширение типов обновлений, которые поддерживает представление).

Типы триггеров DML

- триггеры AFTER (или FOR):
 - могут применяться только в таблицах;
 - выполняются после выполнения инструкций INSERT, UPDATE или DELETE;
 - выполняются после проверки ограничений;
 - может быть более одного для каждого действия (UPDATE, DELETE или INSERT), можно задать выполнение в первую и в последнюю очередь;
 - не могут работать со столбцами типа text, ntext и image;
- триггеры INSTEAD OF:
 - используются в таблицах и представлениях (с одной или более базовыми таблицами);
 - выполняются вместо обычных действий, запускающих триггеры (например, дополнительная обработка может предотвращать нарушения ограничений), при повторном вызове действия триггер не срабатывает;
 - выполняются до проверки ограничений;
 - единственный триггер на каждое из запускающих триггеры действие (UPDATE, DELETE или INSERT);
 - не допускаются в таблицах, в которых используется каскадное обновление (для UPDATE и DELETE);
 - могут работать со столбцами типа text, ntext и image.

Специальные средства, доступные в триггерах

- таблицы `inserted` и `deleted` (соответственно, вставленных и удаленных значений), формат таблиц соответствует формату таблицы или представления, для которых задан триггер;
- функция `COLUMNS_UPDATED()` возвращает битовую маску обновленных столбцов;
- функция `UPDATE(column_name)` возвращает `TRUE`, если столбец был обновлен.

Создание триггера DML

```
CREATE TRIGGER trigger_name
  ON { table | view }
  { FOR | AFTER | INSTEAD OF }
  { [ DELETE ] [ , ] [ INSERT ] [ , ] [ UPDATE ] }
AS
  sql_statement [ ...n ]
```

```
CREATE TRIGGER TableAInsertTrig ON TableA
  INSTEAD OF INSERT AS ...
```

```
CREATE TRIGGER TableBDeleteTrig ON TableB
  AFTER DELETE AS ...
```

-- uses the FOR keyword to generate an AFTER trigger.

```
CREATE TRIGGER TableCUpdateTrig ON TableC
  FOR UPDATE AS ...
```

Использование триггеров DML: пример

```
CREATE TABLE my_table (a int NULL, b int NULL)
GO
```

```
CREATE TRIGGER my_trig
    ON my_table
    FOR INSERT
    AS
    IF UPDATE(b)
        PRINT 'Column b Modified'
GO
```

```
CREATE TRIGGER my_trig2
    ON my_table
    FOR INSERT
    AS
    IF ( COLUMNS_UPDATED() & 2 = 2 )
        PRINT 'Column b Modified'
GO
```

Триггеры DML: работа с одной и несколькими строками

```
CREATE TRIGGER NewPODetail
ON Purchasing.PurchaseOrderDetail
AFTER INSERT
AS
    UPDATE PurchaseOrderHeader
    SET SubTotal = SubTotal + LineTotal
    FROM inserted
    WHERE PurchaseOrderHeader.PurchaseOrderID = inserted.PurchaseOrderID;
```

```
CREATE TRIGGER NewPODetail2
ON Purchasing.PurchaseOrderDetail
AFTER INSERT
AS
    UPDATE PurchaseOrderHeader SET SubTotal = SubTotal +
        (SELECT SUM(LineTotal) FROM inserted WHERE
         PurchaseOrderHeader.PurchaseOrderID = inserted.PurchaseOrderID)
    WHERE PurchaseOrderHeader.PurchaseOrderID
    IN (SELECT PurchaseOrderID FROM inserted);
```


Триггеры DDL

- триггеры DDL - особый вид триггеров, которые срабатывают при выполнении инструкций языка DDL (CREATE, ALTER, DROP и т.п.)
- триггеры DDL могут применяться при выполнении административных задач, например, для аудита и регулирования операций в базе данных:
 - для предотвращения внесений определенных изменений в схему базы данных;
 - для выполнения в базе данных некоторых действий в ответ на изменения в схеме базы данных;
 - для записи изменений или событий схемы базы данных;
- триггеры DDL срабатывают только после выполнения соответствующих инструкций DDL;
- триггер DDL и инструкция, приводящая к его срабатыванию, выполняются в одной транзакции;
- для одной инструкции Transact-SQL можно создать несколько триггеров DDL.

Триггеры DDL: пример

```
CREATE TRIGGER safety
ON DATABASE
FOR DROP_TABLE, ALTER_TABLE
AS
    PRINT 'You must disable Trigger "safety" to drop or alter tables!'
    ROLLBACK ;
```

```
CREATE TRIGGER ddl_trig_login
ON ALL SERVER
FOR DDL_LOGIN_EVENTS
AS
    PRINT 'Login Event Issued.'
    SELECT EVENTDATA().value('(/EVENT_INSTANCE/TSQLCommand/CommandText)[1]',
        'nvarchar(max)')
GO
```

```
DROP TRIGGER ddl_trig_login
ON ALL SERVER
GO;
```

Вопросы к экзамену

- Триггеры: назначение, типы. Триггеры DDL.
- Триггеры DML: триггеры типа INSTEAD OF.
- Триггеры DML: триггеры типа AFTER.
- Триггеры DML: сравнение триггеров типа INSTEAD OF и AFTER. Дополнительные средства управления данными, доступные в триггерах.

Лабораторная работа №9.

Триггеры DML.

1. Для одной из таблиц пункта 2 задания 7 создать триггеры на вставку, удаление и добавление, при выполнении заданных условий один из триггеров должен инициировать возникновение ошибки (RAISERROR / THROW).
2. Для представления пункта 2 задания 7 создать триггеры на вставку, удаление и добавление, обеспечивающие возможность выполнения операций с данными непосредственно через представление.

Базы данных

Тема 11

Транзакции

Управление параллельной обработкой в базах данных

- управление параллельной обработкой (управление параллелизмом, *concurrency control*) направлено на то, чтобы исключить *непредусмотренное* влияние действий одного пользователя на действия другого;
- управление параллельной обработкой, как правило, предполагает компромисс между уровнем изоляции различных операций друг от друга и производительностью системы;
- транзакция (*transaction*) является последовательностью операций, выполненных как одна логическая единица работы (*logical unit of work*), т.е. либо все действия внутри транзакции выполняются успешно, либо не выполняется ни одно из них:
 - BEGIN TRANSACTION;
 - COMMIT | ROLLBACK TRANSACTION (фиксация / откат).

Параллельная обработка транзакций

- параллельные транзакции (*parallel transactions*) – две или более транзакций, обрабатываемые одновременно;
- проблемы параллельной обработки:
 - *грязное / несогласованное чтение* (dirty / inconsistent reads);
 - *невоспроизводимое / фантомное чтение* (unrepeatable / phantom reads);
 - *потерянное обновление* (lost / concurrent update);
- необходимо упорядочить операции внутри транзакций таким образом, чтобы предотвратить *нежелательное влияние* одной транзакции на другую.

Проблема грязного чтения

- грязное чтение (dirty read)

Write-Read Conflict

T_1 : WRITE(A)

T_1 : ABORT

T_2 : READ(A)

Проблема несогласованного чтения

- несогласованное чтение (read skew)

Write-Read Conflict

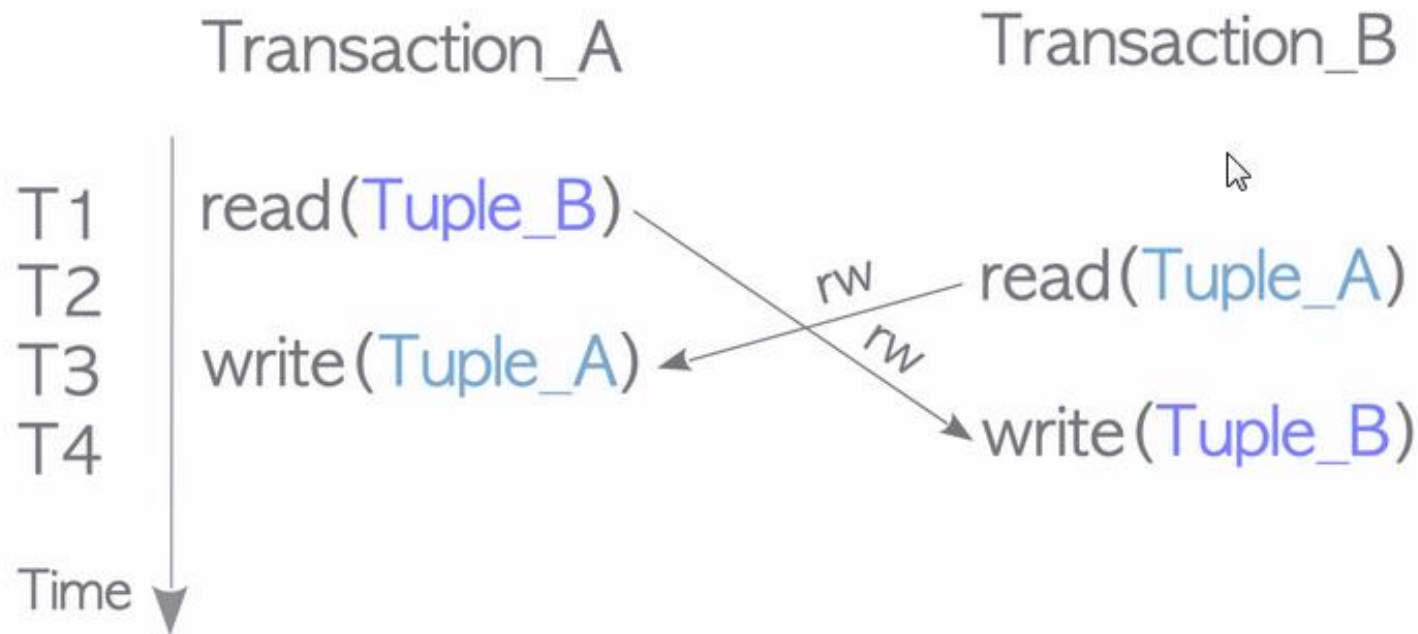
T_1 : $A := 20$; $B := 20$;
 T_1 : WRITE(A)

T_1 : WRITE(B)

T_2 : READ(A);
 T_2 : READ(B);

Проблема несогласованной записи

- несогласованная запись (write skew)



Проблема невоспроизводимого чтения

- невоспроизводимое чтение
(non-repeatable read)

Read-Write Conflict

T_1 : WRITE(A)

T_2 : READ(A);

T_2 : READ(A);

Проблема потерянного обновления

User A

1. Read item 100
(item count is 10).
2. Reduce count of items by 5.
3. Write item 100.

User B

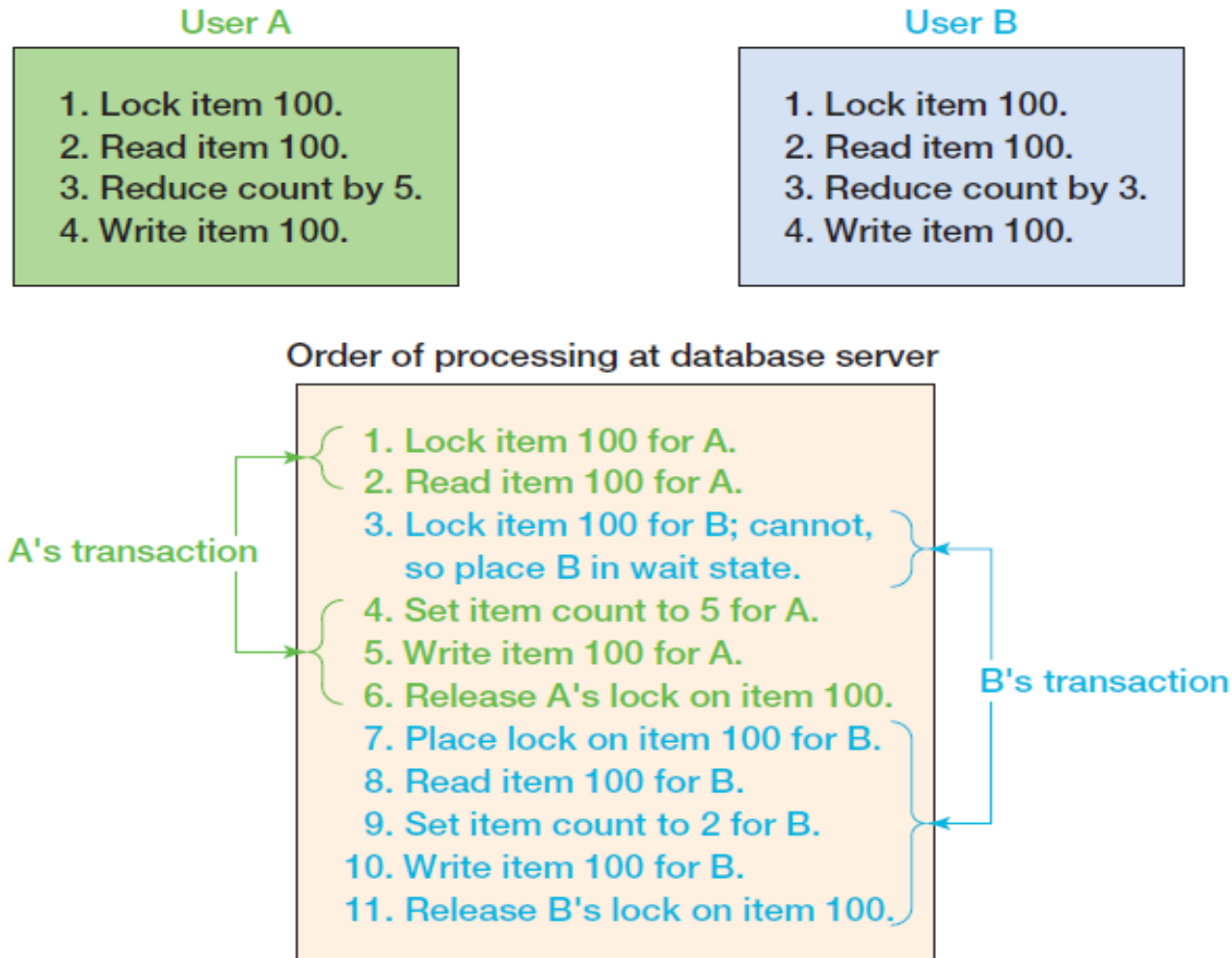
1. Read item 100
(item count is 10).
2. Reduce count of items by 3.
3. Write item 100.

Order of processing at database server

1. Read item 100 (for A).
2. Read item 100 (for B).
3. Set item count to 5 (for A).
4. Write item 100 for A.
5. Set item count to 7 (for B).
6. Write item 100 for B.

Note: The change and write in steps 3 and 4 are lost.

Решение проблемы потерянного обновления с помощью транзакций



Сериализуемость транзакций

- идеальный случай: последовательное (*serial*) выполнение, при котором исход транзакции зависит только от состояния данных перед её началом и от действий внутри самой транзакции;
- транзакция – последовательность операций чтения и записи (неделимые абстрактные операции над объектами базы данных);
- конфликт двух операций возникает, если:
 - они обращаются к одному и тому же объекту;
 - хотя бы одна из них является операцией записи;
- последовательности операций (*schedule*) являются эквивалентными относительно конфликтов (*conflict equivalence*), если конфликтующие операции выполняются в одинаковом порядке;
- сериализуемость транзакций (*serializable transactions*) – такая схема обработки транзакций, при которой результат их параллельной обработки такой же, как если бы они выполнялись последовательно (то есть, существует эквивалентная относительно конфликтов последовательность операций, соответствующая последовательному выполнению транзакций).

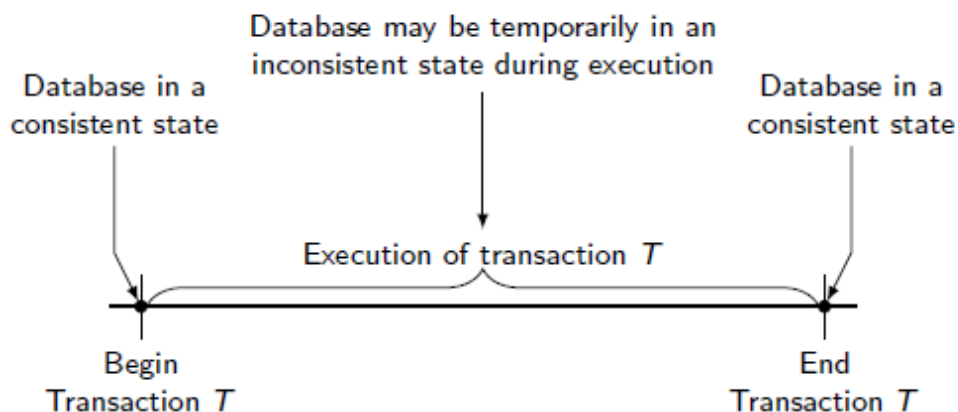
Свойства транзакций

- транзакция обладает четырьмя свойствами (ACID):
 - *атомарность (atomicity)*: транзакция должна быть атомарной единицей работы; должны быть выполнены либо все входящие в нее модификации данных, либо ни одно из них не должно быть выполнено;
 - *согласованность (consistency)*: по завершении транзакция должна оставить все данные в согласованном состоянии (включая все внутренние структуры данных), для этого модификации должны применяться к данным в том состоянии, в котором они были на момент начала операции:
 - на уровне оператора (*statement-level consistency*);
 - на уровне транзакции (*transaction-level consistency*);
 - *изоляция (isolation)*: выполняемые транзакцией модификации должны быть (могут быть) изолированы от любых модификаций, проводимых другими транзакциями;
 - *устойчивость (durability)*: после завершения транзакции произведенные ею действия фиксируются в системе (даже в случае системного сбоя).

Согласованность транзакций

Согласованность (consistency): по завершении транзакция должна оставить все данные в согласованном состоянии (включая все внутренние структуры данных), для этого модификации должны применяться к данным в том состоянии, в котором они были на момент начала операции:

- на уровне оператора (*statement-level consistency*);
- на уровне транзакции (*transaction-level consistency*).



```
BEGIN TRANSACTION;  
UPDATE CUSTOMER  
    SET AreaCode = '425'  
    WHERE ZipCode = '98050';  
COMMIT TRANSACTION;
```

```
BEGIN TRANSACTION;  
UPDATE CUSTOMER  
    SET AreaCode = '425'  
    WHERE ZipCode = '98050';
```

```
UPDATE CUSTOMER  
    SET Discount = 0.05  
    WHERE AreaCode = '425';  
COMMIT TRANSACTION;
```


Изоляция транзакций

- цель изоляции транзакций (сериализации) – предотвращение чтения неполных результатов другими транзакциями, что предотвращает появление различных аномалий (таких как потерянные обновления и т.п.) и необходимость каскадных откатов (*cascade aborts*);
- управление параллельной обработкой, как правило, предполагает компромисс между уровнем изоляции различных операций друг от друга и производительностью системы;
- фактически изоляция транзакций означает, что клиент СУБД может устанавливать требуемый уровень изоляции, а СУБД, соответственно, обеспечивает управление параллельным выполнением для достижения нужного уровня изоляции.

Стандарт SQL-92: аномалии изоляции транзакций

- проблемы (аномалии, *phenomena*) изоляции транзакций, рассматриваемые стандартом SQL-92:
 - «грязное» чтение (*dirty read*) – чтение транзакцией записи, измененной другой транзакцией, при этом эти изменения еще не зафиксированы;
 - невозпроизводимое чтение (*non-repeatable read*) – при повторном чтении транзакция обнаруживает измененные или удаленные данные, зафиксированные другой транзакцией;
 - фантомное чтение (*phantom read*) – при повторном чтении транзакция обнаруживает новые строки, вставленные другой завершенной транзакцией;

Стандарт SQL-92: уровни изоляции транзакций

- стандарт SQL-92 определяет четыре уровня изоляции транзакций:
 - незавершенное чтение (*read uncommitted*);
 - завершенное чтение (*read committed*);
 - воспроизводимое чтение (*repeatable read*);
 - сериализуемость (*serializable [anomaly-serializable]*);

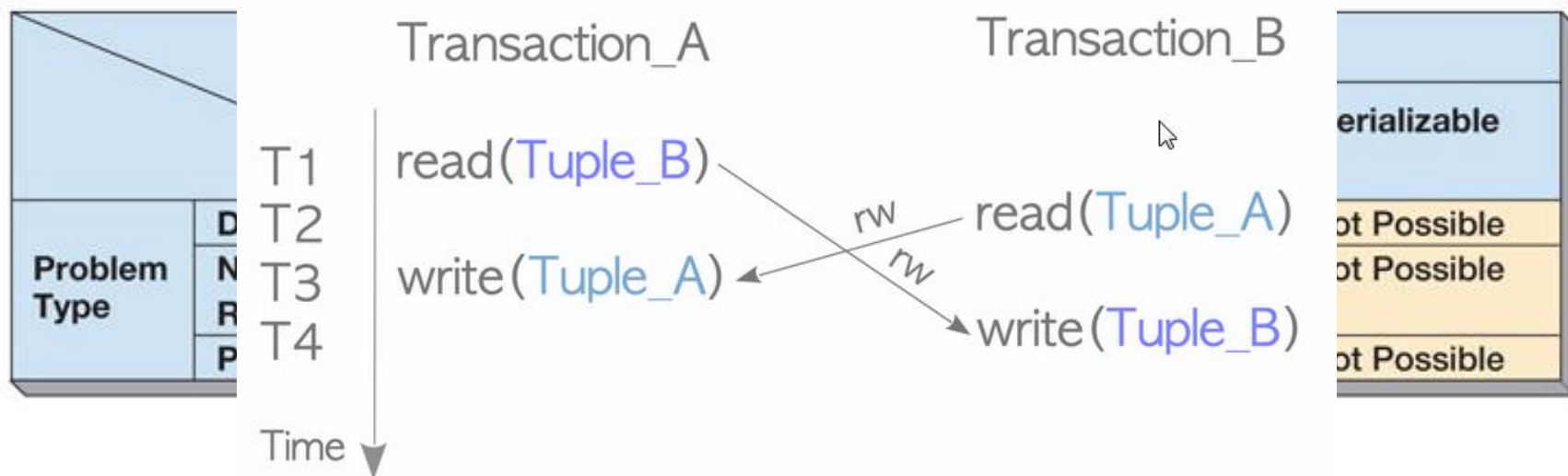
		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not Possible	Not Possible	Not Possible
	Nonrepeatable Read	Possible	Possible	Not Possible	Not Possible
	Phantom Read	Possible	Possible	Possible	Not Possible

Платформы и уровни изоляции

	Read uncommitted	Read committed	Cursor stability	Repeatable read	Snapshot isolation	Serializable
Oracle		+				+
Db2	+		+	+		+
Microsoft SQL Server	+	+		+	+	+
PostgreSQL	+	+		+		+
MySQL	+	+		+		+
MongoDB ²	+	+			+	
SQLite					+	
CockroachDB						+
YugabyteDB		+			+	+

Изоляция моментальных снимков

- изоляция моментальных снимков (snapshot isolation) позволяет транзакциям получать согласованный набор данных, существовавший на момент начала транзакции – версионирование строк (*row versioning*);
- таким образом, чтение данных никогда не блокируется записью, но данные могут быть неактуальными;
- фиксация транзакции происходит только если изменяемые данные не были изменены (зафиксированы) другой транзакцией.

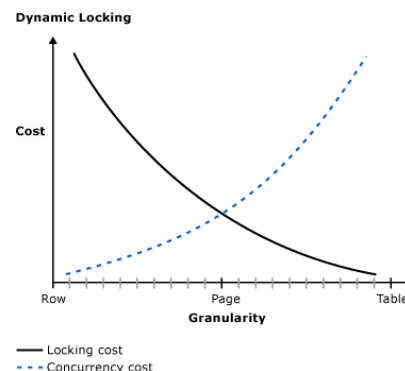


Стратегии и алгоритмы управления параллелизмом

- механизмы синхронизации:
 - блокировка (*resource-locking*): взаимоисключающий доступ к разделяемым данным;
 - упорядочивание транзакций в соответствии с набором правил (*transaction ordering*), в том числе упорядочивание по временным меткам (*TO, timestamp ordering*);
- стратегии, обеспечивающие изоляцию транзакций:
 - пессимистическая (*pessimistic concurrency control*): обнаружение и предотвращение конфликтов в процессе выполнения транзакции;
 - оптимистическая блокировка (*optimistic concurrency control*): анализ конфликтов в момент фиксации транзакции.

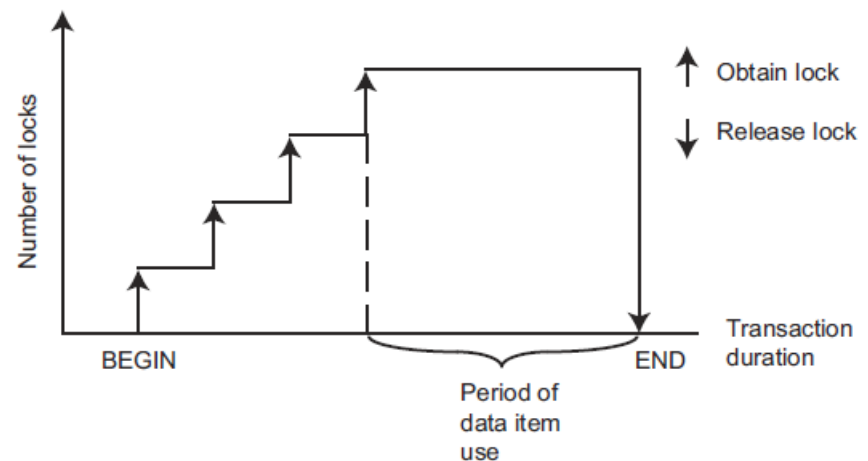
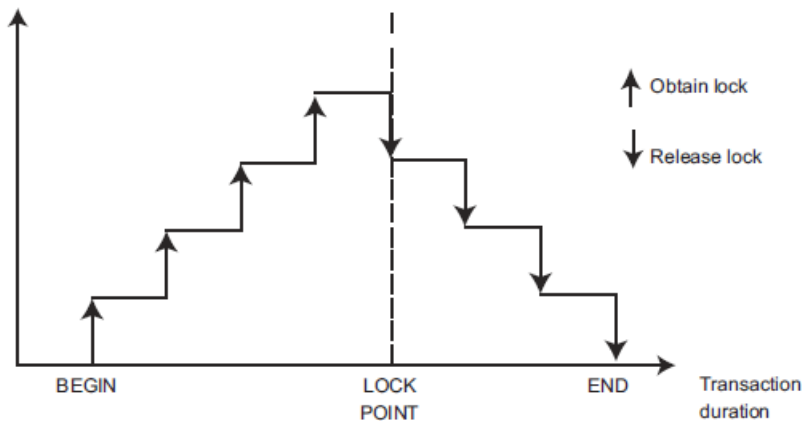
Блокировка ресурсов

- инициатор блокировки:
 - СУБД: неявная блокировка (*implicit lock*);
 - программа / запрос: явная блокировка (*explicit lock*);
- степень (глубина) детализации блокировки (*locking granularity*):
 - на уровне базы данных;
 - на уровне таблицы;
 - на уровне страницы;
 - на уровне строки;
- режим блокировки (*lock mode*):
 - *монопольный* (*exclusive lock / write lock*): блокируются все виды доступа к элементу;
 - *разделяемый* (*shared lock / read lock*): блокируется только запись.



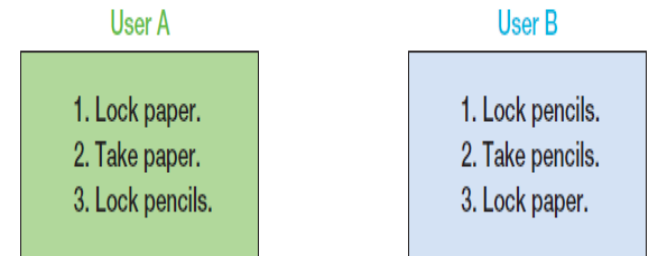
Двухфазная блокировка

- в общем случае типы и длительность удержания блокировок определяют уровень изоляции транзакций;
- один из способов достижения *сериализуемости* – использование *двухфазной блокировки* (two-phase locking):
 - *фаза нарастания* (growing phase): транзакция может налагать блокировки по мере необходимости, до тех пор, пока не снимается хотя бы одна блокировка;
 - *фаза сжатия* (shrinking phase): после снятия одной из блокировок никакие другие накладываться уже не могут;
- модификация двухфазной блокировки (S2PL – strict 2PL): снятие блокировок непосредственно в момент фиксации / отката транзакции (предотвращает каскадные откаты).

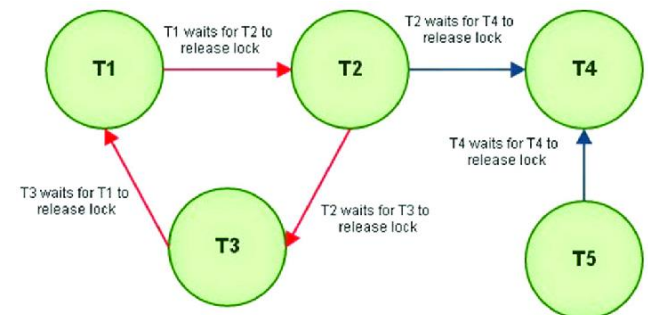
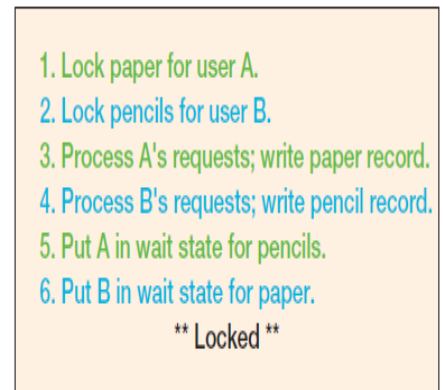


Взаимная блокировка

- взаимная блокировка (*deadlock*, *deadly embrace*) – состояние взаимного ожидания транзакциями освобождения ресурсов, заблокированных друг другом;
- анализ: граф ожидания (*WFG*, *wait-for graph*);
- обработка взаимной блокировки:
 - предотвращение (*deadlock prevention*): блокирование всех ресурсов одновременно в начале операции;
 - избежание (*deadlock avoidance*): требование блокировки в одинаковом порядке;
 - обнаружение и устранение (*deadlock detection and resolution*): откат одной или нескольких транзакций, вызвавших блокировку (*victim transaction*) – оценка стоимости отката / стоимости завершения / количества разрешаемых блокировок (циклов).



Order of processing at database server



Упорядочивание транзакций

- алгоритмы упорядочивания транзакций выбирают порядок сериализации априори – в соответствии с уникальной и монотонно возрастающей меткой транзакции (*transaction timestamp*);
- правило упорядочивания: конфликтующая операция более «старой» транзакции имеет приоритет;
- при наличии конфликтов в очередь операций может быть добавлена новая операция только той транзакции, которая является самой последней из конфликтующих; в противном случае производится откат / перезапуск транзакции;
- для реализации данного алгоритма каждый объект должен также иметь метки чтения и записи, соответствующие максимальным меткам транзакций, осуществлявшим чтение / запись данного объекта.

Многоверсионное управление параллельным доступом (MVCC, Multi-Version Concurrency Control)

- множественные версии объекта («снимки», *snapshots*);
- завершённое чтение (read committed):
 - при чтении транзакция получает самую последнюю версию объекта *на момент начала очередной инструкции*;
- изоляция моментальных снимков (SI, snapshot isolation):
 - при чтении транзакция получает самую последнюю версию объекта *на момент начала данной транзакции*;
 - фиксация транзакции происходит только если изменяемые данные не были изменены (зафиксированы) другой транзакцией (“first committer wins”);
- сериализуемая изоляция моментальных снимков (SSI, serializable snapshot isolation):
 - при чтении транзакция получает самую последнюю версию объекта *на момент начала данной транзакции*;
 - фиксация транзакции происходит только если изменяемые данные не были изменены (зафиксированы) или прочитаны другой транзакцией;
- собственные изменения видны в транзакции.

Многоверсионное управление параллельным доступом: сериализуемая изоляция моментальных снимков

- множественные версии объекта («снимки»);
- при чтении транзакция получает самую последнюю версию объекта *на момент начала данной транзакции*;
- при записи:

$$TS(T_i) < RTS(X_k) = TS(T_j) \Rightarrow cancel T_i$$

$$X_k^t \rightarrow X_k^{t+1}; RTS(X_k^{t+1}) = WTS(X_k^{t+1}) = TS(T_i)$$

- условие удаления «устаревших» версий:

$$\exists X_k^{t+1} | TS(X_k^{t+1}) > TS(X_k^t) \ \& \ \forall T_i \ TS(T_i) > TS(X_k^{t+1})$$



Уровни изоляции в популярных реляционных СУБД

 PostgreSQL	Dirty Read	Lost Update	Phantom	Nonrepeatable read	Skewed write
Read Uncommitted	No	Yes	Yes	Yes	Yes
Read Committed					
Repeatable Read	No	No	No	No	Yes
Serializable	No	No	No	No	No

 Microsoft SQL Server	Dirty Read	Lost Update	Phantom	Nonrepeatable read	Skewed write
Read Uncommitted	Yes	Yes	Yes	Yes	Yes
Read Committed	No	Yes	Yes	Yes	No
Repeatable Read	No	No	Yes	No	No
Snapshot isolation	No	No	No	No	Yes
Serializable	No	No	No	No	No

 ORACLE	Dirty Read	Lost Update	Phantom	Nonrepeatable read	Skewed write
Read Uncommitted	-				
Read Committed	No	Yes	Yes	Yes	Yes
Repeatable Read	-				
Serializable	No	No	No	No	Yes

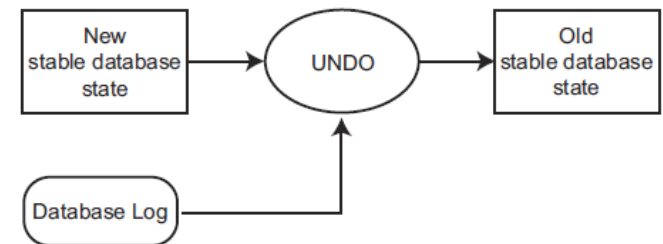
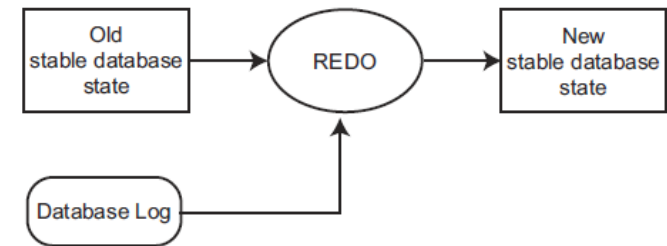
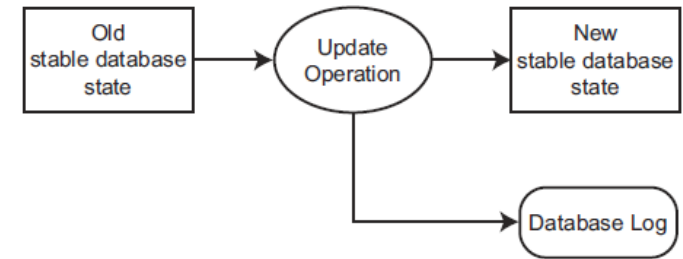
 MySQL	Dirty Read	Lost Update	Phantom	Nonrepeatable read	Skewed write
Read Uncommitted	Yes	Yes	Yes	Yes	Yes
Read Committed	No	Yes	Yes	Yes	Yes
Repeatable Read	No	Yes	No	No	Yes
Serializable	No	No	No	No	No

Восстановление баз данных

- три типа сбоев в базах данных:
 - сбои транзакций (*transaction failures*) – откаты;
 - системные сбои (*system failures*) – потеря содержимого оперативной памяти;
 - сбои носителей (*media failures*) – потеря данных на диске;
- для предотвращения катастрофических последствий при сбое носителей:
 - дублирование дисков;
 - создание резервных копий базы данных (полных или инкрементных копий);
- восстановление после сбоев транзакций и системных сбоев зависит от организации записи обновлений основных данных в СУБД:
 - обновление с перезаписью (*in-place updating*);
 - обновление с заменой (*out-of-place updating*);
- для восстановления в СУБД с перезаписью необходим журнал базы данных (*database log*):
 - синонимы: журнал транзакций (*transaction log*), журналом завершённых транзакций (*commit log*), журнал повторного выполнения (*redo log*) или журнал опережающей записи (*write-ahead log*, *WAL*);
 - содержит информацию для восстановления (*recovery information*), которая необходима для выполнения набора идемпотентных операций.

Восстановление баз данных: подходы

- повторная обработка (reprocessing):
периодические снимки базы данных и записи о всех операциях со времени последнего сохранения резервной копии – нереализуемый подход в силу отсутствия времени и асинхронного характера первоначальных изменений;
- откат-накат (rollback-rollforward):
 - периодические снимки базы данных и журнал транзакций со времени последнего сохранения;
 - rollforward: выполнение всех корректных транзакций (для повторного выполнения транзакции журнал должен содержать копию записей базы данных после их изменения – конечные образы, *after-images*);
 - rollback: отмена изменений, вызванных некорректными или незавершенными транзакциями (для отмены транзакции журнал должен содержать копию записей базы данных до их изменения – исходные образы, *before-images*);



СУБД SQL Server 2025 – Транзакции и восстановление.

Управление параллельной обработкой

- определение уровня изоляции транзакции:

```
SET TRANSACTION ISOLATION LEVEL
```

```
{ READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ |  
  SNAPSHOT | SERIALIZABLE }
```

- определение характеристик курсора:

```
DECLARE cursor_name CURSOR
```

```
[ LOCAL | GLOBAL ]
```

```
[ FORWARD_ONLY | SCROLL ]
```

```
[ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
```

```
[ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
```

```
[ TYPE_WARNING ]
```

```
FOR select_statement
```

```
[ FOR UPDATE [ OF column_name [ ,...n ] ] ]
```

- блокировочные подсказки инструкций (SELECT, INSERT, UPDATE, DELETE):

```
SELECT ... FROM ... WITH
```

Определение уровня изоляции транзакций

- возможные уровни изоляции (блокировка / версионирование):
 - READ UNCOMMITTED;
 - **READ COMMITTED;**
 - READ_COMMITTED_SNAPSHOT (ON | OFF);
 - REPEATABLE READ;
 - SNAPSHOT – согласованность на уровне транзакции;
 - SERIALIZABLE;
- возможно переключение уровня изоляции внутри транзакции (кроме → SNAPSHOT, правила блокировок применяются на основании текущего уровня изоляции);
- при выходе из хранимой процедуры / триггера уровень изоляции восстанавливается до уровня, актуального на момент вызова.

		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not Possible	Not Possible	Not Possible
	Nonrepeatable Read	Possible	Possible	Not Possible	Not Possible
	Phantom Read	Possible	Possible	Possible	Not Possible

Уровни изоляции транзакций (пример)

- разделяемые блокировки удерживаются все время до завершения транзакции.

```
USE AdventureWorks;  
GO
```

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
BEGIN TRANSACTION;
```

```
SELECT * FROM HumanResources.EmployeePayHistory;  
SELECT * FROM HumanResources.Department;
```

```
COMMIT TRANSACTION;
```

Блокировочные подсказки инструкций

SELECT/UPDATE/INSERT/DELETE ... WITH (...)

- HOLDLOCK (= SERIALIZABLE)
- NOLOCK (= READUNCOMMITTED)
- READCOMMITTED
- READCOMMITTEDLOCK
- READPAST (пропуск заблокированных другими транзакциями записей)
- READUNCOMMITTED
- REPEATABLEREAD
- SERIALIZABLE
- PAGLOCK | ROWLOCK | TABLOCK
- TABLOCKX
- XLOCK

```
UPDATE Production.Product
```

```
    WITH (TABLOCK)
```

```
    SET ListPrice = ListPrice * 1.10
```

```
    WHERE ProductNumber LIKE 'BK-%'
```

Блокировочные подсказки инструкций (пример)

```
USE AdventureWorks;  
GO
```

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
BEGIN TRANSACTION;
```

```
SELECT Title FROM HumanResources.Employee WITH (NOLOCK);
```

```
-- Get information about the locks held by the transaction.
```

```
SELECT resource_type, resource_subtype, request_mode FROM  
    sys.dm_tran_locks WHERE request_session_id = @@spid;
```

```
ROLLBACK;
```

```
GO
```

Определение характеристик курсора

```
DECLARE cursor_name CURSOR
  [ LOCAL | GLOBAL ]
  [ FORWARD_ONLY | SCROLL ]
  [ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
  [ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
  [ TYPE_WARNING ]
  FOR select_statement
  [ FOR UPDATE [ OF column_name [ ,...n ] ] ]
```

- FORWARD_ONLY | SCROLL – возможность прокрутки курсора;
- STATIC | KEYSET | DYNAMIC | FAST_FORWARD – тип курсора;
- READ_ONLY | SCROLL_LOCKS | OPTIMISTIC (WITH VALUES / WITH ROW VERSIONING) – управление изменениями и блокировками записей;
- возможно неявное изменение типа курсора, если формирующая его инструкция выборки содержит элементы, не соответствующие требованиям курсора.

Режимы транзакций

- явные транзакции:
 - запускаются посредством инструкции `BEGIN TRANSACTION`;
 - завершение транзакции осуществляется с использованием инструкций `COMMIT TRANSACTION / WORK` (фиксация) или `ROLLBACK TRANSACTION / WORK` (откат);
 - управление транзакциями также может осуществляться через функции API – ADO, ODBC, OLEDB (`ITransactionLocal::StartTransaction()`, `ITransaction::Commit()` и `ITransaction::Abort()`);
- автоматически фиксируемые транзакции (режим по умолчанию):
 - каждая отдельная инструкция фиксируется после завершения;
- неявные транзакции:
 - режим запускается через инструкцию `SET IMPLICIT_TRANSACTIONS ON` или через соответствующие функции API (ODBC, OLEDB);
 - очередная инструкция `ALTER TABLE`, `INSERT`, `CREATE`, `OPEN`, `DELETE`, `REVOKE`, `DROP`, `SELECT`, `FETCH`, `TRUNCATE TABLE`, `GRANT`, `UPDATE` автоматически запускает новую транзакцию;
 - необходимо только фиксировать или выполнять откат каждой транзакции (`COMMIT / ROLLBACK`), после завершения транзакции следующая инструкция (из перечисленного списка) запускает новую транзакцию.

Неявные транзакции: пример (T-SQL)

```
CREATE TABLE ImplicitTran
    (Cola int PRIMARY KEY,
     Colb char(3) NOT NULL);
GO

SET IMPLICIT_TRANSACTIONS ON;

-- First implicit transaction started by an INSERT statement.
INSERT INTO ImplicitTran VALUES (1, 'aaa');
INSERT INTO ImplicitTran VALUES (2, 'bbb');

-- Commit first transaction.
COMMIT TRANSACTION;

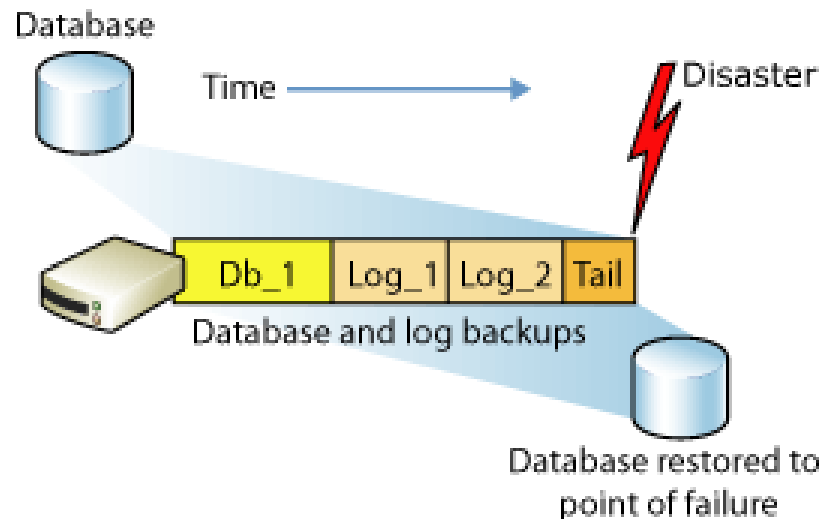
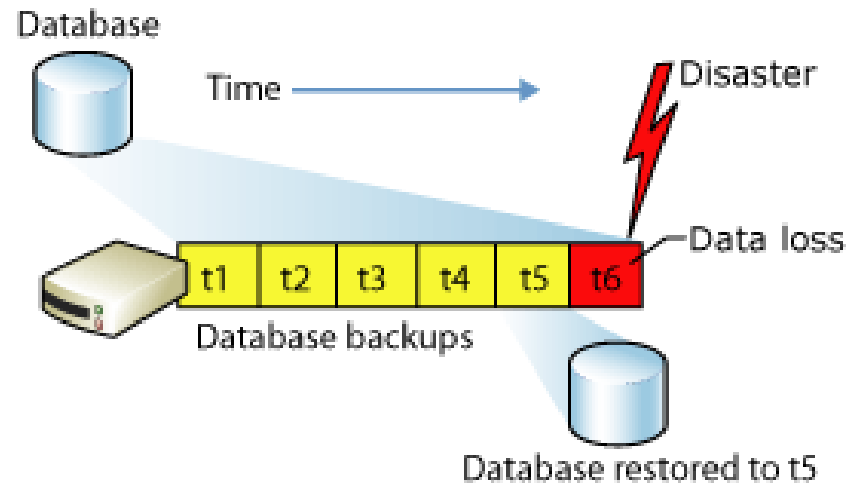
-- Second implicit transaction started by a SELECT statement.
SELECT COUNT(*) FROM ImplicitTran;
INSERT INTO ImplicitTran VALUES (3, 'ccc');
SELECT * FROM ImplicitTran;

-- Commit second transaction.
COMMIT TRANSACTION;

SET IMPLICIT_TRANSACTIONS OFF;
```


Восстановление

- создание резервной копии (данных и журнала транзакций):
 - полные (full backup);
 - частичные (differential backup);
- модели ВОССТАНОВЛЕНИЯ:
 - простая (simple recovery model);
 - полная (full recovery model);
 - частичная (bulk-logged recovery model).



Вопросы к экзамену

- Управление параллельной обработкой в БД. Проблемы (аномалии) параллельной обработки транзакций.
- Свойства транзакций. Сериализуемость транзакций.
- Стандарт SQL-92: аномалии изоляции транзакций и уровни изоляции.
- Стратегии и механизмы управления параллелизмом. Блокировка ресурсов. Алгоритм двухфазной блокировки. Взаимная блокировка.
- Стратегии и механизмы управления параллелизмом. Многоверсионное управление параллельным доступом.
- Восстановление баз данных.
- Управление параллельной обработкой в СУБД MS SQL Server.
- Транзакции в СУБД MS SQL Server. Режимы транзакций.
- Восстановление баз данных. Восстановление в СУБД MS SQL Server.

Лабораторная работа №10.

Режимы выполнения транзакций

1. Исследовать и проиллюстрировать на примерах различные уровни изоляции транзакций MS SQL Server, устанавливаемые с использованием инструкции **SET TRANSACTION ISOLATION LEVEL**
2. Накладываемые блокировки исследовать с использованием **sys.dm_tran_locks**

Лабораторная работа №11.

Создание базы данных и запросов к ней в СУБД SQL Server

1. создать базу данных, спроектированную в рамках лабораторной работы №4, используя изученные в лабораторных работах 5-10 средства SQL Server 2012:
 - поддержания создания и физической организации базы данных;
 - различных категорий целостности;
 - представления и индексы;
 - хранимые процедуры, функции и триггеры;
2. создание объектов базы данных должно осуществляться средствами DDL (CREATE/ALTER/DROP), в обязательном порядке иллюстрирующих следующие аспекты:
 - добавление и изменение полей;
 - назначение типов данных;
 - назначение ограничений целостности (PRIMARY KEY, NULL/NOT NULL/UNIQUE, CHECK и т.п.);
 - определение значений по умолчанию;

Лабораторная работа №11

(продолжение)

3. в рассматриваемой базе данных должны быть тем или иным образом (в рамках объектов базы данных или дополнительно) созданы запросы DML для:
 - выборки записей (команда SELECT);
 - добавления новых записей (команда INSERT), как с помощью непосредственного указания значений, так и с помощью команды SELECT;
 - модификации записей (команда UPDATE);
 - удаления записей (команда DELETE);
4. запросы, созданные в рамках пп.2,3 должны иллюстрировать следующие возможности языка:
 - удаление повторяющихся записей (DISTINCT);
 - выбор, упорядочивание и именование полей (создание псевдонимов для полей и таблиц / представлений);
 - соединение таблиц (INNER JOIN / LEFT JOIN / RIGHT JOIN / FULL OUTER JOIN);
 - условия выбора записей (в том числе, условия NULL / LIKE / BETWEEN / IN / EXISTS);
 - сортировка записей (ORDER BY - ASC, DESC);
 - группировка записей (GROUP BY + HAVING, использование функций агрегирования – COUNT / AVG / SUM / MIN / MAX);
 - объединение результатов нескольких запросов (UNION / UNION ALL / EXCEPT / INTERSECT);
 - вложенные запросы.

Базы данных

Тема 12

Параллельные и распределенные базы данных

Нефункциональные требования

- производительность:
 - пропускная способность (*throughput*) – количество действий за единицу времени
 - время отклика – время выполнения одного действия;
- масштабируемость (*scalability*) – определяет изменения какой-либо характеристики производительности при изменении нагрузки (количества пользователей, объёма накапливаемой информации) и размеров вычислительной системы;
- надежность (*reliability*) и отказоустойчивость (*fault tolerance*);
- доступность (*availability*): отношение времени нормальной работы системы (возможность выполнения запросов на чтение и/или запись) к длине интервала времени измерения.

Параллелизм

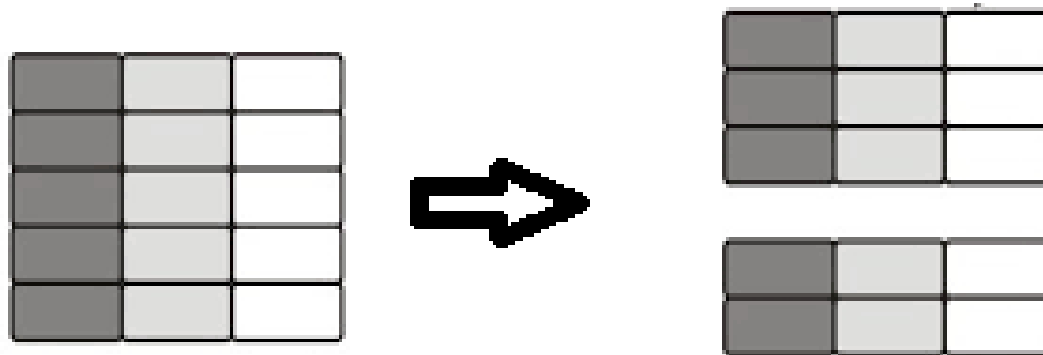
- **гранулярность параллелизма:**
 - между транзакциями;
 - между запросами внутри транзакций;
 - между операциями плана выполнения внутри запросов;
 - в алгоритмах выполнения операций;
- **издержки:**
 - синхронизация между параллельно выполняемыми операциями с данными;
 - проблема равномерного распределения нагрузки;
- **преимущества:** высокая масштабируемость для отдельных типов операций.

Распределение данных

- на физическом уровне (блоки данных):
 - необходимы программные или аппаратные средства управления размещением данных на дисках, фактически замещающие функции файловой системы ОС (например, RAID или распределённые файловые системы с поддержкой интерфейса блочного доступа);
- на логическом уровне (строки/столбцы таблиц и сами таблицы)
 - горизонтальная фрагментация / секционирование (*horizontal partitioning*);
 - вертикальная фрагментация / секционирование (*vertical partitioning*);
 - репликация / тиражирование (*replication*).

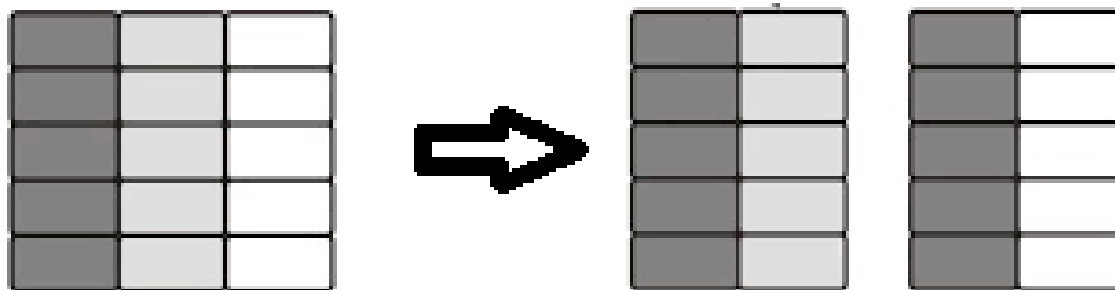
Горизонтальная фрагментация

- горизонтальная фрагментация – разбиение таблицы на совокупность попарно непересекающихся секций (*partition*) по значениям *ключа секционирования*;
- разбиение может определяться:
 - диапазонами значений ключа секционирования;
 - значениями функции хеширования, вычисляемыми на основе ключа секционирования;
 - списками значений ключа секционирования.



Вертикальная фрагментация

- вертикальная фрагментация – разбиение таблицы на части таким образом, чтобы каждая часть содержала только некоторые из столбцов исходной таблицы (разбиение строк);
- каждая из частей содержит первичный ключ, обеспечивающий возможность соединения частей для получения значений полного набора атрибутов.



Репликация

- репликация – создание множественных копий данных (таблиц);
- процедура репликации должна обладать прозрачностью для клиентов СУБД;
- согласованность реплик:
 - строгая (глобальная) согласованность: единая логическая копия;
 - согласованность на уровне отдельных транзакций;
 - локальная согласованность (например, разделение реплик для оперативной и аналитической обработки);
- согласованность реплик определяют правила обновления и распространения изменений по репликам.

Поддержка единой логической копии

- изменения, вносимые любой транзакцией, должны быть выполнены на всех серверах до того, как транзакция будет зафиксирована;
- методы реализации:
 - глобальные транзакции;
 - главная копия: все обновляющие транзакции выполняются на главном (*primary*) сервере, и внесённые изменения до фиксации распространяются на остальные копии, расположенные на резервных (*standby*) серверах;
- недостаток: снижается доступность (как минимум, для записи) и производительность системы.

Репликация главной копии

- различные методы распространения обновлений, влияющие на:
 - доступность системы;
 - отставание состояния резервных серверов;
 - связанную с репликацией нагрузку на серверы;
- синхронное распространение изменений: до фиксации транзакции, выполнившей изменения на главном сервере;
- асинхронное распространение изменений, предполагающее отставание состояния резервных серверов:
 - (повторение операторов SQL);
 - обновление логических записей;
 - на уровне страниц баз данных;
 - распространение записей журнала.

Репликация главной копии: варианты использования

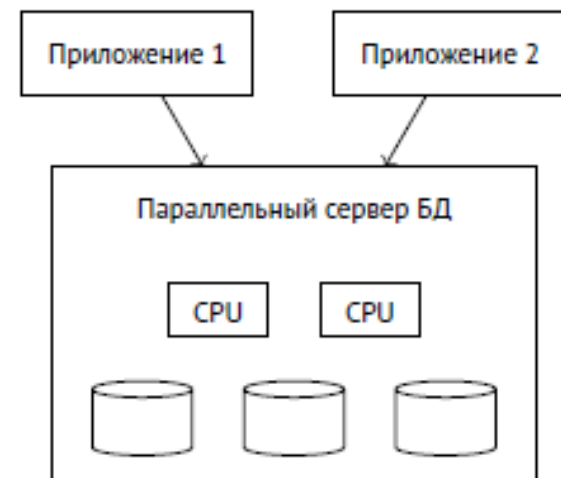
- создание копий, отражающих состояние базы данных на основном сервере на определённый момент времени (без последующего распространения изменений);
- перераспределение нагрузки на главный сервер (перераспределение операций чтения на резервные серверы);
- перераспределение запросов в зависимости от типов нагрузки (оперативная и аналитическая обработка данных);
- создание резервных копий базы данных для переключения в случае отказа.

Масштабируемость

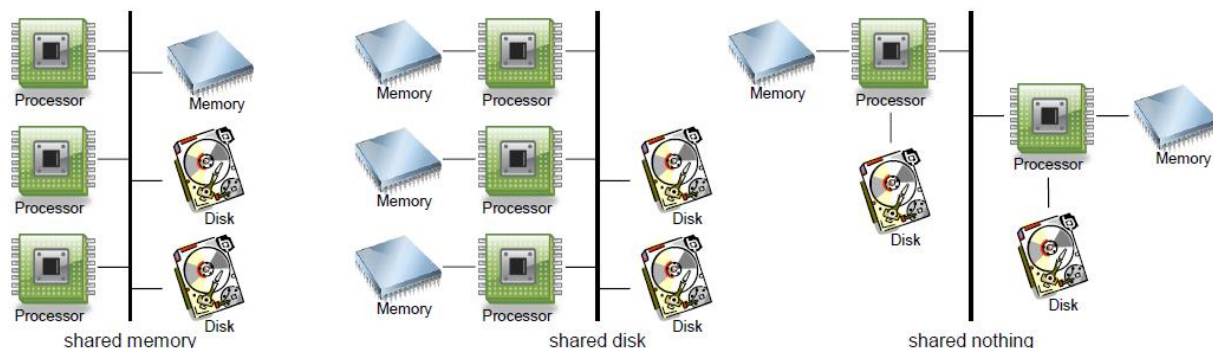
- централизованные / монолитные СУБД
 - реляционные СУБД: Oracle, PostgreSQL, MS SQL Server, ... ;
 - вертикальное масштабирование (*scale-up, vertical scaling*);
- параллельные СУБД (*parallel database systems*):
 - Teradata, Oracle Real Application Cluster, ... ;
 - вертикальное / горизонтальное масштабирование;
- распределенные СУБД (*distributed database systems*);
 - на базе монолитных СУБД / нереляционные СУБД (MongoDB, Cassandra, HBase, Google BigTable, Neo4j, Redis, Memcached, Tarantool,...);
 - горизонтальное масштабирование (*scale-out, horizontal scaling*).

Параллельные СУБД

- основная цель: обеспечение высокой производительности за счет выполнения запроса (транзакции) или отдельных операций на нескольких вычислительных узлах (устройствах);
- с точки зрения клиента не отличается от централизованной СУБД;
- единая схема данных (для каждой базы данных);
- проблема балансировки нагрузок между вычислительными узлами как с точки зрения выполнения запросов, так и с точки зрения распределения хранимых данных (для архитектуры SN из-за неравномерности скорости доступа к носителям данных).



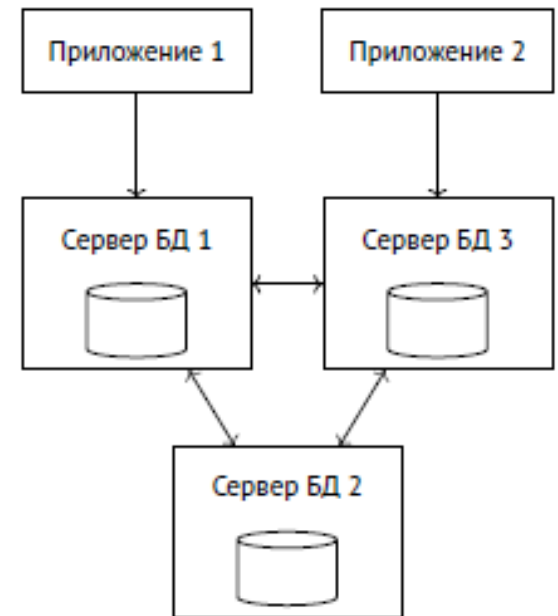
Аппаратная конфигурация параллельных СУБД



- SM (shared memory):
 - многопроцессорные системы с разделяемой памятью;
 - масштабируемость ограничена возможным количеством устанавливаемых процессоров в одной вычислительной системе и пропускной способностью;
- SD (shared disk):
 - вычислительные устройства взаимодействуют между собой и с устройствами дисковой памяти через коммуникационную сеть;
 - масштабируемость ограничена необходимостью синхронизации для предотвращения конфликтующих изменений страниц;
- SN (shared nothing):
 - каждая вычислительная система функционирует автономно и взаимодействие через вычислительную сеть.

Распределённые СУБД

- основные цели:
 - повышение доступности;
 - горизонтальная масштабируемость;
 - интеграция разнородных систем;
- совокупность серверов баз данных, каждый из которых может работать независимо от остальных серверов;
- отдельный сервер поддерживает свою схему данных, но может выполнять запросы к данным, размещённым на нескольких серверах (прозрачный доступ к данным);
- с точки зрения используемых на серверах СУБД могут быть однородными и неоднородными;
- возможные ограничения реализации:
 - доступность некоторых данных;
 - выполнение требований ACID для распределённых транзакций.



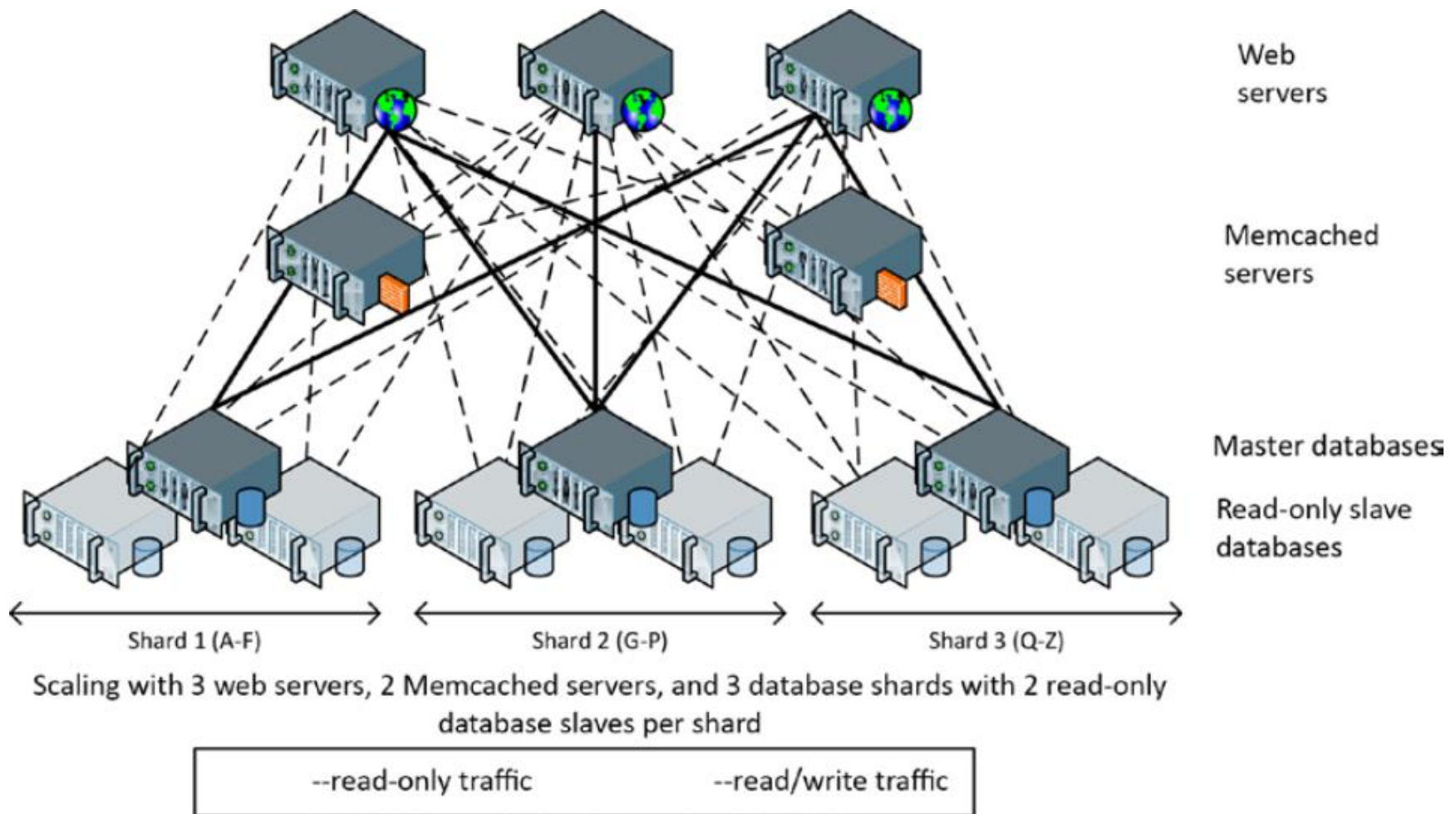
Согласованность в распределённых системах

- глобальные (распределённые) транзакции – содержат операции обработки элементов данных на разных серверах:
 - должны удовлетворять требуемому уровню изоляции;
 - должны завершаться одинаково (фиксацией или откатом);
- локальные транзакции – обрабатывают данные, находящиеся на одном сервере;
- глобальные протоколы управления транзакциями должны обеспечивать высокую доступность, отсутствие централизованных механизмов управления и горизонтальную масштабируемость → ослабленные критерии согласованности в реализациях распределённых систем;
- протоколы на основе меток времени: распределённый протокол изоляции моментальных снимков.

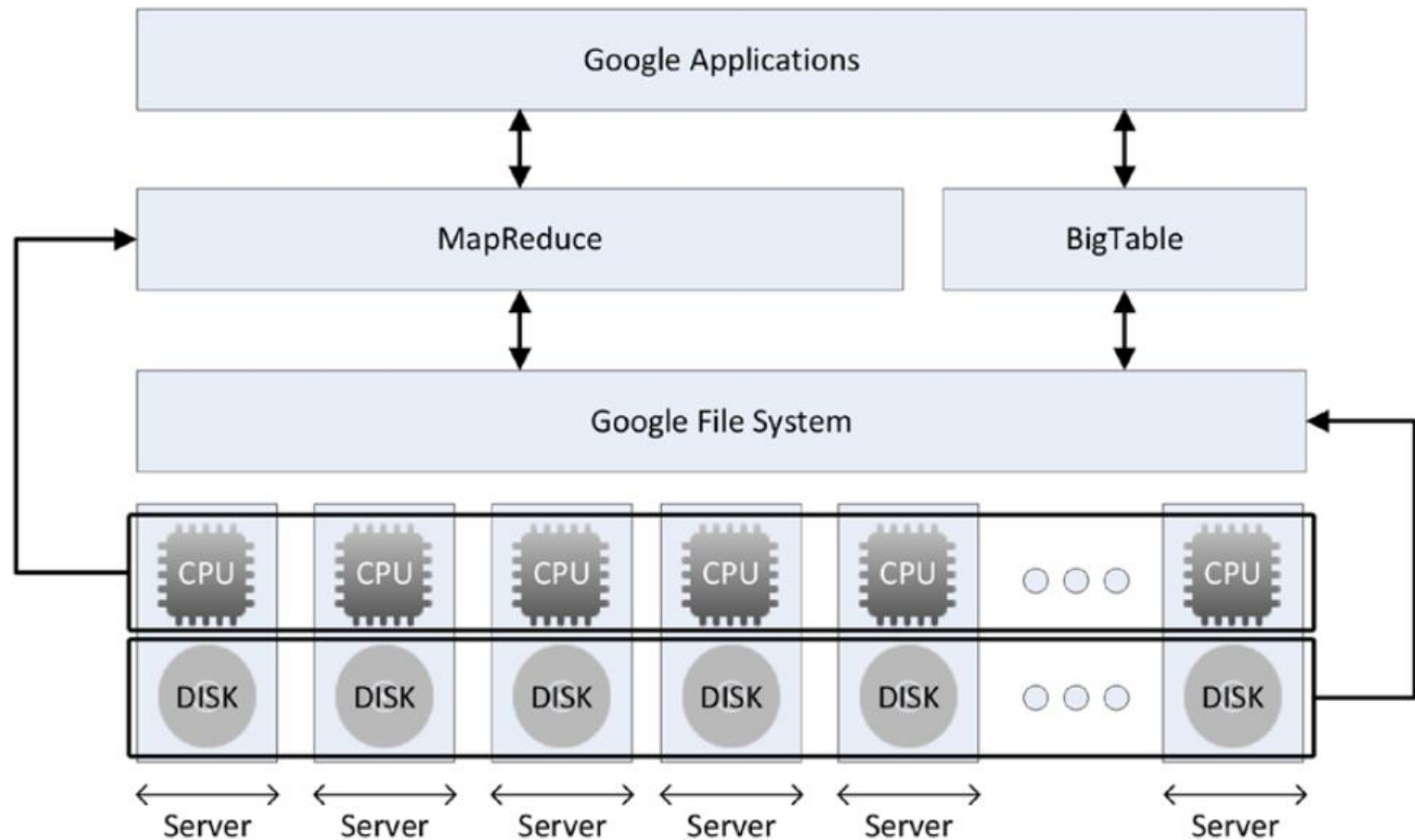
Двухфазная фиксация транзакций

- протокол двухфазной фиксации транзакций (*2PC – two-phase commit protocol*) определяет алгоритм завершения распределённых транзакций
- участники распределенной транзакции:
 - два или более сервера, которые называются *диспетчерами ресурсов*;
 - *диспетчер транзакций* (координатор распределенных транзакций), который обеспечивает управление транзакцией путем координации между диспетчерами ресурсов;
- двухфазная фиксация транзакций:
 - фаза подготовки: каждый из диспетчеров ресурсов обеспечивает устойчивость транзакции, в частности, записывая журнал транзакции на диск;
 - фаза фиксации: начинается в случае успешного завершения подготовки всеми диспетчерами ресурсов и завершается после получения диспетчером транзакций подтверждений об успешной фиксации от каждого из диспетчеров ресурсов;

Пример: сегментирование данных в web-системах



Пример: программно-аппаратная архитектура Google (Google BigTable / HBase)



СУБД SQL Server 2025 – Создание распределённых баз данных.

Секционирование в СУБД SQL Server

- предпочтительным способом локального секционирования (в рамках одной базы данных) является определение секционированных таблиц и индексов (*partitioned tables and indexes*);
- секционированные представления поддерживают объединение горизонтально секционированных данных, распределённых между базовыми таблицами, находящимися на одном или нескольких серверах;
 - локальное / распределенное секционированное представление (*local / distributed partitioned views*);
 - федеративные серверы баз данных (*federated database servers*);
- поддерживается несколько режимов репликации для различных вариантов использования.

Проектирование распределенных секционированных представлений

- при проектировании набора распределенных секционированных представлений необходимо выполнить следующие процедуры:
 - определить шаблон инструкций SQL, выполняемых приложением;
 - определить, каким образом таблицы связаны друг с другом (через внешние ключи или неявно через столбцы, используемые в соединениях);
 - сопоставить частоту инструкций SQL с секциями, определенными при анализе связей;
 - определить правила маршрутизации инструкций SQL;
- цель проектирования: локальное размещение большей части данных, необходимых для выполнения запросов, обрабатываемых конкретным сервером (хотя итоговый набор секций и таблиц может быть асимметричным);
- пример: секционирование данных по регионам.

Горизонтальное секционирование

- распределённое секционированное представление формируется как набор строк из отдельных таблиц-элементов, объединённых с помощью UNION ALL.
- ограничения для базовых таблиц:
 - не должны входить в запрос более одного раза;
 - не могут иметь индексов, созданных на любых вычисляемых столбцах;
 - все ограничения первичного ключа таблиц-элементов должны быть на одинаковом количестве столбцов;
 - должна быть одна и та же настройка дополнения ANSI (параметр конфигурации ANSI_PADDING);
- ограничения для списка выборки:
 - все столбцы в каждой таблице-элементе должны быть включены в список выборки;
 - на столбцы нельзя ссылаться в списке выборки больше одного раза;
 - столбцы должны быть одного типа, идти в одном и том же порядке и встречаться только один раз.

```
SELECT
    <select_list1>
FROM T1
UNION ALL
SELECT
    <select_list2>
FROM T2
UNION ALL
...
SELECT
    <select_listn>
FROM Tn;
```

Ограничения для столбца секционирования

- для секционирования может использоваться только один столбец (столбец секционирования – *partitioning column*), который должен присутствовать в каждой таблице-элементе и находиться в одинаковом порядковом расположении в списках выборки всех инструкций SELECT в представлении (возможно использование SELECT *);
- столбец секционирования:
 - должен входить в первичный ключ;
 - не должен быть определён как вычисляемый, с автоматическим приращением, со значением по умолчанию или типа timestamp;
 - должен иметь единственное ограничение CHECK с операторами [IN, BETWEEN, AND, OR, <, <=, >, >=, =];
- диапазоны значений ключа секционирования, определённые во всех таблицах, не должны пересекаться.

Создание распределенных секционированных представлений: пример

```
-- On Server1:
CREATE TABLE Customers_33
  (CustomerID INTEGER PRIMARY KEY
    CHECK (CustomerID BETWEEN 1 AND 32999),
  ... -- Additional column definitions)

-- On Server2:
CREATE TABLE Customers_66
  (CustomerID INTEGER PRIMARY KEY
    CHECK (CustomerID BETWEEN 33000 AND 65999),
  ... -- Additional column definitions)

-- On Server3:
CREATE TABLE Customers_99
  (CustomerID INTEGER PRIMARY KEY
    CHECK (CustomerID BETWEEN 66000 AND 99999),
  ... -- Additional column definitions)

-- On EACH server:
CREATE VIEW Customers AS
  SELECT * FROM CompanyDatabase.TableOwner.Customers_33
  UNION ALL
  SELECT * FROM Server2.CompanyDatabase.TableOwner.Customers_66
  UNION ALL
  SELECT * FROM Server3.CompanyDatabase.TableOwner.Customers_99
```

Изменение данных в распределённых секционированных представлениях

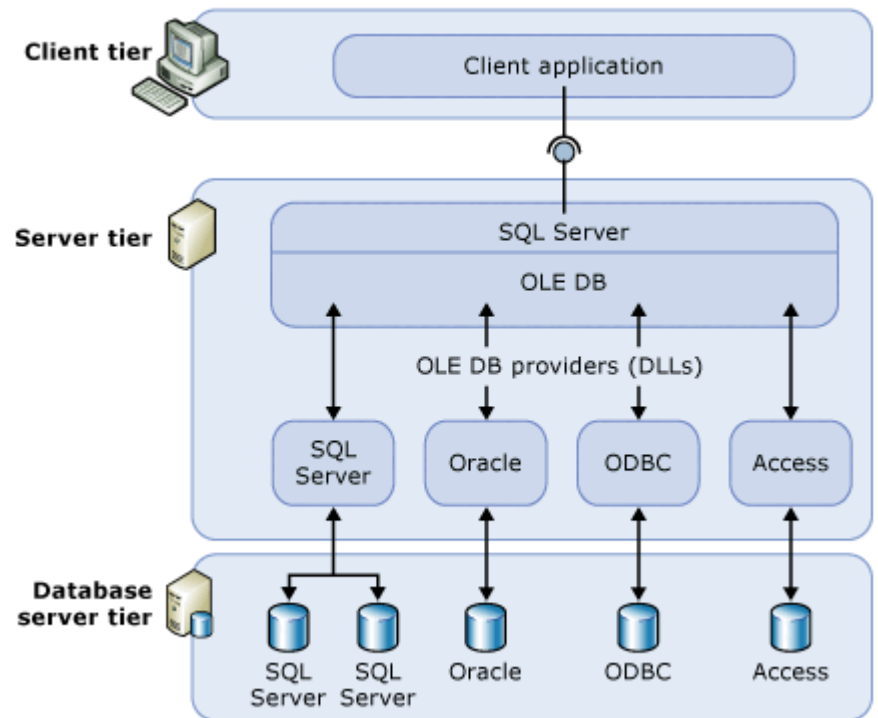
- в инструкцию INSERT должны включаться все столбцы (кроме столбцов с автоматическим приращением), даже если в базовой таблице значением столбца может быть NULL или если для него определено ограничение DEFAULT, также при вставке или обновлении не могут указываться значения по умолчанию (с помощью ключевого слова DEFAULT);
- значение столбца секционирования должно соответствовать одному из заданных ограничений;
- изменение данных через распределённое секционированное представление невозможно, если:
 - какая-либо из базовых таблиц содержит столбец типа timestamp (кроме инструкции DELETE);
 - для какой-либо из базовых таблиц определён триггер или ограничение ON UPDATE CASCADE/SET NULL/SET DEFAULT or ON DELETE CASCADE/SET NULL/SET DEFAULT;
 - есть самосоединение с тем же самым представлением или с одной из базовых таблиц.

Репликация

- СУБД SQL Server поддерживает следующие механизмы репликации:
 - транзакционная репликация (*transactional replication*)
начинается с создания исходного моментального снимка объектов и данных базы данных публикации, последующие изменения данных и схемы на издателе обычно доставляются подписчику без задержек (практически в реальном времени), а изменения данных применяются на подписчике в том же порядке и в тех же рамках транзакций, в которых они выполнялись у издателя (в пределах публикации гарантируется согласованность транзакций);
 - репликация слиянием (*merge replication*);
 - репликация моментальных снимков (*snapshot replication*)
распространяет данные подписчикам целиком и точно в том виде, в котором они были представлены в момент синхронизации, и не контролирует обновления этих данных.

Распределенные запросы

- распределенные запросы используются для доступа к данным, внешним по отношению к экземпляру СУБД SQL Server и расположенным в возможно гетерогенных источниках данных:
 - в других экземплярах SQL Server;
 - в различных реляционных и нереляционных источниках данных, представляющих данные в табличных объектах, именуемых наборами строк (*rowset*);
- доступ к внешним данным осуществляется посредством поставщиков данных OLE DB (*OLEDB providers*).



Взаимодействие с поставщиками OLE DB

- поставщики OLE DB для различных источников данных могут в зависимости от реализации поддерживать следующие функции:
 - только открытие и чтение набора строк базовой таблицы;
 - выполнение инструкций SELECT, INSERT, UPDATE, DELETE;
 - выполнение транзакций;
 - сканирование индексов;
 - использование курсоров, в том числе для изменения и удаления данных;
 - использование параметризованных запросов;
- при обработке распределённых запросов экземпляр SQL Server также выполняет следующие действия:
 - подключение к источнику данных;
 - получение необходимых метаданных;
 - управление выполнением транзакций;
 - преобразование типов данных (между типами OLE DB и встроенными типами);
 - обработку ошибок;
 - управление правами доступа.

Выполнение и оптимизация распределённых запросов

- SQL Server максимально делегирует выполнение распределенного запроса поставщику данных в зависимости от поддерживаемых им возможностей;
- на степень делегирования в том числе влияют:
 - уровень диалекта (грамматики SQL-запросов – SQL Minimum, ODBC Core, SQL-92 Entry level, SQL Server) и дополнительные возможности синтаксиса запросов, поддерживаемые поставщиком;
 - совместимость параметров сопоставления строк (*collation*);
- при оптимизации плана выполнения запроса при наличии поддержки со стороны поставщика данных также могут использоваться:
 - сканирование индексов удалённых таблиц;
 - использование статистик удалённых таблиц;
 - выполнение параметризованных запросов;
 - информация об ограничениях, определённых в базовых таблицах.

Распределённые транзакции

- SQL Server обеспечивает выполнение и управление удалёнными (локальными для источника данных) и распределёнными транзакциями в том случае, если транзакции поддерживаются источником данных и поставщиком (в противном случае операции чтения/записи могут быть запрещены или ограничены);
- распределённая транзакция инициируется явно (инструкцией `BEGIN DISTRIBUTED TRANSACTION`) или неявно (выполнением распределённого запроса или удалённой хранимой процедуры внутри локальной транзакции);
- процедура завершения распределённой транзакции выполняется в соответствии с протоколом двухфазной фиксации, причём в качестве диспетчера транзакций может выступать координатор распределённых транзакций MS DTC (Distributed Transaction Coordinator) или любой другой диспетчер транзакций, поддерживающий спецификацию X/Open XA.

Связанный сервер

- связанным сервером (*linked server*) называется зарегистрированный на экземпляре СУБД SQL Server источник данных с указанием соответствующего поставщика OLE DB и параметров подключения;
- в инструкциях языка SQL (SELECT, INSERT, UPDATE, DELETE) к объектам связанного сервера можно обращаться путём указания их полного имени (*< linked-server >.<catalog>.<schema>.<object>*), включающего имя связанного сервера;
- параметры *catalog*, *schema* и *object_name* представляют собой ссылку на объект источника данных, представляемый как набор строк, и передаются поставщику OLE DB для идентификации конкретного объекта данных (для СУБД SQL Server параметр *catalog* соответствует базе данных);
- доступ к связанному серверу также может осуществляться следующими способами:
 - через команду OPENQUERY для выполнения сквозного запроса (*pass-through query*) на связанном сервере;
 - через инструкцию EXECUTE AT *Linked_server_name*, позволяющую выполнить произвольный динамически сформированный параметризованный запрос на удалённом сервере.

Операции со связанными серверами

- получение списка зарегистрированных серверов:
 - `sp_linkedservers`;
 - `sys.servers` (представление системного каталога);
- добавление связанного сервера:
 - `sp_addlinkedserver`;
- удаление связанного сервера:
 - `sp_dropserver`;
- добавление / удаление имени входа (сопоставление имени и пароля удалённого входа для заданного локального входа):
 - `sp_addlinkedsrvlogin`
 - `sp_droplinkedsrvlogin`.

```
EXEC sp_addlinkedserver  
    N'SEATTLESales', N'SQL Server'
```

```
EXECUTE sp_addlinkedserver  
    @server = N'S1_instance1',  
    @srvproduct = N'',  
    @provider = N'MSOLEDBSQL',  
    @datasrc = N'S1\instance1';
```

```
EXEC sp_addlinkedserver  
    @server = 'LONDON Mktg',  
    @srvproduct = 'Oracle',  
    @provider = 'MSDAORA',  
    @datasrc = 'MyServer'
```

```
EXEC sp_droplinkedsrvlogin  
    @rmtsrvname = 'ServerOne',  
    @locallogin = NULL
```

```
EXECUTE master.dbo.sp_addlinkedsrvlogin  
    @rmtsrvname = N'LinkServerName',  
    @locallogin = NULL,  
    @useself = N'False',  
    @rmtuser = N'myUser',  
    @rmtpassword = N'*****';
```

Нерегистрируемые серверы (специальные серверы, ad hoc servers)

- команды OPENROWSET и OPENDATASOURCE обеспечивают возможность доступа к удалённым источникам данных без регистрации связанного сервера;
- команда OPENDATASOURCE:
 - может использоваться там, где синтаксис Transact-SQL позволяет указать имя связанного сервера;
 - требует указания параметров подключения к источнику данных OLE DB (поставщика данных и строки подключения);
- команда OPENROWSET:
 - может применяться везде, где в инструкции языка Transact-SQL используется ссылка на таблицу или представление;
 - требует указания параметров подключения к источнику данных OLE DB (поставщика данных и строки подключения), а также имени объекта или запроса, которые должны формировать набор строк;
- команды OPENROWSET и OPENDATASOURCE не поддерживают передачу переменных языка Transact-SQL в качестве аргументов;
- команды OPENROWSET и OPENDATASOURCE рекомендуется использовать только для единичных обращений к источникам данных, так как они не предоставляют всех возможностей определений связанных серверов, а также при каждом использовании требуют указания всей информации, необходимой для подключения (в том числе паролей).

Нерегистрируемые серверы: пример

```
SELECT GroupName, Name, DepartmentID
FROM OPENDATASOURCE('MSOLEDBSQL',
    'Server=Seattle1;Database=AdventureWorks2022;TrustServerCertificate=
    Yes;Trusted_Connection=Yes;').HumanResources.Department
ORDER BY GroupName, Name;
```

```
SELECT d.* FROM OPENROWSET(
    'MSOLEDBSQL',
    'Server=Seattle1;Trusted_Connection=yes;',
    AdventureWorks2022.HumanResources.Department
) AS d;
```

```
SELECT a.*
FROM OPENROWSET(
    'MSOLEDBSQL', 'Server=Seattle1;Trusted_Connection=yes;',
    'SELECT GroupName, Name, DepartmentID
        FROM AdventureWorks2022.HumanResources.Department
        ORDER BY GroupName, Name'
) AS a;
```

Сквозные запросы

- команда **OPENQUERY** позволяет выполнить передаваемый запрос к указанному связанному серверу:
 - на команду **OPENQUERY** как на имя таблицы можно ссылаться из предложения **FROM** инструкции **SELECT**, а также в инструкциях **INSERT**, **UPDATE** или **DELETE**;
 - в качестве аргументов нельзя использовать переменные;
 - команда **OPENQUERY** не может быть использована для выполнения расширенных хранимых процедур на связанном сервере.

```
SELECT * FROM OPENQUERY (OracleSvr,  
    'SELECT name FROM joe.titles WHERE name = ''NewTitle''');
```

```
INSERT OPENQUERY (OracleSvr, 'SELECT name FROM joe.titles')  
    VALUES ('NewTitle');
```

```
UPDATE OPENQUERY (OracleSvr, 'SELECT name FROM joe.titles WHERE id = 101')  
    SET name = 'ADifferentName';
```

```
DELETE OPENQUERY (OracleSvr,  
    'SELECT name FROM joe.titles WHERE name = ''NewTitle''');
```


Вопросы к экзамену

- Распределение данных в СУБД. Горизонтальная и вертикальная фрагментация.
- Распределение данных в СУБД: репликация. Согласованность реплик и распространение изменений. Репликация в СУБД SQL Server.
- Параллелизм в СУБД. Параллельные СУБД.
- Распределённые СУБД.
- Согласованность в распределённых системах. Протокол двухфазной фиксации транзакций. Распределённые транзакции в СУБД SQLServer.
- Горизонтальное секционирование в СУБД SQL Server. Проектирование и создание распределённых секционированных представлений.
- Распределённые запросы. Взаимодействие с поставщиками данных. Выполнение и оптимизация распределённых запросов.
- Связанные серверы. Нерегистрируемые серверы (OPENDATASOURCE, OPENROWSET). Сквозные запросы (OPENQUERY).

Лабораторная работа №13.

Создание распределенных баз данных на основе секционированных представлений

1. Создать две базы данных на одном экземпляре СУБД SQL Server.
2. Создать в базах данных п.1. горизонтально фрагментированные таблицы.
3. Создать секционированные представления, обеспечивающие работу с данными таблиц (выборку, вставку, изменение, удаление).

Лабораторная работа №14.

Создание вертикально фрагментированных таблиц средствами СУБД SQL Server

1. Создать в базах данных пункта 1 задания 13 таблицы, содержащие вертикально фрагментированные данные.
2. Создать необходимые элементы базы данных (представления, триггеры), обеспечивающие работу с данными вертикально фрагментированных таблиц (выборку, вставку, изменение, удаление).

Лабораторная работа №15.

Создание распределенных баз данных со связанными таблицами средствами СУБД SQL Server

1. Создать в базах данных пункта 1 задания 13 связанные таблицы.
2. Создать необходимые элементы базы данных (представления, триггеры), обеспечивающие работу с данными связанных таблиц (выборку, вставку, изменение, удаление).

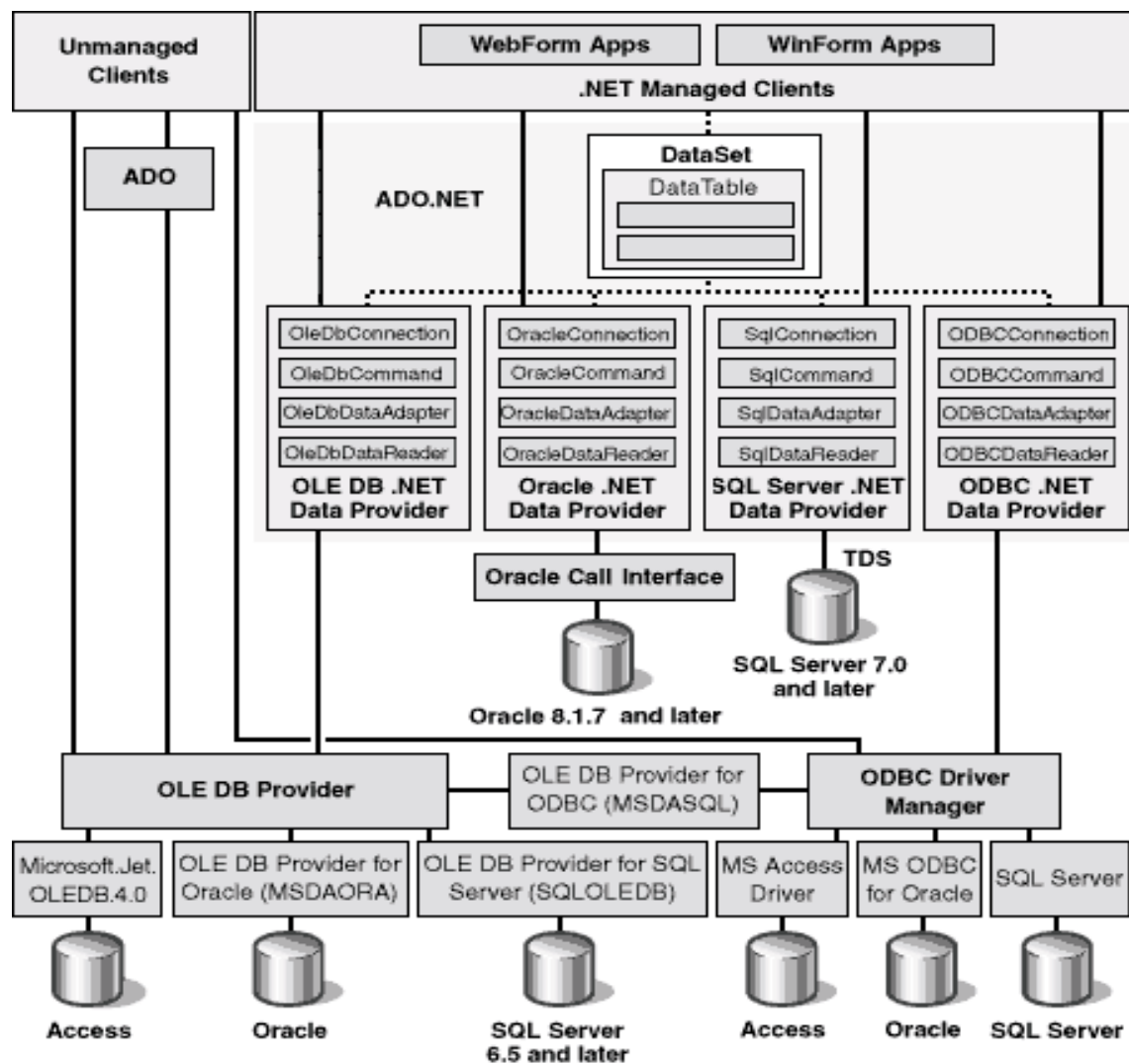
Базы данных

Тема 13

Технология доступа к данным ADO.NET

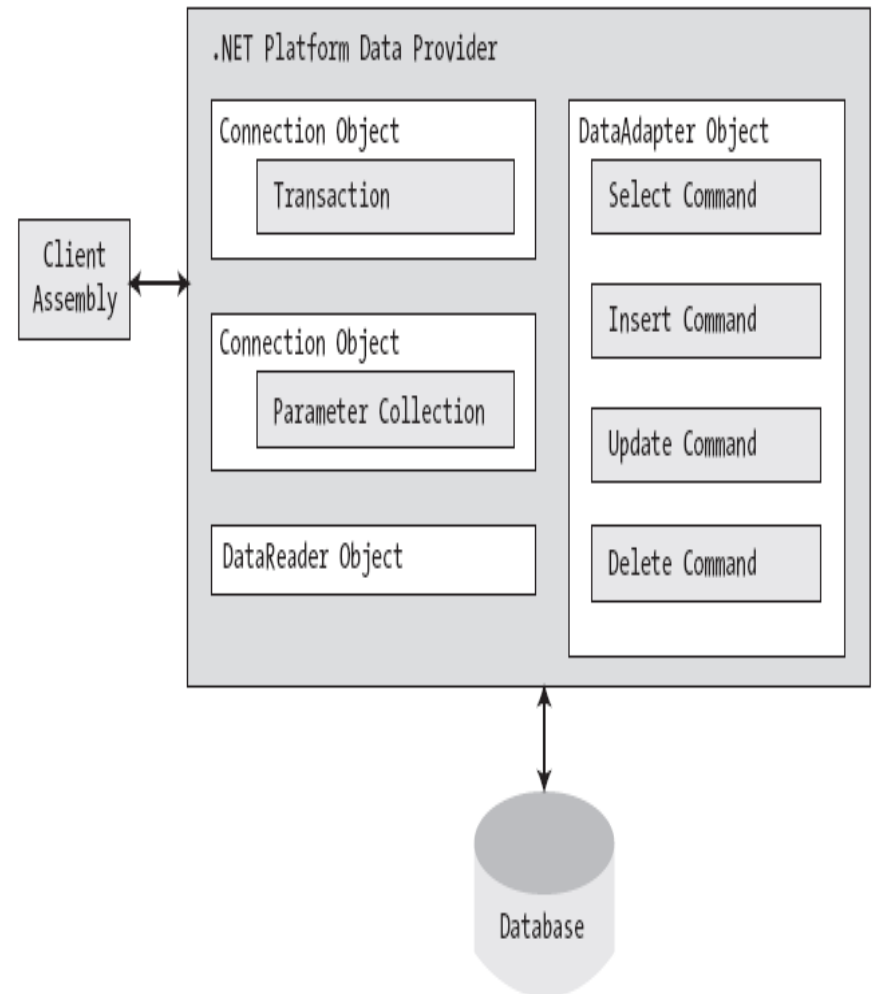
Стек доступа к данным

Доступ к данным
в технологии
ADO.NET
осуществляется
путем
обращения к
специальным
библиотекам -
поставщикам
(провайдерам,
providers),
реализующим
определенный
набор типов
(прежде всего,
классов и
интерфейсов)
доступа к
данным

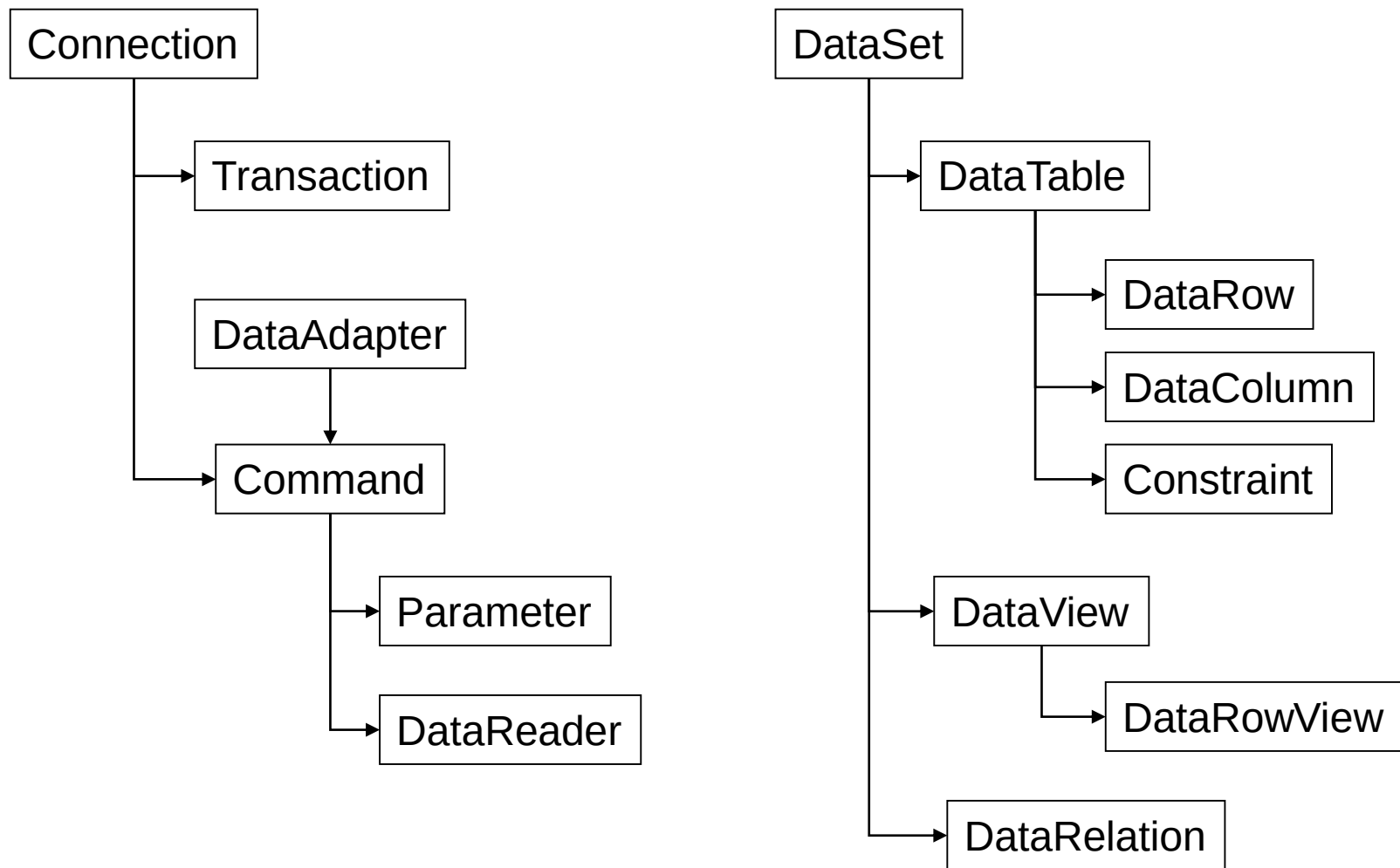


Режимы работы ADO.NET

- режим с поддержанием соединения (связный уровень, *connected layer*):
 - осуществляется непосредственное, явное подключение к источнику данных и отключение от него;
 - используются объекты **Connection**, **Command**, **DataReader**;
- режим без поддержания соединения (несвязный уровень, *disconnected layer*):
 - операции с данными производятся над локальной копией данных источника, представленной объектом **DataSet** и связанными объектами (**DataTable** etc.);
 - используется объект **DataAdapter** для обеспечения взаимодействия приложения и источника данных, при этом операции открытия и закрытия соединения выполняются автоматически по мере необходимости;
 - фактически операции с объектами несвязного уровня можно выполнять вообще без подключения к источнику данных.



Объектная модель ADO.NET



Основные объекты поставщиков данных ADO.NET

Объект	Базовый класс	Реализуемые интерфейсы	Назначение
Connection	DbConnection	IDbConnection	Обеспечивает подключение к источнику данных
Command	DbCommand	IDbCommand	Представляет запрос к источнику данных
DataReader	DbDataReader	IDataReader IDataRecord	Обеспечивает однонаправленный просмотр данных (на основе курсора, открытого на сервере)
DataAdapter	DbDataAdapter	IDataAdapter IDbDataAdapter	Обеспечивает обмен данными между источниками данных и объектами наборов данных (DataSet)
Parameter	DbParameter	IDataParameter IDbDataParameter	Представляет именованный параметр в параметризованном запросе
Transaction	DbTransaction	IDbTransaction	Инкапсулирует транзакцию

Пространство имен System.Data

содержит типы, общие для всех поставщиков данных:

- типы примитивов баз данных (таблицы, ряды, столбцы, ограничения и т.п.);
- интерфейсы, реализуемые поставщиками данных;
- специфические исключения, возникающие при работе с БД (NotNullAllowedException, RowNotInTableException, MissingPrimaryKeyException etc.).

Тип	Назначение
DataSet	Набор данных – локальный кэш данных источника
DataTable	Таблица (в DataSet)
DataRow	Строка (в DataTable)
DataColumn	Столбец (в DataTable)
DataRelation	Отношение (связь) между таблицами
Constraint	Ограничение (для объекта DataColumn)
DataView	Представление, построенное для объекта DataTable
IDataAdapter	Основной интерфейс объекта DataAdapter
IDbDataAdapter	Интерфейс объекта DataAdapter, описывающий дополнительную функциональность
IDataParameter	Интерфейс объекта Parameter
IDataReader	Интерфейс объекта DataReader
IDbCommand	Интерфейс объекта Command
IDbTransaction	Интерфейс объекта Transaction

Интерфейс IDbConnection

```
public interface IDbConnection : IDisposable
{
    string ConnectionString { get; set; }
    int ConnectionTimeout { get; }
    string Database { get; }
    ConnectionState State { get; }

    IDbTransaction BeginTransaction();
    IDbTransaction BeginTransaction(IsolationLevel il);
    void ChangeDatabase(string databaseName);
    void Close();
    IDbCommand CreateCommand();
    void Open();
}
```

Интерфейс IDbTransaction

```
public interface IDbTransaction : IDisposable
{
    IDbConnection Connection { get; }
    IsolationLevel IsolationLevel { get; }

    void Commit();
    void Rollback();
}
```

Интерфейс IDbCommand

```
public interface IDbCommand : IDisposable
{
    string CommandText { get; set; }
    int CommandTimeout { get; set; }
    CommandType CommandType { get; set; }
    IDbConnection Connection { get; set; }
    IDataParameterCollection Parameters { get; }
    IDbTransaction Transaction { get; set; }
    UpdateRowSource UpdatedRowSource { get; set; }

    void Cancel();
    IDataParameter CreateParameter();
    int ExecuteNonQuery();
    IDataReader ExecuteReader();
    IDataReader ExecuteReader(CommandBehavior behavior);
    object ExecuteScalar();
    void Prepare();
}
```

Интерфейсы IDbDataParameter и IDataParameter

```
public interface IDbDataParameter : IDataParameter
{
    byte Precision { get; set; }
    byte Scale { get; set; }
    int Size { get; set; }
}
```

```
public interface IDataParameter
{
    DbType DbType { get; set; }
    ParameterDirection Direction { get; set; }
    bool IsNullable { get; }
    string ParameterName { get; set; }
    string SourceColumn { get; set; }
    DataRowVersion SourceVersion { get; set; }
    object Value { get; set; }
}
```

Интерфейсы IDbDataAdapter и IDataAdapter

```
public interface IDbDataAdapter : IDataAdapter
{
    IDbCommand DeleteCommand { get; set; }
    IDbCommand InsertCommand { get; set; }
    IDbCommand SelectCommand { get; set; }
    IDbCommand UpdateCommand { get; set; }
}
```

```
public interface IDataAdapter
{
    MissingMappingAction MissingMappingAction { get; set; }
    MissingSchemaAction MissingSchemaAction { get; set; }
    ITableMappingCollection TableMappings { get; }

    int Fill(System.Data.DataSet dataSet);
    DataTable[] FillSchema(DataSet dataSet, SchemaType schemaType);
    IDataParameter[] GetFillParameters();
    int Update(DataSet dataSet);
}
```

Интерфейсы IDataReader и IDataRecord

```
public interface IDataReader :  
    IDisposable, IDataRecord  
{  
    int Depth { get; }  
    bool IsClosed { get; }  
    int RecordsAffected { get; }  
  
    void Close();  
    DataTable GetSchemaTable();  
    bool NextResult();  
    bool Read();  
}
```

```
public interface IDataRecord  
{  
    int FieldCount { get; }  
    object this[ string name ] { get; }  
    object this[ int i ] { get; }  
  
    bool GetBoolean(int i);  
    byte GetByte(int i);  
    char GetChar(int i);  
    DateTime GetDateTime(int i);  
    Decimal GetDecimal(int i);  
    float GetFloat(int i);  
    short GetInt16(int i);  
    int GetInt32(int i);  
    long GetInt64(int i);  
    ...  
    bool IsDBNull(int i);  
}
```


Категории провайдеров

- провайдеры непосредственного доступа (native providers) - обеспечивают доступ к данным путем непосредственного обращения к соответствующим источникам данных с использованием их собственных протоколов доступа:
 - **SQL Client .NET Data Provider** (MS .Net Framework, MS SQL Server 7.0 и выше);
 - **Microsoft SQL Server Mobile** (MS .Net Framework, MS SQL Server Mobile Edition);
 - **Oracle Client .NET Data Provider** (MS .Net Framework, Oracle 8.1.7 и выше);
 - **Oracle Data Provider, ODP.NET** (.Net Framework Data Provider for Oracle, Oracle 8.1.7 и выше);
- провайдеры-посредники (bridge providers) - обеспечивают доступ к данным путем использования других (более ранних технологий доступа):
 - **OLE DB .NET Data Provider** (MS .Net Framework, источники данных OLEDB):
 - SQLOLEDB - Microsoft OLE DB Provider for SQL Server;
 - MSDAORA - Microsoft OLE DB Provider for Oracle;
 - Microsoft.Jet.OLEDB.4.0 - OLE DB Provider for Microsoft Jet;
 - **ODBC .NET Data Provider** (MS .Net Framework, источники данных ODBC):
 - SQL Server;
 - Microsoft ODBC for Oracle;
 - Microsoft Access Driver (*.mdb).

<http://www.databasedrivers.com/ado/>

Пространства имен ADO.NET

- пространства имен ADO.NET можно разделить на следующие категории:
 - пространства имен, специфические для конкретного поставщика данных:
 - основное пространство имен (System.Data.SqlClient, Oracle.DataAccess.Client);
 - вспомогательные пространства имен (System.Data.Sql, System.Data.SqlTypes, Oracle.DataAccess.Types);
 - пространства имен, содержащие базовые типы библиотеки ADO.NET:
 - System.Data (определение общих типов, не связанных с конкретным источником данных - DataSet, DataTable, DataRow, IDataAdapter, IDataReader, IDataParameter, IDbTransaction etc.);
 - System.Data.Common (определение абстрактных базовых классов DbXXX).

Именованние объектов доступа к данным и пространства имен

- Connection
- Command
- CommandBuilder
- DataReader
- DataAdapter
- Transaction
- Parameter

Каждый поставщик данных, помимо базовых функций, как правило реализует также некоторые дополнительные функции, специфические для конкретного источника данных (СУБД)

Провайдер	Пространство имен	Префикс
Базовые классы	System.Data.Common (System.Data.dll)	<i>Db...</i>
.NET Framework Data Provider for SQL Server	System.Data.SqlClient (System.Data.dll)	<i>Sql...</i>
.NET Framework Data Provider for SQL Server Mobile	System.Data.SqlServerCe (System.Data.SqlServerCe.dll)	<i>SqlCe...</i>
.NET Framework Data Provider for Oracle	System.Data.OracleClient (System.Data.OracleClient.dll)	<i>Oracle...</i>
Oracle Data Provider, ODP.NET	Oracle.DataAccess.Client (Oracle.DataAccess.dll)	<i>Oracle...</i>
.NET Framework Data Provider for OLE DB	System.Data.OleDb (System.Data.dll)	<i>OleDb...</i>
.NET Framework Data Provider for ODBC	System.Data.Odbc (System.Data.dll)	<i>Odbc...</i>

Доступ к данным: пример (SQL Server)

```
{  
    // Create and open a connection.  
    SqlConnection cn = new SqlConnection();  
    cn.ConnectionString =  
        @"Data Source=(local)\SQLEXPRESS;Integrated Security=SSPI;" +  
        "Initial Catalog=AutoLot";  
    cn.Open();  
  
    // Create a SQL command object.  
    string strSQL = "Select * From Inventory";  
    SqlCommand myCommand = new SqlCommand(strSQL, cn);  
  
    // Obtain a data reader  
    SqlDataReader dr;  
    myDataReader = myCommand.ExecuteReader(CommandBehavior.CloseConnection);  
  
    // Loop over the results.  
    while (dr.Read())  
    {  
        Console.WriteLine("-> Make: {0}, Color: {1}.",  
            dr["Make"].ToString().Trim(), dr["Color"].ToString().Trim());  
    }  
  
    dr.Close();  
}
```

Доступ к данным: пример (Oracle)

```
using System;
using Oracle.DataAccess.Client;
class Sample
{
    static void Main()
    {
        // Connect to Oracle
        string constr = "User Id=scott;Password=tiger;Data Source=oracle";
        OracleConnection con = new OracleConnection(constr);

        con.Open();

        // Display Version Number
        Console.WriteLine("Connected to Oracle " + con.ServerVersion);

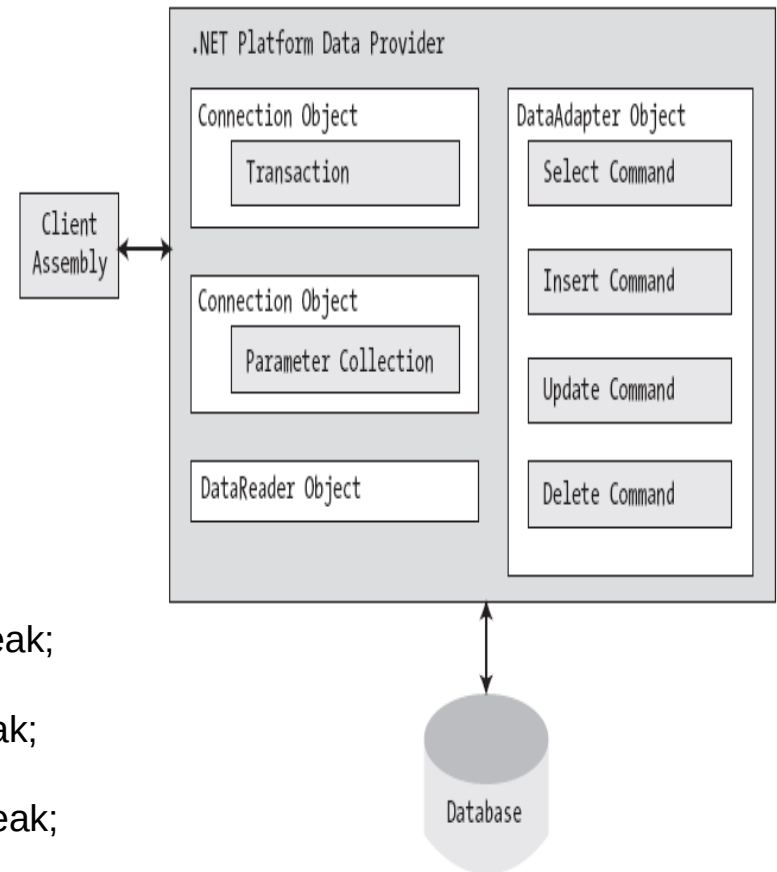
        // Close and Dispose OracleConnection
        con.Close();
        con.Dispose();
    }
}
```

Использование общих базовых интерфейсов

```
public static void OpenConnection(IDbConnection cn)
{
    // Open the incoming connection for the caller.
    cn.Open();
}

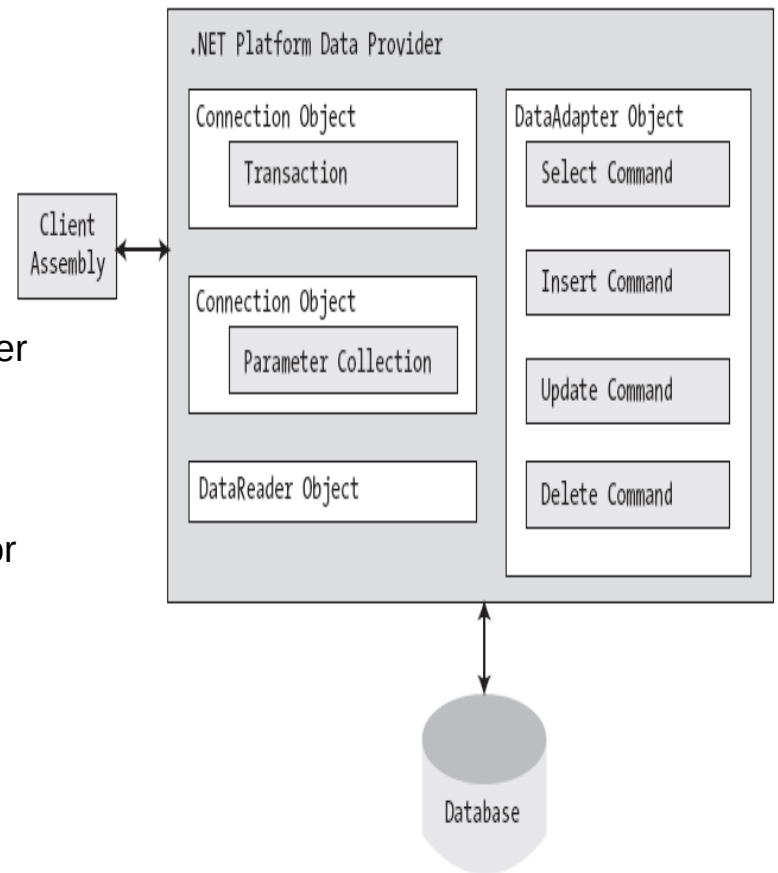
static IDbConnection GetConnection(DataProvider dp)
{
    IDbConnection conn = null;
    switch (dp)
    {
        case DataProvider.SqlServer:
            conn = new SqlConnection();break;
        case DataProvider.OleDb:
            conn = new OleDbConnection();break;
        case DataProvider.Odbc:
            conn = new OdbcConnection();break;
        case DataProvider.Oracle:
            conn = new OracleConnection();break;
    }

    return conn;
}
```



Программирование в ADO.NET 2.0+ : использование общих базовых классов (Oracle)

OracleClientFactory DbProviderFactory
OracleCommand DbCommand
OracleCommandBuilder DbCommandBuilder
OracleConnection DbConnection
OracleConnectionStringBuilder DbConnectionStringBuilder
OracleDataAdapter DbDataAdapter
OracleDataReader DbDataReader
OracleDataSourceEnumerator DbDataSourceEnumerator
OracleException DbException
OracleParameter DbParameter
OracleParameterCollection DbParameterCollection
OracleTransaction DbTransaction



Использование общих базовых классов и функций фабрики объектов поставщика (ADO.NET 2.0): DbProviderFactories.GetFactory() и DbProviderFactory

```
public abstract class DbProviderFactory
```

```
{
```

```
    ...
```

```
    public virtual DbCommand CreateCommand();
```

```
    public virtual DbCommandBuilder CreateCommandBuilder();
```

```
    public virtual DbConnection CreateConnection();
```

```
    public virtual DbConnectionStringBuilder CreateConnectionStringBuilder();
```

```
    public virtual DbDataAdapter CreateDataAdapter();
```

```
    public virtual DbDataSourceEnumerator CreateDataSourceEnumerator();
```

```
    public virtual DbParameter CreateParameter();
```

```
}
```

```
using System.Data.Common;
```

```
// Get the factory for the SQL data provider.
```

```
DbProviderFactory sqlFactory = DbProviderFactories.GetFactory("System.Data.SqlClient");
```

```
...
```

```
// Get the factory for the Oracle data provider.
```

```
DbProviderFactory oracleFactory = DbProviderFactories.GetFactory("System.Data.OracleClient");
```


Использование общих базовых классов и функций фабрики объектов поставщика (ADO.NET 2.0) : пример

```
{
    string constr = "user id=scott;password=tiger;data source=oracle";
    DbProviderFactory factory = DbProviderFactories.GetFactory("Oracle.DataAccess.Client");
    DbConnection conn = factory.CreateConnection();
    try
    {
        conn.ConnectionString = constr;
        conn.Open();
        DbCommand cmd = factory.CreateCommand();
        cmd.Connection = conn;
        cmd.CommandText = "select * from emp";
        DbDataReader reader = cmd.ExecuteReader();
        while (reader.Read()) Console.WriteLine(reader["ENAME"]);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}
```

Регистрация провайдеров ADO.NET 2.0 : файлы конфигурации (machine.config, app.config)

```
<system.data>
  <DbProviderFactories>
    <add name="Odbc Data Provider" invariant="System.Data.Odbc"
description=".Net Framework Data Provider for Odbc" type="System.Data.Odbc.OdbcFactory,
System.Data, Version=2.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089"/>
    <add name="OleDb Data Provider" invariant="System.Data.OleDb"
description=".Net Framework Data Provider for OleDb" type="System.Data.OleDb.OleDbFactory,
System.Data, Version=2.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089"/>
    <add name="OracleClient Data Provider"
invariant="System.Data.OracleClient" description=".Net Framework Data Provider for Oracle"
type="System.Data.OracleClient.OracleClientFactory, System.Data.OracleClient, Version=2.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089"/>
    <add name="SqlClient Data Provider" invariant="System.Data.SqlClient"
description=".Net Framework Data Provider for SqlServer"
type="System.Data.SqlClient.SqlClientFactory, System.Data, Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089"/>
    <add name="Microsoft SQL Server Compact Data Provider"
invariant="System.Data.SqlServerCe.3.5" description=".NET Framework Data Provider for
Microsoft SQL Server Compact" type="System.Data.SqlServerCe.SqlCeProviderFactory,
System.Data.SqlServerCe, Version=3.5.0.0, Culture=neutral,
PublicKeyToken=89845dcd8080cc91"/>
  </DbProviderFactories>
</system.data>
```

Связный уровень ADO.NET: общий сценарий работы

1. создание, настройка и открытие объекта соединения (Connection);
2. создание и настройка объекта запроса (Command), в том числе:
 - определение объекта соединения для объекта запроса;
 - определение строки запроса;
3. вызов метода ExecuteReader() объекта DataReader для получения результатов запроса;
4. прокрутка результатов выполнения запроса (DataReader.Read()).

```
SqlConnection cn = new SqlConnection();  
  
cn.ConnectionString = @"Data  
Source=(local)\SQLEXPRESS;Integrated Security=SSPI;  
Initial Catalog=AutoLot";  
  
cn.Open();  
  
  
string strSQL = "Select * From Inventory";  
  
SqlCommand myCommand = new SqlCommand(strSQL,  
cn);  
  
  
SqlDataReader myDataReader;  
  
myDataReader = myCommand.ExecuteReader();  
  
  
while (myDataReader.Read())  
{ ...}  
  
myDataReader.Close();  
  
cn.Close()
```

Connection

- содержит необходимую информацию для соединения с сервером;
- содержит информацию о текущем состоянии и позволяет его изменять;
- может использоваться пул соединений.

- BeginTransaction()
- ChangeDatabase()
- ConnectionTimeout
- Database
- DataSource
- GetSchema()
- State

```
using System.Data;
```

```
using System.Data.SqlClient;
```

```
string connectionString = "Data Source=localhost;" +
```

```
"Initial Catalog=Northwind;Integrated Security=SSPI;"
```

```
SqlConnection conn = new SqlConnection(connectionString);
```

```
conn.Open();
```

```
    // работа с базой данных
```

```
conn.Close();
```

Параметры соединения

MS SQL Server

Advanced	
MultipleActiveResultSets	False
Network Library	
Packet Size	8000
Transaction Binding	Implicit Unbind
Type System Version	Latest
Context	
Application Name	.Net SqlClient Data Provider
Workstation ID	
Initialization	
Asynchronous Processing	False
Connect Timeout	15
Current Language	
Pooling	
Enlist	True
Load Balance Timeout	0
Max Pool Size	100
Min Pool Size	0
Pooling	True
Replication	
Replication	False
Security	
Encrypt	False
Integrated Security	True
Password	
Persist Security Info	False
TrustServerCertificate	False
User ID	
Source	
AttachDbFilename	
Context Connection	False
Data Source	.\SQLEXPRESS
Failover Partner	
Initial Catalog	Northwind
User Instance	False

Oracle

Data	
ConnectionString	DATA SOURCE=oracle;USER ID=s
Misc	
Connection Life Time	
Connection Timeout	15
Context Connection	False
Data Source	oracle
DBA Privilege	
Decrement pool size	
Enlist	true
HAEvents	
Increment pool size	
Load Balancing	False
Max Pool Size	100
metadata pooling	True
Min Pool Size	1
Password	*****
Persist Security Info	False
Pooling	True
Proxy Password	
Proxy User	
Statement Cache Purge	False
Statement Cache Size	0
User ID	scott
Validate Connection	False

<http://www.connectionstrings.com/>

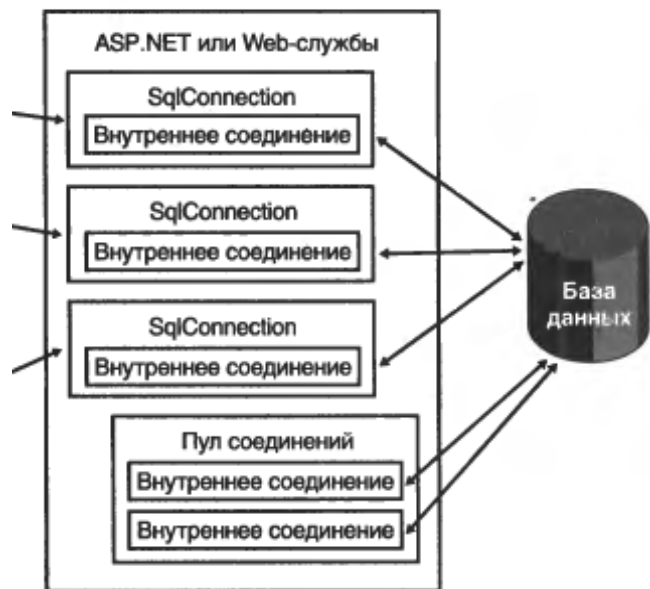
Формирование строки подключения

- непосредственное перечисление необходимых пар имя параметра – значение в строке;
- использование конструктора строк подключения (ConnectionStringBuilder), который обеспечивает строгую типизацию передаваемых параметров для конкретного поставщика данных.

```
SqlConnectionStringBuilder cnStrBuilder = new  
SqlConnectionStringBuilder();  
  
cnStrBuilder.InitialCatalog = "AutoLot";  
  
cnStrBuilder.DataSource = @"(local)\SQLEXPRESS";  
  
cnStrBuilder.ConnectTimeout = 30;  
  
cnStrBuilder.IntegratedSecurity = true;  
  
SqlConnection cn = new SqlConnection();  
cn.ConnectionString = cnStrBuilder.ConnectionString;  
  
....
```

Пул соединений

- пул соединений позволяет улучшить производительность за счет повторного использования установленного физического соединения с источником;
- пул соединений поддерживается поставщиками подключений ADO.NET и по умолчанию используется (*Pooling = true*);



- в момент соединения поставщик определяет:
 - существует ли пул для данной строки подключения;
 - есть ли в данном пуле свободные (неактивные) соединения;
- минимальный размер пула определяется параметром (*MinPoolSize = 0*):
 - в момент открытия первого соединения пула поставщик устанавливает необходимое число физических подключений к источнику;
 - поставщик поддерживает минимальное число соединений в пуле независимо от того, являются ли они активными;
- максимальный размер пула определяется параметрами (*MaxPoolSize = 100*), а в случае, когда достигнут максимальный размер пула и предпринимается попытка открытия нового соединения:
 - производится ожидание освобождения соединения пула в течении *ConnectionTimeout = 15*;
 - по истечении таймаута возникает исключение *InvalidOperationException*;
- при закрытии соединения и уничтожении объекта соединения внутреннее соединение с СУБД не разрывается, а помещается в пул для дальнейшего использования;
- соединение в пуле, которое не использовалось, уничтожается по истечении таймаута;
- возможна принудительная очистка пулов путем вызова функций *ClearPool()* и *ClearAllPools()*.

Использование файлов конфигурации (app.config) для выбора провайдера и настройки подключения

```
<?xml version="1.0" encoding="utf-8" ?>
<connectionStrings>
  <add name="ASClientCS"
        providerName="System.Data.SqlClient"
        connectionString="Server=192.168.0.92\sqli2005; Database=OneToManyMerge; User ID=sa;
                          Password=Hc6X"/>
</connectionStrings>
```

// Get the factory provider.

```
DbProviderFactory df =
DbProviderFactories.GetFactory(ConfigurationManager.ConnectionStrings["ASClientCS"].ProviderName);
```

// Now make connection object.

```
DbConnection cn = df.CreateConnection();
cn.ConnectionString = ConfigurationManager.ConnectionStrings["ASClientCS"].ConnectionString;
cn.Open();
```

// Make command object.

```
DbCommand cmd = df.CreateCommand();
cmd.Connection = cn;
cmd.CommandText = "Select * From Inventory";
```

```
DbDataReader dr = cmd.ExecuteReader(CommandBehavior.CloseConnection);
while (dr.Read()) {...}
dr.Close();
```


Command

- ПОЗВОЛЯЕТ ВЫПОЛНИТЬ:
 - команду языка SQL (*CommandType == Text*);
 - хранимую процедуру (*CommandType == StoredProcedure*);
- с точки зрения возвращаемых данных запросы могут быть отнесены к одному из трех типов:
 - не возвращающие результатов [INSERT, UPDATE, DELETE] : *ExecuteNonQuery()*;
 - возвращающие одиночное (скалярное) значение [SELECT COUNT(...)...]: *ExecuteScalar()*;
 - возвращающие набор результатов : *ExecuteReader()*;
- SqlCommand позволяет организовать асинхронный доступ к данным:
 - *BeginExecuteReader()* / *EndExecuteReader()*;
 - *BeginExecuteNonQuery()* / *EndExecuteNonQuery()*;
 - *BeginExecuteXmlReader()* / *EndExecuteXmlReader()*;

- *CommandTimeout* (= 30)
- *Connection*
- *Parameters*
- *Cancel()*
- *ExecuteReader()*
- *ExecuteNonQuery()*
- *ExecuteScalar()*
- *ExecuteXmlReader()*
- *Prepare()*

```
public enum
CommandType
{
    StoredProcedure,
    TableDirect,
    Text    // Default value.
}
```

ExecuteNonQuery() : пример

```
string connectionString = "Data Source=localhost;" +  
"Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);  
conn.Open();  
  
SqlCommand cmd = conn.CreateCommand();  
cmd.CommandText =  
    "DELETE FROM Customers WHERE CustomerID = 'ALFKI'";  
Console.WriteLine(  
    "RecordsAffected" + cmd.ExecuteNonQuery());  
  
conn.Close();
```

ExecuteScalar() : пример

```
string connectionString = "Data Source=localhost;" +  
"Initial Catalog=Northwind;Integrated Security=SSPI;"  
  
SqlConnection conn = new SqlConnection(connectionString);  
conn.Open();  
  
SqlCommand cmd = conn.CreateCommand();  
cmd.CommandText = "SELECT COUNT(*) FROM Customers";  
Console.WriteLine(Convert.ToInt32(cmd.ExecuteScalar()));  
  
conn.Close();
```

ExecuteReader() : пример

```
string connectionString = "Data Source=localhost;" +  
"Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);  
conn.Open();  
  
SqlCommand cmd = new SqlCommand(  
    "SELECT CustomerID, CompanyName FROM Customers", conn);  
  
SqlDataReader rdr = cmd.ExecuteReader();  
while (rdr.Read()) {  
    Console.WriteLine(  
        rdr["CustomerID"] + " - " + rdr["CompanyName"]);  
}  
rdr.Close();  
conn.Close();
```

DataReader

- обеспечивает последовательный доступ к результатам запроса;
- требует наличия постоянного активного подключения к базе данных в процессе работы;
- не позволяет возвращаться к предыдущим записям;
- не позволяет вносить изменения в данные;
- позволяет прочитать несколько наборов результатов;
- перегруженный индексатор позволяет осуществлять доступ к полям записи как по индексу, так и по имени.

DataReader : пример

```
string connectionString = "Data Source=localhost;" +  
"Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);  
conn.Open();  
  
SqlCommand cmd = new SqlCommand(  
    "SELECT CustomerID, CompanyName FROM Customers;" +  
    "SELECT OrderID, CustomerID FROM Orders", conn);  
  
SqlDataReader rdr = cmd.ExecuteReader();  
// SqlDataReader rdr = cmd.ExecuteReader(CommandBehavior.CloseConnection);  
  
do {  
    while (rdr.Read()) {  
        Console.WriteLine(rdr[0] + " - " + rdr[1]);  
    }  
    Console.WriteLine();  
} while (rdr.NextResult());  
  
rdr.Close();  
  
conn.Close();
```

Использование параметризованных запросов: Parameter

```
string sql = string.Format("Insert Into Inventory" +  
"(CarID, Make, Color) Values (@CarID, @Make, @Color)");
```

// This command will have internal parameters.

```
using(SqlCommand cmd = new SqlCommand(sql, this.sqlCn))  
{
```

// Fill params collection.

```
SqlParameter param = new SqlParameter();  
param.ParameterName = "@CarID";  
param.Value = id;  
param.SqlDbType = SqlDbType.Int;  
cmd.Parameters.Add(param);
```

```
param = new SqlParameter();  
param.ParameterName = "@Make";  
param.Value = make;  
param.SqlDbType = SqlDbType.Char;  
param.Size = 10;  
cmd.Parameters.Add(param);
```

```
param = new SqlParameter();  
param.ParameterName = "@Color";  
param.Value = color;  
param.SqlDbType = SqlDbType.Char;  
param.Size = 10;  
cmd.Parameters.Add(param);
```

```
cmd.ExecuteNonQuery();
```

```
}
```

- DbType
- Direction
- IsNullable
- ParameterName
- Size
- Value

Выполнение хранимых процедур

```
using (SqlCommand cmd = new SqlCommand("GetPetName", this.sqlCn))
{
    cmd.CommandType = CommandType.StoredProcedure;

    SqlParameter param = new SqlParameter();
    param.ParameterName = "@carID";
    param.SqlDbType = SqlDbType.Int;
    param.Value = carID;
param.Direction = ParameterDirection.Input;
    cmd.Parameters.Add(param);

    param = new SqlParameter();
    param.ParameterName = "@petName";
    param.SqlDbType = SqlDbType.Char;
    param.Size = 10;
param.Direction = ParameterDirection.Output;
    cmd.Parameters.Add(param);

    cmd.ExecuteNonQuery();

    Name = (string)cmd.Parameters["@petName"].Value;
}
```

```
CREATE PROCEDURE GetPetName
    @carID int,
    @petName char(10)
        output
AS
SELECT @petName = PetName
from Inventory
where CarID = @carID
```


Transaction

```
using (SqlConnection connection = new SqlConnection(connectionString))
{
    connection.Open();
    SqlCommand command = connection.CreateCommand();
    SqlTransaction transaction;

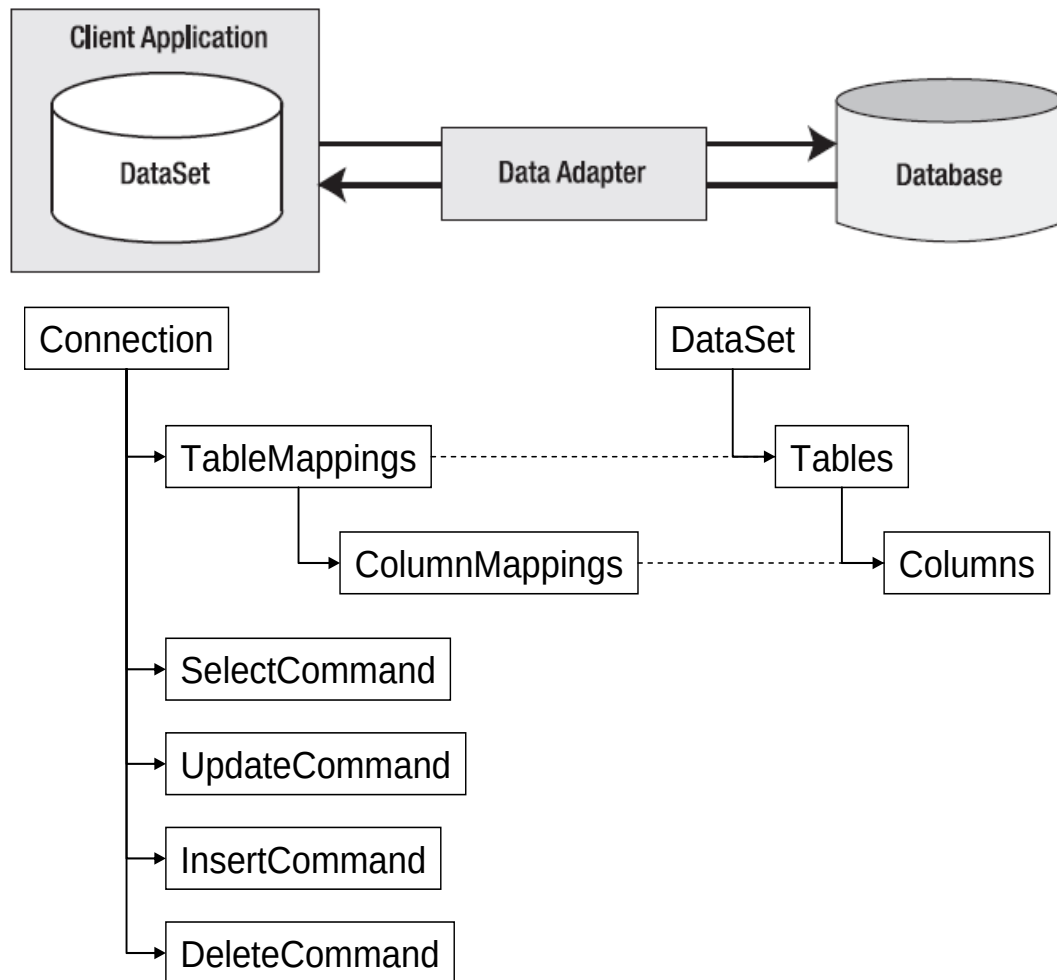
    transaction = connection.BeginTransaction("SampleTransaction");
    command.Connection = connection;
    command.Transaction = transaction;

    try
    {
        command.CommandText = "Insert into Reg (RegID, RegDesc) VALUES (1, 'D1')";
        command.ExecuteNonQuery();
        command.CommandText = "Insert into Reg (RegID, RegDesc) VALUES (2, 'D2')";
        command.ExecuteNonQuery();

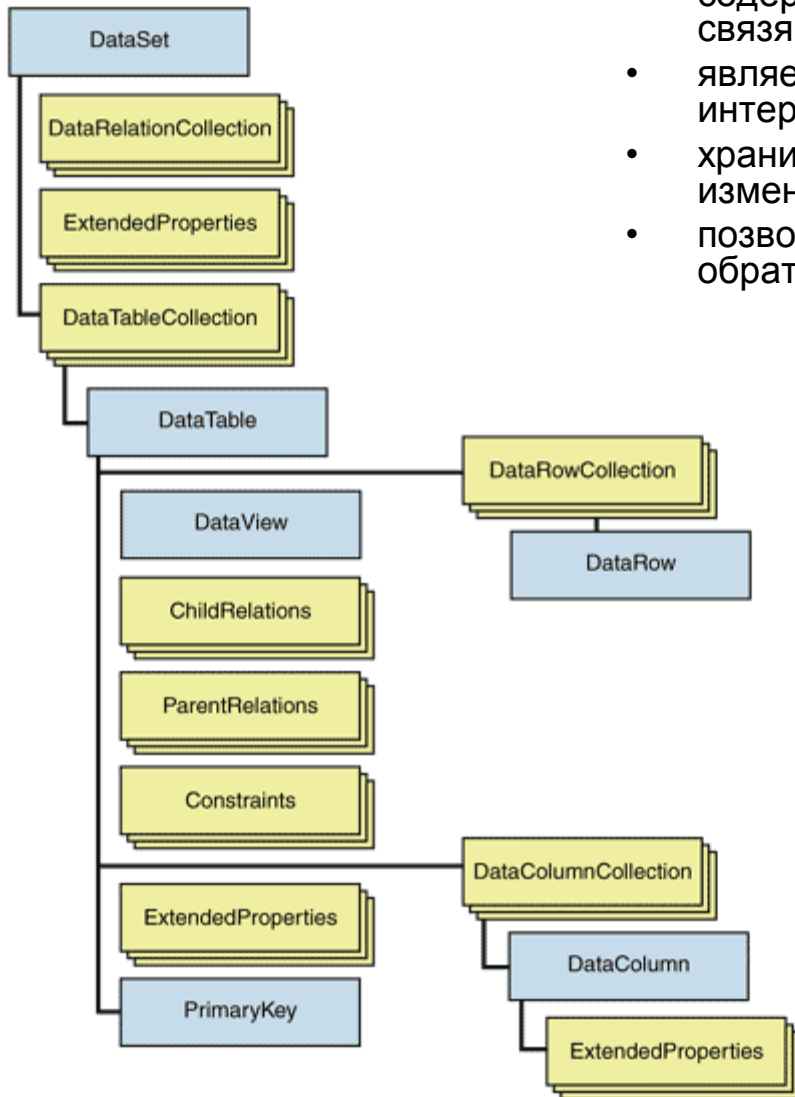
        transaction.Commit();
    }
    catch (Exception ex)
    {
        Console.WriteLine(" Message: {0}", ex.Message);
        try
        {
            transaction.Rollback();
        }
        catch (Exception ex2)
        {
            Console.WriteLine("Rollback Exception Msg: {0}", ex2.Message);
        }
    }
}
```

Несвязный уровень ADO.NET: DataAdapter

- автономно управляет подключением к источнику данных;
- получает результаты запроса и записывает их в DataSet;
- переносит в БД изменения, внесенные в DataSet;
- устанавливает соответствие между результатами запроса и таблицами / столбцами в DataSet.



DataSet



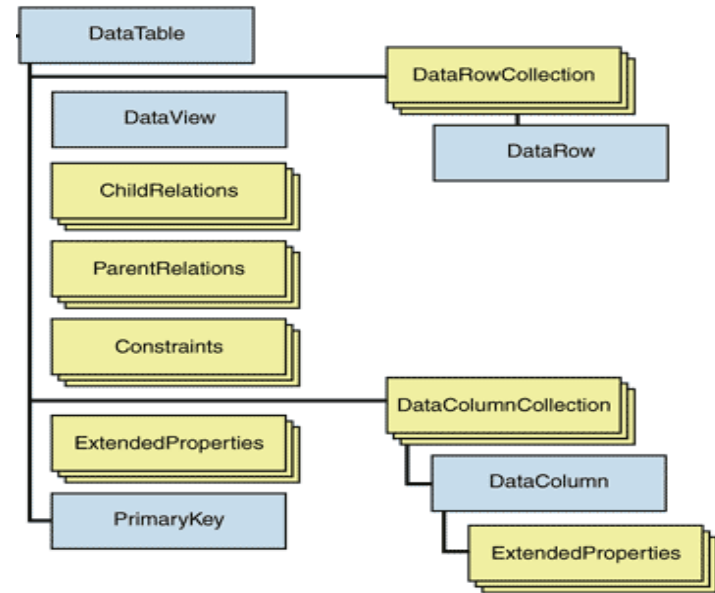
- содержит данные в виде набора таблиц с возможными связями между ними;
- является источником данных для элементов управления интерфейса пользователя;
- хранит локальную копию исходных данных и внесенных изменений;
- позволяет преобразовывать данные в XML документ и обратно, а также в двоичный формат;

- CaseSensitive
- DataSetName
- EnforceConstraints
- HasErrors
- RemotingFormat
- AcceptChanges() / RejectChanges()
- Clear()
- Clone() / Copy()
- GetChanges()
- GetChildRelations()
- GetParentRelations()
- HasChanges()
- Merge()
- ReadXml() / ReadXmlSchema()
- WriteXml() / WriteXmlSchema()

DataTable

- содержит данные в виде набора записей, состоящих из одинаковых наборов полей;
- каждая запись хранится как в исходном варианте, так и в текущем – с учетом внесенных изменений.

- CaseSensitive
- ChildRelations / ParentRelations
- Constraints / PrimaryKey
- Copy()
- DataSet
- DefaultView
- MinimumCapacity
- RemotingFormat
- Select()
- TableName
- ReadXml() / ReadXmlSchema()
- WriteXml() / WriteXmlSchema()



```
DataSet ds = new DataSet();
DataTable tbl = ds.Tables.Add("Customers");

DataColumn col = tbl.Columns.Add("CustomerID");
col.AllowDBNull = false;
col.MaxLength = 5;
col.Unique = true;

DataColumn col = tbl.Columns.Add("CompanyName");
col.MaxLength = 40;
```

DataColumn

- определяет столбец в таблице и его свойства: тип, значение по умолчанию, может ли содержать NULL, может ли значение быть изменено;
- задает выражение для вычисляемых столбцов.

- AllowDBNull (*true*) / Unique
- AutoIncrement / AutoIncrementSeed / AutoIncrementStep
- Caption
- ColumnMapping
- ColumnName (*Column1, Column2, ...*).
- DataType
- DefaultValue
- Expression
- Ordinal
- ReadOnly (*false*)
- Table

```
DataColumn carIDColumn = new DataColumn("CarID", typeof(int));  
carIDColumn.Caption = "Car ID";  
carIDColumn.ReadOnly = true;  
carIDColumn.AllowDBNull = false;  
carIDColumn.Unique = true;  
  
DataColumn carMakeColumn = new DataColumn("Make", typeof(string));  
DataColumn carColorColumn = new DataColumn("Color", typeof(string));
```

DataRow

- соответствует одной записи в таблице;
- может быть в одном из состояний:
 - `Unchanged`;
 - `Detached`;
 - `Added`;
 - `Modified`;
 - `Deleted`;
- содержит следующие версии значений полей:
 - `Original`;
 - `Current`;
 - `Proposed`;
 - `Default`.

- `HasErrors` / `GetColumnsInError()` / `GetColumnError()` / `ClearErrors()` / `RowError`
- `ItemArray`
- `RowState`
- `Table`
- `AcceptChanges()` / `RejectChanges()`
- `BeginEdit()` / `EndEdit()` / `CancelEdit()`
- `Delete()`
- `IsNull()`

```
DataTable inventoryTable = new
    DataTable("Inventory");

inventoryTable.Columns.AddRange(new
    DataColumn[]
    { carIDColumn, carMakeColumn,
      carColorColumn,
    carPetNameColumn });
```

```
carRow = inventoryTable.NewRow();
carRow[0] = 0;
carRow[1] = "Saab";
carRow[2] = "Red";
carRow[3] = "Sea Breeze";
inventoryTable.Rows.Add(carRow);
```

Состояния строк объекта DataRow

```
// temp DataTable for testing.  
DataTable temp = new DataTable("Temp");  
temp.Columns.Add(new DataColumn("TempColumn", typeof(int)));
```

// RowState = Detached.

```
DataRow row = temp.NewRow();
```

// RowState = Added.

```
temp.Rows.Add(row);
```

// RowState = Added.

```
row["TempColumn"] = 10;
```

// RowState = Unchanged.

```
temp.AcceptChanges();
```

// RowState = Modified.

```
row["TempColumn"] = 11;
```

// RowState = Deleted.

```
temp.Rows[0].Delete();
```

Объекты несвязного уровня : пример

```
DataSet carsInventoryDS = new DataSet("Car Inventory");

carsInventoryDS.ExtendedProperties["TimeStamp"] = DateTime.Now;
carsInventoryDS.ExtendedProperties["DataSetID"] = Guid.NewGuid();
carsInventoryDS.ExtendedProperties["Company"] = "Intertech";

DataColumn carIDColumn = new DataColumn("CarID", typeof(int));
carIDColumn.Caption = "Car ID";
carIDColumn.ReadOnly = true;
carIDColumn.AllowDBNull = false;
carIDColumn.Unique = true;
DataColumn carMakeColumn = new DataColumn("Make",
    typeof(string));
DataColumn carColorColumn = new DataColumn("Color",
    typeof(string));

DataTable inventoryTable = new DataTable("Inventory");
inventoryTable.Columns.AddRange(new DataColumn[]
    { carIDColumn, carMakeColumn, carColorColumn, carPetNameColumn
    });

carsInventoryDS.Tables.Add(inventoryTable);
```


DataRelation

- устанавливает связь один-к одному, один-ко-многим или многие-ко-многим между двумя столбцами или наборами столбцов;
- по умолчанию, для родительских столбцов автоматически добавляется ограничение *UniqueConstraint*, а для дочерних – *ForeignKeyConstraint*;
- позволяет определять родительские и дочерние записи для указанной записи:
 - **DataRow.GetParentRow();**
 - **DataRow.GetParentRows();**
 - **DataRow.GetChildRows().**

```
public DataRelation ( string relationName,  
                    DataColumn parentColumn,  
                    DataColumn childColumn,  
                    bool createConstraints )      // = true
```

```
DataRelation dr = new DataRelation("CustomerOrder",  
autoLotDS.Tables["Customers"].Columns["CustID"],           // parent table  
autoLotDS.Tables["Orders"].Columns["CustID"]);             // child table  
autoLotDS.Relations.Add(dr);
```

Constraint

- *Primary Key* – указывает столбцы, входящие в первичный ключ;
- *Unique* – указывает столбцы, значения которых должно быть уникально в таблице;
- *Foreign Key* – указывает столбцы, входящие во внешний ключ, а также задает правила поведения ограничений ссылочной целостности (*Cascade*, *SetNull*, *SetDefault*, *None*).

```
ForeignKeyConstraint custOrderFK = new ForeignKeyConstraint("CustOrderFK",  
    custDS.Tables["CustTable"].Columns["CustomerID"],  
    custDS.Tables["OrdersTable"].Columns["CustomerID"]);  
  
custOrderFK.DeleteRule = Rule.None;  
  
custDS.Tables["OrdersTable"].Constraints.Add(custOrderFK);
```

```
DataTable custTable = custDS.Tables["Customers"];  
  
UniqueConstraint custUnique = new UniqueConstraint(new DataColumn[]  
    {custTable.Columns["CustomerID"], custTable.Columns["CompanyName"]});  
  
custDS.Tables["Customers"].Constraints.Add(custUnique);
```

DataTableReader

- является аналогом DataReader для таблиц несвязного уровня (DataTable)

```
static void PrintTable(DataTable dt)
{
    // Get the DataTableReader type.
    DataTableReader dtReader = dt.CreateDataReader();

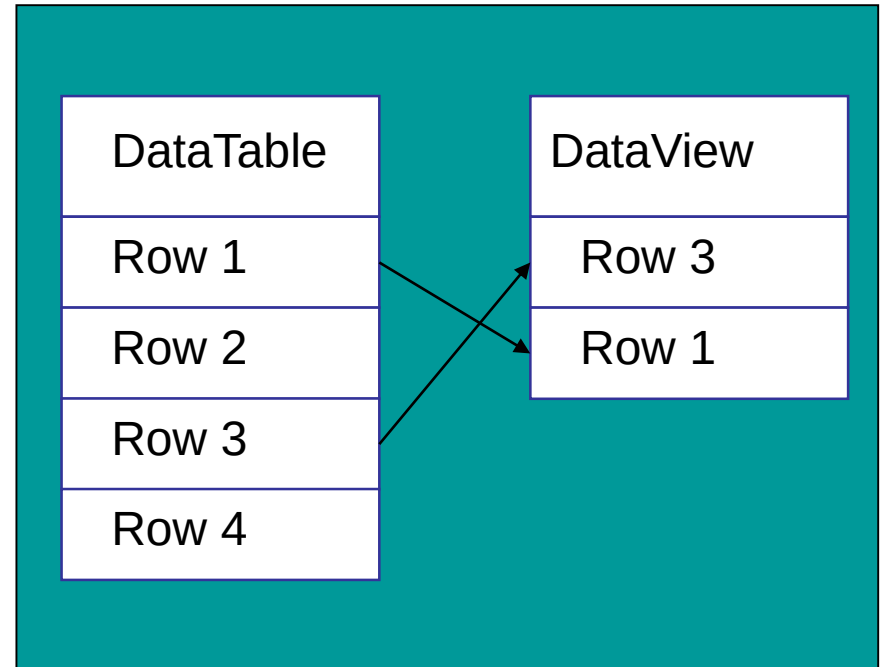
    // The DataTableReader works just like the DataReader.
    while (dtReader.Read())
    {
        for (int i = 0; i < dtReader.FieldCount; i++)
        {
            Console.Write("{0}\t",
                           dtReader.GetValue(i).ToString());
        }

        Console.WriteLine();
    }

    dtReader.Close();
}
```

DataView

- отбирает записи, удовлетворяющие заданному условию;
- отбирает записи, находящиеся в определенном состоянии;
- сортирует записи по значениям указанных столбцов;
- МОЖЕТ ОСНОВЫВАТЬСЯ ТОЛЬКО на одной таблице.



```
coltsOnlyView = new DataView(inventoryTable);  
// Now configure the views using a filter.  
coltsOnlyView.RowFilter = "Make = 'Colt'";  
// Bind to the new grid.  
dataGridColtsView.DataSource = coltsOnlyView;
```

Создание DataView

```
DataTable tbl = new DataTable("Customers");
```

```
// Инициализация таблицы Customers ...
```

```
DataViewRowState dvrs =  
    DataViewRowState.ModifiedOriginal |  
    DataViewRowState.Deleted;
```

```
DataView view = new DataView();  
view.Table = tbl;  
view.RowFilter = "Country = 'USA'";  
view.Sort = "City DESC";  
view.RowStateFilter = dvrs;
```

```
DataView view2 = new DataView(tbl,  
    "Country = 'USA'", "City DESC", dvrs);
```

Загрузка данных из БД и отправка измененных данных

```
string connectionString = "Data Source=localhost;" +  
    "Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);  
  
SqlDataAdapter daCustomers = new SqlDataAdapter("SELECT * FROM Customers", conn);  
SqlDataAdapter daOrders = new SqlDataAdapter("SELECT * FROM Orders", conn);  
SqlDataAdapter da = new SqlDataAdapter(  
    "SELECT OrderID, ProductID, Quantity, UnitPrice FROM [Order Details]", conn);  
  
ds = new DataSet();  
SqlCommandBuilder cb = new SqlCommandBuilder(da);  
  
daCustomers.Fill(ds);  
daOrders.Fill(ds);  
da.Fill(ds);  
  
...  
  
da.Update(tbl);
```

- Fill()
- SelectCommand
- InsertCommand
- UpdateCommand
- DeleteCommand
- Update()

Настройка запросов DataAdapter

```
SqlCommand cmd = new SqlCommand(  
    "UPDATE [Order Details] " +  
    "SET OrderID = ?, ProductID = ?, " +  
    "Quantity = ?, UnitPrice = ? " +  
    "WHERE OrderID = ? AND ProductID = ? AND " +  
    "Quantity = ? AND UnitPrice = ?", conn);
```

```
SqlParameterCollection pc = cmd.Parameters;  
pc.Add("OrderID_New", OleDbType.Integer, 0, "OrderID");  
pc.Add("ProductID_New", OleDbType.Integer, 0, "ProductID");  
pc.Add("Quantity_New", OleDbType.SmallInt, 0, "Quantity");  
pc.Add("UnitPrice_New", OleDbType.Currency, 0, "UnitPrice");
```

```
SqlParameter param;  
param = pc.Add("OrderID_Orig", OleDbType.Integer, 0, "OrderID");  
param.SourceVersion = DataRowVersion.Original;  
param = pc.Add("ProductID_Orig", OleDbType.Integer, 0, "ProductID");  
param.SourceVersion = DataRowVersion.Original;  
param = pc.Add("Quantity_Orig", OleDbType.SmallInt, 0, "Quantity");  
param.SourceVersion = DataRowVersion.Original;  
param = pc.Add("UnitPrice_Orig", OleDbType.Currency, 0, "UnitPrice");  
param.SourceVersion = DataRowVersion.Original;
```

```
dataAdapter.UpdateCommand = cmd;
```

Настройка запросов DataAdapter: использование CommandBuilder

```
string = "Data Source=localhost;" +  
    "Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);
```

```
SqlDataAdapter da = new SqlDataAdapter(  
    "SELECT OrderID, ProductID, Quantity, UnitPrice FROM [Order Details]",  
    connectionString);
```

```
ds = new DataSet();  
SqlCommandBuilder cb = new SqlCommandBuilder(da);
```

```
da.Fill(ds);
```

```
...
```

```
da.Update(tbl);
```

- ограничения использования CommandBuilder:
 - в запросе выборки участвует единственная таблица;
 - для данной таблицы определен первичный ключ;
 - поля первичного ключа содержатся в запросе выборки и объекте DataTable.

Вопросы к экзамену

- ADO.NET: режимы работы, объектная модель, базовые интерфейсы и базовые классы. Поставщики данных.
- Связный уровень ADO.NET. Объекты связанного уровня и общий сценарий работы.
- Связный уровень ADO.NET. Общий сценарий работы. Пул подключений.
- Связный уровень ADO.NET. Поставщики данных. Создание программного кода, не зависящего от поставщика.
- Связный уровень ADO.NET. Параметризованные запросы.
- Несвязный уровень ADO.NET. Объекты несвязного уровня и общий сценарий работы.
- Несвязный уровень ADO.NET. Настройка запросов для синхронизации с источником данных.

Лабораторная работа №12.

Программирование клиентского приложения доступа к данным

1. Создать пользовательское приложение с использованием технологии ADO.NET для выполнения следующих операций:
 - просмотра содержимого таблиц;
 - вставки, удаления, изменения данных в этих таблицах.
2. Сравнить на практике использование для некоторой таблицы:
 - несвязного уровня ADO.NET;
 - связанного уровня ADO.NET.
3. Для хранения строки подключения к источнику данных использовать конфигурационный файл приложения (app.config).

Базы данных

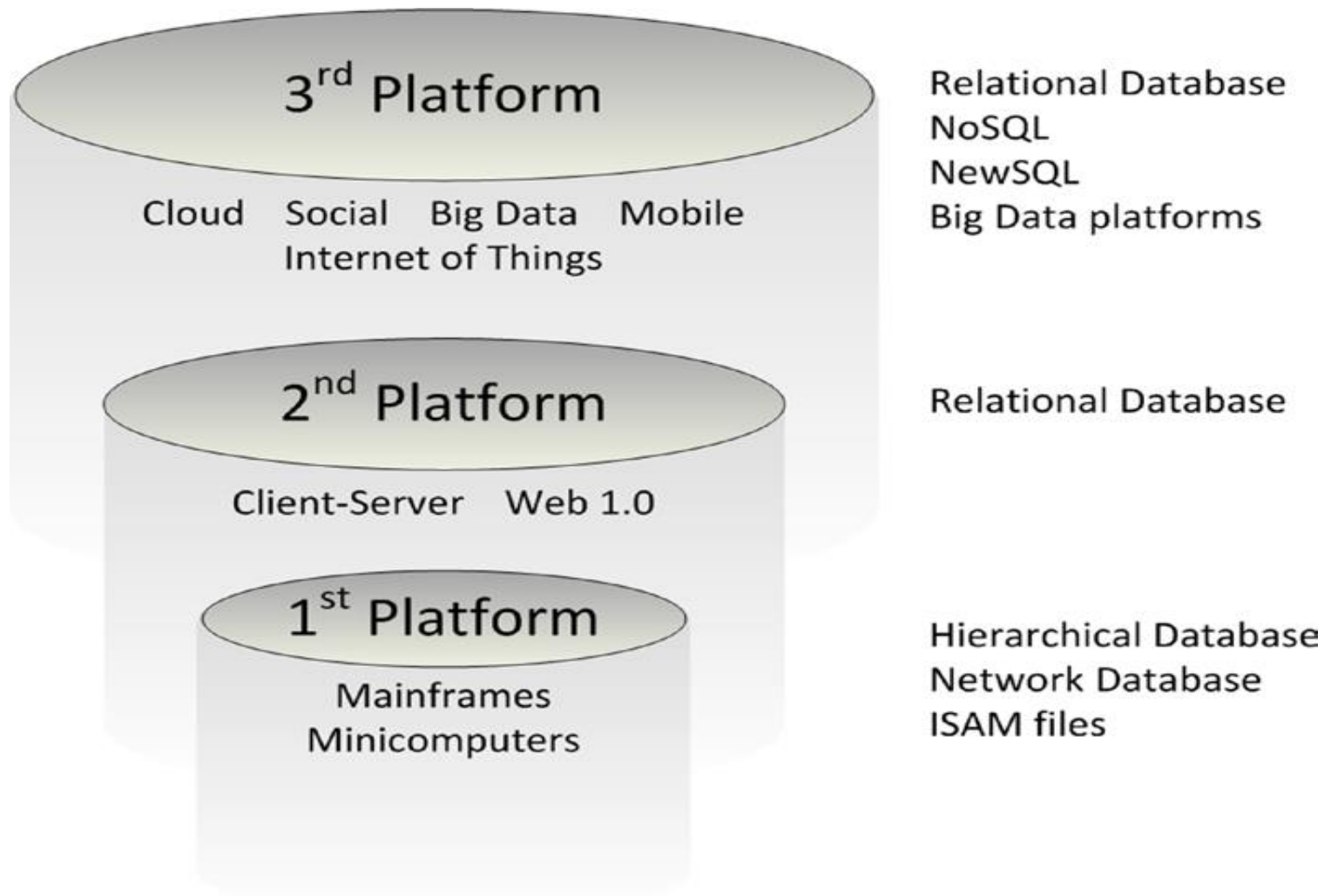
Тема 14

Логические модели баз данных

История развития баз данных

- крупные БД (системы обработки транзакций масштаба крупных организаций) → настольные БД → сетевые БД → интернет:
 - до 1970-х – переход от ведения записей вручную к системам обработки файлов (file-processing systems);
 - 1970-1980 – появление первых баз данных (dBase, ADABAS, Total, System2000, IDMS, IMS);
 - 1978-1985 – появление реляционных баз данных (DB2, Oracle);
 - 1982-1992 – развитие баз данных для настольных компьютеров (dBase-II, Paradox, Access, dBase-IV, FoxPro);
 - 1985-2000 – появление и развитие объектно-ориентированных баз данных;
 - 1995-н.в. – интеграция технологий баз данных и интернет-технологий (в т.ч. NoSQL);

Базы данных и технологические платформы

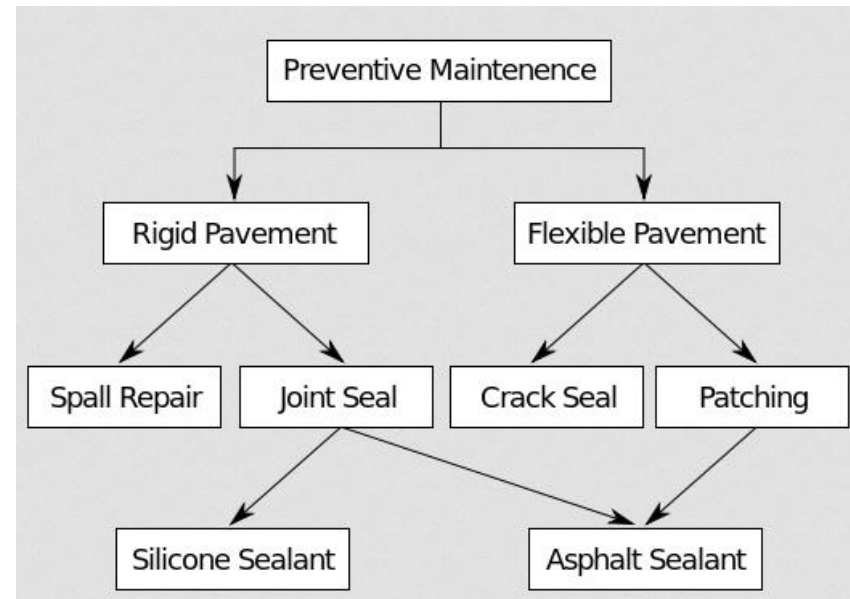
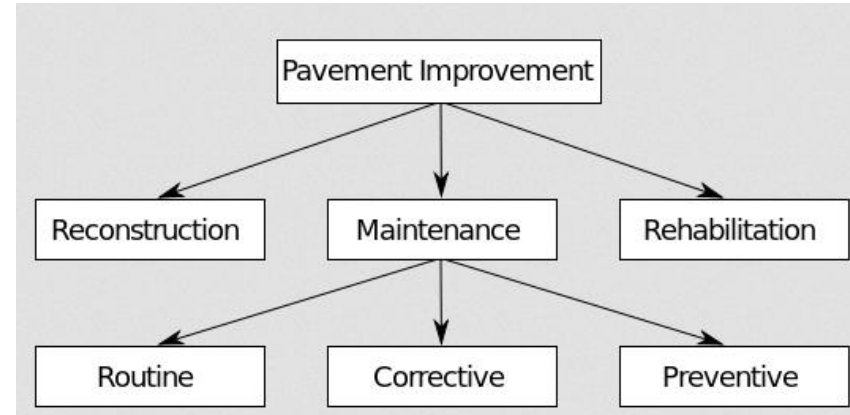


Логические модели баз данных

- логическая модель (*logical data model, information model*) базы данных определяет:
 - набор поддерживаемых типов структур данных;
 - набор допустимых операций над поддерживаемыми структурами данных;
 - набор общих правил целостности данных, явно или неявно определяющих корректные состояния базы данных или их изменения;
- модели баз данных:
 - иерархическая (*hierarchical model*);
 - сетевая (*network model*);
 - реляционная (*relational model*);
 - объектно-ориентированные (*object-oriented model*);
 - пост-реляционные модели, NoSQL (Not only SQL) базы данных:
 - хранилища ключей и значений (*KV-store, key-value store, KVP, key-value pairs*);
 - документо-ориентированные базы данных (*document-oriented databases*);
 - графовые базы данных (*graph databases*);
 - столбцовые базы данных (*column store, wide-column store*);
 - временные базы данных (*time-series databases*);
- ряд СУБД поддерживают несколько логических моделей данных.

Иерархическая и сетевая модели

- иерархическая модель (hierarchical model) – модель данных, в которой данные представляют собой древовидную структуру;
 - пример: реестр Windows (Windows Registry);
- сетевая модель (network model) – модель данных с сетевой структурой, возможно наличие циклов (как на уровне модели данных, так и на уровне самих данных).



Реляционная модель

- реляционная модель (*relational database model*)
 - введена в 1970г. Э.Ф.Коддом (E.F.Codd, A Relational Model of Data for Large Shared Databanks);
 - основана на применении концепции реляционной алгебры к проблеме хранения больших объемов данных;
 - определяет способ хранения данных в виде таблиц со строками и столбцами, при этом минимизируется дублирование и исключаются определенные типы ошибок обработки;
 - дает стандартный способ структурирования и обработки данных;
- нормализация (*normalization*) – последовательность преобразований, обеспечивающих определенный набор требований.

EmpNum	EmpName	DeptNum	DeptName
100	Jones	10	Accounting
150	Lau	20	Marketing
200	McCauley	10	Accounting
300	Griffin	10	Accounting

(a) One-Table Design

OR?

DeptNum	DeptName
10	Accounting
20	Marketing

EmpNum	EmpName	DeptNum
100	Jones	10
150	Lau	20
200	McCauley	10
300	Griffin	10

(b) Two-Table Design

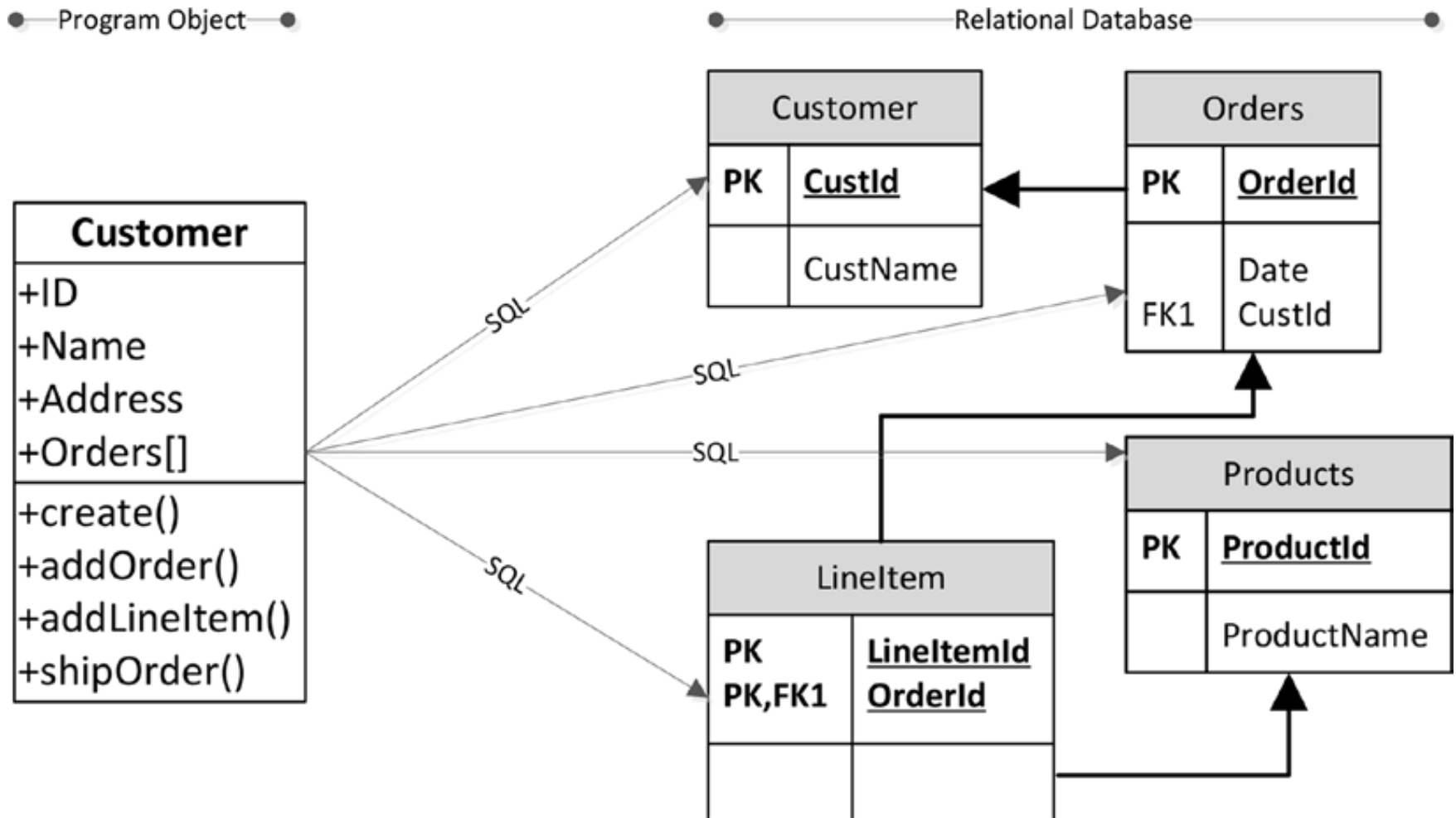
Отношение

- отношение (*relation*) – двумерная таблица, обладающая следующими свойствами:
 - каждая строка содержит данные, описывающие некоторую сущность или ее часть;
 - каждый столбец представляет один из атрибутов сущности;
 - каждая ячейка содержит одиночное значение;
 - все значения в одном столбце имеют один и тот же тип;
 - каждый столбец имеет уникальное имя;
 - не важен порядок строк;
 - не важен порядок столбцов;
 - не может существовать двух идентичных строк.

Объектно-ориентированные базы данных

- object-oriented programming, объектно-ориентированное программирование (конец 1980х);
- объектно-ориентированные СУБД, object-oriented DBMS – предназначены для прозрачной и унифицированной обработки структур данных ООП, так как реляционная модель для этого не вполне подходит:
 - сложность переноса накопленных данных;
 - не обладают достаточной универсальностью (редкой является реализация поддержки наследования);
 - примеры: Cache, ObjectDB;
- Oracle: object-relational databases;
- системы ORM (*Object-relational mapping*, объектно-реляционное отображение) – системы преобразования объектов (в терминах ООП) в форму, которая подходит для их хранения в файлах и базах данных:
 - примеры: NHibernate.

Объектно-ориентированные базы данных → ORM (Object-Relational Mapping)



Постреляционные СУБД: предпосылки

- реляционные базы данных в первую очередь ориентированы на обработку транзакций, то есть выполнение большого числа незначительных (относительно общего объёма хранимой информации) модификаций данных:
 - построчная организация физического хранения данных;
 - применение строгих механизмов обеспечения свойств атомарности, согласованности, изоляции и отказоустойчивости транзакций (англ. ACID – *Atomicity, Consistency, Isolation, Durability*);
- теорема CAP (*Consistency, Availability, Partition Tolerance*): в распределённой компьютерной системе невозможно постоянно и строго гарантировать выполнение свойств согласованности, доступности и устойчивости к потере связи между узлами:
 - *согласованность* означает получение в каждый момент времени одинаковых данных всеми пользователями, осуществляющими их чтение;
 - *доступность* означает возможность пользователя выполнить операцию в конкретный момент времени;
 - *устойчивость к потере связности* – полное или частичное сохранение работоспособности хранилища в случае разрыва соединения между различными группами его узлов (учитывая необходимость обеспечить заданное время отклика при высокопроизводительной обработке данных).

Реляционные базы данных: хранение

Block	ID	Name	DOB	Salary	Sales	Expenses
1	1001	Dick	21/12/1960	67,000	78980	3244
2	1002	Jane	12/12/1955	55,000	67840	2333
3	1003	Robert	17/02/1980	22,000	67890	6436
4	1004	Dan	15/03/1975	65,200	98770	2345
5	1005	Steven	11/11/1981	76,000	43240	3214

Row-oriented storage

ID	Name	DOB	Salary	Sales	Expenses
1001	Dick	21/12/1960	67,000	78980	3244
1002	Jane	12/12/1955	55,000	67840	2333
1003	Robert	17/02/1980	22,000	67890	6436
1004	Dan	15/03/1975	65,200	98770	2345
1005	Steven	11/11/1981	76,000	43240	3214

Tabular data

Columnar storage

Block	Dick	Jane	Robert	Dan	Steven
1					
2	21/12/1960	12/12/1955	17/02/1980	15/03/1975	11/11/1981
3	67,000	55,000	22,000	65,200	76,000
4	78980	67840	67890	98770	43240
5	3244	2333	6436	2345	3214

Block	ID	Name	DOB	Salary	Sales	Expenses
1	1001	Dick	21/12/1960	67,000	78980	3244
2	1002	Jane	12/12/1955	55,000	67840	2333
3	1003	Robert	17/02/1980	22,000	67890	6436
4	1004	Dan	15/03/1975	65,200	98770	2345
5	1005	Steven	11/11/1981	76,000	43240	3214

Storage in row format

```
SELECT SUM(salary)
FROM saleperson
```

Storage in columnar format

Block	Dick	Jane	Robert	Dan	Steven
1					
2	21/12/1960	12/12/1955	17/02/1980	15/03/1975	11/11/1981
3	67,000	55,000	22,000	65,200	76,000
4	78980	67840	67890	98770	43240
5	3244	2333	6436	2345	3214

Block	ID	Name	DOB	Salary	Sales	Expenses
1	1001	Dick	21/12/1960	67,000	78980	3244
2	1002	Jane	12/12/1955	55,000	67840	2333
3	1003	Robert	17/02/1980	22,000	67890	6436
4	1004	Dan	15/03/1975	65,200	98770	2345
5	1005	Steven	11/11/1981	76,000	43240	3214

Row-oriented storage

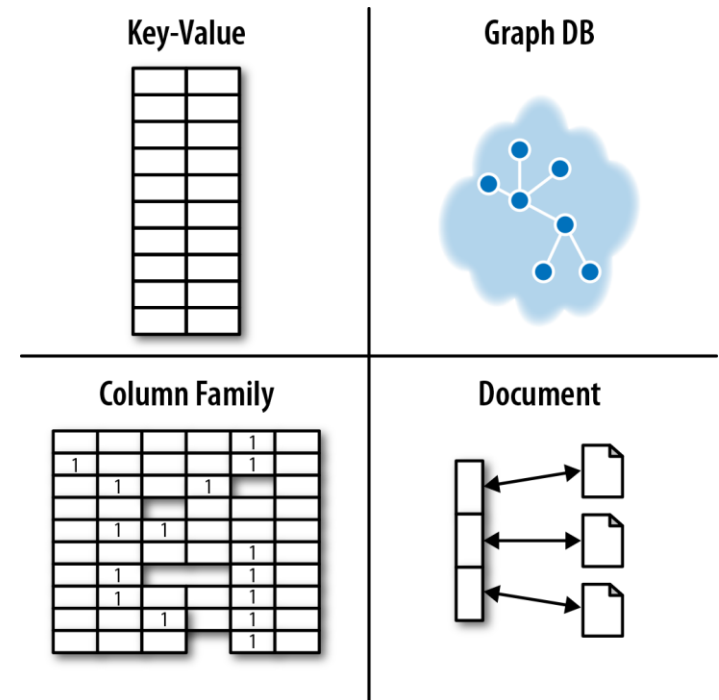
```
INSERT INTO saleperson
```

Columnar storage

Block	Dick	Jane	Robert	Dan	Steven
1					
2	21/12/1960	12/12/1955	17/02/1980	15/03/1975	11/11/1981
3	67,000	55,000	22,000	65,200	76,000
4	78980	67840	67890	98770	43240
5	3244	2333	6436	2345	3214

Постреляционные СУБД

- восполняют недостаток функциональности в традиционных (реляционных СУБД)
 - хранилища ключей и значений (KV-store, key-value store, KVP, key-value pairs);
 - документо-ориентированные (document-oriented model);
 - графовые базы данных (graph databases);
 - столбцовые базы данных (column store, wide-column store);
 - временные базы данных (time-series databases);
- многостороннее хранение (polyglot persistence).

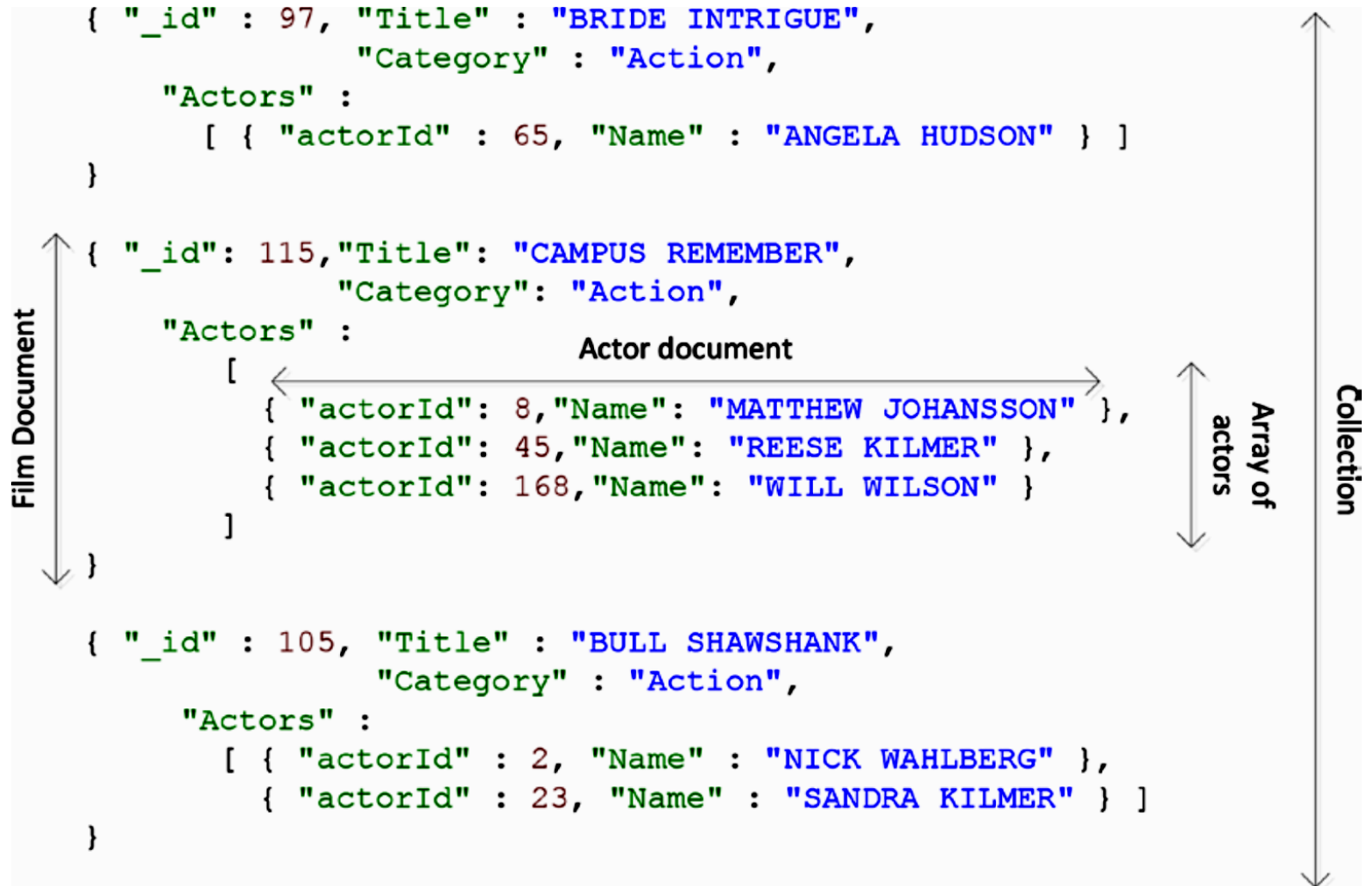


Хранилища ключей и значений

- KV-store, key-value store (KVP, key-value pairs);
- хранилища сопоставляют значения ключам;
- примеры: Redis, Memcached, Amazon DynamoDB, Riak KV, Aerospike // memcached (memcachedb, membase), MS Velocity, Tokyo Cabinet, MongoDB, Tarantool, Apache Cassandra, Google BigTable;
- дополнительные возможности:
 - поддержка различных типов значений: текст, изображения, XML-документы, составные типы данных (например, списки, отсортированные множества)
- могут рассматриваться более сложные способы организации данных на уровне хранения (тройки /triplestore//subject-predicate-object, entity-attribute-value model/ и т.д.).

Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

Документо-ориентированные базы данных: пример документа (JSON)



Документо-ориентированные базы данных

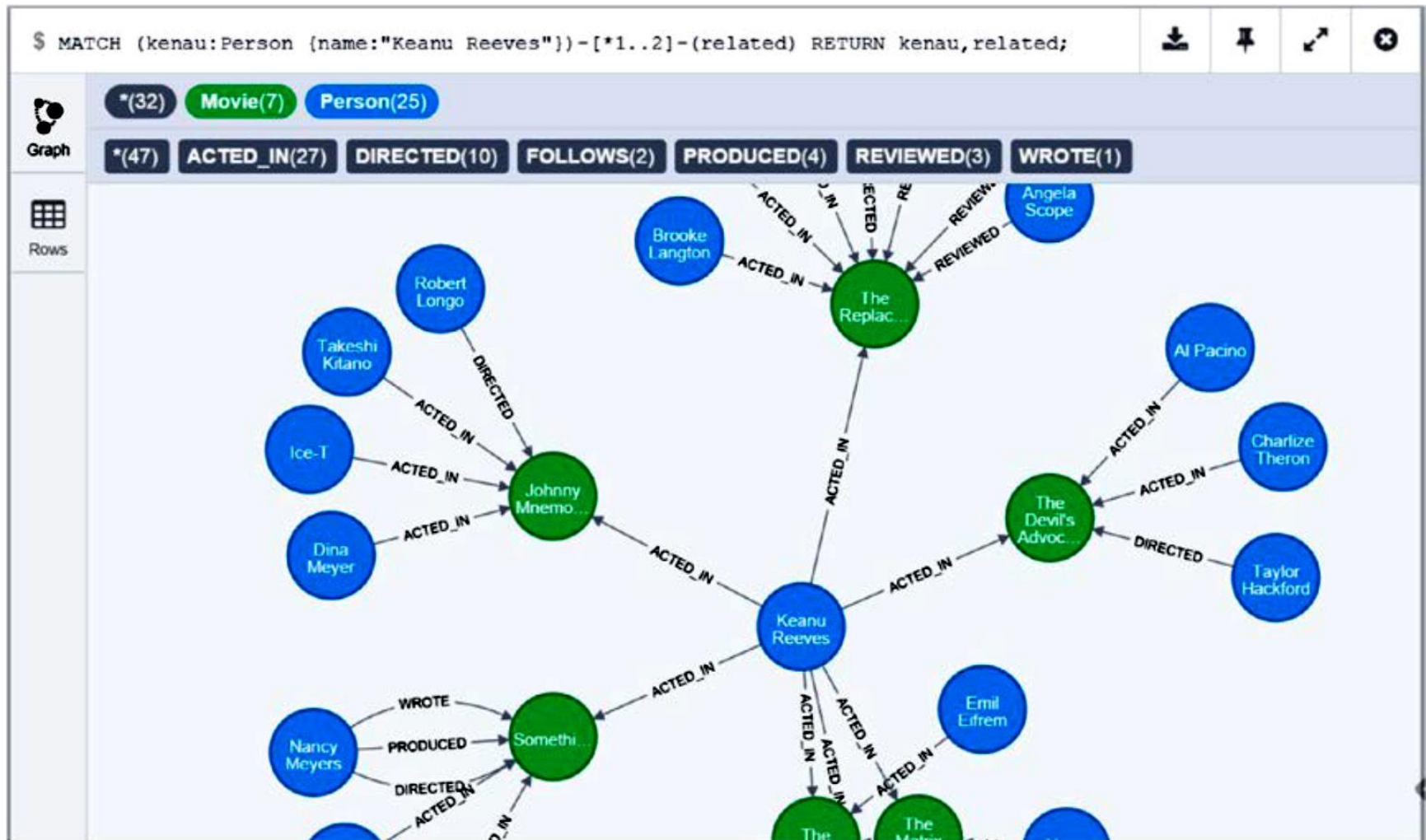
- документо-ориентированные базы данных (document-oriented databases) предназначены для хранения и обработки документо-ориентированной (полуструктурированной, semi-structured) информации;
- основаны на понятии *документа*, как сущности, включающей в себя некоторый набор данных, закодированных в стандартных форматах (XML, YML, JSON, BSON, PDF, MS Office);
- документы могут содержать вложенные структуры;
- каждый из документов может содержать в себе как общие для некоторого набора документов поля, так и уникальные, что делает структуру документа более гибкой по сравнению с реляционной моделью;
- база данных предоставляет средства извлечения документов (document retrieval) на основе содержимого документов (content) в виде API или языка запросов (query language);
- примеры: CouchDB, MongoDB, MarkLogic, Amazon DynamoDB, LotusNotes, OrientDB, ...

Столбцовые базы данных

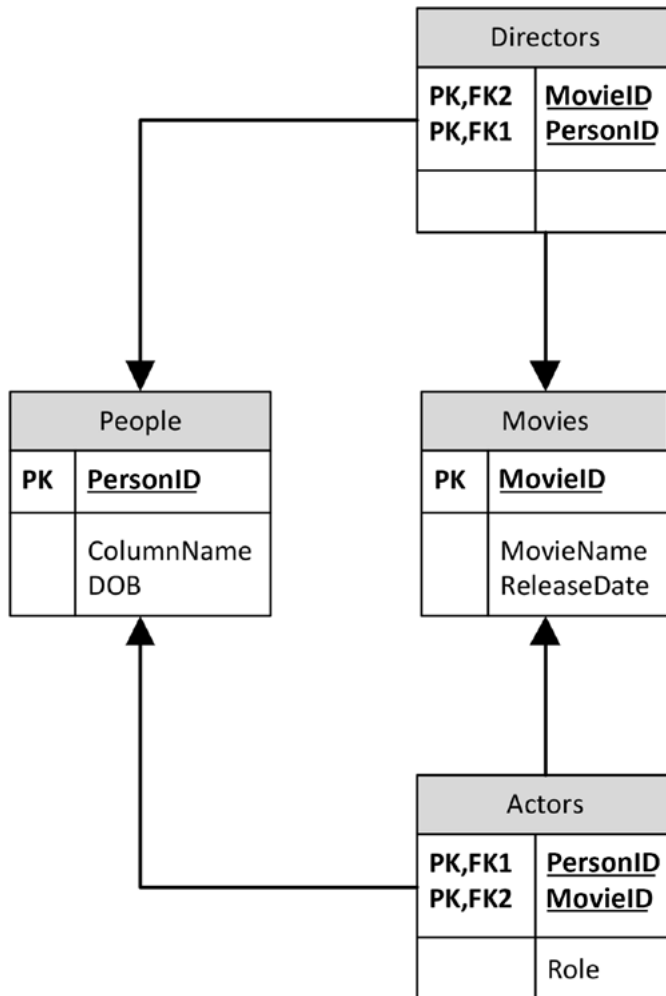
- столбцовые базы данных (column-oriented databases) ориентированы на хранение данных по столбцам, в отличие от реляционных баз данных (хранение по строкам):
 - несущественные накладные расходы на добавление нового столбца к уже существующим данным;
 - гибкость схемы данных (набор столбцов у различных строк может быть разным);
 - возможна высокая степень сжатия данных для столбцов с повторяющимися значениями;
- примеры: Apache HBase, Apache Cassandra, Accumulo, Google BigTable, ...

	INFO column Family	FRIENDS column Family												
Rowkey=Guy	<table><tr><td>Name</td></tr><tr><td>Guy Harrison</td></tr></table>	Name	Guy Harrison	<table><tr><td>Jo</td><td>John</td></tr><tr><td>jo@gmail</td><td>john@hotmail</td></tr></table>	Jo	John	jo@gmail	john@hotmail						
Name														
Guy Harrison														
Jo	John													
jo@gmail	john@hotmail													
Rowkey=Jo	<table><tr><td>Name</td></tr><tr><td>Joanna S</td></tr></table>	Name	Joanna S	<table><tr><td>John</td><td>Paul</td><td>Ringo</td><td>George</td></tr><tr><td>john@hotmail</td><td>paul@yahoo</td><td>ringo@aol</td><td>george@gmail</td></tr></table>	John	Paul	Ringo	George	john@hotmail	paul@yahoo	ringo@aol	george@gmail		
Name														
Joanna S														
John	Paul	Ringo	George											
john@hotmail	paul@yahoo	ringo@aol	george@gmail											

Графовые данные



Графовые данные в реляционной СУБД



```
1 SELECT p2.personname, m1.movieName
2 FROM people p1
3 JOIN actors a1 ON (p1.personid = a1.personid)
4 JOIN movies m1 ON (a1.movieid = m1.movieid)
5 JOIN actors a2 ON (a2.movieid = m1.movieid)
6 JOIN people p2 ON (p2.personid = a2.personid)
7 WHERE p1.personname = 'Keanu Reeves';
8
```

Графовые базы данных

- графовые базы данных (*graph databases*) предназначены для хранения структуры графа: узлов и связей между ними;
- как с узлами, так и со связями могут быть ассоциированы свойства (атрибуты), представляющие собой пару ключ-значение и позволяющие хранить дополнительные данные;
- язык запросов или API графовых баз данных, а также оптимизация производительности ориентированы на предоставление возможности максимально удобного и быстрого обхода узлов по связям (поиск в ширину, нахождение подграфов и т.д.);
- примеры: Neo4j, OrientDB, AllegroGraph, HypergraphDB, FlockDB, DEX, SonesDB ...
- реализации вычислительных библиотек (*graph computing engines*) для выполнения графовых операций в хранилищах данных различных типов (Apache Giraph, Titan, Apache GraphX, Oracle Graph)

Вопросы к экзамену

- Модели баз данных: иерархическая и сетевая модели, реляционная модель. Построчное и столбцовое хранение в реляционных СУБД.
- Модели баз данных: обзор постреляционных моделей (объектно-ориентированные базы данных, хранилища ключей и значений, документо-ориентированные, графовые и столбцовые базы данных).